

Practical Verification of Quantum Circuit Transformations

*Vérification Pratique des Transformations de
Circuits Quantiques*

Thèse de doctorat de l'université Paris-Saclay et du CEA/LIST

École doctorale n° 580, Sciences et technologies de l'information et de la
communication (STIC)

Spécialité de doctorat: informatique

Graduate School : Informatique et sciences du numérique. Référent : **à remplir**

Thèse préparée au **CEA-LIST, LMF**,
sous la direction de **Benoît VALIRON**, professeur à CentraleSupélec,
et le co-encadrement de **Christophe CHARETON**, ingénieur de recherche au CEA-LIST

Thèse soutenue à Paris-Saclay, le 04 juin 2026, par

Jérôme RICCIARDI

Composition du jury

Membres du jury avec voix délibérative

Miriam BACKENS

ISFP, INRIA, Université de Lorraine

Sylvie BOLDOT

Directrice de recherche, UPS, INRIA, CNRS, ENS, LMF

Aleks KISSINGER

Professor, University of Oxford

Alessandra DI PIERRO

Associate Professor, University of Verona

Titre: Vérification pratique des transformations de circuits quantiques

Mots clés: Calcul quantique - Circuit hybride - Transformation de circuits - Vérification

Résumé: L'informatique quantique, qui repose sur les principes de la physique quantique, promet des gains significatifs en termes de puissance de calcul. Cependant, elle introduit de nouveaux défis, notamment pour la vérification de programmes. En effet, les techniques de vérification classiques ne s'appliquent pas directement à l'informatique quantique. Cela s'explique principalement par l'utilisation de la mémoire quantique (unité d'information non standard) et de son opération de mesure, qui est le seul moyen de récupérer une information à partir d'un bit quantique et introduit un aspect probabiliste.

Dans cette thèse, nous nous concentrons sur le modèle de calcul quantique par circuits et sur un aspect pratique de ce modèle : les transformations de circuit. Un circuit quantique peut être purement quantique (circuit unitaire) ou inclure du contrôle classique et de la mesure (circuit hybride). Les transformations de circuits se produisent tout au long du processus de calcul quantique, qu'il s'agisse de compiler un circuit décrit dans un langage de haut niveau vers un langage de bas niveau, d'optimiser un circuit pour un matériel particulier (transpilation), de changer de modèle de calcul (one-way measurement [29]), ou de transférer un état quantique (téléportation).

Chacune de ces transformations peut altérer le comportement du circuit ; nous avons donc besoin d'outils permettant de vérifier sa préservation. Mais la plupart des outils actuels sont limités au cas unitaire, donc limités à vérifier des transformations qui ne changent pas le type de circuit: un circuit unitaire est transformé en circuit unitaire. Et ceux qui s'aventurent dans le cas hybride sont limités à l'hybridation : un circuit unitaire est transformé en un circuit hybride. Alors que le vrai défi survient lors de l'introduction de fils auxiliaires, tels que les ancillas, ou les *discards*^a, car il faut s'assurer que ces fils auxiliaires

n'affectent pas les autres.

Pour vérifier un circuit, plusieurs techniques existent (simulation, exécution symbolique, preuve), qui diffèrent par leur automatisation et leur scalabilité. La simulation, bien qu'automatisable, manque de scalabilité. L'exécution symbolique, automatisable, passe mieux à l'échelle. La preuve, insensible à l'échelle, est difficile à automatiser. Nous avons ainsi opté pour la preuve et pour l'exécution symbolique. Nous avons utilisé la preuve pour certifier une passe de compilation d'un langage de haut niveau vers un langage de bas niveau. Et l'exécution symbolique, en développant un outil de vérification d'équivalence de circuits hybrides avec *discard*.

Pour nous aider dans la preuve, nous avons utilisé l'outil de preuve semi-automatique Qbricks [16], capable de prouver des circuits unitaires paramétrés basés sur les sommes de chemins [2], un formalisme permettant une modélisation compacte et encodable des circuits.

En préalable au développement de notre outil de vérification de circuits hybrides avec *discard* nous avons défini des relations d'équivalence partielles pour les circuits unitaires, ainsi qu'un mécanisme pour étendre les outils de vérification unitaires aux circuits hybrides. La combinaison de ces deux composants permet de traiter les circuits hybrides avec *discard*.

Pour évaluer notre approche, nous avons implémenté dans l'outil SQbricks nos relations d'équivalence ainsi que notre mécanisme d'extension des outils limités aux unitaires. Nous avons démontré que les outils unitaires étendus au cas hybride sans *discard* surpassent, pour la plupart, largement l'état de l'art. Et qu'avec l'ajout d'un test de séparation au niveau unitaire, nous pouvons traiter des classes beaucoup plus vastes.

^a*Discard*: Mesures intermédiaires non incluses dans la sortie du circuit.

Title: Practical verification of quantum circuit transformations

Keywords: Quantum computing - Hybrid circuit - Circuit transformation - Verification

Abstract: Quantum computing, based on the principles of quantum physics, promises significant gains in computational power. However, it introduces new challenges, particularly for program verification. Classical verification techniques do not apply directly to quantum computing. This is mainly due to the use of quantum memory (non-standard information unit) and its measurement operation, which is the only way to retrieve information from a quantum bit and introduces a probabilistic aspect.

In this thesis, we focus on the circuit-based quantum computing model and a practical aspect of it: circuit transformations. A quantum circuit can be purely quantum (unitary circuit) or include classical control and measurement (hybrid circuit). Circuit transformations occur throughout the quantum computation process, whether compiling a circuit described in a high-level language to a low-level language, optimising a circuit for specific hardware (transpilation), changing the computation model (one-way measurement [29]), or transferring a quantum state (teleportation).

Each of these transformations can alter the circuit's behaviour. Therefore, tools are needed to verify that the behaviour is preserved. However, current tools are mostly limited to the unitary case, thus restricted to verifying transformations that do not change the type of circuit: a unitary circuit is transformed into a unitary circuit. Those that venture into the hybrid case are limited to hybridisation: a unitary circuit is transformed into a hybrid. The real challenge arises with the introduction of auxiliary wires, such as ancillas or *discards*^a

for hybrid circuits, as it must be ensured that these auxiliary wires do not affect the others.

Several techniques exist for verifying a circuit (simulation, symbolic execution, proof), differing in their automation and scalability. Simulation, while automatable, lacks scalability. Symbolic execution, which can be automated, scales better. Proof, while scalable, is challenging to automate. We thus chose to use both proof and symbolic execution. We use proof to certify a compilation pass from a high-level language to a low-level language. We also use symbolic execution by developing a tool to verify the equivalence of hybrid circuits using *discard*.

To assist us in the proof process, we utilise the semi-automatic proof tool Qbricks [16], which is capable of proving parametrised unitary circuits based on path sums [2]. This formalism allows for compact and encodable modelling of circuits.

As a prerequisite for developing our verification tool for hybrid circuits with *discard*, we define partial equivalence relations for unitary circuits and a mechanism to extend unitary verification tools to hybrid circuits. The combination of these two components enables the handling of hybrid circuits with *discard*.

To evaluate our approach, we implement in the tool SQbricks our equivalence relations and our mechanism for extending tools limited to unitary circuits. We have demonstrated that unitary tools extended to the hybrid case without *discard* vastly surpass the state-of-the-art. Moreover, by adding a separation test at the unitary level, we can handle much larger classes.

^a*Discard*: Intermediate measurements not included in the circuit output.

Contents

1	Introduction	7
1.1	Quantum Circuit Transformation	8
1.2	Verification of Circuit Transformation	9
1.3	Contributions	12
2	Background	15
2.1	Notations	15
2.2	Quantum Circuit	17
2.2.1	Quantum State	17
2.2.2	Measurement (M)	25
2.2.3	Unitary Matrices	26
2.2.4	Unitary Circuits	27
2.2.5	Hybrid Circuits	32
2.2.6	Deferred Measurement (DM)	39
2.3	Path Sums (PS)	40
2.3.1	Prerequisites	40
2.3.2	Semantics	40
2.3.3	Reduction Rules	42
2.4	Quantum Verification Tools	43
2.4.1	Simulation	46
2.4.2	Symbolic Execution	48
2.4.3	Proof	50
2.4.4	Tools Overview	51
2.5	Quantum Circuit Transformations	51
2.5.1	Compilation	52
2.5.2	Transpilation	53
2.5.3	Computational Model Shift	53
2.5.4	Teleportation Insertion	55
2.5.5	Hybridisation	57
3	Compilation Proof Example	61
3.1	Qbricks	61
3.1.1	Algebraic Data Type	62
3.1.2	Semantics	62
3.1.3	Useful Lemmas	66
3.2	From Qbricks To OpenQASM	67
3.2.1	OpenQASM	68
3.2.2	Overview	70

3.2.3	Parallel Compositions	71
3.2.4	Ternary Gates	75
3.2.5	Sub-Circuit Placement	83
3.2.6	Sub-Circuit Control	89
3.2.7	Complete Conversion	97
3.3	Experience	99
3.3.1	Practical Implementation	100
3.3.2	Compilation Usage Examples	103

4 Circuit Equivalences 105

4.1	Classifications	107
4.1.1	Circuit Transformations	107
4.1.2	Equivalence Classes	108
4.1.3	Circuit Views	108
4.1.4	Equivalence Approaches	109
4.2	Complete Equivalence (Comp)	110
4.2.1	Unitary-Complete-Equivalences (Unit-Comp)	111
4.2.2	Hybrid-Complete-Equivalences (Hyb-Comp)	117
4.3	Partial Equivalences (Part)	117
4.3.1	Prerequisites	119
4.3.2	Unitary Partial Equivalences (Unit-Part)	122
4.3.3	Path-Sum Partial Equivalences (PS-Part)	128
4.3.4	Hybrid-Partial-Equivalences (Hyb-Part)	140
4.3.5	Reduction from Hyb-Part to Unit-Part (Hyb-Part-DM)	145
4.4	Relating Complete and Partial Equivalences	165
4.4.1	Unitary-Equivalences	165
4.4.2	Hybrid-Equivalence	167

5 Implementation 169

5.1	Circuit Transformations Correctness	170
5.1.1	One-Way Measurement (O)	170
5.1.2	Teleportation Insertion (T)	183
5.2	Hyb-Part-DM Implementation: SQbricks	183
5.2.1	PS-Part Implementation: SQbricks-Verif	185
5.2.2	DM Implementation: SQbricks-Lift	192
5.2.3	Usage	194
5.3	Evaluation Protocol	195
5.3.1	Experimental setup	195
5.3.2	Considered tools	195
5.3.3	Circuit Collection	196
5.3.4	Circuit Equivalence Challenges	196
5.4	Experimentation Evaluation	197
5.4.1	Preliminary Experiment: Sanity Check	197

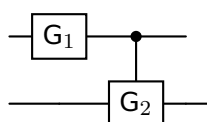
5.4.2	RQ1: Evaluation on Hybrid Equivalence DisFree	198
5.4.3	RQ2: Comparison with prior work on DisFree	200
5.4.4	RQ3: Hybrid equivalence evaluation beyond DisFree.	201
5.4.5	Additional Findings	202
5.5	Discussion	202
5.5.1	Future Extensions and Research Directions	203
5.5.2	New insights for the path-sum approach	203
5.5.3	Insights on other approaches	204
6	Conclusion	205
A	Tables of Symbols and Notations	217

Chapter 1

Introduction

Quantum computing is a computational model based on the principles of quantum physics. In particular, it leverages the *superposition* principle, where a quantum bit (*qubit*) can exist in two states simultaneously, and the *entanglement* principle, which states that qubits can be inherently correlated in a non-classical way. These properties have the potential to enhance computing capabilities significantly compared to classical machines. In the best-case scenario, a quantum computer with n qubits could evaluate all 2^n possibilities in a single operation, whereas a classical computer would require 2^n separate computations. Several algorithms have been developed to exploit these capabilities. The most famous is Shor's algorithm [71], which can factor large integers in polynomial time, potentially breaking encryption systems like RSA [66]. Other notable algorithms include Grover's algorithm [36], Quantum Fourier Transform [23] (QFT), Quantum Teleportation [11], Variational Quantum Eigensolver [56] (VQE), Quantum Approximate Optimization Algorithm [33] (QAOA), HarrowHassidimLloyd [37] (HHL), Quantum Key Distribution [10] (QKD), and Quantum Random Generators [39] (QRNG). While there were significant doubts about the feasibility of quantum computers a few years ago, recent advancements suggest we are moving closer to developing usable machines [5, 85].

To perform quantum computing, physics provides us with *unitary operations*, which evolve (transform) a quantum state and are reversible. They do not alter the number of qubits, and no information is lost. Quantum gates can represent these operations, which are similar to logic gates in classical computing. We can connect these gates to create circuits that represent computations. For instance, the following quantum circuit represents the sequence of a gate G_1 followed by a controlled gate G_2 , where the black dot (•) represents the control:



The second type of operation in quantum computing is measurement, which is the only operation that allows us to retrieve information from a qubit. However, measurement

causes the loss of quantum properties (it collapses the superposition), and the qubit, which was in two states simultaneously, fixes its state probabilistically.

A quantum algorithm is a sequence of initialisations, unitary operations, and measurements. However, this ideal functioning, with unitary computation and final measurements, requires quantum computers with capabilities that are currently unattainable. One way to circumvent this problem is to use intermediate measurements, which allow qubits to be reused within their limited coherence time (quantum lifespan). However, introducing intermediate measurements requires managing classical data, classical control, and the probabilistic aspect of measurement. We refer to a purely quantum circuit, which contains only unitary operations, as a *unitary circuit*, and a circuit that contains unitary operations, measurements, and classical control as a *hybrid circuit*.

Quantum properties introduce new challenges in software development. In particular, they make classical program verification methods difficult to apply [65, 84]. Indeed, because they are probabilistic, testing a quantum program requires running it multiple times to obtain a meaningful probability distribution. Additionally, exhaustive verification requires testing all possible input data, making testing particularly challenging in the context of quantum computing. Similarly, program assertions are challenging to use because measuring a qubit (the only operation that allows retrieving information from it) is destructive. Once measured, a qubit loses its quantum properties, requiring the program to be re-executed each time a variable's state is read. Program verification addresses these issues by utilising underlying formalisms to compute the program's output state without executing it directly. However, it requires finding a way to model quantum properties effectively. Standard methods for modelling quantum programs involve complex matrices, which are difficult to encode, from which useful information is not easily accessed, and whose size grows exponentially with the number of qubits. Therefore, it is essential to adapt these methods or develop new ones. This work is situated within this context.

1.1 Quantum Circuit Transformation

To execute a quantum circuit on a quantum computer, we must transform it. This transformation is essential to adapt the circuit to the constraints of the computer's technology, architecture, and languages. To describe a quantum circuit, we need a representation language. Similar to classical computing, there are high-level languages that facilitate program design by defining operations for controlling sub-circuits, and low-level languages closer to the hardware's instruction set, such as OpenQASM [27]. Thus, we need to compile these high-level constructs into low-level instructions. This process is quite similar to compilation in classical computing, which is why we refer to it here as *compilation*. Additionally, we must ensure the compiled circuit is compatible with the machine's capabilities, including the gate set and qubit mapping. Therefore, we need to transform the circuit to match the hardware's constraints, a process known as *transpilation*. As previously discussed, unitary circuits are not currently executable on

today’s computers. One solution is to insert intermediate measurements. This transformation changes the circuit type from unitary to hybrid (*hybridisation*). However, there are even more complex transformations that go beyond changing the circuit type. These transformations introduce auxiliary wires not part of the circuit’s output (*observed* qubits); they are used only for internal computation. When these wires are located after a measurement, they are called *discard* wires; otherwise, they are called *ancilla* wires. For simplicity, in this thesis, we do not make this distinction and will refer to both as *discard*. Examples of such transformations include Quantum Error Correction [31, 68] (QEC), Measurement-Based Quantum Computing [29] (MBQC), and Quantum Teleportation. Here, we will focus on a type of MBQC called *One-Way Measurement* (OWM) and a *Teleportation Insertion*, which inserts a Teleportation on each wire. Each quantum circuit transformation can alter the circuit’s behaviour. Therefore, we must ensure that the behaviour remains the same.

1.2 Verification of Circuit Transformation

There is a large body of theoretical work on quantum program verification, but in our work, we focus on practical approaches. To verify quantum circuit transformations, we can use methods from classical program verification. Approaches include testing or assertion checking [44, 51]; here, we use approaches inspired by Hoare logic [41], which associate input conditions (preconditions) and output conditions (postconditions) with a program. If a program is executed in a state satisfying the preconditions, then the resulting state must satisfy the postconditions. To apply Hoare logic, we require a *semantics*, that is, a formalism for representing program states and their evolution.

From this perspective, we distinguish three verification approaches: proof, simulation, and symbolic execution. All three rely on a semantic representation of the program. In this sense, they can be regarded as forms of static analysis, since even in the case of simulation or symbolic execution, the program itself is not executed; rather, its semantic representation is.

Proof corresponds to a direct application of Hoare logic. It involves proving, manually or with proof assistants such as Coq [25] or Isabelle/HOL [53], that a program is correct for all inputs satisfying the preconditions, without execution. Deductive verification platforms, such as Why3 [35] and frama-C [28], extend these proof tools by enabling semi-automated proofs via SMT solvers such as CVC5 [8], Alt-Ergo [22], and Z3 [30].

Exhaustive simulation (refer here to *simulation*) consists of simulating a program, via its semantics, on a bounded but exhaustive set of inputs using, for example, test case enumerators such as SmallCheck [69] or AutoCheck [32]. This distinguishes simulation from testing, where the actual program is executed. Exhaustive simulation involves executing the semantic representation of a program on concrete inputs that satisfy the preconditions and verifying that the resulting states satisfy the postconditions. When such a simulation is exhaustive over all possible inputs, it constitutes a

proof by enumeration.

Symbolic execution [46] lies between proof and simulation. It operates on a symbolic, semantic representation of the program rather than the program itself. It executes it on symbolic representations of inputs, which may represent multiple concrete executions simultaneously. When symbolic execution soundly and completely represents all states satisfying the preconditions, it can be used to establish formal correctness guarantees.

To verify a quantum circuit transformation, we can either formally prove that the transformation is correct or establish that the transformed circuit is semantically equivalent to the original one. For simple transformations that preserve the type, such as compilation or transpilation, we can use most state-of-the-art tools. These include fully manual proof tools like CoqQ [86] or SQIR [40], and semi-automatic tools like Qbricks [16]. To our knowledge, current automatic simulation tools do not natively support equivalence verification. We can use tools that employ automatic symbolic execution, such as AutoQ [1, 20, 21], Feynman [2], PyZX [47], QCEC [12, 14], SliQEC [18], or VeriQC [42]. For transformations that do not preserve the type and do not introduce discard, such as hybridisation, we are limited to CoqQ and SQIR for proof, and QCEC and VeriQC for automatic symbolic execution. For transformations that introduce discard, we can use only CoqQ and SQIR. VeriQC attempted to handle discards by defining equivalence relations but did not implement them. Thus, for simple cases, we have many tools, but as cases become more complex, the choices diminish. Moreover, in the most complex cases involving discard, only manual proof tools remain, which require long and difficult proofs.

What we would like is a way to utilise all the state-of-the-art tools for more complex cases, with automated tools prioritised. This would enable us to leverage each tool's strengths. To achieve this, a natural approach is to use deferred measurement, which enables treating a hybrid circuit as a unitary circuit with final measurement(s).

Deferred measurement is a well-known circuit transformation that preserves the original behaviour of the circuit by replacing classical measurements and controls with quantum controls and relocating all measurements to the end of the circuit. It is the inverse process of hybridisation. To lift unitary tools to the hybrid case, we first apply deferred measurement to isolate the unitary parts of the two circuits. Then, we determine the equivalence of the original circuits by checking the equivalence of their unitary parts. However, this process shifts the difficulty from managing measurements and classical controls to managing quantum multi-control. Figure 1.1a shows a hybrid circuit featuring two measurements ( and $\oplus$) and two classical controls ($\bullet$, a symbol we use for both the quantum and classical control). By applying the principle of deferred measurement to this circuit, as illustrated in Figure 1.1b, we reveal the quantum multi-control. In addition, for hybrid circuits with discard, an additional step is necessary. We must perform a separation test (to ensure the absence of entanglement) between the observed qubits and the discarded qubits (e.g., qubits measured and then forgotten). This test ensures that the discarded wires do not affect the observed qubits.



Figure 1.1: Applying Deferred Measurement reveals Quantum Multi-Control

Thus, this process introduces two difficulties:

1. Managing multi-qubit quantum control.
2. Testing the separation between observed qubits and discarded qubits.

Current automatic tools do not allow the verification of the separation between qubits. Consequently, they are limited to being lifted (by deferred measurement) to the hybrid case without discard. To verify transformations that involve discards, we require a formalism that is both compact and easily encodable. This formalism should enable the implementation of multi-qubit quantum control and separation testing by providing direct access to the symbolic decomposition of the state. A particularly suitable formalism is that of path-sums, as used in Feynman [2] and Qbricks [16]. To illustrate this point, consider the standard semantics of a unitary circuit acting on a quantum state $|x\rangle$:

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} |x\rangle = \frac{1}{\sqrt{2}} \sum_{y \in \{0,1\}} e^{2\pi i \frac{x \cdot y}{2}} |y\rangle.$$

This transformation can be described using three parameters in the *path-sum* formalism: a path variable y representing the amplitude, a phase term $\frac{x \cdot y}{2}$, and the output state $|y\rangle$:

$$\left\langle y, \frac{x \cdot y}{2}, |y\rangle \right\rangle.$$

This example is deliberately simple. In general, matrix representations grow exponentially with the number of qubits, and their coefficients can be complex numbers. Nevertheless, the advantages of path-sums are already apparent: compact representations, direct access to parameters, and native symbolic formalism.

Another important point is that each tool uses different methods to verify circuit transformations. These methods can include specific equivalence relations, such as AutoQ, Feynman, PyZX, QCEC, or VeriQC, or direct proofs, such as CoqQ, SQIR, or Qbricks, for transformation correctness. Most equivalence checker tools perform strict equivalence checks, such as AutoQ, Feynman, PyZX, QCEC, VeriQC. However, some consider global phase equivalence, such as Feynman, QCEC, SliQEC, while others can verify

equivalence as if the entire output register were measured, such as QCEC [61], SliQEC. Additionally, certain tools handle high-level circuit descriptions of hybrid circuits with loops, such as AutoQ-2, CoqQ, SQIR, while others are limited to unitary circuits, such as AutoQ, Feynman, PyZX, Qbricks, SliQEC. Some tools can verify families of circuits, such as AutoQ, CoqQ, SQIR, Qbricks, while others are limited to individual circuits, such as AutoQ, Feynman, PyZX, Qbricks, SliQEC (summarised in Table 2.2). These differences in approaches and capabilities make comparisons very challenging. We must therefore establish a classification and evaluation protocol.

1.3 Contributions

To verify circuit transformations, we need tools that are easy to use, that either confirm the correctness of the transformation or show equivalence between the transformed and original circuits. Handling complex transformations requires accounting hybrid circuit with discarded qubits. However, most user-friendly tools, whether automatic or semi-automatic, are limited to unitary cases. Only two tools handle simple hybrid cases without discard, revealing a gap in hybrid circuit tools. This gap widens when considering discard. Finally, to assess our approach and previous methods, we need a classification system and an evaluation protocol. We summarise the challenges related to verifying circuit transformations:

1. Handling of hybrid circuits.
2. Management of discard.
3. Verification methods and tools classification.
4. Practical tool implementation.
5. Tools evaluation.

Using our method with deferred measurement, these challenges translate into:

1. Handling of quantum multi-control.
2. Separation test definition.
3. Verification methods and tools classifications.
4. Practical tool implementation.
5. Tools evaluation.

Our contributions address these challenges as follows:

- Challenge 1: In Section 4.3.3.2, we extend the path-sums formalism to quantum multi-control. We used this extension in the work presented at the workshop *Services and Quantum Software*¹ (SQS).
- Challenge 2: In Section 4.3, we define a separation test using both a standard and the path-sums formalism. We prove that the test using the path-sum correctly implements the standard version. We presented the separation test in the article published in SQS.
- Challenge 3: In Sections 2.4 and 4.1, we classify verification methods, and in Sections 4.2, and 4.3, we define equivalence relations according to our classification using standard formalism. We partially used these classifications for the paper presented at SQS.
- Challenge 4: In Section 4.3.3, we define equivalence relations using path-sums and provide a proof of correctness with standard relations². In Section 4.3.5.4, we present the definition and correctness proof of our method for lifting unitary equivalence relations to the hybrid case. In Section 5.2, we implement in OCaml³ a practical tool that integrates our method: partial unitary equivalence relations, and the lifting of unitary tools to hybrid cases. We partially presented equivalence relations, proof, and a practical tool at SQS.
- Challenge 5: Sections 2.5.3 and 2.5.4 define the One-Way Measurement [29] (O) and the Teleportation Insertion (T) functions, with correctness proofs provided in Sections 5.1.1 and 5.1.2. We implement these in OCaml. In Section 5.3, we develop an evaluation protocol (scripts and a Docker⁴ container) for equivalence verification tools in the context of circuit transformation verification. In Section 5.4, we evaluate our approach through our implementation of the test protocol. We partially presented this work at SQS.

Additionally, in Section 3, we illustrate a transformation proof using the semi-automatic tool Qbricks based on the verification platform Why3. This proof was partially submitted as a case study for a tool paper (in collaboration with Jad Issa, Mathieu Nguyen, and Sébastien Bardin) to CAV⁵ last year and has since been resubmitted and is now under review.

¹SQS: Services and Quantum Software, <https://sqs2025.spilab.es/>.

²We define *full-circuit equivalence* as the case where all circuit output wires are measured, treating the circuit as a whole: an input is given, a computation is performed, and only the final measurement result is observed. We include this relation to complete the landscape of equivalence notions, as tools such as QCEC support it. However, we do not implement it due to its poor scalability (Remark 4.3.38). Moreover, our experimental evaluation indicates that it is not required for our main objective, namely verifying circuit transformations.

³OCaml: <https://ocaml.org/>.

⁴Docker: <https://www.docker.com/>.

⁵CAV: International Conference on Computer-Aided Verification, <https://i-cav.org/>.

Chapter 2

Background

This section provides the background necessary to understand the verification of quantum circuit transformations. It begins with some common notations (Section 2.1) and essential concepts for quantum circuits in Section 2.2. It then introduces the path-sum formalism in Section 2.3, which is well suited for encoding and symbolic representation, making it central to our work. We survey existing quantum circuit verification tools in Section 2.4 and identify a gap in the state of the art: partial hybrid equivalence checking (hybrid with discard). Finally, we conclude with the definitions of the quantum circuit transformations employed in our work in Section 2.5.

2.1 Notations

This section introduces the main notations used in this document. We start with Boolean notations in Notation 2.1.1, followed by set notations in Notation 2.1.2, and conclude with bit-string notations in Notation 2.1.3.

Notation 2.1.1 (Boolean). We introduce notations for Boolean values and their operations:

1. Values (0, 1): The value 1 represents the logical truth value `true`, and the value 0 represents `false`.
2. Addition, ($\oplus$): The operation $\oplus$ represents binary addition, also known as the XOR (exclusive OR) operation. It outputs `true` only when the number of `true` inputs is odd. The operation is defined as follows: $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$.
3. Multiplication ($\cdot$): The operations $\cdot$ represent Boolean multiplication. The operation outputs `true` if and only if all the operands are `true`. The Boolean multiplication operation is defined as follows: $0 \cdot 0 = 0$, $0 \cdot 1 = 0$, $1 \cdot 0 = 0$, $1 \cdot 1 = 1$.
4. Symbolic: $b \in \{0, 1\}$.

5. Negation ($\neg b$ or $\bar{b}$): The operation $\neg b$ and $\bar{b}$ represent Boolean negation. This operation is defined as follows: $\bar{0} = \neg 0 = 1$, $\bar{1} = \neg 1 = 0$.

Notation 2.1.2 (Set). Let s , s_1 , and s_2 be three sets of integers with a total order. We denote by:

1. $s = \{i_0, i_1, \dots, i_{n-1}\}$, a set of size n .
2. $\emptyset$: the empty set.
3. $|s|$: the cardinality of the set s .
4. $s_1 \cup s_2$: the union of sets s_1 and s_2 .
5. $s_1 \cap s_2$: the intersection of sets s_1 and s_2 .
6. $s_1 \subseteq s_2$: the inclusion of set s_1 in set s_2 .
7. $s_1 = s_2$, the equality of sets.
8. $s_1 \setminus s_2$: the set difference of s_1 and s_2 .
9. $s_1 \simeq s_2$: the sets which are the same size, $|s_1| = |s_2|$.
10. $\min(s)$: a pair (m, s') where m is the smallest element of the set s and s' is the set s without its smallest element.

For example, $\min(\{3, 1, 2\}) = (1, \{2, 3\})$.

Notation 2.1.3 (Bit-string). Let $x = x_0x_1 \dots x_{n-1} \in \{0, 1\}^n$ be a bit-string of size n , and $y = y_0y_1 \dots y_{m-1} \in \{0, 1\}^m$ be a bit-string of size m . Let $s, s' \subseteq \{0, 1, \dots, n-1\}$ be two sets of bit indices, such that $|s| = |s'|$. We denote by:

1. xy : the concatenation of bit-strings x and y .
2. $|x|$: the size of the bit-string, here n .
3. $\emptyset$: the bit-string of size 0, $|\emptyset| = 0$.
4. x_i : the access to the bit at index i .
5. $x \cong_{(s,s')} y$: the equivalence of bit-strings x and y up to a mapping defined by a canonical permutation. Specifically, there exists a unique permutation $\sigma : s \rightarrow s'$ such that for all $i \in s$, $x_i = y_{\sigma(i)}$, where σ maps the indices of s in increasing order to those of s' in increasing order.

For example, let $s = \{0, 1\}$ and $s' = \{1, 2\}$, and let σ be the bijection such that $\sigma(0) = 1$ and $\sigma(1) = 2$. Then the bit-string $x = 11$ is equivalent to the bit-string $y = 011$, i.e., $11 \cong 011$.

6. $x_{[i \leftarrow b]}$: the substitution of a bit at index i by $b \in \{0, 1\}$.
For example, if $i = 1$, $x = x_0x_1x_2 \in \{0, 1\}^3$, and $b \in \{0, 1\}$, then $x_{[i \leftarrow b]} = x_0bx_2 \in \{0, 1\}^3$.
7. $x_{[cb \leftarrow b]}$: the substitution of a set of bits at indices cb , by $b \in \{0, 1\}^{|cb|}$, in increasing order.
For example, if $cb = \{1, 2\}$, $x = x_0x_1x_2 \in \{0, 1\}^3$, and $b = b_0b_1 \in \{0, 1\}^2$, then: $x_{[cb \leftarrow b]} = x_0b_0b_1 \in \{0, 1\}^3$.
8. $*$: an undefined bit.
9. $x_{|s}$: a sub-bit-string of x over the qubits indices in $s = \{i_0, i_1 \dots, i_k\}$, where $k < n$.
Formally: $x_{|s} = x_{i_0}x_{i_1} \dots x_{i_k}$.
For example, if $x = 101011$ and $s = \{0, 2, 4\}$, then $x_{|s} = 111$.
10. $\text{Indices}(x)$: a function that returns the set of indices of the bits appearing in a bit-string.
For example, $\text{Indices}(x_0x_2x_5) = \{0, 2, 5\}$.

2.2 Quantum Circuit

This chapter introduces the concept of quantum circuits, the computational model used in this work. It is organised into six sections. Section 2.2.1 explains quantum states. Section 2.2.2 introduces measurement, the only way to retrieve information from a quantum bit. Section 2.2.3 introduces unitary matrices, the standard formalism of quantum operations. Section 2.2.4 delves into unitary circuits, which contain only quantum operations. Section 2.2.5 extends the discussion to hybrid circuits, which contain unitary operations, measurement, and classical operations. Finally, Section 2.2.6 introduces deferred measurement, a key technique for utilising unitary circuit tools in hybrid circuit verification.

2.2.1 Quantum State

In classical computing, bits store information, with each bit being in one of two states: 0 or 1 (Notation 2.1.1). In quantum computing, qubits (short for quantum bits) store information. Unlike classical bits, a qubit can simultaneously be in states 0 and 1, a property known as *superposition*. In this section, we first provide the mathematical prerequisites for defining quantum states (Section 2.2.1.1). We then define quantum states (Section 2.2.1.2) and discuss equality relations beneficial for circuit equivalences (Section 2.2.1.3). Additionally, we introduce the notion of quantum entanglement (Section 2.2.1.4), which is central to our approach to equivalence relations. In commonly envisioned applications today, quantum and classical operations are mixed. Therefore,

we need a formalism to represent states that mix quantum and classical information. To this end, we define a notion of mixed state that retains classical information (extending mixed states) in Section 2.2.1.5. We conclude by summarising all the important notations of this section in Section 2.2.1.6.

2.2.1.1 Prerequisites

A quantum state can be represented as a vector with complex coefficients, so we start by giving some mathematical prerequisites about complex numbers (Definition 2.2.1), then introduce Dirac notation (Definition 2.2.2) which is a compact way of representing a vector, and then give three definitions of products on vectors, the tensor product (Definition 2.2.3), the inner product (Definition 2.2.4) and the outer product (Definition 2.2.5).

Definition 2.2.1 (Modulus of a Complex Number). The *modulus of a complex number* $z = a + bi$, where a and b are real numbers, is defined as: $|z| = \sqrt{a^2 + b^2}$.

For instance, using the result from Definition 2.2.4, we can calculate the modulus of the complex number $12 + 4i$. $|12 + 4i| = \sqrt{12^2 + 4^2} = \sqrt{144 + 16} = \sqrt{160} = 4\sqrt{10}$. Therefore, the modulus of the complex number $12 + 4i$ is $4\sqrt{10}$.

Notation 2.2.2 (Dirac Notation). The *Dirac notation* provides a compact representation of vectors in a complex vector space. Let $\mathbb{C}^n$ be an n -dimensional complex vector space, and let $|v\rangle \in \mathbb{C}^n$. The *ket* $|v\rangle$ represents the column vector:

$$|v\rangle = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix}, \quad \text{where } v_i \in \mathbb{C} \text{ for all } i.$$

The *bra* $\langle v|$ represents its conjugate transpose: $\langle v| = (v_1^* \ v_2^* \ \cdots \ v_n^*)$.

Definition 2.2.3 (Tensor Product ($\otimes$)). The *tensor product* $\otimes$ is a way to combine two vectors, the tensor product is calculated as:

$$\begin{pmatrix} v_1 \\ \vdots \\ v_n \end{pmatrix} \otimes \begin{pmatrix} w_1 \\ \vdots \\ w_m \end{pmatrix} = \begin{pmatrix} v_1 w_1 \\ \vdots \\ v_1 w_m \\ \vdots \\ v_n w_1 \\ \vdots \\ v_n w_m \end{pmatrix}.$$

Here is an example of a tensor product between two vectors:

$$\begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}.$$

In the following, we will often denote the tensor product $|v\rangle \otimes |w\rangle$ by $|v\rangle |w\rangle$ or $|vw\rangle$.

Definition 2.2.4 (Inner Product). Let

$$|v\rangle = \begin{pmatrix} v_0 \\ v_1 \\ \vdots \\ v_{n-1} \end{pmatrix}, \quad \text{and} \quad |w\rangle = \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_{n-1} \end{pmatrix}$$

be two complex vectors in $\mathbb{C}^n$. The *inner product* of these two vectors, denoted as $\langle v|w\rangle$ is defined as:

$$\langle v|w\rangle = \sum_{i \in [0, n-1]} v_i^* w_i,$$

where v_i and w_i are the components of the vectors $|v\rangle$ and $|w\rangle$, respectively, and v_i^* denotes the complex conjugate of v_i .

For example, consider two vectors in $\mathbb{C}^2$: $|v\rangle = \begin{pmatrix} 1+i \\ 3-i \end{pmatrix}$ and $|w\rangle = \begin{pmatrix} 2-i \\ 4+i \end{pmatrix}$. The inner product $\langle v|w\rangle$ is calculated as follows: $\langle v|w\rangle = (1-i)(2-i) + (3+i)(4+i)$. First, compute each term: $(1-i)(2-i) = 1 - 3i$ and $(3+i)(4+i) = 11 + 7i$. Then, sum the results: $\langle v|w\rangle = (1 - 3i) + (11 + 7i) = 12 + 4i$. Therefore, the inner product $\langle v|w\rangle$ is $12 + 4i$.

Definition 2.2.5 (Outer Product). Let

$$|v\rangle = \begin{pmatrix} v_0 \\ v_1 \\ \vdots \\ v_{n-1} \end{pmatrix}, \quad \text{and} \quad |w\rangle = \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_{n-1} \end{pmatrix}$$

be two complex vectors in $\mathbb{C}^n$. The *outer product* of these two vectors, denoted as $|v\rangle\langle w|^1$, is defined as:

$$|v\rangle\langle w| := \begin{pmatrix} v_0 w_0^* & v_0 w_1^* & \cdots & v_0 w_{n-1}^* \\ v_1 w_0^* & v_1 w_1^* & \cdots & v_1 w_{n-1}^* \\ \vdots & \vdots & \ddots & \vdots \\ v_{n-1} w_0^* & v_{n-1} w_1^* & \cdots & v_{n-1} w_{n-1}^* \end{pmatrix}$$

For example, with $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, then $|0\rangle\langle 0| = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$.

2.2.1.2 Quantum State

A quantum state can be represented as a sum of complex vectors in a Hilbert space. We begin by defining the Hilbert space (Definition 2.2.6), followed by the basis states (Definition 2.2.7), and then proceed to the general definition of a quantum state (Definition 2.2.8). Finally, we define the standard computational basis (Definition 2.2.9).

¹ $|v\rangle\langle v|$ is a *projector* onto the subspace spanned by $|v\rangle$.

Definition 2.2.6 (Hilbert Space). The Hilbert space of n qubits, denoted $\mathcal{H}^{\otimes n}$, is the 2^n -dimensional complex vector space $\mathbb{C}^{2^n}$ equipped with an inner product (Definition 2.2.4).

Definition 2.2.7 (Basis States ($|x\rangle$)). A ket vector of n qubits, denoted by $|\psi\rangle \in \mathcal{H}^{\otimes n}$, represents a quantum state, while a bra vector $\langle\psi|$ represents its dual. *Basis states* form an orthonormal set that defines the state space of a quantum system.

For a single qubit, the basis states ($|0\rangle \in \mathcal{H}$) and ($|1\rangle \in \mathcal{H}$) can be represented by the vectors: $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$. For a system of n qubits, basis states are denoted by $|x\rangle \in \mathcal{H}^{\otimes n}$, where $(x = x_0x_1 \dots x_{n-1})$ is an n -bit-string with each $(x_i \in \{0, 1\})$, representing a quantum state of the n -qubit system.

The orthonormality of basis states is expressed through their inner product (Definition 2.2.4): $\langle x|x'\rangle = \begin{cases} 1 & \text{if } x = x', \\ 0 & \text{otherwise.} \end{cases}$

Definition 2.2.8 (Quantum State ($|\psi\rangle$)). A *quantum state* ($|\psi\rangle \in \mathcal{H}^{\otimes n}$) of an n -qubit system is a superposition of basis states (Definition 2.2.7):

$$|\psi\rangle = \sum_x \alpha_x |x\rangle, \quad \sum_x |\alpha_x|^2 = 1$$

where x is an n -bit-string and α_x are complex amplitudes.

Definition 2.2.9 (Standard Computational Basis). The standard computational basis for a circuit of n qubits is the set of all possible states where each qubit is either in the state $|0\rangle$ or $|1\rangle$. This set is denoted by $\{|x\rangle\}_{x \in \{0,1\}^n}$, where x is a n -bit-string representing the state of each qubit.

2.2.1.3 Quantum States Equivalence

Although this content is simple and well-known, it is essential to formalise it for the subsequent definitions of our equivalence relations. These notions of equivalence serve as the basis for the equivalence relations of quantum circuits discussed in Chapter 4. This section introduces one notion of equality and two notions of equivalence for quantum states. In quantum circuit analysis, where states are explicitly represented as vectors, strict vector equality requires all components of the state vectors to be identical (Notation 2.2.10). A standard notion in quantum computing is equivalence up to a global phase (Definition 2.2.12), where states differing only by a complex phase factor have the same observable physical properties. A more general relation, we called equivalence up to the conditional phase (Definition 2.2.14) captures cases where phase differences occur among the different terms of the superposition. Lemmas 2.2.13 and 2.2.15 formalise the implications between these notions, which are helpful to prove the link between the different equivalence relations.

Notation 2.2.10 (Vector Equality (Vect-Equal)). Let $\mathcal{H}^{\otimes n}$ be a Hilbert space of n qubits, and let $|\psi\rangle, |\phi\rangle \in \mathcal{H}^{\otimes n}$ be two quantum state. We denote by $|\psi\rangle = |\phi\rangle$ the vector equality (Vect-Equal).

Notation 2.2.11 (Global Phase (GP)). Let $\alpha \in \mathbb{R}$. A *global-phase* is represented by a complex factor of the form: $e^{i\pi\alpha}$.

Definition 2.2.12 (Equivalence Up to Global Phase (GP-Equiv)). Let $\mathcal{H}^{\otimes n}$ be a Hilbert space (Definition 2.2.6) of n qubits. Two quantum states $|\psi\rangle, |\phi\rangle \in \mathcal{H}^{\otimes n}$ are *equal up to global-phase* (GP-Equiv), denoted $|\psi\rangle \approx_{\text{gp}} |\phi\rangle$, if there exists $\alpha \in \mathbb{R}$ such that $|\psi\rangle = e^{i\pi\alpha} |\phi\rangle$.

For example, let $\mathcal{H}$ be a Hilbert space (Definition 2.2.6) of 1 qubit, and $(|\psi\rangle, |\phi\rangle) \in \mathcal{H}$ two quantum states. Let $|\psi\rangle = |0\rangle$ and $|\phi\rangle = -|0\rangle$. We can define a global-phase $\alpha = 1$ such that $|\psi\rangle = e^{i\pi\alpha} |\phi\rangle$ (Definition 2.2.10). Thus, by Definition 2.2.12, the states $|\psi\rangle$ and $|\phi\rangle$ are equal up to global-phase, denoted by: $|\psi\rangle \approx_{\text{gp}} |\phi\rangle$.

Lemme 2.2.13 (Vect-Equal Implies GP-Equiv). Let $\mathcal{H}^{\otimes n}$ be a Hilbert space (Definition 2.2.6) of n qubits, and $|\psi\rangle, |\phi\rangle \in \mathcal{H}^{\otimes n}$ two quantum states. If $|\psi\rangle$ and $|\phi\rangle$ are vector equal (Definition 2.2.10), then they are also equal up to global-phase (Definition 2.2.12), denoted by $|\psi\rangle = |\phi\rangle \implies |\psi\rangle \approx_{\text{gp}} |\phi\rangle$.

Proof. Assume $|\psi\rangle = |\phi\rangle$, then $|\psi\rangle = e^{i\pi \cdot 0} |\phi\rangle$. Thus, by Definition 2.2.12, $|\psi\rangle \approx_{\text{gp}} |\phi\rangle$. $\square$

Definition 2.2.14 (Equivalence Up to Conditional Phase (CP-Equiv)). Let $\mathcal{H}^{\otimes n}$ be a Hilbert space (Definition 2.2.6) of n qubits, two quantum states $|\psi\rangle, |\phi\rangle \in \mathcal{H}^{\otimes n}$ are said to be *equal up to conditional phase* (CP-Equiv), denoted by $|\psi\rangle \approx_{\text{cp}} |\phi\rangle$, if there exists a set of real-valued phases $\{\alpha_j\}_{j \in \{0,1\}^n}$ such that when both states are expressed in the basis $\{|j\rangle\}_{j \in \{0,1\}^n}$ as $|\psi\rangle = \sum_{j \in \{0,1\}^n} c_j |j\rangle$, $|\phi\rangle = \sum_{j \in \{0,1\}^n} d_j |j\rangle$, they satisfy the relation $\forall j \in \{0,1\}^n, c_j = e^{i\pi\alpha_j} d_j$.

For example, These states are not GP-Equiv (Definition 2.2.11): $(|0\rangle + |1\rangle)/\sqrt{2} \not\approx_{\text{gp}} (|0\rangle - |1\rangle)/\sqrt{2}$. But, they are CP-Equiv (Definition 2.2.14): $(|0\rangle + |1\rangle)/\sqrt{2} \approx_{\text{cp}} (|0\rangle - |1\rangle)/\sqrt{2}$.

Lemme 2.2.15 (GP-Equiv Implies CP-Equiv). Let $\mathcal{H}^{\otimes n}$ be a Hilbert space (Definition 2.2.6) of n qubits, and $|\psi\rangle, |\phi\rangle \in \mathcal{H}^{\otimes n}$ two quantum states. If $|\psi\rangle$ and $|\phi\rangle$ are GP-Equiv (Definition 2.2.12), then they are also CP-Equiv (Definition 2.2.14), denoted by $|\psi\rangle \approx_{\text{gp}} |\phi\rangle \implies |\psi\rangle \approx_{\text{cp}} |\phi\rangle$.

Proof. Assume $|\psi\rangle \approx_{\text{gp}} |\phi\rangle$. By Definition 2.2.12, there exists $\alpha \in \mathbb{R}$ such that $|\psi\rangle = e^{i\pi\alpha} |\phi\rangle$. By Definition 2.2.14, two states are CP-Equiv if there exists a family $(\alpha_j)_{j \in \{0,1\}^n}$ of real numbers such that, in the computational basis, each amplitude differs by a phase $e^{i\pi\alpha_j}$. Taking $\alpha_j = \alpha$ for all j immediately yields $|\psi\rangle \approx_{\text{cp}} |\phi\rangle$. $\square$

2.2.1.4 Quantum Entanglement

Quantum entanglement is a feature of quantum systems where the state of a spatially composite system cannot be separated into independent subsystem states. This section introduces entanglement by first defining separability (Definition 2.2.16) and then characterising entanglement as the absence of this separation (Definition 2.2.17).

Definition 2.2.16 (Quantum Separability). Let $\mathcal{H}^{\otimes n}$ and $\mathcal{H}^{\otimes m}$ be two Hilbert spaces (Definition 2.2.6) of n and m qubits respectively, let $|\psi\rangle \in \mathcal{H}^{\otimes n}$, and $|\phi\rangle \in \mathcal{H}^{\otimes m}$ two quantum states. $|\psi\rangle$ and $|\phi\rangle$ are *separable* if we can express the system state as $|\psi\rangle \otimes |\phi\rangle \in \mathcal{H}^{\otimes(n+m)}$.

Definition 2.2.17 (Quantum Entanglement). Two quantum states are called *entangled* if they are not separable (Definition 2.2.16).

Example 2.2.18 (Bell State). An example of an entangled state is Bell's state [52, Section 1.3.6]: $\beta_{00} = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. It is said to be entangled because it cannot be written in the form: $|\phi\rangle \otimes |\psi\rangle$. Let us assume that we can, and then define $|\phi\rangle = \alpha_1|0\rangle + \beta_1|1\rangle$ and $|\psi\rangle = \alpha_2|0\rangle + \beta_2|1\rangle$. Then, the product state would be: $|\phi\rangle \otimes |\psi\rangle = \alpha_1\alpha_2|00\rangle + \alpha_1\beta_2|01\rangle + \beta_1\alpha_2|10\rangle + \beta_1\beta_2|11\rangle$. By comparing coefficients with β_{00} , we get the following equations: $\alpha_1\alpha_2 = \frac{1}{\sqrt{2}}$, $\alpha_1\beta_2 = 0$, $\beta_1\alpha_2 = 0$, $\beta_1\beta_2 = \frac{1}{\sqrt{2}}$. These equations have no solution, confirming that β_{00} is not separable and thus is entangled.

2.2.1.5 Extended Mixed State

Hybrid circuits operate with both quantum and classical data. The standard way to represent this type of circuit is with density matrices [52, Section 2.4]. However, density matrices are difficult to use for program verification because their size grows exponentially with the number of qubits, and they intertwine classical and quantum data. Despite these drawbacks, density-matrix-based approaches are used in several verification frameworks, such as [40, 86]. For our work, we need a compact and easily manageable representation. It should provide straightforward access to both classical and quantum data. To achieve this, we propose extending the concept of mixed states. The standard version of mixed state is a probability distribution of quantum states. Our extension of mixed states retains measurement results in a classical register (Definition 2.2.19). This yields extended mixed states, which are distributions over quantum and classical states, as detailed in Definition 2.2.20.

Definition 2.2.19 (Classical Register and Classical Data). A *classical register* of n bits is a register that stores *classical data*, denoted as a bit-string $c \in \{0, 1\}^n$. In the context of extended mixed states, classical data refers to the information obtained from measuring a quantum system.

Definition 2.2.20 (Extended Mixed States). We define an *extended mixed state* as an extension of the standard mixed state. This is achieved by retaining measurement

results within a probability distribution of quantum states. Let n be the size of the classical register, $c \in \{0, 1\}^n$ be a bit-string representing a *classical state*, $\mathcal{H}^{\otimes m}$ be a Hilbert space of m qubits, $|\psi_c\rangle \in \mathcal{H}^{\otimes m}$ be the quantum state indexed by c , and $p_c \in [0, 1]$ be the probability of being in the classical state c . An extended mixed state is defined as a probability distribution over tuples of classical and quantum states:

$$\rho = \sum_{c \in \{0,1\}^n} (p_c, |\psi_c\rangle, c).$$

This distribution must satisfy the following constraints:

1. Non-negativity: $\forall c \in \{0, 1\}^n, p_c \geq 0$,
2. Normalisation: $\sum_{c \in \{0,1\}^n} p_c = 1$.

Notation 2.2.21 (Extended Mixed States). Let $\rho = \sum_{c \in \{0,1\}^n} (p_c, |\psi_c\rangle, c)$ be an extended mixed state (Definition 2.2.20). Then, we denote it by:

$$\rho = \{p_c, |\psi_c\rangle, c\}_{c \in \{0,1\}^n}.$$

For example, consider a system with one qubit and one classical bit. An extended mixed state (as in Definition 2.2.20) could be as follows: $\{(\frac{1}{2}, |0\rangle, 0), (\frac{1}{2}, |1\rangle, 1)\}$.

Definition 2.2.22 (Equality of Extended Mixed States). Let n be the size of the classical register. Let $\forall i \in \{1, 2\}, \rho_i = \{p_{c_i}, |\psi_{c_i}\rangle, c_i\}_{c_i \in \{0,1\}^{m_i}}$ be extended mixed states (Definition 2.2.20). We denote by $\rho_1 = \rho_2$ the equality of extended mixed states. We say that $\rho_1 = \rho_2$ if and only if

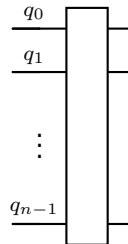
$$\forall c_1 \in \{0, 1\}^{m_1}, \exists c_2 \in \{0, 1\}^{m_2}, \quad c_1 = c_2 \quad \text{and} \quad p_{c_1} = p_{c_2} \quad \text{and} \quad |\psi_{c_1}\rangle = |\psi_{c_2}\rangle.$$

2.2.1.6 Notations

In this section, we recapitulate all the important notations for the quantum state.

Notation 2.2.23 (Quantum state). Let n be the size of the quantum register.

1. Qubits can be denoted by i or q_i , where i is an index ranging from 0 to $n - 1$. Qubits are typically represented with the smallest index at the top, increasing downward, reflecting their order in the register.



2. Qubits are indexed to denote their initial position in the register. $|x_i\rangle$, where $i \in \{0, \dots, n-1\}$, denotes the initial value of the qubit at position i . To illustrate the notation of qubit indices, consider the following example: $|x_2x_0x_1\rangle$ represents a quantum system that was initially in the state $|x_0x_1x_2\rangle$.

Let s_1 and s_2 be two sets of integers with a total order. Let σ be the unique permutation (acting as identity outside s_1) mapping the elements of s_1 in increasing order to those of s_2 in increasing order, assuming $|s_1| = |s_2|$.

For example, let $s_1 = \{0, 1, 2\}$ and $s_2 = \{3, 5, 0\}$ be two sets of qubit indices. The permutation σ is defined as $\sigma(0) = 0$, $\sigma(1) = 3$, and $\sigma(2) = 5$. We denote by:

1. $|x\rangle = |x_0x_1 \dots x_{n-1}\rangle$: a basis state, where $x_i \in \{0, 1\}$. x is always indexed in the same way.
2. $|x\rangle_i$: the state of the qubit indexed by i , where $|x\rangle$ is a basis state.
3. $|\psi\rangle$, $|\phi\rangle$, or $|\xi\rangle$: a general state.
4. $|\psi\phi\rangle$: a separable state, equivalent to $|\psi\rangle \otimes |\phi\rangle$.
5. $|\psi\rangle \sim_{(\sigma')} |\phi\rangle$: the vector equivalence (Notation 2.2.10) of $|\psi\rangle$ and $|\phi\rangle$ modulo a qubit indices permutation σ' . Formally,

$$\text{if } |\psi\rangle = \sum_i \alpha_i |x_0^i \dots x_{n-1}^i\rangle, \text{ then } |\phi\rangle = \sum_i \alpha_i |x_{\sigma'(0)}^i \dots x_{\sigma'(m-1)}^i\rangle$$

(Definition 2.2.8)

6. $|\psi\rangle \sim_{(s_1, s_2)} |\phi\rangle$: the vector equivalence of $|\psi\rangle$ and $|\phi\rangle$ modulo a qubit indices permutation σ (Vect-Equiv-Map). Formally, $|\psi\rangle \sim_{(s_1, s_2)} |\phi\rangle := |\psi\rangle \sim_{(\sigma)} |\phi\rangle$.
For example, let $s_1 = \{0, 1, 2\}$ and $s_2 = \{3, 5, 0\}$. The permutation σ is defined as $\sigma(0) = 0$, $\sigma(1) = 3$, and $\sigma(2) = 5$. Then, $|\psi\rangle = \alpha |001\rangle + \beta |010\rangle + \gamma |100\rangle$ is equal to $|\phi\rangle = \alpha |011001\rangle + \beta |110110\rangle + \gamma |001010\rangle$ under σ , and thus $|\psi\rangle \sim_{(s_1, s_2)} |\phi\rangle$.
7. $|\psi\rangle \sim_{(s_1, s_2, \text{gp})} |\phi\rangle$: the global phase equivalence (Definition 2.2.12) of $|\psi\rangle$ and $|\phi\rangle$ modulo a qubit indices permutation σ (GP-Equiv-Map).
8. $|\psi\rangle \sim_{(s_1, s_2, \text{cp})} |\phi\rangle$: the conditional phase equivalence (Definition 2.2.14) of $|\psi\rangle$ and $|\phi\rangle$ modulo a qubit indices permutation σ (CP-Equiv-Map).

Let $|\xi^1\rangle_{\text{Inp}_1}$ be the input quantum state of the circuit C_1 and $|\xi^2\rangle_{\text{Inp}_2}$ be the input quantum state of the circuit C_2 . We denote by:

1. Inp, Init, Obs, and Di the sets (Notation 2.1.2) of Q corresponding to input, initialisation, observed, and discard (garbage) qubits, respectively. These satisfy:

$$Q = \text{Inp} \cup \text{Init} = \text{Obs} \cup \text{Di}, \quad \text{Inp} \cap \text{Init} = \emptyset, \quad \text{Obs} \cap \text{Di} = \emptyset.$$

2. Inp (resp. Obs): in the context of circuit equivalence between C_1 and C_2 , the set Inp_1 (resp. Obs_1) if $\text{Inp}_1 \simeq \text{Inp}_2$ (resp. $\text{Obs}_1 \simeq \text{Obs}_2$) (Notation 2.1.2).
3. $|\xi\rangle_{\text{Inp}}$, in the context of circuit equivalence between C_1 and C_2 , the state $|\xi^1\rangle_{\text{Inp}_1}$ if $|\xi^1\rangle_{\text{Inp}_1} \sim_{(\text{Inp}_1, \text{Inp}_2)} |\xi^2\rangle_{\text{Inp}_2}$.
4. $|\psi\rangle_{\text{Obs}}$, in the context of circuit equivalence between C_1 and C_2 , the state $|\psi^1\rangle_{\text{Obs}_1}$ if $|\psi^1\rangle_{\text{Obs}_1} \sim_{(\text{Obs}_1, \text{Obs}_2)} |\psi^2\rangle_{\text{Obs}_2}$.

2.2.2 Measurement (M)

Measurement operations are the only way to extract information from quantum states. While unitary operations evolve quantum states deterministically, measurement operations introduce probabilistic outcomes. This marks the fundamental transition from deterministic quantum evolution to the probabilistic nature of measurement outcomes. In Definition 2.2.24, we define complete register measurement, where all qubits in a quantum register are measured, resulting in a probability distribution of pure states with all information transferred to classical registers. Conversely, in Definition 2.2.26, we define partial measurement, which involves measuring only a subset of qubits, yielding extended mixed states (Definition 2.2.20) where part of the quantum information may remain in the quantum register, potentially in superposition, while the rest is stored in classical registers.

Definition 2.2.24 (Complete Register Measurement). Let $\mathcal{H}^{\otimes n}$ be a Hilbert space of n qubits, assume that $|\psi\rangle \in \mathcal{H}^{\otimes n}$. Upon measurement, $|\psi\rangle$ is projected onto one of the states in the standard computational basis (Definition 2.2.9). The probability of measuring the outcome $c \in \{0, 1\}^n$ is given by $|\langle c | \psi \rangle|^2$ (Definition 2.2.4). The *complete register measurement* is the probability distribution $d(\psi)$ over all possible outcomes:

$$d(\psi) = \{ |\langle c | \psi \rangle|^2, |c\rangle, c \}_{c \in \{0, 1\}^n}$$

Example 2.2.25 (Complete Register Measurement). Consider a quantum state $|\psi\rangle = \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle$, where $\alpha, \beta, \gamma, \delta \in \mathbb{C}$. We calculate the probabilities of measuring each basis state as $p(00) = |\alpha|^2$, $p(01) = |\beta|^2$, $p(10) = |\gamma|^2$, and $p(11) = |\delta|^2$. According to Definition 2.2.24, we derive the complete register measurement of $|\psi\rangle$ as

$$d(\psi) = \{ (|\alpha|^2, |00\rangle, 00), (|\beta|^2, |01\rangle, 01), (|\gamma|^2, |10\rangle, 10), (|\delta|^2, |11\rangle, 11) \}.$$

Definition 2.2.26 (Partial Register Measurement). The *partial register measurement* generalises the complete register measurement (Definition 2.2.24) by allowing measurement of only a subset of qubits. Let $\mathcal{H}^{\otimes n}$ be a Hilbert space of n qubits, and let $|\psi\rangle \in \mathcal{H}^{\otimes n}$ be a quantum state. Suppose we measure $m < n$ qubits, leaving the remaining $n - m$ qubits unmeasured. For a given measurement outcome $c \in \{0, 1\}^m$ on the measured qubits, the probability of this outcome is given by:

$$p_c = \sum_{z \in \{0, 1\}^{n-m}} |\langle cz | \psi \rangle|^2$$

where z represents all possible states of the $n - m$ unmeasured qubits. The resulting state of the entire system after obtaining outcome c is:

$$|\psi_c\rangle = \frac{\sum_{z \in \{0,1\}^{n-m}} \langle zc | \psi \rangle |cz\rangle}{\sqrt{p_c}}$$

Thus, the partial register measurement result is: $\{p_c, |\psi_c\rangle, c\}_{c \in \{0,1\}^m}$.

For example, consider the same quantum state as in Example 2.2.25: $|\psi\rangle = \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle$, where $\alpha, \beta, \gamma, \delta \in \mathbb{C}$. Now, let's measure only the first qubit (a partial measurement). According to Definition 2.2.26, we calculate:

- For outcome 0 on the first qubit: $p_0 = |\alpha|^2 + |\beta|^2$, and $|\psi_0\rangle = \frac{\alpha|00\rangle + \beta|01\rangle}{\sqrt{p_0}}$.
- For outcome 1 on the first qubit: $p_1 = |\gamma|^2 + |\delta|^2$, and $|\psi_1\rangle = \frac{\gamma|10\rangle + \delta|11\rangle}{\sqrt{p_1}}$.
- Thus, the partial register measurement of $|\psi\rangle$ when measuring only the first qubit is: $\{(p_0, |\psi_0\rangle), (p_1, |\psi_1\rangle)\}$.

2.2.3 Unitary Matrices

In quantum computing, the standard way to represent the evolution of a quantum state (Definition 2.2.8) is to use unitary matrices. In this section, we provide the minimal definitions required to define the notion of unitary matrix, concluding with their formal definition (Definition 2.2.32).

Definition 2.2.27 (Matrix Conjugate ($\overline{M}$)). The *conjugate* of a complex matrix M is a matrix where all the complex coefficients $z = a + ib$ are replaced by their conjugates $\overline{z} = a - ib$.

For example, if $M = \begin{pmatrix} 1+i & 0 \\ 2 & -3i \end{pmatrix}$, the conjugate of M is $\overline{M} = \begin{pmatrix} 1-i & 0 \\ 2 & 3i \end{pmatrix}$.

Definition 2.2.28 (Matrix Transposed (M^T)). The *transpose* of a matrix M is a matrix obtained by exchanging its rows with columns. If M is an $m \times n$ matrix, its transpose M^T is an $n \times m$ matrix.

For example, if $M = \begin{pmatrix} 1+i & 0 \\ 2 & -3i \end{pmatrix}$, the transpose of M is $M^T = \begin{pmatrix} 1+i & 2 \\ 0 & -3i \end{pmatrix}$.

Definition 2.2.29 (Matrix Adjoint ($M^\dagger$)). The *adjoint* of a matrix M , denoted $M^\dagger$, is the transpose of the conjugate of M . It is obtained by first taking the conjugate of each element of M and then transposing the resulting matrix.

For example, if $M = \begin{pmatrix} 1+i & 0 \\ 2 & -3i \end{pmatrix}$, the adjoint of M is $M^\dagger = \begin{pmatrix} 1-i & 2 \\ 0 & 3i \end{pmatrix}$.

Definition 2.2.30 (Square Matrix). A *square matrix* is a matrix with an equal number of rows and columns. A square matrix of size $n \times n$ has n rows and n columns.

Definition 2.2.31 (Identity Matrix (I)). An *identity matrix* is a square matrix in which all the elements of the principal diagonal are ones, and all other elements are zeros.

For example, let I be an identity matrix of size 2×2 . Then, $I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$.

Definition 2.2.32 (Unitary Matrix). A *unitary matrix* U is a complex square matrix that satisfies the condition: $UU^\dagger = U^\dagger U = I$, where $U^\dagger$ is the adjoint (Definition 2.2.29) of U , and I is the identity matrix (Definition 2.2.31). This property ensures that the matrix preserves the norm of vectors and that the operation it represents is reversible.

For example, the Hadamard operation is a fundamental single-qubit quantum transformation represented by the following unitary matrix:

$$H = H^\dagger = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}. \quad (\text{Definition 2.2.29})$$

To verify that H is unitary, we check that $HH^\dagger = H^\dagger H = I$:

$$HH^\dagger = H^\dagger H = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$$

We can use the unitary matrix associated with the Hadamard gate to evolve the quantum state $|0\rangle$: $H|0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$.

2.2.4 Unitary Circuits

This section introduces the concept of unitary circuits and presents three standard representations commonly used in the field of quantum computing. These representations are particularly relevant to our work. Firstly, we discuss the graphical representation in Section 2.2.4.1, which is widely used for describing small circuits in a highly readable manner. This representation is employed throughout this document to provide examples and simplify explanations. Secondly, we explore the bricks-based representation in Section 2.2.4.2, a common approach well-suited for formal verification. This representation is particularly useful in our work for translating circuits from Qbricks to OpenQASM, as detailed in Chapter 3. Finally, we examine the QASM representation in Section 2.2.4.3, a popular choice for programming and formal verification.

2.2.4.1 Graphical Representation

A unitary circuit can be represented as a series of gates connected by wires. Each gate corresponds to a unitary matrix that transforms the state of the system. We introduce these gates using basis states (Definition 2.2.7). These definitions generalise by linearity to any quantum state (Definition 2.2.8).

In the following, we discuss the relevant gates for our work. The *Hadamard gate* (H) creates a state in *superposition*:

$$|0\rangle \xrightarrow{\text{H}} \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \quad \text{and} \quad |1\rangle \xrightarrow{\text{H}} \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle).$$

The Z gate, also known as the *phase flip gate*, inverts the phase of the state $|1\rangle$:

$$|0\rangle \xrightarrow{\text{Z}} |0\rangle \quad \text{and} \quad |1\rangle \xrightarrow{\text{Z}} -|1\rangle.$$

The X gate, also known as the *bit flip gate*, inverts the state of a qubit:

$$|0\rangle \xrightarrow{\text{X}} |1\rangle \quad \text{and} \quad |1\rangle \xrightarrow{\text{X}} |0\rangle.$$

Definition 2.2.33 (Controlled Quantum Gates). A *controlled quantum gate* is a multi-qubit gate that applies a specific quantum operation to one or more *target qubits* conditionally based on the state of one or more control qubits. If all *control qubits* are in the state $|1\rangle$, the gate operation is applied to the target qubits. Otherwise, the target qubits remain unchanged. The control qubits themselves are not affected by the operation.

The *Controlled Not gate*, also known as the CNOT or CX gate, is a controlled quantum gate (Definition 2.2.33) that apply a X gate to the target qubit (X) if the control qubit is in the state $|1\rangle$. The control qubit remains unchanged. The action of the CNOT gate can be visualized with the following quantum circuits:

$$\begin{array}{c} |0\rangle \xrightarrow{\bullet} |0\rangle \\ |0\rangle \xrightarrow{\text{X}} |0\rangle \end{array}, \quad \begin{array}{c} |0\rangle \xrightarrow{\bullet} |0\rangle \\ |1\rangle \xrightarrow{\text{X}} |1\rangle \end{array}, \quad \begin{array}{c} |1\rangle \xrightarrow{\bullet} |1\rangle \\ |0\rangle \xrightarrow{\text{X}} |1\rangle \end{array}, \quad \text{and} \quad \begin{array}{c} |1\rangle \xrightarrow{\bullet} |1\rangle \\ |1\rangle \xrightarrow{\text{X}} |0\rangle \end{array}.$$

In a quantum circuit, the identity gate can be visualised as:

$$|\psi\rangle \xrightarrow{\text{I}} |\psi\rangle$$

The identity gate indicates that the state of the qubit ($|\psi\rangle$, Notation 2.2.23) remains unchanged.

We now informally introduce three multi-qubit gates using basis states (Definition 2.2.7). These gates are: the Swap gate, the Toffoli gate, and the Fredkin gate. These definitions generalise by linearity to any quantum state (Notation 2.2.23).

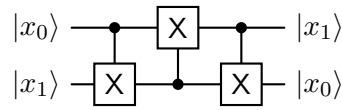
Definition 2.2.34 (Toffoli Gate (CCNOT, CCX)). The *Toffoli gate* is a three-qubit controlled gate (Definition 2.2.33) that apply a X gates to a target qubit (X) if and only if both control qubits are in the state $|1\rangle$. The state of the control qubits remains unchanged. The Toffoli gate can be represented by the following quantum circuit:

$$\begin{array}{c} |x_0\rangle \xrightarrow{\bullet} |x_0\rangle \\ |x_1\rangle \xrightarrow{\bullet} |x_1\rangle \\ |x_2\rangle \xrightarrow{\text{X}} |x_0 \cdot x_1 \oplus x_2\rangle \end{array}$$

Definition 2.2.35 (Swap Gate (SWAP)). The *Swap gate* is a two-qubit gate that differs from controlled gates (Definition 2.2.33) in that it exchanges the quantum states of two qubits. Specifically, if the input states are $|x_0\rangle$ and $|x_1\rangle$, the Swap gate outputs $|x_1\rangle$ and $|x_0\rangle$ respectively. It can be represented by the following quantum circuit:

$$\begin{array}{c} |x_0\rangle \text{ --- } \times \text{ --- } |x_1\rangle \\ |x_1\rangle \text{ --- } \times \text{ --- } |x_0\rangle \end{array}$$

Example 2.2.36 (Swap Gate Implementation with CNOT Gates). The Swap gate can be implemented using three CNOT gates. The following quantum circuit demonstrates this implementation:



Definition 2.2.37 (Fredkin Gate (CSWAP)). The *Fredkin gate* is a three-qubit controlled gate that applies a SWAP gate between the two target qubits if the control qubit is in the state $|1\rangle$. The state of the control qubit remains unchanged. The Fredkin gate can be represented by the following quantum circuit:

$$\begin{array}{c} |x_0\rangle \text{ --- } \bullet \text{ --- } |x_0\rangle \\ |x_1\rangle \text{ --- } \times \text{ --- } |x_0 \cdot x_2 \oplus \neg x_0 \cdot x_1\rangle \\ |x_2\rangle \text{ --- } \times \text{ --- } |x_0 \cdot x_1 \oplus \neg x_0 \cdot x_2\rangle \end{array}$$

2.2.4.2 Brick-Based Representation

This section defines a syntax (Definition 2.2.38) that allows for a general representation of quantum circuits. This syntax is well-suited to matrix semantics (Definition 2.2.40), which is characterised by both temporal and spatial composition. The set of gates we have selected is universal, offering great flexibility in circuit design that aligns with the circuit transformations we are considering.

Definition 2.2.38 (Brick-Based Unitary Circuit (U)). We define a *brick-based representation of a unitary circuit* as a temporal (—) and spatial (/) composition of instructions. The gates used in this representation are as follows:

- Rotation gates $R_x(\theta)$, $R_y(\theta)$, $R_z(\theta)$, $U_1(\theta)$ are parametrised by an angle, $\theta \in \mathbb{R}$.
- Specifically, $U_1(\theta)$ is equivalent to $R_z(\theta)$ up to a global phase.
- The gates X, Y, Z, S, T are specialized rotation gates.
- The gate GP (θ) applies a global phase.
- I represents the identity gate, which leaves the qubit state unchanged.
- CX flips the target qubit if the control qubit is in state $|1\rangle$.

The syntax is:

$$\begin{aligned} G &:= H \mid R_x(\theta) \mid R_y(\theta) \mid R_z(\theta) \mid U_1(\theta) \mid GP(\theta) \mid X \mid Y \mid Z \mid S \mid T \mid I \mid CX \\ U &:= G \mid G \text{ -- } U \mid G // U \end{aligned}$$

A brick-based unitary circuit is *well-formed* if it does not contain temporal composition of spatial compositions, denoted by $U_1 \text{ -- } (U_2 // U_3)$.

We define the inversion of unitary circuits, which is used to verify the equivalence of unitary circuits as presented in Chapter 4.

Definition 2.2.39 (Inverse). The *brick-based representation of unitary circuit inverse* is obtained by reversing the temporal order of component unitaries and applying the adjoint operation to each. For a unitary circuit $U = U_1 \text{ -- } U_2 \text{ -- } \dots \text{ -- } U_n$, its inverse is defined as $U^\dagger = U_n^\dagger \text{ -- } \dots \text{ -- } U_2^\dagger \text{ -- } U_1^\dagger$.

For example, consider a unitary circuit defined as $U = (CX \text{ -- } H // I)$. In this case, both the Hadamard gate and the CX gate are self-inverse, so the inverse circuit is constructed by simply reversing the temporal order of operations: $U^\dagger = (H // I) \text{ -- } CX$.

Definition 2.2.40 (Semantics). The *semantics of the brick-based representation of unitary circuits* is defined using unitary matrices (Definition 2.2.32) to evolve quantum states (Definition 2.2.8). This semantics employs matrix multiplication ($\cdot$) for temporal composition (--) and the tensor product ($\otimes$, Definition 2.2.3) for parallel composition ($//$). Each rotation and global phase gate is parametrised by an angle, $\theta \in \mathbb{R}$. This semantics is defined in the following:

$$\begin{aligned} \llbracket H \rrbracket &:= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} & \llbracket R_x(\theta) \rrbracket &:= \begin{pmatrix} \cos \frac{\theta}{2} & -i \sin \frac{\theta}{2} \\ -i \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix} \\ \llbracket R_y(\theta) \rrbracket &:= \begin{pmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix} & \llbracket R_z(\theta) \rrbracket &:= \begin{pmatrix} e^{-i\frac{\theta}{2}} & 0 \\ 0 & e^{i\frac{\theta}{2}} \end{pmatrix} \\ \llbracket U_1(\theta) \rrbracket &:= \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix} & \llbracket GP(\theta) \rrbracket &:= e^{i\theta} \\ \llbracket X \rrbracket &:= i \llbracket R_x(\pi) \rrbracket = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} & \llbracket Y \rrbracket &:= i \llbracket R_y(\pi) \rrbracket = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \\ \llbracket Z \rrbracket &:= \llbracket U_1(\pi) \rrbracket = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} & \llbracket S \rrbracket &:= \llbracket U_1\left(\frac{\pi}{2}\right) \rrbracket = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} \\ \llbracket T \rrbracket &:= \llbracket U_1\left(\frac{\pi}{4}\right) \rrbracket = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\frac{\pi}{4}} \end{pmatrix} & \llbracket I \rrbracket &:= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \\ \llbracket CX \rrbracket &:= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} & \llbracket G \text{ -- } U \rrbracket &:= \llbracket U \rrbracket \cdot \llbracket G \rrbracket \\ & & \llbracket G // U \rrbracket &:= \llbracket G \rrbracket \otimes \llbracket U \rrbracket \end{aligned}$$

2.2.4.3 QASM representation of Unitary Circuits

This section presents a QASM representation of unitary circuits that extends the brick-based representation (Section 2.2.4.2) by introducing a more flexible notation for gate application. The QASM syntax for unitary circuits is defined in Definition 2.2.41, while its associated semantics are detailed in Definition 2.2.44.

Definition 2.2.41 (QASM Unitary Circuit (U)). The *QASM representation of unitary circuit* is defined as a sequence of instructions applied to a quantum register of n qubits:

$$\begin{aligned} G &:= H \mid R_x(\theta) \mid R_y(\theta) \mid R_z(\theta) \mid U_1(\theta) \mid GP(\theta) \mid X \mid Y \mid Z \mid S \mid T \mid I \\ GA &:= \text{Apply}(G, qb, qb') & \text{Ins} &:= E \mid GA \\ U &:= \text{Ins} \mid \text{Ins} \text{ -- } U & U &:= \text{Ins} \mid \text{Ins} \text{ // } U \end{aligned}$$

Informally:

1. $\text{Apply}(G, qb, qb')$ applies G to qubits in the set qb' , controlled by qubits in qb .
2. We represent temporal composition by (--) , and the spatial composition by (//) .
3. Finally, (E) denotes an empty circuit.

In the QASM representation of unitary circuits, a circuit is considered *well-formed* if every gate application of the form $\text{Apply}(G, qb, qb')$ satisfies the following conditions:

1. The sets of qubit indices qb and qb' have no common elements, $qb \cap qb' = \emptyset$. This condition enforces the no-cloning theorem [52, Box 12.1].
2. A gate must be applied to at least one qubit, i.e., $|qb'| > 0$.
3. All qubit indices must be positive integers less than n : $\forall i \in qb \cup qb', 0 \leq i < n$.

Notation 2.2.42 (Gate Application (GA)). To simplify the notation and when the context is clear, we denote by $\text{Apply}(G, i, j)$, $\text{Apply}(G, \{i\}, \{j\})$.

Notation 2.2.43 (Unary Application by Permutation). We define the application of a quantum gate G to a set of qubit indices qb . Let G be a quantum gate, $qb \subseteq \{0, 1, \dots, n-1\}$, and n the number of qubits in the register. The application of G to the qubits indexed by qb is defined recursively, with the identity operation applied to all other qubits.

- **Basis case:** $G_{\{i\}} := I^{\otimes i-1} // G // I^{\otimes n-1}$.
- **Inductive case:**

$$G_{qb} := \begin{cases} E & \text{if } qb = \emptyset, \\ G_{\{i\}} \text{ -- } G_{qb'} & \text{if } (i, qb') = \min(qb). \end{cases} \quad (\text{Notation 2.1.2})$$

Definition 2.2.44 (Semantics). Let $\mathcal{H}^{\otimes n}$ be a Hilbert space (Definition 2.2.6) and let $|x\rangle \in \mathcal{H}^{\otimes n}$ be a basis state. The semantics of the QASM representation of unitary circuits is defined as follows:

$$\begin{aligned} \llbracket \text{Apply} (G, \mathbf{qb}, \mathbf{qb}') \rrbracket (|x\rangle) &= \begin{cases} \llbracket G_{\mathbf{qb}'} \rrbracket (|x\rangle) & \text{if } \forall i \in \mathbf{qb}, x_i = 1, \\ |x\rangle & \text{otherwise,} \end{cases} & \llbracket E \rrbracket (|x\rangle) &= |x\rangle, \\ \llbracket G \rrbracket, \llbracket \text{Ins} // U \rrbracket, \llbracket \text{Ins} - U \rrbracket & & & \end{aligned} \quad (\text{Definition 2.2.40})$$

The extension to arbitrary quantum states follows by linearity.

2.2.4.4 Ancillas

This section introduces ancillas whose role is to facilitate internal computations.

Definition 2.2.45 (Ancilla Qubit). An *ancilla* qubit (Anc) is an auxiliary qubit used in quantum computations, initialised in a known state, typically $|0\rangle$, and facilitates operations by storing intermediate results.

Example 2.2.46 (Ancilla Qubit). To illustrate the concept of ancillas (Definition 2.2.45), consider a X gate controlled by three qubits, as shown in Figure 2.1a. By using an ancilla qubit, we can implement the same functionality with three X gates, each controlled by only two qubits (Toffoli gates, Definition 2.2.34), as illustrated in Figure 2.1b. When

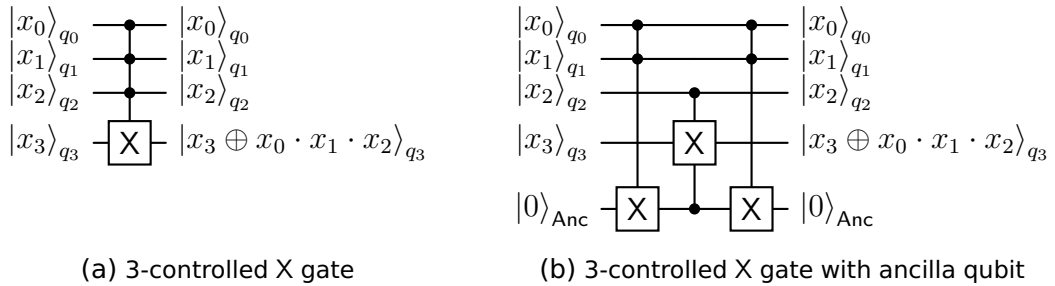


Figure 2.1: Example of using an ancilla qubit (Anc)

using an ancilla qubit, an additional Toffoli gate is added at the end to reset it to its initial state. This step ensures that the ancilla qubit does not affect the system's final state.

2.2.5 Hybrid Circuits

The commonly accepted current model of quantum computation is a hybrid model. A hybrid model mixes classical and quantum operations. More precisely, the central part of the computation is done with a classical processor, and the quantum processor handles specific computations. This model transitions from purely unitary, deterministic

quantum evolution to a probabilistic framework with measurements and classical control. In the hybrid model, measurement results condition the rest of the calculation. In this section, we formalise this hybrid computational model in three parts. First, we define a syntax of hybrid circuits (Section 2.2.5.1), next a semantics (Section 2.2.5.2), and finally, we introduce the concept of discard (Section 2.2.5.3), which is central to our study.

2.2.5.1 Hybrid Circuit Syntax

Our syntax for hybrid circuits extends the QASM representation of unitary circuits (Section 2.2.4.3) to include non-unitary operations such as measurement and classical control. We also add a classical register alongside the quantum register, and include qubit initialisation and the classical NOT gate to manage the circuit transformations discussed in Section 2.5.

Definition 2.2.47 (Hybrid Circuit (C)). A *hybrid circuit* extends the QASM representation of unitary circuits (Definition 2.2.41) by including a classical register to store measurement results and incorporating classical operations such as:

- Initialising qubits indexed by qb to $|0\rangle$: $\text{InitQ}(qb)$.
- Measuring qubits qb and storing the result in classical bits cb : $\text{Meas}(qb, cb)$.
- Applying GA controlled by classical bits cb : $\text{if}(cb, GA)$.
- Flipping the bits indexed by cb : $\text{Not}(cb)$.

Our syntax for hybrid circuits is defined as follows:

$$\begin{aligned}
 G &:= H \mid R_x(\theta) \mid R_y(\theta) \mid R_z(\theta) \mid U_1(\theta) \mid GP(\theta) \mid X \mid Y \mid Z \mid S \mid T \mid I \\
 GA &:= \text{Apply}(G, qb, qb') \\
 \text{Ins} &:= E \mid GA \mid \text{InitQ}(qb) \mid \text{if}(cb, GA) \mid \text{Meas}(qb, cb) \mid \text{Not}(cb) \\
 C &:= \text{Ins} \mid \text{Ins} \text{ -- } C \qquad \qquad \qquad C := \text{Ins} \mid \text{Ins} \text{ // } C
 \end{aligned}$$

A hybrid circuit is considered *well-formed* if it satisfies the well-formed condition of the QASM representation for unitary circuits (Definition 2.2.41) as well as the following additional conditions:

1. Each classical wire receives at most one measurement result.
2. After measuring on a qubit, no operation acting on that qubit such that GA , $\text{InitQ}(qb)$, $\text{if}(cb, GA)$, or $\text{Meas}(qb, cb)$ can be applied to it.
3. $\text{InitQ}(qb)$ does not follow any other operation on the same wire.
4. If we measure a qubit and store the result in cb and if cb is not in the discarded part of the register, it is forbidden to apply $\text{Not}(cb)$.

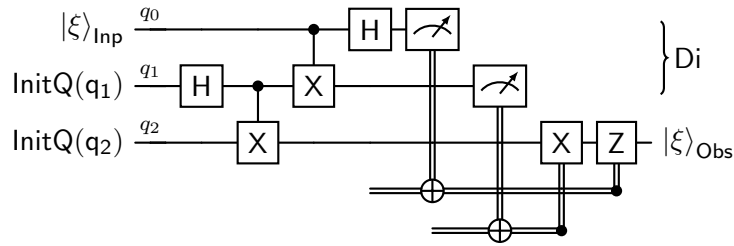
Notation 2.2.48 (Classical Operations). To simplify the notation and when the context is clear, we denote $\text{InitQ}(\{i\})$, $\text{Meas}(\{i\}, \{j\})$, $\text{if}(\{i\}, \text{GA})$, and $\text{Not}(\{i\})$ by $\text{InitQ}(i)$, $\text{Meas}(i, j)$, $\text{if}(i, \text{GA})$, and $\text{Not}(i)$ respectively.

Example 2.2.49 (Illustrating Both Well-Formed and Ill-Formed Conditions). In this example we illustrate both well-formed (✓) and ill-formed conditions (✗):

$C_1 := \text{InitQ}(q_0) \text{ -- Apply } (G, \emptyset, q_0) \text{ -- Meas}(q_0, c_0) \text{ -- if}(c_0, \text{Apply } (G', \emptyset, q_1))$	(✓)
$C_2 := \text{Apply } (G, \emptyset, q_0) \text{ -- InitQ}(q_0) \text{ -- Meas}(q_0, c_0) \text{ -- if}(\textcolor{red}{c}_1, \text{Apply } (G, \emptyset, q_1))$	(✗)
$C_3 := \text{InitQ}(q_0) \text{ -- Meas}(q_0, c_0) \text{ -- Apply } (G, \emptyset, \textcolor{red}{q}_0)$	(✗)
$C_4 := \text{Apply } (G, \emptyset, q_1) \text{ -- InitQ}(q_0) \text{ -- Meas}(q_1, c_0) \text{ -- Apply } (G, \emptyset, q_0)$	(✓)
$C_5 := \text{Apply } (H, q_0, q_1) \text{ -- Init}(\textcolor{red}{q}_0) \text{ -- Meas}(q_0, c_0)$	(✗)

The intermediate operations and well-formedness of these circuits are as follows: C_1 is well-formed (✓). C_2 is not well-formed because a classical wire ($\textcolor{red}{c}_1$) that has not received a measurement result is read in a classically controlled instruction (✗). C_3 is not well-formed because of the application of a quantum operation after measurement on the same quantum wire ($\textcolor{red}{q}_0$) (✗). C_4 is well-formed, with $\text{Meas}(q_1, c_0)$, as the operations on the same wire are in the correct order (✓). C_5 is not well-formed because the initialisation occurs after a Hadamard operation on the same wire ($\textcolor{red}{q}_0$) (✗).

Example 2.2.50 (Quantum Teleportation). A classic example of hybrid circuits is quantum teleportation [11], which is a protocol that enables the "teleportation" of a quantum state ($|\xi\rangle \in \mathcal{H}$) from the input q_0 to the output q_2 of the circuit without any transfer of matter. The graphical representation is:



The associated description is:

$C :=$	
$\text{InitQ}(q_1) \text{ -- InitQ}(q_2) \text{ --}$	(1, ✓)
$\text{Apply } (H, \emptyset, q_1) \text{ -- Apply } (X, q_1, q_2) \text{ --}$	
$\text{Apply } (X, q_0, q_1) \text{ -- Apply } (H, \emptyset, q_0) \text{ --}$	
$\text{Meas}(q_0, c_0) \text{ -- Meas}(q_1, c_1) \text{ --}$	(2, ✓)
$\text{if}(c_1, \text{Apply } (X, \emptyset, q_2)) \text{ -- if}(c_0, \text{Apply } (Z, \emptyset, q_2))$	(3, ✓)

The quantum teleportation circuit respects the *well-formation* conditions of hybrid circuits: All initializations ($\text{InitQ}(q_1)$) and ($\text{InitQ}(q_2)$) are at the beginning of the circuit (1, ✓). After the measurements ($\text{Meas}(q_0, c_0)$) and ($\text{Meas}(q_1, c_1)$), there are no further operations on q_0 and q_1 (2, ✓). The classical controls ($\text{if}(c_1, \text{Apply}(X, \emptyset, q_2))$) and ($\text{if}(c_0, \text{Apply}(Z, \emptyset, q_2))$) use c_0 and c_1 which have received measurement results (3, ✓).

2.2.5.2 Hybrid Circuits Semantics

In this section, we begin by defining the semantics of measurement in Definition 2.2.51. Next, we define qubit initialisation in Definition 2.2.52. Finally, we define the semantics of hybrid circuits in Definition 2.2.53, based on evolving extended mixed states.

Definition 2.2.51 (Measurement Operation). We define the measurement of the set of qubits indexed by (qb) into the set bit indexed by (cb). Let $\rho = \{p_c, |\xi_c\rangle, c\}_c$ be an extended mixed state (Definition 2.2.20). Let $n \in \mathbb{N}$ be the number of measurements, let qb be a set of qubit indices indicating the qubits to be measured, let $c \in \{0, 1\}^n$ be a bit-string representing the measurement results, and let cb be a set of bit indices indicating the bits where the results are stored. We define the semantics of measuring qubits indexed by qb into bits indexed by cb by induction, as follows:

- **Basis case**, Measurement of qubit indexed by i into bit indexed by j is:

1. Measurement of qubit indexed by 0 into bit indexed by j :

$$\begin{aligned} & \llbracket \text{Meas}(0, j) \rrbracket_{\mathbb{H}}(\rho) \\ &= \left\{ p_c \cdot \sum_{z \in \{0, 1\}^{n-1}} |\langle zx \mid \xi_c \rangle|^2, \frac{\sum_{z \in \{0, 1\}^{n-1}} \langle zx \mid \xi_c \rangle |xz\rangle}{\sqrt{\sum_{z \in \{0, 1\}^{n-1}} |\langle zx \mid \xi_c \rangle|^2}}, c' \right\}_{c' = c_{[j \leftarrow x]}} \end{aligned}$$

(Definition 2.2.4, Notation 2.1.3)

2. Measurement of qubit indexed by i into bit indexed by j :

$$\text{Meas}(i, j) := I^{\otimes i-1} // \text{Meas}(0, j) / I^{\otimes n-1}.$$

- **Inductive case**:

$$\llbracket \text{Meas}(\text{qb}, \text{cb}) \rrbracket_{\mathbb{H}}(\rho) := \begin{cases} \rho & \text{if } \text{qb} = \emptyset \\ \llbracket \text{Meas}(i, j) \rrbracket_{\mathbb{H}}(\rho) & \text{if } \min(\text{qb}) = (i, \text{qb}') \\ \llbracket \text{Meas}(\text{qb}', \text{cb}') \rrbracket_{\mathbb{H}}(\rho) & \text{and } \min(\text{cb}) = (j, \text{cb}') \end{cases}$$

The operation is *well-formed* when $|\text{qb}| = |\text{cb}| = n > 0$.

Definition 2.2.52 (Qubit Initialisation). We define the qubit initialisation to $|0\rangle$ of a set of qubits indexed by (qb). Let $\rho = \{p_c, |\xi_c\rangle, c\}_c$ be an extended mixed state (Definition 2.2.20). Let $n \in \mathbb{N}$ be the number of measurements, let qb be a set of qubit indices indicating the qubits to be measured. We define the semantics of initialising qubits indexed by qb to $|0\rangle$ by induction, denoted $\text{InitQ}(\text{qb})$, as follows:

- **Basis case:** Initialisation of qubit indexed by i to $|0\rangle$ is defined by:

$$\llbracket \text{InitQ}(0) \rrbracket_{\mathcal{H}}(\rho) = \{p_c, I^{\otimes i-1} // |0\rangle\langle 0| // I^{\otimes n-1} |\psi\rangle, c\}_c \quad (\text{Definition 2.2.5})$$

- **Inductive case:**

$$\llbracket \text{InitQ}(\text{qb}) \rrbracket_{\mathcal{H}}(\rho) := \begin{cases} \rho & \text{if } \text{qb} = \emptyset \\ \llbracket \text{InitQ}(i) \rrbracket_{\mathcal{H}}(\rho) \text{---} \llbracket \text{InitQ}(\text{qb}') \rrbracket_{\mathcal{H}}(\rho) & \text{if } \min(\text{qb}) = (i, \text{qb}') \end{cases}$$

Definition 2.2.53 (Hybrid Circuit Semantics). Let $\rho = \{p_c, |\xi_c\rangle, c\}_c$ be an extended mixed state (Definition 2.2.20). Let qb be a set of qubit indices, $c \in \{0, 1\}^n$ be a bit-string, and cb be a set of bit indices, we define a *hybrid circuit semantics* as follows:

$$\llbracket \text{Not}(\text{cb}) \rrbracket_{\mathcal{H}}(\rho) = \{p_{c'}, |\xi_{c'}\rangle, c'\}_{c' = c_{[\text{cb} \leftarrow \overline{c(\text{cb})}]}} \quad (\text{Notation 2.1.3})$$

$$\llbracket \text{if}(\text{cb}, \text{Apply}(\text{co}, \text{ta}, \text{G})) \rrbracket_{\mathcal{H}} := \begin{cases} \llbracket \text{E} \rrbracket_{\mathcal{H}} & \text{if } \forall i \in \text{cb}, c_i = 0 \\ \llbracket \text{Apply}(\text{co}, \text{ta}, \text{G}) \rrbracket_{\mathcal{H}} & \text{if } \forall i \in \text{cb}, c_i = 1 \end{cases} \quad (\text{Notation 2.1.3})$$

$$\llbracket \text{Meas}(\text{qb}, \text{cb}) \rrbracket_{\mathcal{H}}(\rho) \quad (\text{Definition 2.2.51})$$

$$\llbracket \text{InitQ}(\text{qb}) \rrbracket_{\mathcal{H}}(\rho) \quad (\text{Definition 2.2.52})$$

$$\llbracket \text{G} \rrbracket_{\mathcal{H}} := \llbracket \text{G} \rrbracket, \quad \llbracket \text{Ins---C} \rrbracket_{\mathcal{H}} := \llbracket \text{Ins---C} \rrbracket, \quad \llbracket \text{Ins//C} \rrbracket_{\mathcal{H}} := \llbracket \text{Ins//C} \rrbracket \quad (\text{Definition 2.2.40})$$

$$\llbracket \text{Apply}(\text{G}, \text{qb}, \text{qb}') \rrbracket_{\mathcal{H}} := \llbracket \text{Apply}(\text{G}, \text{qb}, \text{qb}') \rrbracket, \quad \llbracket \text{E} \rrbracket_{\mathcal{H}} := \llbracket \text{E} \rrbracket \quad (\text{Definition 2.2.44})$$

Example 2.2.54 (Hybrid Circuit Semantics: Quantum Teleportation). To illustrate our hybrid circuit semantics, we use the example of quantum teleportation (Example 2.2.50). Let $\mathcal{H}$ be a Hilbert space of 1 qubit, let $(*)$ be a classical undefined bit, and let the input state be defined as: $|\xi\rangle = \sum_{i \in \{0,1\}} \alpha_i |i\rangle_{q_0} \in \mathcal{H}$, $|\phi_0\rangle, |\phi_1\rangle \in \mathcal{H}$, where $\sum_{i \in \{0,1\}} |\alpha_i|^2 = 1$. The protocol consists of several steps:

1. Initializations:

$$\begin{aligned} & \llbracket \text{InitQ}(q_1) \rrbracket_{\mathcal{H}} \text{---} \llbracket \text{InitQ}(q_2) \rrbracket_{\mathcal{H}} \left(\left\{ 1, \sum_{i \in \{0,1\}} \alpha_i |i\rangle_{q_0} |\phi_0\rangle_{q_1} |\phi_1\rangle_{q_2}, *_{c_0} *_{c_1} \right\} \right) \\ &= \left\{ 1, \sum_{i \in \{0,1\}} \alpha_i |i\rangle_{q_0} |0\rangle_{q_1} |0\rangle_{q_2}, *_{c_0} *_{c_1} \right\} \end{aligned}$$

2. Preparation:

$$\begin{aligned}
& \llbracket \text{Apply}(\mathbf{H}, \emptyset, \mathbf{q}_1) \rrbracket_{\mathbf{H}} \left(\left\{ 1, \sum_{i \in \{0,1\}} \alpha_i |i\rangle_{q_0} |0\rangle_{q_1} |0\rangle_{q_2}, *_{c_0} *_{c_1} \right\} \right) \\
&= \left\{ 1, \frac{1}{\sqrt{2}} \sum_{i,j \in \{0,1\}} \alpha_i |i\rangle_{q_0} |j\rangle_{q_1} |0\rangle_{q_2}, *_{c_0} *_{c_1} \right\} \\
& \llbracket \text{Apply}(\mathbf{X}, \mathbf{q}_1, \mathbf{q}_2) \rrbracket_{\mathbf{H}} \left(\left\{ 1, \frac{1}{\sqrt{2}} \sum_{i,j \in \{0,1\}} \alpha_i |i\rangle_{q_0} |j\rangle_{q_1} |0\rangle_{q_2}, *_{c_0} *_{c_1} \right\} \right) \\
&= \left\{ 1, \frac{1}{\sqrt{2}} \sum_{i,j \in \{0,1\}} \alpha_i |i\rangle_{q_0} |j\rangle_{q_1} |j\rangle_{q_2}, *_{c_0} *_{c_1} \right\}
\end{aligned}$$

3. Alice sends the quantum state $|\xi\rangle$:

$$\begin{aligned}
& \llbracket \text{Apply}(\mathbf{X}, \mathbf{q}_0, \mathbf{q}_1) \rrbracket_{\mathbf{H}} \left(\left\{ 1, \frac{1}{\sqrt{2}} \sum_{i,j \in \{0,1\}} \alpha_i |i\rangle_{q_0} |j\rangle_{q_1} |j\rangle_{q_2}, *_{c_0} *_{c_1} \right\} \right) \\
&= \left\{ 1, \frac{1}{\sqrt{2}} \sum_{i,j \in \{0,1\}} \alpha_i |i\rangle_{q_0} |i \oplus j\rangle_{q_1} |j\rangle_{q_2}, *_{c_0} *_{c_1} \right\} \\
& \llbracket \text{Apply}(\mathbf{H}, \emptyset, \mathbf{q}_0) \rrbracket_{\mathbf{H}} \left(\left\{ 1, \frac{1}{\sqrt{2}} \sum_{i,j \in \{0,1\}} \alpha_i |i\rangle_{q_0} |i \oplus j\rangle_{q_1} |j\rangle_{q_2}, *_{c_0} *_{c_1} \right\} \right) \\
&= \left\{ 1, \frac{1}{2} \sum_{i,j,k \in \{0,1\}} \alpha_i (-1)^{i \cdot k} |k\rangle_{q_0} |i \oplus j\rangle_{q_1} |0\rangle_{q_2}, *_{c_0} *_{c_1} \right\} \\
& \llbracket \text{Meas}(\mathbf{q}_0, c_0) \rrbracket_{\mathbf{H}} \left(\left\{ 1, \frac{1}{2} \sum_{i,j,k \in \{0,1\}} \alpha_i (-1)^{i \cdot k} |k\rangle_{q_0} |i \oplus j\rangle_{q_1} |j\rangle_{q_2}, *_{c_0} *_{c_1} \right\} \right) \\
&= \left\{ \frac{1}{2}, \frac{1}{\sqrt{2}} \sum_{i,j,k \in \{0,1\}} \alpha_i (-1)^{i \cdot k} |k\rangle_{q_0} |i \oplus j\rangle_{q_1} |j\rangle_{q_2}, [k]_{c_0} *_{c_1} \right\} \\
& \llbracket \text{Meas}(\mathbf{q}_1, c_1) \rrbracket_{\mathbf{H}} \left(\left\{ \frac{1}{2}, \frac{1}{\sqrt{2}} \sum_{i,j,k \in \{0,1\}} \alpha_i (-1)^{i \cdot k} |k\rangle_{q_0} |i \oplus j\rangle_{q_1} |j\rangle_{q_2}, [k]_{c_0} *_{c_1} \right\} \right) \\
&= \left\{ \frac{1}{4}, \sum_{i,j,k \in \{0,1\}} \alpha_i (-1)^{i \cdot k} |k\rangle_{q_0} |i \oplus j\rangle_{q_1} |j\rangle_{q_2}, [k]_{c_0} [i \oplus j]_{c_1} \right\}
\end{aligned}$$

4. Bob receives the state $|\xi\rangle$ with probability 1, as expected:

$$\begin{aligned}
& \llbracket \text{if}(c_1, \text{Apply}(X, \emptyset, q_2)) \rrbracket_H \left(\left\{ \frac{1}{4}, \sum_{i,j,k \in \{0,1\}} \alpha_i (-1)^{i \cdot k} |k\rangle_{q_0} |i \oplus j\rangle_{q_1} |j\rangle_{q_2}, [k]_{c_0} [i \oplus j]_{c_1} \right\} \right) \\
&= \left\{ \frac{1}{4}, \sum_{i,j,k \in \{0,1\}} \alpha_i (-1)^{i \cdot k} |k\rangle_{q_0} |i \oplus j\rangle_{q_1} |i\rangle_{q_2}, [k]_{c_0} [i \oplus j]_{c_1} \right\} \\
& \llbracket \text{if}(c_0, \text{Apply}(Z, \emptyset, q_2)) \rrbracket_H \left(\left\{ \frac{1}{4}, \sum_{i,j,k \in \{0,1\}} \alpha_i (-1)^{i \cdot k} |k\rangle_{q_0} |i \oplus j\rangle_{q_1} |i\rangle_{q_2}, [k]_{c_0} [i \oplus j]_{c_1} \right\} \right) \\
&= \left\{ \frac{1}{4}, \sum_{i,j,k \in \{0,1\}} \alpha_i |k\rangle_{q_0} |i \oplus j\rangle_{q_1} |i\rangle_{q_2}, [k]_{c_0} [i \oplus j]_{c_1} \right\} \\
&= \left\{ \frac{1}{4}, \sum_{i,j,k \in \{0,1\}} \alpha_i |k\rangle_{q_0} |j\rangle_{q_1} |i\rangle_{q_2}, [k]_{c_0} [j]_{c_1} \right\} \quad (i \oplus j \text{ replaced by } j) \\
&= \left\{ \frac{1}{4}, \sum_{j,k \in \{0,1\}} |k\rangle_{q_0} |j\rangle_{q_1} \otimes \sum_{i \in \{0,1\}} \alpha_i |i\rangle_{q_2}, [k]_{c_0} [j]_{c_1} \right\}.
\end{aligned}$$

2.2.5.3 Discards

The concept of discard is central to our work because transformations that add discard qubits are the most difficult to verify. The discard operation simplifies a quantum circuit by forgetting qubits that are no longer useful. Typically, these are qubits used for intermediate calculations. At worst, this allows us to simplify the circuit by focusing only on the observed qubits. At best, it allows us to reuse these qubits. However, before discarding a qubit, we must ensure it is not entangled with the rest of the register. This is done in Example 2.2.46 by adding a Toffoli gate at the end. Without this precaution, the discard operation can disturb the rest of the computation. In our study, we use the discard notation as a marker to distinguish these qubits from the observed qubits, without necessarily applying the operation itself. We begin by defining discard qubits (Definition 2.2.55). Next, we introduce the different registers used here (Definition 2.2.56). Finally, we provide the Example 2.2.57 for highlighting the importance of checking the separation between discard (Di) and observed (Obs) qubits.

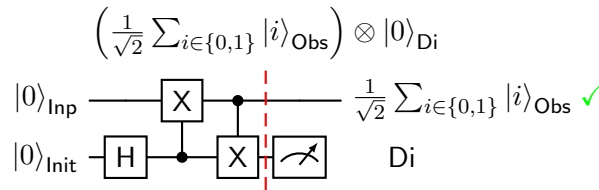
Definition 2.2.55 (Discard). In quantum computing, the concept of *discard* (Di) refers to the act of measuring a qubit and then ignoring the result. This is done when the qubit's state is no longer needed for subsequent computations. Discarding helps simplify the quantum state by removing qubits that have fulfilled their role.

Definition 2.2.56 (Registers). Let $Q = \{0, 1, \dots, n-1\}$ be a set (Notation 2.1.2) of qubit indices composing a quantum register. We define the following partitions: *Input qubits*

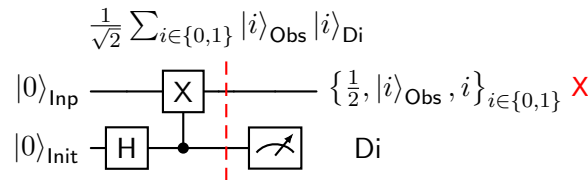
(Inp), *Observed qubits* (Obs), *Ancillas qubits* (Anc), *Initialised qubits* (Init), *Discarded qubits* (Di). Q satisfies: $Q = \text{Init} \cup \text{Inp} = \text{Di} \cup \text{Obs}$, with $\text{Init} \cap \text{Inp} = \emptyset$ and $\text{Di} \cap \text{Obs} = \emptyset$.

Remark: In this document, qubits indexed by Init are typically initialised to $|0\rangle$. Additionally, Init can be considered as a type of Anc. If we revisit Example 2.2.50 of quantum teleportation, we see that in Example 2.2.50, qubits q_1 and q_2 initially serve as ancillas (Init). The process involves measuring qubits q_0 and q_1 at the end. Their measurement outcomes are used to apply necessary corrections to q_2 . Following these corrections, q_0 and q_1 are no longer required, so we can note them as discarded (Di). Initially an ancilla (denoted Anc, or Init), q_2 becomes an output qubit (Obs) and ultimately carries the final information.

Example 2.2.57 (The Necessity of Separation Checks). To facilitate the implementation of multi-qubit operations in a quantum circuit, auxiliary qubits may be introduced (Init). These qubits can be measured during computation but do not form part of the observed qubits, noted Di. The following example shows that it is necessary to ensure that the Di qubit is separate from the observed qubit, noted Obs. To do this, the Di qubit can be reset. With reset, Di qubit does not influence the computation result on qubit Obs:



However, without resetting Di causes the Obs to collapse, affecting the outcome:



2.2.6 Deferred Measurement (DM)

The principle of deferred measurement is utilised in a several approaches, such as QCEC [14], to manage hybrid circuits. This principle asserts that we can always postpone measurements until the end in a hybrid circuit without altering the computation. Informally, we introduce a *deferred measurement function* (DM) that rearranges a quantum circuit by moving all measurement operations to the end and replacing classical controls with quantum controls. DM ensures that all quantum gates are applied before any measurements. For example, Figure 2.2a illustrates a hybrid circuit C with its output state, and Figure 2.2b its deferred measurement version with its output state. Note that C and DM(C) have the same output state. Definition 4.3.69 provides a formal definition of this function.

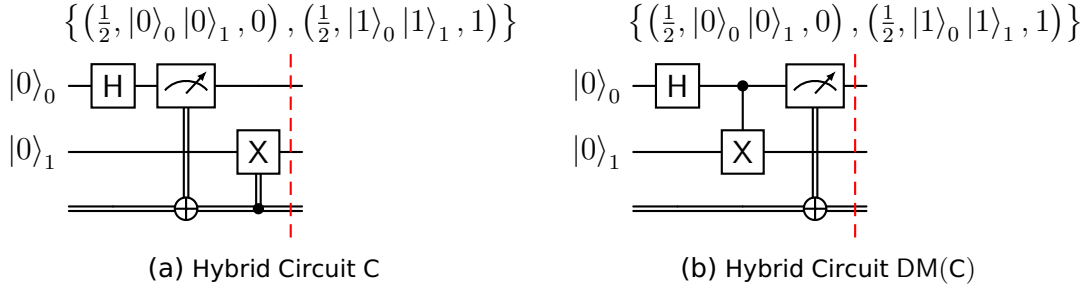


Figure 2.2: Example: Deferred Measurement (DM)

2.3 Path Sums (PS)

As a reminder, our goal is to verify quantum circuit transformations, which requires a compact, easy-to-encode formalism for representing quantum circuits. This section presents the path-sum formalism [2], a framework for representing the unitary evolution of quantum circuits. We start with the mathematical prerequisites in Section 2.3.1. We then present the path-sum semantics in Section 2.3.2. Finally, we discuss the reduction rules used in our work in Section 2.3.3.

2.3.1 Prerequisites

This section introduces two important concepts in path-sum formalism: dyadic numbers (Definition 2.3.1) and Boolean polynomial transformation (Definition 2.3.2). We use Dyadic numbers to represent real numbers using integers, allowing them to be encoded accurately and compactly.

Definition 2.3.1 (Dyadic ($\mathbb{D}$)). A *dyadic number* ($\mathbb{D}$) is a real number of the form $\frac{a}{2^k}$ where $a \in \mathbb{Z}$ and $k \in \mathbb{N}$.

Definition 2.3.2 (Boolean Polynomial Transformation). As presented in [2, Section 2.1], the *Boolean polynomial transformation* for Boolean (Notation 2.1.1) polynomials Q_1 and Q_2 is given by $\overline{Q_1 \oplus Q_2} = \overline{Q_1} + \overline{Q_2} - 2 \cdot \overline{Q_1} \cdot \overline{Q_2}$.

For example, with $Q_1 = Q_2 = 1$ yields equivalent results through two approaches: first, directly obtaining $\overline{1 \oplus 1} = \overline{0} = 0$; second, using the definition's formula $\overline{1 \oplus 1} = \overline{1} + \overline{1} - 2 \cdot \overline{1} \cdot \overline{1} = 1 + 1 - 2 = 0$.

2.3.2 Semantics

This section presents the semantics of path-sums, structured as follows: We first introduce the algebraic form of path-sums in Definition 2.3.3, followed by their concretisation into quantum states in Definition 2.3.4. We then present the composition rule in Definition 2.3.5, the tensor product of path-sums in Definition 2.3.6, and establish the

semantics themselves in Definition 2.3.7. The section concludes with Theorem 2.3.8, demonstrating the equivalence between path-sum and matrix semantics.

Definition 2.3.3 (Path Sum Algebraic Form). A *path-sum* is a triplet composed of:

- An input signature as a bit-string: $x = x_0x_1 \dots x_{n-1} \in \{0, 1\}^n$,
- Path-variables as a bit-string: $y = y_0y_1 \dots y_{m-1} \in \{0, 1\}^m$,
- A phase polynomial $p(x, y)$ with dyadic coefficients, as defined in Definition 2.3.1,
- An output signature is a set of Boolean functions. By abuse of notation, we concatenate them when clear: $f(x, y) = f_0(x, y)f_1(x, y) \dots f_{n-1}(x, y)$.

A path-sum (PS) is defined as: $\langle y, p(x, y), f(x, y) \rangle$.

Definition 2.3.4 (Path Sum Concretisation to Quantum State). To bridge the gap between the algebraic path-sum formalism and its concrete representation as a quantum state, we introduce a *concretisation function* which maps the algebraic path-sums to their corresponding quantum state. Let $\langle y, p(x, y), f(x, y) \rangle$ be a path-sum, its concretisation as a quantum state is defined as follows:

$$\mathcal{V}(\langle y, p(x, y), f(x, y) \rangle) : x \mapsto \frac{1}{\sqrt{2^{|y|}}} \sum_{y \in \{0, 1\}^{|y|}} e^{2\pi i p(x, y)} |f(x, y)\rangle.$$

Definition 2.3.5 (Path Sum Composition). Let U and U' be two unitary circuits such that: $\llbracket U \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle) = \langle y, p(x, y), f(x, y) \rangle$ and $\llbracket U' \rrbracket_{\text{PS}}(\langle \emptyset, 0, x' \rangle) = \langle y', p'(x', y'), f'(x', y') \rangle$. The *path sum composition* $\llbracket U' \rrbracket_{\text{PS}} \circ \llbracket U \rrbracket_{\text{PS}}$ is then defined as:

$$\llbracket U' \rrbracket_{\text{PS}} \circ \llbracket U \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle) = \langle yy', p(x, y) + p'_{[y'_i \leftarrow y_{i+m}, x'_i \leftarrow f_i]}(x, y), f'_{[y'_i \leftarrow y_{i+m}, x'_i \leftarrow f_i]}(x, y) \rangle.$$

where the substitution $y'_i \leftarrow y_{i+m}$ and $x'_i \leftarrow f_i$ indicates variable replacements.

Definition 2.3.6 (Tensor Product of Path Sum). Let U and U' be two unitary circuits such that:

$$\begin{aligned} \llbracket U \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle) &= \langle y, p(x, y), f(x, y) \rangle, \\ \llbracket U' \rrbracket_{\text{PS}}(\langle \emptyset, 0, x' \rangle) &= \langle y', p'(x', y'), f'(x', y') \rangle. \end{aligned}$$

The *path sum tensor product* (Definition 2.2.3) $\llbracket U \rrbracket_{\text{PS}} \otimes \llbracket U' \rrbracket_{\text{PS}}$ is then defined as:

$$\llbracket U \rrbracket_{\text{PS}} \otimes \llbracket U' \rrbracket_{\text{PS}}(\langle \emptyset, 0, xx' \rangle) = \langle yy', p(x, y) + p'(x', y'), f(x, y)f'(x', y') \rangle. \quad (\text{Notation 2.1.3})$$

Definition 2.3.7 (Path Sum Semantics). The *Path Sum* semantics for a universal gate set, specifically Clifford+ $R_z(k)$ (as referenced in [2, Definition 2.9]), are defined as:

$$\begin{aligned} \llbracket I \rrbracket_{\text{PS}}(\langle \emptyset, 0, x_0 \rangle) &= \langle \emptyset, 0, x_0 \rangle & \llbracket H \rrbracket_{\text{PS}}(\langle \emptyset, 0, x_0 \rangle) &= \left\langle y_0, \frac{x_0 \cdot y_0}{2}, y_0 \right\rangle \\ \llbracket R_z(k) \rrbracket_{\text{PS}}(\langle \emptyset, 0, x_0 \rangle) &= \left\langle \emptyset, \frac{x_0}{2^k}, x_0 \right\rangle & \llbracket R_z(k)^\dagger \rrbracket_{\text{PS}}(\langle \emptyset, 0, x_0 \rangle) &= \left\langle \emptyset, -\frac{x_0}{2^k}, x_0 \right\rangle \\ \llbracket CX \rrbracket_{\text{PS}}(\langle \emptyset, 0, x_0 \rangle) &= \langle \emptyset, 0, x_0(x_0 \oplus x_1) \rangle \\ \llbracket U_1 // U_2 \rrbracket_{\text{PS}} &:= \llbracket U_1 \rrbracket_{\text{PS}} \otimes \llbracket U_2 \rrbracket_{\text{PS}}. & \llbracket U_1 - U_2 \rrbracket_{\text{PS}} &:= \llbracket U_2 \rrbracket_{\text{PS}} \circ \llbracket U_1 \rrbracket_{\text{PS}} \end{aligned}$$

Additionally, we included the semantics of I and $//$ to enhance flexibility.

We can revisit Example 2.2.18 using path-sums. The semantics of the Hadamard gate followed by a CNOT gate is given by the composition of their path-sums:

$$\begin{aligned} \llbracket (H/I) - - CX \rrbracket_{\text{PS}}(\langle \emptyset, 0, x_0 x_1 \rangle) &= \llbracket CX \rrbracket_{\text{PS}} \circ (\llbracket H \rrbracket_{\text{PS}} \otimes \llbracket I \rrbracket_{\text{PS}})(\langle \emptyset, 0, x_0 x_1 \rangle) \\ &= \left\langle y_0, \frac{x_0 \cdot y_0}{2}, y_0(x_1 \oplus y_0) \right\rangle \quad (\text{Definition 2.3.7}) \end{aligned}$$

Remark: This semantics is given for a fragment of the syntax of circuits based on bricks (Definition 2.2.38). To use our method for lifting unitary relations to the hybrid case, introduced in Section 1.2, we need to extend the path-sums formalism to quantum multi-control. We achieve this in Section 4.3.3.2.

Theorem 2.3.8 (Equivalence Semantics between Path Sum and Unitary Matrix). Let U be a unitary circuit with $\llbracket U \rrbracket$ being its standard unitary semantics (Definition 2.2.40) and $\llbracket U \rrbracket_{\text{PS}}$ its path sum semantics (Definition 2.3.3). Then the two semantics are functionally equivalent because they produce the same state transformation for all input states. Let $\mathcal{H}^{\otimes n}$ a Hilbert space (Definition 2.2.6) of n qubits, the following equality holds:

$$\forall |\psi\rangle \in \mathcal{H}^{\otimes n}, \llbracket U \rrbracket(|\psi\rangle) = \mathcal{V}(\llbracket U \rrbracket_{\text{PS}})(|\psi\rangle).$$

Proof. The proof has been established in Qbricks [16], specifically detailed in the file available at <https://github.com/Qbricks/qbricks.github.io/blob/main/Qbricks/qbricks.mlw>. $\square$

2.3.3 Reduction Rules

Path-sums are not unique; two different path-sums can represent the same quantum state. For example:

1. Consider the composition of path-sums representing the sequence of two Hadamard gates applied to a basis state: $\llbracket H - - H \rrbracket_{\text{PS}}(\langle \emptyset, 0, |x_0\rangle \rangle)$.
2. This composition results in the path-sum: $\langle y_0 y_1, \frac{x_0 \cdot y_0 + y_0 \cdot y_1}{2}, |y_1\rangle \rangle$.
3. Whose concretisation is: $\frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{2\pi i \frac{x_0 \cdot y_0 + y_0 \cdot y_1}{2}} |y_1\rangle = |x_0\rangle$.

As shown in the previous example, path-sums are not unique (Lemma 2.3.9), which justifies the need for an equational theory. Path-sums are provided with an equational theory that is complete for the Clifford group (composed of H , S , and CNOT). In the following, we focus on a subset of the reduction rules from the path-sums equational theory that are relevant to our subsequent discussion (Definition 2.3.10). Specifically, we omit the *Case* and ω rules. For the complete set of reduction rules, including those not shown here, refer to [2].

Lemme 2.3.9 (Path-Sum Concretisation Non-Injectivity). The concretisation function is not injective, meaning two distinct path-sums can yield the same concretisation. Consequently, different path-sums can represent the same quantum state. Let PS_1 and PS_2 be two path-sums (Definition 2.3.3), $\exists PS_1, PS_2, PS_1 \neq PS_2 \wedge \mathcal{V}(PS_1) = \mathcal{V}(PS_2)$.

Proof. The proof follows directly from the example at the beginning of this section. $\square$

Definition 2.3.10 (Reduction Rules). As defined in [2, Figure 2], let Q and R be two Boolean polynomials. We use only these *reduction rules*:

$$\begin{aligned} \langle y_0 y, p(x, y), f(x, y) \rangle &\rightarrow \langle y, p(x, y), f(x, y) \rangle & (Elim) \\ \left\langle y_0 y, \frac{1}{2} y_0 \cdot (y_i + Q(x, y)) + R(x, y), f(x, y) \right\rangle &\rightarrow \langle y_0 y, R[y_i \leftarrow \overline{Q}](x, y), f[y_i \leftarrow Q](x, y) \rangle \\ & & (HH) \end{aligned}$$

Revisiting the example from Definition 2.3.7, if we compose it with the inverse of the initial circuit, we obtain the identity path-sum, which is the expected result.

$$\begin{aligned} \llbracket (H/I) \text{---} CX \text{---} CX \text{---} (H/I) \rrbracket_{PS}(\langle \emptyset, 0, |x_0 x_1\rangle \rangle) &= \left\langle y_0 y_1, \frac{x_0 y_0 + y_0 y_1}{2}, |y_1 x_1\rangle \right\rangle \\ &\rightarrow \langle y_0 y_1, 0, |x_0 x_1\rangle \rangle & (HH) \\ &\rightarrow \langle \emptyset, 0, |x_0 x_1\rangle \rangle & (Elim) \end{aligned}$$

2.4 Quantum Verification Tools

As a reminder, our goal is to verify the correctness of quantum circuit transformations. So we want to ensure that for a given circuit, or family of circuits, if we transform it, its behaviour will remain the same. There are many tools available for verifying quantum circuits and, by extension, quantum circuit transformations, but it is not easy to compare them because each has unique characteristics. In this chapter, we explore and classify various methods for verification of quantum circuits, focusing on three main categories of tools: (1) simulators, (2) symbolic execution tools, and (3) proof-based tools.

1. *Simulators* verify a given circuit instance for a fixed input. To ensure that a transformation does not alter the circuit's behaviour, it is necessary to verify that the original and the transformed circuits are equivalent. In this context, simulation refers to analysing the circuit's behaviour by representing it as a mathematical object. This differs from testing, which consists of executing the program. Thus, simulation can be regarded as a form of static analysis. Moreover, to prove circuit equivalence using simulation, we must consider all possible inputs. For a family of circuits with a number of qubits ranging from 1 to n , this requires simulating all possible input state combinations for each circuit size. The total number of simulations required follows a geometric series pattern: $\sum_{i=1}^n 2^i = 2^{n+1} - 2$. For

instance, when $n = 20$, this results in: $2^{21} - 2 = 2097150$ simulations. This exponential growth highlights the scalability limitations of simulation-based verification for larger circuits. This limitation likely explains why the simulation tools we have encountered do not include equivalence verification capabilities to ensure that the transformed circuit behaves identically to the original circuit.

2. *Symbolic execution tools* automatically can verify a circuit instance for all possible inputs in a single pass. To ensure that a transformation does not alter the circuit's behaviour, the process is the same as in simulation: verify that the original and transformed circuits are equivalent. At the difference of the simulation, for a family of circuits, it suffices to execute each instance once, requiring only n symbolic execution. This represents a significant reduction compared to the $2^{n+1} - 2$ simulations.
3. *Proof-based tools* are scale-insensitive and can verify all circuit instances for all possible inputs in one go. However, unlike automated symbolic execution and simulation, proof-based verification requires manual intervention and can be time-consuming and complex.

Table 2.1: Performance comparison of quantum circuit verification methods.

Method	Complexity	Automation
Simulation	$2^{n+1} - 2$	✓
Symbolic Execution	n	✓
Proof	Not Applicable	✗

Table 2.1 summarises the comparative analysis of these verification methods, highlighting the differences in terms of complexity and the degree of automation. Figure 2.3 illustrates this comparison for a concrete example of a 3-qubit circuit family. The figure represents how each verification method addresses this specific case, complementing the quantitative analysis presented in the table. To verify the correctness of this circuit, we require either 1 proof, 3 symbolic executions, or 14 simulations.

To go further, we need to refine our notion of a quantum circuit. In Section 2.2.4, we defined the concept of a unitary circuit (U), while in Section 2.2.5, we introduced the concept of a hybrid circuit (C). These are both specific instances of the broader notion of a *quantum circuit* (QC). In this section, we present the overarching concept of a parametrised quantum circuit (Definition 2.4.1) and the notion of a quantum program (Definition 2.4.2), which subsumes all the aforementioned concepts.

Definition 2.4.1 (Parametrised Quantum Circuit (PQC, PUC, PHC)). A *parametrised quantum circuit* (PQC) is defined as a quantum circuit that depends on a parameter, typically its size or number of qubits. From this generic definition, we derive the notions of *parametrised unitary circuit* (PUC) and *parametrised hybrid circuit* (PHC). Fig-

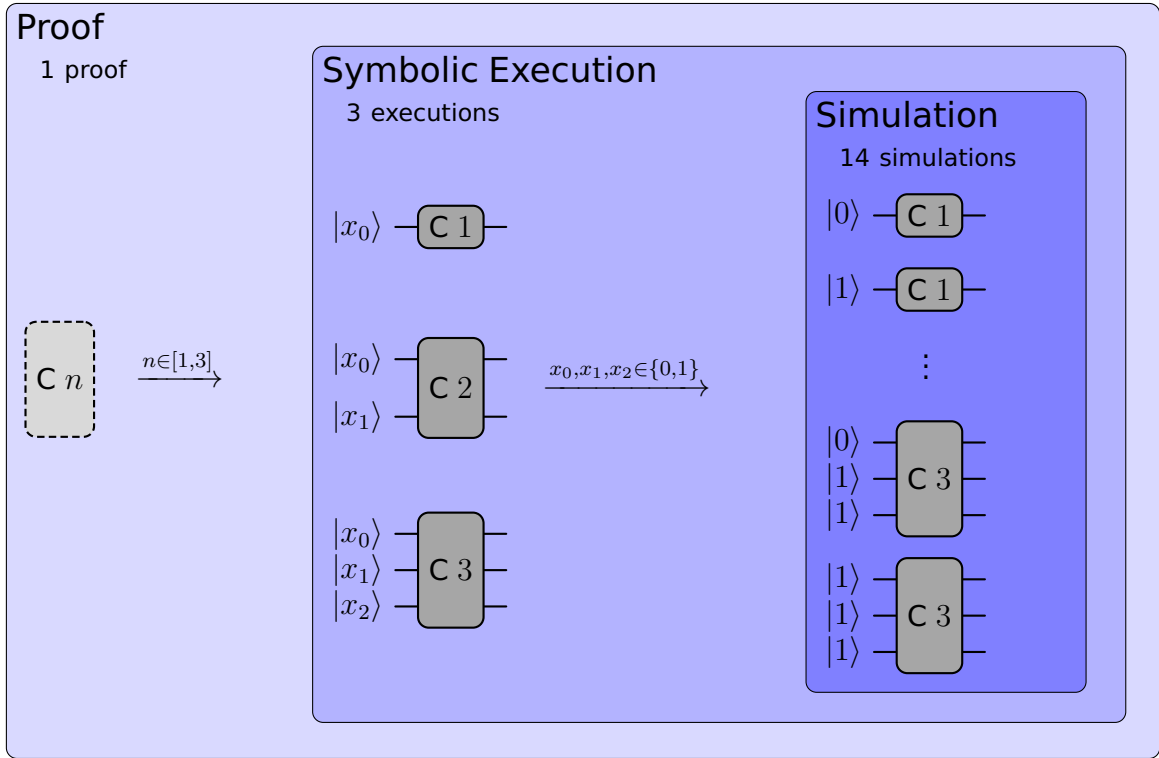


Figure 2.3: Overview of Quantum Circuit Verification.

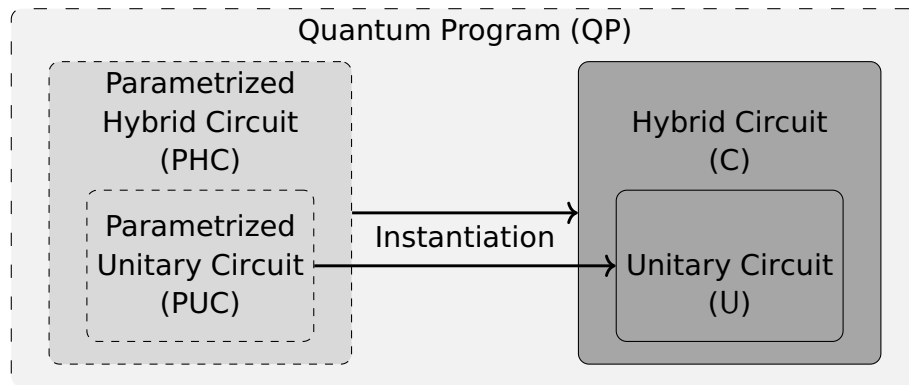


Figure 2.4: Classification Hierarchy of Quantum Programs.

ure 2.4 illustrates the hierarchy of quantum programs. A hybrid circuit C (resp. U) is an instantiation of a parametrised quantum circuit PHC (resp. PUC).

Definition 2.4.2 (Quantum Program (QP)). We define a *quantum program* as a hybrid circuit, parametrised and containing at least one loop structure.

In the following sections, we present several quantum verification tools from the prior work: simulation tools (Section 2.4.1), symbolic execution tools (Section 2.4.2), proof tools (Section 2.4.3), and an tools overview (Section 2.4.4).

2.4.1 Simulation

In this section, we present simulators. For each, we detail the possibility of checking equivalence and separation between qubits. These two aspects are crucial in our work. Equivalence checking allows us to verify if the transformed circuit exhibits the same behaviour as the original. Separation checking allows us to examine only a portion of the register, which is necessary for verifying complex circuit transformations. Additionally, we provide another important characteristic: the kind of circuit it is possible to check (unitary (Section 2.2.4) or hybrid (Definition 2.2.47)).

2.4.1.1 Quasimodo

Quasimodo [72, 63] is a verification tool designed for unitary circuits (U) that employs simulation (Sim) techniques. It utilises Binary Decision Diagrams (BDD) and their variants, such as Weighted BDD (WBDD), where each decision is assigned a weight, and Context-Free-Language Ordered BDD (CFLOBDD), which can provide exponential compression.

To illustrate how Quasimodo handles quantum states, consider the following example: the quantum state $0.5 |0, 0, 0\rangle + 0.5 |0, 1, 0\rangle + 0.5 |0, 1, 1\rangle + 0.5 |1, 1, 0\rangle$. The BDD representation of this state is given by the vector $[0.5, 0, 0.5, 0.5, 0, 0, 0.5, 0]$. Upon measurement, this results in a classical state probability distribution of $\{(0.25, 000), (0.25, 010), (0.25, 011), (0.25, 110)\}$, where each pair consists of a probability and its corresponding classical state. Using this representation, Quasimodo can efficiently simulate quantum circuits and verify properties of their outputs. For example, let p be the output probability; properties such as $p < 0.3$ can be tested. This capability enables debugging of quantum circuits by checking their behaviour against expected outcomes. Furthermore, Quasimodo supports multiple backends such as WBDD and CFLOBDD, providing flexibility in simulating and verifying quantum states. However, it does not offer equivalence checking capabilities. Quasimodo is capable of managing circuits on the order of 4000 qubits, demonstrating its scalability.

2.4.1.2 QuiZX

QuiZX [48, 49, 64] is a quantum circuit simulator designed for unitary circuits (U) that employs simulation (Sim) techniques based on ZX calculus. It enables the optimisation and simulation of circuits, particularly through stabiliser decompositions and the ZX graphical formalism. Designed for performance, QuiZX is implemented in Rust and presents itself as a more efficient version of PyZX (Section 2.4.2.4), although with fewer features such as equivalence checking or visualisation. This tool can efficiently simulate quantum circuits and verify properties of their outputs. However, it does not offer equivalence checking capabilities. QuiZX is capable of managing circuits on the order of 100 qubits, demonstrating its scalability within this range.

2.4.1.3 SliQSim

SliQSim [77, 74] is a quantum circuit simulator designed for unitary circuits (U) that employs simulation (Sim) techniques leveraging Binary Decision Diagrams (BDDs). Its approach is notably original in two aspects. First, it represents complex numbers α in an exact algebraic form: $\alpha = \frac{1}{\sqrt{2^k}}(a\omega^3 + b\omega^2 + c\omega + d)$, which allows avoiding precision loss. Second, it employs a "bit slicing" technique, decomposing state vectors into four integer vectors and a scalar. This technique, combined with the first aspect, enables replacing expensive matrix computations with Boolean calculations. These methods allow SliQSim to simulate unitary circuits with up to 20,000 qubits or more than 3 million gates. However, it does not offer equivalence checking capabilities.

2.4.1.4 Twist

Twist [83, 78] is a verification tool designed for unitary circuits (U) that employs simulation (Sim) techniques and a type system to ensure that certain parts of a quantum register are not entangled with the rest. It can verify that a register, or a portion of a register, contains no entanglement.

Twist introduces a *pure* type to prove that an expression of this type contains no entanglement. To achieve this, Twist provides two complementary verification assertions: `cast` for static analysis of the circuit and `split` for runtime verification. For example, consider the quantum teleportation protocol with deferred measurement (as illustrated in Example 2.2.50). In this context, Twist faces substantial challenges in the static analysis of extended mixed states, as noted in [38]. Specifically, when Twist attempts to verify the separation with `cast` between the output register (Obs) and the discarded register (Di), the tool rejects the program due to the inherent complexity of statically analysing extended mixed states. Consequently, runtime analysis (`split`) is essential for accurately verifying the separation in such scenarios. However, Twist does not offer equivalence checking capabilities, and its scalability is limited, as it handles circuits with up to 10 qubits.

2.4.2 Symbolic Execution

In this section, we present tools using symbolic execution. As with the simulators, we detail the possibility of checking equivalence and separation between qubits. These capabilities are essential for our work. Equivalence checking verifies that the transformed circuit behaves like the original. Separation checking enables us to focus on a specific part of the register, which is essential for complex circuit transformations. We also specify the type of circuit or program each tool can verify: unitary (Section 2.2.4), hybrid (Definition 2.2.47), or quantum program (Definition 2.4.2).

2.4.2.1 AutoQ

AutoQ [21, 1, 7] is a verification tool for unitary circuits (U) that employs Tree Automata (TA) and symbolic execution (SE) techniques. It facilitates both specification verification and equivalence checking. Moreover, AutoQ supports the verification of probabilistic correctness properties. For instance, AutoQ can verify probabilistic properties related to the correctness of the output state in Grover’s algorithm [36]. AutoQ can verify circuits of the order of 200 qubits or more than 2000 gates. AutoQ does not support a separation check.

2.4.2.2 AutoQ-2

The second version of AutoQ [20, 6], AutoQ-2 represents a significant evolution from the first version, transitioning from unitary circuit (Definition 2.2.38) verification to quantum program (Definition 2.4.2) verification. It is a verification tool for quantum programs (QP) that employs symbolic execution (SE) techniques and Tree Automata (TA), specifically introducing Level-Synchronised Tree Automata (LSTA) to support advanced features such as loop constructs and parametrisation. Among the practical applications enabled by these advancements is the verification of complex circuits, such as the Repeat Until Success (RUS) pattern, which repeatedly executes an operation until a success condition is met. AutoQ-2 facilitates both specification verification and equivalence checking, although the latter is limited to unitary circuits (U). It can verify quantum programs with up to a hundred qubits and more than 800 gates. AutoQ-2 does not support separation check.

2.4.2.3 Feynman

Feynman [2, 4] is a specialised verification tool designed to check the equivalence of unitary circuits (U) using path-sum semantics (Definition 2.3.7). It employs symbolic execution (SE) techniques to facilitate this process. In its original implementation, the tool offered extended capabilities, including specification verification. However, these additional functionalities are not currently incorporated in the available version [34]. Feynman can check circuits with around 200 qubits. Feynman does not support quantum multi-control and separation check.

2.4.2.4 PyZX

PyZX [47, 58, 57] is a quantum circuit verification and optimisation tool designed for unitary circuits (U) that employs the ZX-calculus graphical language and symbolic execution (SE) techniques. The ZX-calculus provides a framework for representing and reasoning about quantum circuits. The tool offers three main features: quantum circuit optimisation, interactive visualisation of ZX diagrams, and equivalence verification for unitary circuits. PyZX can manage circuits with several thousand gates. PyZX does not support a separation check.

2.4.2.5 QCEC

QCEC [12, 14, 60, 62] is a verification tool designed for unitary circuits (U) that employs symbolic execution (SE) and simulation (Sim) techniques, utilising both ZX-calculus and Decision Diagrams (DD). It facilitates equivalence checking and can handle circuits without discard operations (Definition 2.2.55). To manage hybrid circuits (C), QCEC utilizes deferred measurement techniques (Section 2.2.6). The tool offers three complementary verification approaches: Decision Diagrams, ZX-calculus, and simulation. These methods can be applied in parallel or individually, allowing users to adapt the approach to their specific verification requirements. QCEC can verify the equivalence of circuits with tens of thousands of gates. QCEC does not support separation check.

2.4.2.6 SliQEC

SliQEC [81, 19, 73] is a verification tool designed for unitary circuits (U) that employs symbolic execution (SE) techniques and Decision Diagrams (DD) to verify partial equivalence by comparing probability distributions on measured qubits (final measurements). The authors aim to demonstrate how partial equivalence can be checked by examining these probability distributions. SliQEC can verify the equivalence or non-equivalence of circuits with a few hundred qubits and several hundred gates. SliQEC does not support separation check.

2.4.2.7 symQV

symQV [9, 76] is a symbolic verification framework designed for hybrid circuits (C) that employs symbolic execution (SE) and simulation (Sim) techniques, utilising Satisfiability Modulo Theories (SMT) solvers. It introduces a model with symbolic qubits, states, and operations, where qubits are encoded as 4-tuples subject to constraints, and common gates are mapped directly at the SMT level. Verification involves encoding the program and specification into an SMT formula. An abstraction technique relaxes qubit constraints for improved scalability. symQV can verify programs with up to 24 symbolic qubits and has been evaluated for algorithms such as teleportation, the QFT, and Grover. However, it does not offer equivalence checking capabilities.

2.4.2.8 VeriQC

VeriQC [42, 80] is a verification tool designed for hybrid circuits (C) that employs symbolic execution (SE) techniques and Tensor Decision Diagrams (TDD) to facilitate equivalence checking. It defines a general equivalence relation [42, Definition 4], which uses a partial trace that discards qubits that are not outputs, allowing the distinction between inputs/outputs and initialisation/garbage wires. However, in their equivalence tests, the authors limit themselves to cases where the garbage and initialisation qubits are empty, as the partial trace is a complex operation.

2.4.3 Proof

In this section, we present proof-based tools. These tools verify that transformations preserve circuit behaviour in one proof for all circuits; however, the process is lengthy and challenging. We specify for each tool which kind of program (Definition 2.4.2) or parametrised circuit (Definition 2.4.1) it is possible to manage.

2.4.3.1 CoqQ

CoqQ [86, 24] is a verification tool developed using Rock [25, 67] for proving quantum programs (QP). It positions itself as a foundational tool using standard formalisms of quantum computing, such as matrices to model the evolution of pure states (Section 2.2.1.5) and density operators [52, Section 2.4, page 98] to model the evolution of extended mixed states (Section 2.2.1.5). To facilitate its use, it allows writing specifications in Dirac notation [52, Section 1.2, page 13].

2.4.3.2 SQIR

SQIR [40, 55, 75] is a verification tool for proving quantum programs (QP) using the prover Rocq [25, 67]. It employs proof-based verification (P) techniques and utilises density operators (DOs) along with matrices to manage hybrid programs. It allows certifying quantum programs such as Shor’s algorithm [71]. Additionally, SQIR can generate certified circuits by extracting OCaml code from Rocq. For instance, in [55], a certified circuit representing an instance of Shor’s algorithm [71] in OpenQASM with 29 qubits and 11k gates is extracted, followed by a reorganisation pass.

2.4.3.3 Qbricks

Qbricks [16, 59], detailed in Section 3.1, is a semi-automatic proof tool for parametrised unitary circuits (PUC) that utilizes the verification platform Why3 [35, 82]. It employs path-sum semantics and, through Why3, allows the use of SMT solvers to automatically discharge part of the proofs.

2.4.4 Tools Overview

Table 2.2 sets and categorises these verification tools based on several criteria, including the types of circuits they can handle (unitary (U) and hybrid (C), parametrised unitary (PUC), parametrised hybrid (PHC), and quantum programs (QP)), the verification techniques they employ (simulation (Sim), symbolic execution (SE), and proof (P)), the underlying formalism, and their capability to perform equivalence checking.

Table 2.2: Verification Tools (Status as of November 30, 2024)

Tool	Circuit	Technique	Formalism	Eq.	Qubit/Gates
AutoQ [21, 1, 7]	U	SE	TA	✓	$10^2/10^3$
AutoQ-2 [20, 6]	QP	SE	LSTA	✓ (U)	$10^2/10^2$
CoqQ [86, 24]	QP	P	M,DO	NA	NA
Feynman [2, 4, 34]	U	SE	PS	✓	$10^2/\text{NI}$
PyZX [47, 58]	U	SE	ZX	✓	$10^3/\text{NI}$
Qbricks [16, 59]	PUC	P	PS	NA	NA
QCEC [12, 14]	C	SE, Sim *	DD, ZX	✓	$\text{NI}/10^3$
Quasimodo [72]	U	Sim	BDD, CFLOBDD	×	$10^3/\text{NI}$
QuiZX [49, 64]	U	Sim	ZX	×	$10^2/\text{NI}$
SliQEC [18, 73]	U	SE	DD	✓	$10^2/10^2$
SliQSim [77, 74]	U	Sim	BDD	×	$10^4/10^6$
SQIR [40, 55, 75]	QP	P	M,DO	NA	NA
symQV [9, 76]	C	Sim, SE	SMT	×	$10/\text{NI}$
Twist [83, 78]	U	Sim	TS	×	$10/\text{NI}$
VeriQC [42, 80]	C	SE	TDD	✓	$10^2/10^2$

U: Unitary Circuit (Definition 2.2.38), C: Hybrid Circuit (Definition 2.2.47),

PUC: Parametrized Unitary Circuit (Definition 2.4.1), QP: Quantum Program (Definition 2.4.2),

P: Proof, SE: Symbolic Execution, Sim: Simulation,

Sim *: QCEC employs simulation among other techniques for equivalence checking, TA: Tree

Automata, DD: Decision Diagram, BDD: Binary Decision Diagram,

CFLOBDD: Context-Free-Language Ordered Binary Decision Diagram, M: Matrix,

DO: Density Operator, PS: Path Sum, TS: Type System, Cat: Categories, ZX: ZX Calculus,

LSTA: Level-Synchronised Tree Automata, Eq: Equivalence Checking, NI: No Information,

Qubits/Gates: This column indicates the order of magnitude for the circuits managed in the provided benchmarks.

2.5 Quantum Circuit Transformations

In this thesis, we seek to verify that quantum circuit transformations preserve circuit behaviour. To this end, we need to define what a quantum circuit transformation is in our context. This chapter is devoted to this definition and to the classification of these transformations. To provide an overview, we classify circuit transformations into

five distinct types: Firstly, *Compilation* involves translating from a high-level to a low-level language (Section 2.5.1). Next, *Transpilation* focuses on adapting or optimising a circuit for a specific backend (Section 2.5.2). Another type is *Computational Model Shift*, which enables the practical implementation of a circuit on a quantum computer (Section 2.5.3). Additionally, *Teleportation Insertion* replaces each output wire of the unitary input circuit with a one-qubit teleportation (Section 2.5.4). Finally, *Hybridisation* involves the insertion of intermediate measurements to facilitate the use of current quantum computers with limited capabilities.

2.5.1 Compilation

We define here what we mean by *compilation* in the context of quantum circuits. Our notion of compilation translates a circuit from a high-level language to a low-level one. The resulting circuit remains independent of any specific physical backend. This transformation preserves the computational behaviour of the circuit and maintains its type: unitary circuits remain unitary (Section 2.2.4) and hybrid circuits remain hybrid (Definition 2.2.47). Compilation does not introduce discards (Definition 2.2.55). It does not aim to adapt to a particular backend but targets a lower-level language. Chapter 3 illustrates this definition with a concrete compilation pipeline example. Figure 2.5 illustrates a part of this pipeline, the decomposition of a X gate applied to q_2 and controlled twice by the qubits q_0 and q_1 (Toffoli, Figure 2.5a). This decomposition uses unary gates, specifically Hadamard gates and rotations along the Z axis ($R_z(k)$ and $R_{zp}(k)$), as well as CNOT gates², the result is given in Figure 2.5b.

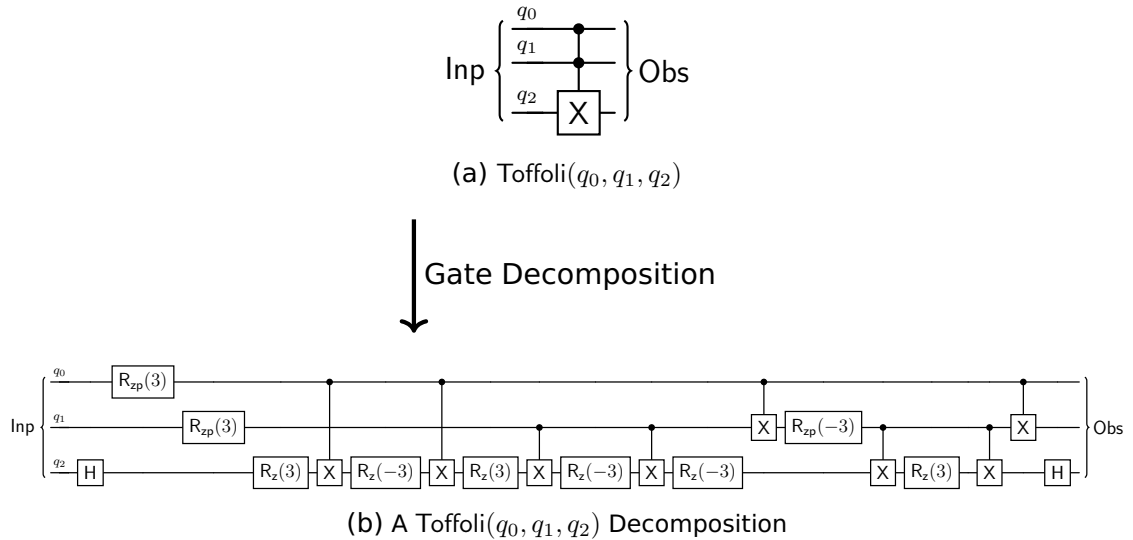


Figure 2.5: Example of Gate Decomposition

²For more details on these gates, see Definition 3.1.1.

2.5.2 Transpilation

Unlike our notion of compilation, *transpilation* adapts a circuit to the physical machine, whether a simulator or a quantum computer. Transpilation involves several circuit modifications, including instruction reordering, simplification, gate decomposition, and qubit mapping. In this thesis, we use the `generate_preset_pass_manager`³ from Qiskit. In the following, any reference to the `pass_manager` refers to this setting:

```
gates = [
    "h", "x", "z", "s", "t", "rz", "u1",
    "sdg", "tdg", "sx", "cx", "cz",
    "cu1", "crz", "ccx" ]

pass_manager = generate_preset_pass_manager(
    optimization_level=3,
    basis_gates=gates )
```

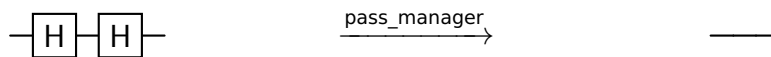


Figure 2.6: Example of Qiskit `pass_manager` Simplification.

Figure 2.6 illustrates circuit simplifications, such as recognising that the Hadamard gate is involutive, making the temporal composition of two Hadamard gates equivalent to identity or a wire. Figure 2.7 shows gate decompositions, such as decomposing a CNOT

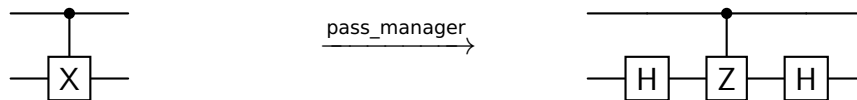


Figure 2.7: Example of Qiskit `pass_manager` Gates Decomposition.

gate into a temporal composition of Hadamard gates and a control Z gate. Figure 2.8 demonstrates gate reordering performed by the `pass_manager`. Finally, Figure 2.9 presents a complete transpilation process, combining reordering, decomposition, and simplification.

2.5.3 Computational Model Shift

A *computational model shift* modifies the computational model, for example, from the standard unitary-based model to a measurement-based model. A well-known computational model shift is *Measurement-Based Quantum Computing* (MBQC). This model

³IBM, Qiskit, `pass_manager`: https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.transpiler.generate_preset_pass_manager

<pre> OPENQASM 2.0; include "qelib1.inc"; qreg q[3]; x q[2]; x q[1]; x q[0]; h q[1]; z q[0]; t q[2]; </pre>	$\xrightarrow{\text{pass_manager}}$	<pre> OPENQASM 2.0; include "qelib1.inc"; qreg q[3]; x q[0]; z q[0]; x q[1]; h q[1]; x q[2]; t q[2]; </pre>
---	--------------------------------------	---

Figure 2.8: Example of Qiskit pass_manager Instruction Reordering.

<pre> OPENQASM 2.0; include "qelib1.inc"; qreg q[3]; x q[2]; x q[1]; x q[0]; h q[1]; z q[0]; t q[2]; cx q[0], q[1]; </pre>	$\xrightarrow{\text{pass_manager}}$	<pre> OPENQASM 2.0; include "qelib1.inc"; qreg q[3]; x q[0]; z q[0]; x q[1]; cz q[0], q[1]; h q[1]; x q[2]; t q[2]; </pre>
--	--------------------------------------	--

Figure 2.9: Example of Qiskit pass_manager Complete Transpilation.

is considered to help to overcome the difficulty of maintaining qubit coherence over time. In this document, we employ the *One-Way Measurement* [29] (OWM), a method for performing MBQC.

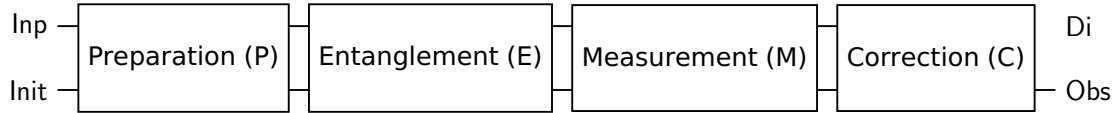


Figure 2.10: One Way Measurement Overview

The principle of OWM involves four phases: *preparation* (initialization of computation qubits (Init) to the $|+\rangle$ state), *entanglement* (creation of entanglement among all qubits), *measurement* in specific bases, and *corrections* (application of corrections based on measurement outcomes on the output qubits (Obs)). Figure 2.10 illustrates this process. Each gate is decomposed following this pattern. The resulting sequence of operations is then reordered so that the entire circuit conforms to the preparation-entanglement-measurement-correction pattern.

The resulting hybrid circuit is equivalent to the original unitary circuit, up to a global phase⁴.

In this work, we use our implementation of OWM⁵: O. This section describes this

⁴We prove this assertion in Section 5.1.1.

⁵Our primary objective is to verify transformations involving discard and demonstrate that our approach is practical for this type of hybrid circuit. The reordering is not essential for this

implementation. OWM involves measuring on different bases. However, in our work, we only consider measurements in the standard basis (Definition 2.2.9):

$$\{|00 \dots 00\rangle, |00 \dots 01\rangle, |00 \dots 10\rangle, \dots, |11 \dots 1\rangle\}.$$

Then, to simulate a measurement in a different basis, we apply a unitary transformation before measuring in the standard basis. For instance, to measure in the Hadamard basis, we apply the Hadamard gate before the measurement.

As primitives, we have chosen a universal gate set compatible with our equivalence relations (Definition 4.3.2): $\{X, H, CX, U_1(\theta)\}$. The function O is presented in Figure 2.11. For each element of our gate set, we provide the hybrid circuit corresponding to the OWM transformation of the gate and its description in our hybrid syntax (Definition 2.2.47). The function O implements OWM. The temporal composition of gates transformed by O requires mapping the observed qubits of the first (Obs_1) to the inputs of the second (Inp_2). Assume that U is a unitary circuit. The O transformation returns two sets of qubit indices (Inp for inputs and Obs for outputs) and a hybrid circuit C :

$$(Inp, Obs, C) = O(U).$$

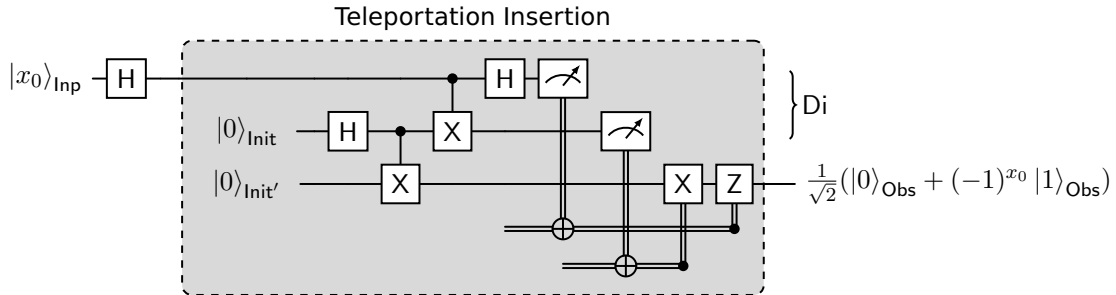
Although the primitives are presented for fixed qubit indices, the implementation generalises to arbitrary indices without loss of generality.

2.5.4 Teleportation Insertion

This work utilises the quantum teleportation protocol presented in Example 2.2.50 as a circuit transformation. We define our *Teleportation Insertion*, denoted by *Tele*, as the application of a one-qubit teleportation (Example 2.2.50) to each output wire of the original circuit. The function T implements this transformation. For a unitary circuit U :

$$(Inp, Obs, C) = T(U),$$

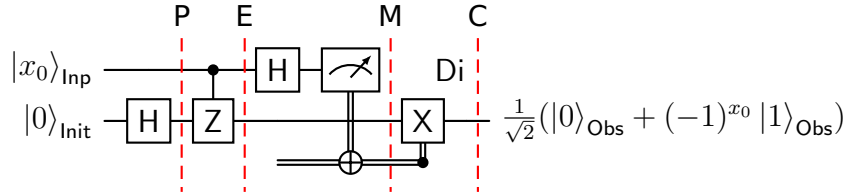
where Inp and Obs are two sets of qubit indices corresponding to the input and observed qubits of the circuit C , respectively. We prove the correctness of our implementation in Section 5.1.2. The following figure presents a simple example of use for a Hadamard gate. This example illustrates the application of teleportation insertion on a single qubit, resulting in the desired output state.



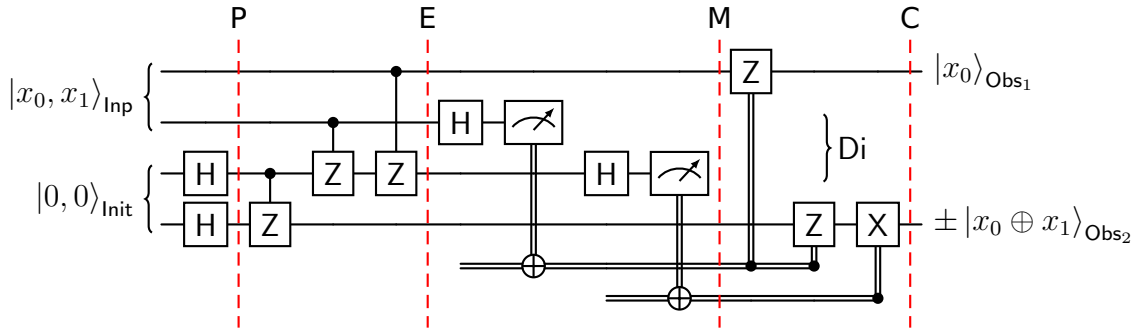
purpose. Thus, we have not implemented the reordering step.

$$|x_0\rangle_{\text{Inp}} \xrightarrow{\text{X}} |1 \oplus x_0\rangle_{\text{Obs}}$$

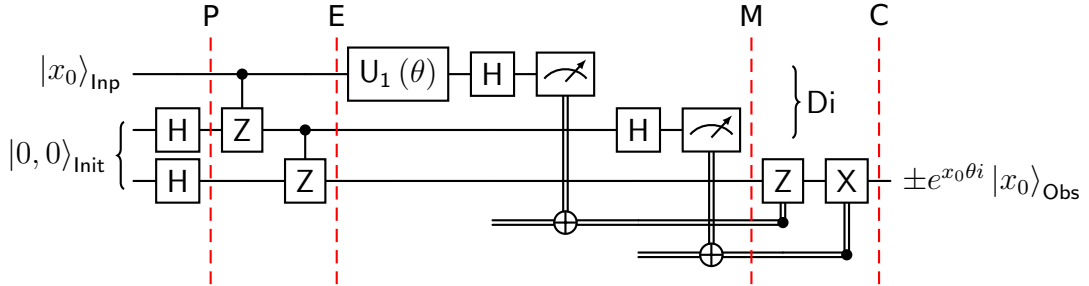
(a) $O(\text{Apply}(\text{X}, \emptyset, \{0\}))$



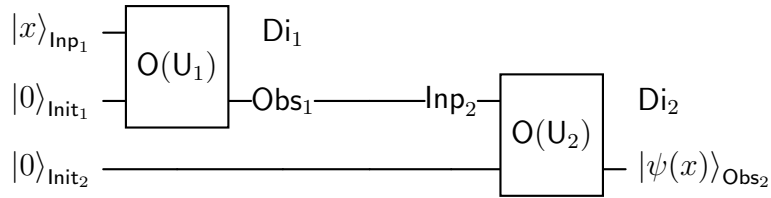
(b) $O(\text{Apply}(\text{H}, \emptyset, \{0\}))$



(c) $O(\text{Apply}(\text{X}, \{0\}, \{1\}))$



(d) $O(\text{Apply}(U_1(\theta), \emptyset, \{0\}))$



(e) $O(U_1 - U_2)$

Figure 2.11: Primitives of our One-Way Measurement implementation, O .

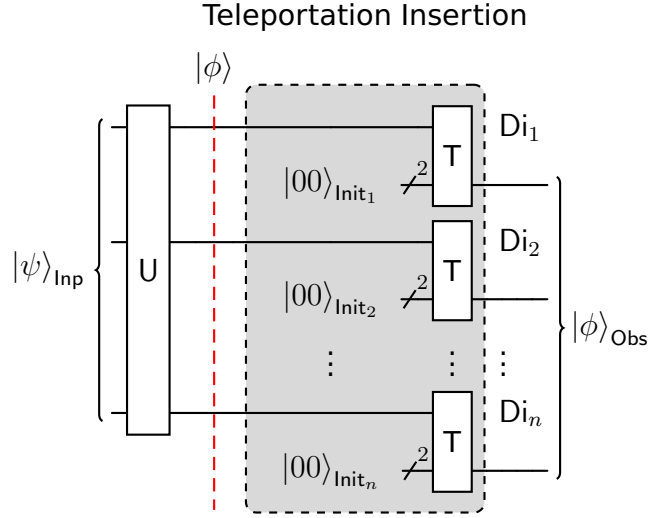
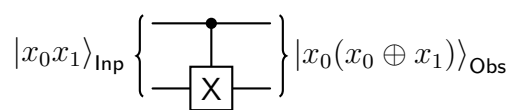


Figure 2.12: Teleportation Insertion applied to any Unitary Circuit, $T(U)$.

Figure 2.12 illustrates the teleportation insertion for an n -qubit circuit, achieved through the spatial composition of individual teleportations applied to each output qubit of the unitary U .

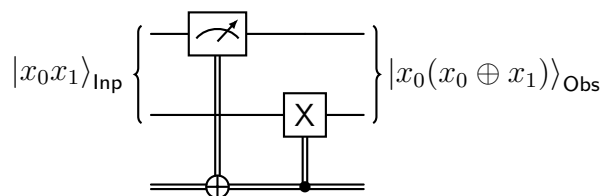
2.5.5 Hybridisation

Hybridisation of unitary circuit [15, 26] is one of the considered means for performing quantum computation on quantum computers. The idea is to reduce the necessary coherence time of qubits by adding intermediate measurements. In practice, this amounts to the inverse operation of deferred measurement (Section 2.2.6). It replaces quantum controls with an intermediate measurement and classical control. Hybridisation is a transformation that does not preserve the type of circuit, as a unitary circuit becomes hybrid, but does not introduce discards. Figure 2.13 illustrates the hybridisation of the CNOT gate. Figure 2.14 shows a hybridisation of an instance of the Quantum Fourier Transform (QFT) algorithm from [17]. Measurements and quantum controls have replaced all quantum controls.



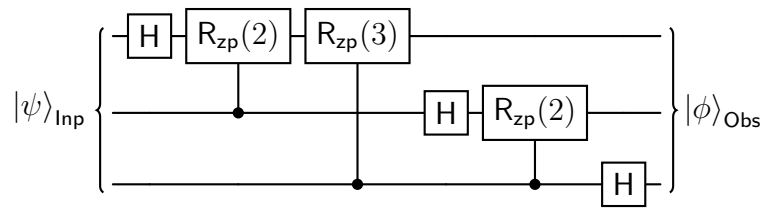
(a) CNOT.

↓
Hybridisation



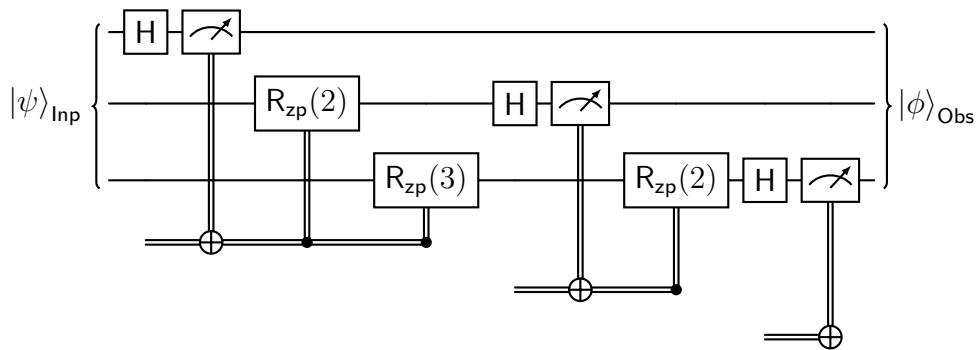
(b) Hybrid CNOT.

Figure 2.13: Hybridisation of CNOT.



(a) Standard 3-qubits QFT

↓
Hybridisation



(b) Hybrid 3-qubits QFT

Figure 2.14: Hybridisation of QFT [17].

Chapter 3

Compilation Proof Example

We begin by recalling the context of this work. In quantum computing with circuits, the circuit is often transformed within a quantum computing pipeline. These transformations, including compilation, can alter the circuit’s behaviour. Our goal is to prove that compiling any instance of a Qbricks parametrised unitary circuit (PUC, Definition 2.4.1) into an OpenQASM unitary circuit does not alter its behaviour. We achieve this by leveraging Qbricks, whose primary purpose is to prove the functional correctness of a PUC, to establish the functional correctness of this compilation pass. We present the certified compilation pass from Qbricks to OpenQASM. Section 3.1 introduces the syntax and semantics of Qbricks and useful lemmas for our proof. The compilation pass is detailed in Section 3.2. Finally, in Section 3.3, we explain how we practically implement and certify our transformation of circuits from Qbricks to OpenQASM. We also provide examples of using this compilation pass in a quantum computing pipeline.

3.1 Qbricks

In this section, we present Qbricks introduced in Section 2.4.3.3 in more detail. Qbricks is a tool designed to verify the functional correctness of parametrised unitary circuits (PUC, Definition 2.4.1). It leverages the verification platform Why3 [35] and a parametrised version of Path-Sums (HOPS) semantics. However, in our study, we do not use HOPS but instead use standard Path-Sums (PS) extended with additional operations that allow for QASM-based operations. The Why3 platform automates part of the proofs, which SMT solvers then handle.

We begin by giving the syntax of Qbricks in Section 3.1.1 and its semantics in Section 3.1.2. We conclude with the lemmas useful for proving the correctness of our compilation pass from Qbricks to OpenQASM (Section 3.1.3). This section is an extract and reformulation of the content from the paper [16], whose code is available on GitHub [82].

3.1.1 Algebraic Data Type

In this section, we reformulate the algebraic data type for Qbricks. We rename the algebraic data type `wired_circuit` from the `wired_circuit` file to `Qb` and reorganise it to simplify the presentation. To simplify encoding and verification, we use integers rather than real numbers to represent angles.

Definition 3.1.1 (Qbricks Unitary Circuit (Qb)). A *Qbricks unitary circuit* (Qb) is a structured representation of a QASM-based unitary circuit (Definition 2.2.41), reformulated and renamed from the original `wired_circuit` algebraic data type. This representation relates the combination of Bricks, parallel and sequential compositions to our brick-based representation (Definition 2.2.38), identifies I with Skip and GP with Phase, adds Toffoli, Fredkin, and swap bricks, and uses integers (k) for angle definitions to simplify encoding and verification. We decompose Qb into Unary, Binary, and Ternary to facilitate the proof. Let co , co' , ta , ta' , and n be five qubit indices, with co and co' as control qubits, ta and ta' as target qubits, and n as the size of the quantum register. The syntax of qbricks is defined as follows:

$$\begin{aligned} \text{Unary} &:= \text{Skip} \mid \text{Phase}(k) \mid \text{R}_x(k) \mid \text{R}_y(k) \mid \text{R}_z(k) \mid \text{R}_{zp}(k) \mid \text{H} \mid \text{S} \mid \text{T} \mid \text{X} \mid \text{Y} \mid \text{Z} \\ \text{Binary} &:= \text{Bricks_Cnot} \mid \text{Bricks_Swap} \mid \text{Cnot}(co, ta, n) \mid \text{Swap}(ta, ta', n) \\ \text{Ternary} &:= \text{Bricks_Toffoli} \mid \text{Bricks_Fredkin} \mid \text{Toffoli}(co, co', ta, n) \mid \text{Fredkin}(co, ta, ta', n) \\ \text{Gate} &:= \text{Unary} \mid \text{Binary} \mid \text{Ternary} \\ \text{Qb} &:= \text{Gate} \mid \text{Place}(\text{Qb}, ta, n) \mid \text{Cont}(\text{Qb}, co, ta, n) \mid \text{Sequence}(\text{Qb}, \text{Qb}) \mid \text{Parallel}(\text{Qb}, \text{Qb}) \end{aligned}$$

A circuit Qb is considered *well-formed* if it satisfies the following conditions, for:

1. $\text{Cnot}(i, j, n)$ and $\text{Swap}(i, j, n)$, then $(i \neq j)$ and $(0 \leq i, j < n)$.
2. $\text{Toffoli}(i, j, k, n)$ and $\text{Swap}(i, j, k, n)$, then $i \neq j$, $i \neq k$, $k \neq j$, and $0 \leq i, j, k < n$.
3. $\text{Place}(U, ta, n)$, where n' is the number of qubits in U , then $0 \leq ta$, $0 \leq ta + n' < n$.
4. $\text{Cont}(U, co, ta, n)$, where n' is the number of qubits in U , then $co \neq ta$, $0 \leq co$, $0 \leq ta \leq n - n'$, and either $(co < ta)$ or $(ta + n' \leq co)$.

Notation 3.1.2 (Infixed Compositions Notation). We denote the temporal composition equivalently by $(--)$ and (Sequence) , and the spatial composition by $(//)$ and (Parallel) .

3.1.2 Semantics

This section introduces the semantics of Qbricks. Qbricks uses path-sum temporal (Definition 2.3.5) and spatial (Definition 2.3.6) composition to construct circuits. This brick-based approach simplifies proofs and has been used to prove the correctness of the semantics of Qbricks with respect to the brick-based semantics (Definition 2.2.40). Figure 3.1 provides an example of a quantum circuit constructed using this model. In this

example, Figure 3.1a demonstrates the parallel composition of a Z with a sequence that includes a H followed by a Bricks_Cnot. This is clearly illustrated in Figure 3.1b. An important aspect of multi-qubit in the Qbricks brick-based approach is that control wires always have lower indices than their target wires, as shown in Figure 3.1b, where the control wire of the Bricks_Cnot gate is positioned above the target wire. Although

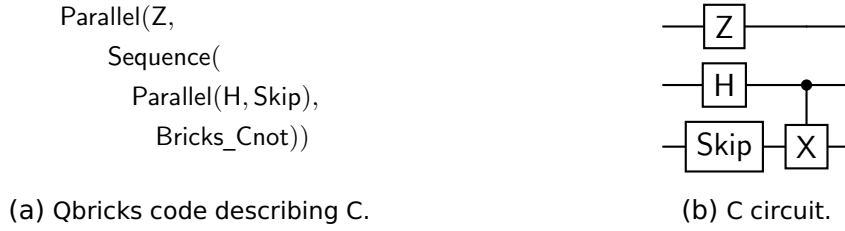


Figure 3.1: Example of a Qbricks Unitary Circuit using only Bricks and Composition.

the approach has evolved into a more structured model, some elements of the initial method remain. Unary gates, such as rotations, global phase, and the Hadamard gate, are examples of bricks. These, along with temporal composition, are fully used in the current version. Higher-level structures have been introduced to enhance usability, allowing for more intuitive and efficient manipulation of quantum circuits. Figures 3.2 illustrate high-level Figure 3.2 illustrates high-level instructions. Figure 3.2a shows QASM instructions over gates. Figures 3.2b and 3.2c illustrate the Place and Cont high-level operations for placing and controlling a sub-circuit (U), respectively.

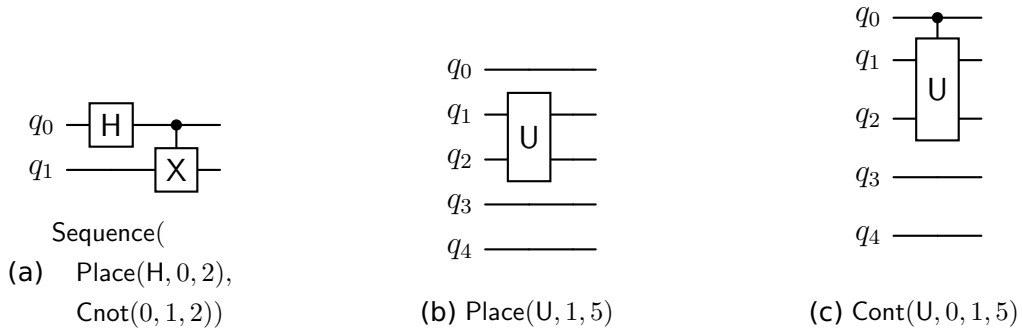


Figure 3.2: Examples of High Level Instructions

We begin by presenting the Qbricks Brick-Based Semantics (Definition 3.1.3). Next, we define the semantics for multi-qubit gates (Definition 3.1.4), followed by the semantics of the higher-order operations Place and Cont (Definition 3.1.6 and Definition 3.1.7, respectively).

Definition 3.1.3 (Qbricks Brick-Based Semantics). We reformulate the semantics interpretations of Qbricks brick-based semantics. Let $\langle \emptyset, 0, |x_0\rangle \rangle$ be a symbolic path-sum

(Definition 2.3.3) representing a basis state. This reformulation is presented below:

$$\begin{aligned}
\llbracket R_x(k) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0\rangle \rangle) &= \left\langle \emptyset, \frac{x_0 \cdot y_0}{2} + \frac{y_0 \cdot y_1}{2} + \frac{1}{2^{k+1}}(-1)^{y_0-1}, |y_0\rangle \right\rangle \\
\llbracket R_y(k) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0\rangle \rangle) &= \left\langle \emptyset, \frac{x_0 \cdot y_0}{2} + \frac{y_0 \cdot y_1}{2} + \frac{1}{2^{k+1}}(-1)^{y_0-1} + \frac{y_1 - x_0}{4}, |y_0\rangle \right\rangle \\
\llbracket R_z(k) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0\rangle \rangle) &= \left\langle \emptyset, \frac{(-1)^{x_0-1}}{2^{k+1}}, |x_0\rangle \right\rangle \\
\llbracket R_{zp}(k) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0\rangle \rangle) &= \left\langle \emptyset, \frac{x_0}{2^k}, |x_0\rangle \right\rangle \\
\llbracket S \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0\rangle \rangle) &= \left\langle \emptyset, \frac{x_0}{4}, |x_0\rangle \right\rangle \\
\llbracket T \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0\rangle \rangle) &= \left\langle \emptyset, \frac{x_0}{8}, |x_0\rangle \right\rangle \\
\llbracket X \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0\rangle \rangle) &= \langle \emptyset, 0, |1 \oplus x_0\rangle \rangle \\
\llbracket Y \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0\rangle \rangle) &= \left\langle \emptyset, \frac{(-1)^{x_0}}{2}, |y_0\rangle \right\rangle \\
\llbracket Z \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0\rangle \rangle) &= \left\langle \emptyset, \frac{x_0}{2}, |x_0\rangle \right\rangle \\
\llbracket \text{Skip} \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0\rangle \rangle) &= \langle \emptyset, 0, |x_0\rangle \rangle \\
\llbracket \text{Phase}(k) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0\rangle \rangle) &= \left\langle \emptyset, \frac{1}{2^k}, |x_0\rangle \right\rangle \\
\llbracket H \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0\rangle \rangle) &= \left\langle \emptyset, \frac{x_0 \cdot y_0}{2}, |y_0\rangle \right\rangle \\
\llbracket \text{Bricks_Cnot} \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0 x_1\rangle \rangle) &= \langle \emptyset, 0, |x_0(x_0 \oplus x_1)\rangle \rangle \\
\llbracket \text{Bricks_Toffoli} \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0 x_1 x_2\rangle \rangle) &= \langle \emptyset, 0, |x_0 x_1(x_0 \cdot x_1 \oplus x_2)\rangle \rangle \\
\llbracket \text{Bricks_Swap} \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0 x_1\rangle \rangle) &= \langle \emptyset, 0, |x_1 x_0\rangle \rangle \\
\llbracket \text{Bricks_Fredkin} \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0 x_1 x_2\rangle \rangle) &= \langle \emptyset, 0, |x_0(x_0 \cdot x_2 \oplus \neg x_0 \cdot x_1)(x_0 \cdot x_1 \oplus \neg x_0 \cdot x_2)\rangle \rangle \\
\llbracket \text{Sequence}(U, U') \rrbracket_{\text{qb}} &:= \llbracket U' \rrbracket_{\text{qb}} \circ \llbracket U \rrbracket_{\text{qb}} \\
\llbracket \text{Parallel}(U, U') \rrbracket_{\text{qb}} &:= \llbracket U \rrbracket_{\text{qb}} \otimes \llbracket U' \rrbracket_{\text{qb}}
\end{aligned}$$

We give this definition on a basis state without loss of generality, thanks to unitarity.

The number of qubits n is essential for verifying well-formedness constraints, as outlined in Definition 3.1.1, ensuring that operations remain within the circuit boundaries. To streamline our presentation, we omit the explicit notation of n when it is not directly relevant to the semantics. This simplification is justified as these well-formed condition checks are generally managed automatically by the proof infrastructure of Qbricks, utilising Why3 and SMT solvers.

Definition 3.1.4 (Qbricks Multi-Qubits Gates Semantics [59]). We reformulate the semantics of the Qbricks multi-qubit gates based on the original `wired_circuit`. Let $\langle \emptyset, 0, x \rangle$ be a symbolic path-sum, as defined in Definition 2.3.3, representing a basis

state $|x\rangle = |x_0, x_1, \dots, x_{n-1}\rangle$, where each $x_i \in \{0, 1\}$ for $i \in [0, n-1]$. We define the semantics of these gates as follows:

$$\begin{aligned}
& \llbracket \text{Cnot}(co, ta) \rrbracket_{\text{Qb}} (\langle \emptyset, 0, x \rangle) \\
&= \langle \emptyset, 0, x_0 \dots x_{ta-1} (x_{co} \oplus x_{ta}) x_{ta+1} \dots x_{n-1} \rangle \\
& \llbracket \text{Toffoli}(co_1, co_2, ta) \rrbracket_{\text{Qb}} (\langle \emptyset, 0, x \rangle) \\
&= \langle \emptyset, 0, x_0 \dots x_{ta-1} (x_{co1} \cdot x_{co2} \oplus x_{ta}) x_{ta+1} \dots x_{n-1} \rangle \\
& \llbracket \text{Swap}(ta_1, ta_2) \rrbracket_{\text{Qb}} (\langle \emptyset, 0, x_0 \dots x_{ta_1} \dots x_{ta_2} \dots x_{n-1} \rangle) \\
&= \langle \emptyset, 0, x_0 \dots x_{ta_2} \dots x_{ta_1}, \dots x_{n-1} \rangle \\
& \llbracket \text{Fredkin}(co, ta_1, ta_2) \rrbracket_{\text{Qb}} (\langle \emptyset, 0, x_0 \dots x_{ta_1} \dots x_{ta_2} \dots x_{n-1} \rangle) \\
&= \langle \emptyset, 0, |x_0 \dots x_{ta_1-1} (x_{co} \cdot x_{ta_2} \oplus \neg x_{co} \cdot x_{ta_1}) x_{ta_1+1} \\
&\quad \dots x_{ta_2-1} (x_{co} \cdot x_{ta_1} \oplus \neg x_{co} \cdot x_{ta_2}) x_{ta_2+1} \dots x_{n-1} \rangle
\end{aligned}$$

The definition uses a basis state, leveraging the unitarity of quantum gates.

Notation 3.1.5 (Qbricks Gate Application). We define the gate application to a qubit index (i). Let G be a quantum gate of type Gate (Definition 3.1.1), i be a qubit index, and n the number of qubits such that $i \in \{0, 1, \dots, n-1\}$. with the identity operation applied to all other qubits in the register, denoted by $\text{Bricks}_{\text{qb}}$:

$$G_i := \text{Swap}(i, 0) \text{---} \underbrace{(G // \text{Skip} // \dots // \text{Skip})}_{n-1} \text{---} \text{Swap}(0, i)$$

Definition 3.1.6 (Sub-Circuit Placement (Place) Semantics [59]). We define the semantics of sub-circuit placement ($\llbracket \text{Place} \rrbracket_{\text{Qb}}$) for a circuit of type Qb . We give this definition using syntactic notation to simplify the presentation. Let U be a well-formed unitary circuit of type Qb , and $\langle \emptyset, 0, x \rangle$ be a symbolic path-sum representing a basis state $|x\rangle = |x_0, x_1, \dots, x_{n-1}\rangle$, where each $x_i \in \{0, 1\}$ for $i \in [0, n-1]$ (Definition 2.3.3). The semantics of Gate is given in Definition 3.1.3. This semantics is:

$$\llbracket \text{Place}(U, ta, n) \rrbracket_{\text{Qb}} := \begin{cases} \llbracket U_{ta} \rrbracket_{\text{Qb}} & \text{if } U \in \text{Gate} \\ \llbracket \text{Place}(U', ta + ta', n) \rrbracket_{\text{Qb}} & \text{if } U = \text{Place}(U', ta', n') \\ & \text{and } ta + n' < n \\ \llbracket \text{Cont}(U, co' + ta, ta' + ta, n) \rrbracket_{\text{Qb}} & \text{if } U = \text{Cont}(U', co', ta', n') \\ & \text{and } ta + n' < n \\ \llbracket \text{Sequence}(\text{Place}(U_1, ta, n), \text{Place}(U_2, ta, n)) \rrbracket_{\text{Qb}} & \text{if } U = \text{Sequence}(U_1, U_2) \\ \llbracket \text{Sequence}(\text{Place}(U_1, ta, n), \text{Place}(U_2, ta + n_1, n)) \rrbracket_{\text{Qb}} & \text{if } U = \text{Parallel}(U_1, U_2) \\ & \text{where } n_1, n_2 \\ & \text{are the sizes of } U_1, U_2 \\ & \text{and } n_1 + n_2 \leq n \end{cases}$$

The definition uses a basis state, leveraging the unitarity of quantum gates.

Definition 3.1.7 (Sub-circuit Control (Cont) [59]). This definition reformulates the semantics of sub-circuit control ($\llbracket \text{Cont} \rrbracket_{\text{qb}}$) for a circuit of type Qb. We give this definition using syntactic notation to simplify the presentation. Let U be a well-formed unitary circuit of type Qb with width n , co a control qubit, ta the target position, and $\langle \emptyset, 0, x \rangle$ a symbolic path-sum representing a basis state $|x\rangle = |x_0 x_1 \dots x_{n-1}\rangle$, where each $x_i \in \{0, 1\}$ for $i \in [0, n-1]$, as defined in Definition 2.3.3. The semantics is given over sub-circuit placement semantics (Definition 3.1.6). The semantics of Cont are defined as:

$$\llbracket \text{Cont}(U, co, ta, n) \rrbracket_{\text{qb}} (\langle \emptyset, 0, x \rangle) = \begin{cases} \llbracket \text{Place}(U, ta, n) \rrbracket_{\text{qb}} (\langle \emptyset, 0, x \rangle) & \text{if } x_{co} = 1 \\ \langle \emptyset, 0, x \rangle & \text{if } x_{co} = 0 \end{cases}$$

The definition uses a basis state, leveraging the unitarity of quantum gates.

3.1.3 Useful Lemmas

To make this document more self-contained, we present the following properties of the Sequence, Place, and Cont operations, available in `circuits_equiv.mlw`¹. These lemmas are crucial for the conversion process discussed in Chapter 3.2 as they guarantee that structural transformations preserve circuit semantics. We adopt the following notation for the semantics equality: For circuits U_1, U_2 of type Qb with n qubits, we write $\llbracket U_1 \rrbracket_{\text{qb}} = \llbracket U_2 \rrbracket_{\text{qb}}$ to denote that U_1 and U_2 are semantically equivalent, i.e., for all basis states $|x\rangle \in \{0, 1\}^n$, $\llbracket U_1 \rrbracket_{\text{qb}} (\langle \emptyset, 0, x \rangle) = \llbracket U_2 \rrbracket_{\text{qb}} (\langle \emptyset, 0, x \rangle)$.

Lemma 3.1.8 (Equivalence Lemmas). Let U, U_1, U_2, U'_1, U'_2 be five circuits of type Qb of sizes n, n_1, n_2, n'_1 and n'_2 , such that $\llbracket U_1 \rrbracket_{\text{qb}} = \llbracket U'_1 \rrbracket_{\text{qb}}$, and $\llbracket U_2 \rrbracket_{\text{qb}} = \llbracket U'_2 \rrbracket_{\text{qb}}$ respectively, then

1. $\llbracket U_1 -- U_2 \rrbracket_{\text{qb}} = \llbracket U'_1 -- U'_2 \rrbracket_{\text{qb}}$,

2. if $n_1 + ta \leq n$ and $n_2 + ta \leq n$, then,

$$\llbracket \text{Place}(U_1 -- U_2, ta, n) \rrbracket_{\text{qb}} = \llbracket \text{Place}(U_1, ta, n) -- \text{Place}(U_2, ta, n) \rrbracket_{\text{qb}},$$

3. if $n_1 + n_2 + ta \leq n$, then,

$$\llbracket \text{Place}(\text{Parallel}(U_1, U_2), ta, n) \rrbracket_{\text{qb}} = \llbracket \text{Place}(U_1, ta, n) -- \text{Place}(U_2, ta + n_{U_1}, n) \rrbracket_{\text{qb}},$$

4. if $ta_2 + n_1 \leq n_2$, then

$$\llbracket \text{Place}(\text{Place}(U, ta_1, n_1), ta_2, n_2) \rrbracket_{\text{qb}} = \llbracket \text{Place}(U, ta_1 + ta_2, n_2) \rrbracket_{\text{qb}},$$

¹https://github.com/Qbricks/qbricks.github.io/blob/main/Qbricks/circuits_equiv.mlw

5. if $ta_2 + n_1 \leq n_2$, then

$$\llbracket \text{Place}(\text{Cont}(U, co_1, ta_1, n_1), ta_2, n_2) \rrbracket_{qb} = \llbracket \text{Cont}(U, co_1 + ta_2, ta_1 + ta_2, n_2) \rrbracket_{qb},$$

6. $\llbracket \text{Place}(U_1, ta_1, n_1) \rrbracket_{qb} = \llbracket \text{Place}(U_2, ta_1, n_1) \rrbracket_{qb}$

7. Let n_{U_1} and n_{U_2} be the size of U_1 and U_2 respectively such that $n_{U_1} + co_1 < n_1$, $n_{U_2} + co_1 \leq n_1$, $n_{U_1} + ta_1 \leq n_1$, $n_{U_2} + ta_1 \leq n_1$, then

$$\begin{aligned} & \llbracket \text{Cont}(\text{Sequence}(U_1, U_2), co_1, ta_1, n_1) \rrbracket_{qb} \\ &= \llbracket \text{Sequence}(\text{Cont}(U_1, co_1, ta_1, n_1), \text{Cont}(U_2, co_1, ta_1, n_1)) \rrbracket_{qb} \end{aligned}$$

8. if $ta_2 + n_1 \leq n_2$, then

$$\llbracket \text{Cont}(\text{Place}(U_1, ta_1, n_1), co_1, ta_2, n_2) \rrbracket_{qb} = \llbracket \text{Cont}(U_1, co_1, ta_1 + ta_2, n_2) \rrbracket_{qb},$$

9. $\llbracket \text{Cont}(U_1, co_1, ta_1, n_1) \rrbracket_{qb} = \llbracket \text{Cont}(U_2, co_1, ta_1, n_1) \rrbracket_{qb}$

Proof. Complete proofs of these lemmas are available in the file `circuits_equiv.mlw`².

□

3.2 From Qbricks To OpenQASM

Our goal is to verify that a circuit transformation preserves the circuit's behaviour. This section presents our compilation pass (Section 2.5.1) that converts circuits from Qbricks (Section 3.1) to OpenQASM (Section 3.2.1), along with its correctness proof. This proof guarantees that any circuit in Qbricks preserves its behaviour after conversion. We target OpenQASM because it is a standard. Our work enables the execution of parametrised unitary circuit instances (PUC, Definition 2.4.1, defined and certified in Qbricks) on both simulators and quantum computers, using frameworks like Qiskit [45]. We begin by presenting the OpenQASM syntax (Section 3.2.1), followed by an overview of our method in Section 3.2.2. The subsequent sections detail each step of this compilation pass: removing spatial compositions (Section 3.2.3), eliminating ternary gates (Section 3.2.4), removing sub-circuit placement (Section 3.2.5), and eliminating generic control (Section 3.2.6). Finally, Section 3.2.7 integrates all these steps into a coherent whole.

²Circuit equivalences lemmas: https://github.com/Qbricks/qbricks.github.io/blob/main/Qbricks/circuits_equiv.mlw

3.2.1 OpenQASM

This chapter introduces OpenQASM as the target language for our circuit transformations. To provide the necessary background, we reproduce below the definitions of the fundamental gates as established in [27]. OpenQASM is directly related to our unitary QASM representation (Section 2.2.4.3). The way gates are applied is the main difference between OpenQASM and our unitary QASM representation. For example, `Apply (H,  $\emptyset$ , {ta})` in our representation is defined in OpenQASM by `H ta` and in Qbricks by `Place(H, ta)`. We begin by introducing the two fundamental gates of OpenQASM in Definition 3.2.1: the rotation gate `U`, which takes real numbers as parameters, and the controlled NOT gate, `Cnot`. Next, we define temporal composition in Definition 3.2.2. To align with the QASM unitary circuit (Definition 2.2.44), we utilize derived operations from Definitions 3.2.4, 3.2.5, and 3.2.6. This enables us to define a data type in OpenQASM syntax.

Definition 3.2.1 (Fundamental Gates in OpenQASM). The arbitrary single-qubit rotation gate `U` is given by:

$$U(\theta, \phi, \lambda) := \begin{pmatrix} e^{-i\frac{\phi+\lambda}{2}} \cos \frac{\theta}{2} & -e^{-i\frac{\phi-\lambda}{2}} \sin \frac{\theta}{2} \\ e^{i\frac{\phi-\lambda}{2}} \sin \frac{\theta}{2} & e^{i\frac{\phi+\lambda}{2}} \cos \frac{\theta}{2} \end{pmatrix}$$

The controlled-NOT gate `Cnot` is defined as:

$$\text{Cnot} := \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Definition 3.2.2 (Temporal Composition (`--`)). In OpenQASM, unitary operations are represented as matrices, and their temporal composition follows the matrix multiplication semantics (Section 2.2.40). While temporal composition is typically denoted by (`;`) in OpenQASM, we use the operator (`--`) in this document to unify the syntax.

Notation 3.2.3 (Gate Application in OpenQASM (Qbricks-style)). To simplify reasoning, we adapt OpenQASM gate applications to Qbricks' qubit convention. Let θ , ϕ , and λ be floating-point parameters, and ta , co be integer qubit indices in an n -qubit system.

- Application of `U` to target qubit ta :

$$U(\theta, \phi, \lambda) \text{ ta} := I^{\otimes(ta-1)} \otimes U(\theta, \phi, \lambda) \otimes I^{\otimes(n-ta)}.$$

- `Cnot` gate with control co and target ta :

$$\text{Cnot } co \text{ ta} \mid x_0 \dots x_{co} \dots x_{ta} \dots x_{n-1} \rangle = \mid x_0 \dots x_{co} \dots (x_{co} \oplus x_{ta}) \dots x_{n-1} \rangle.$$

This notation omits register names (e.g., `q[0]`) for consistency with Qbricks.

Definition 3.2.4 (Unary Gates Derived from U in Qbricks style). For a target qubit ta (integer index) in an n -qubit system, the following unary gates are defined using the U gate with floating-point parameters:

$$\begin{aligned} R_x(\theta) \, ta &:= U\left(\theta, -\frac{\pi}{2}, \frac{\pi}{2}\right) \, ta & R_y(\theta) \, ta &:= U(\theta, 0, 0) \, ta \\ R_z(\lambda) \, ta &:= U(0, 0, \lambda) \, ta & R_{zp}(\lambda) \, ta &:= e^{i\frac{\lambda}{2}} \cdot U\left(0, 0, \frac{\lambda}{2}\right) \, ta \\ H \, ta &:= U\left(\frac{\pi}{2}, 0, \pi\right) \, ta & X \, ta &:= U(\pi, 0, \pi) \, ta \\ Y \, ta &:= U\left(\pi, \frac{\pi}{2}, \frac{\pi}{2}\right) \, ta & \text{Skip } ta &:= U(0, 0, 0) \, ta \end{aligned}$$

Definition 3.2.5 (Unary Gates Derived from R_{zp}). For a target qubit ta (integer index) in an n -qubit system, the following unary gates are defined using the R_{zp} gate with floating-point parameter:

$$Z \, ta := R_{zp}(\pi) \, ta \quad S \, ta := R_{zp}\left(\frac{\pi}{2}\right) \, ta \quad T \, ta := R_{zp}\left(\frac{\pi}{4}\right) \, ta$$

Definition 3.2.6 (SWAP Gate). The Swap gate exchanges the states of two target qubits ta_1 and ta_2 , where ta_1 and ta_2 are integers representing qubit indices:

$$\text{Swap } ta_1 \, ta_2 \left(|x_0 \dots x_{ta_1} \dots x_{ta_2} \dots\rangle \right) = |x_0 \dots x_{ta_2} \dots x_{ta_1} \dots\rangle$$

Remark 3.2.7 (OpenQASM in the Symbolic Context). In OpenQASM, quantum circuits are initialised to the state $|0\rangle$. However, Qbricks (Section 3.1) works with symbolic input variables. To maintain consistency with this symbolic representation, we use the following conventions:

1. A qubit that is **not explicitly initialised** in OpenQASM is considered **symbolic**.
2. The OpenQASM operation (**Reset** ta) **explicitly initialize** the qubit ta to $|0\rangle$.

Definition 3.2.8 (Our Algebraic Data Type for OpenQASM, OQ). We define OQ, an algebraic data type capturing the OpenQASM syntax. Let co , ta , and ta' be three integers representing qubit indices, and θ be a real number representing an angle. We decompose the primitives of OQ into unary (OUnary) and binary (OBinary) gates to simplify the presentation. We define OQ as follows:

OUnary :=

| H ta | Skip ta | $R_x(k) \, ta$ | $R_y(\theta) \, ta$ | $R_z(\theta) \, ta$ | $R_{zp}(\theta) \, ta$ | S ta | T ta | X ta | Y ta | Z ta

OBinary := Cnot $co \, ta$ | Swap $ta \, ta'$ | OQ—OQ

OQ := OUnary | OBinary | OQ—OQ

3.2.2 Overview

This chapter provides an overview of our compilation and certification. Figure 3.3 illustrates the transformation from the Qbricks type Qb (Definition 3.1.1) to the OpenQASM type OQ (Definition 3.2.8). The middle of the figure illustrates the process of converting Qb to OQ. The unitary circuit U (an instance of PUC, Definition 2.4.1) at the top is transformed to fit OQ (U_4) and preserve its semantics. Finally, `to_qasm` outputs the unitary circuit in OQ type. At each step, we verify semantic preservation (e.g., $\llbracket U_1 \rrbracket_{\text{qb}} = \llbracket U \rrbracket_{\text{qb}}$) and ensure that operations not in OQ do not appear (e.g., `parallel_free( $U_1$ )`). We also check that the previous property is maintained (e.g., `parallel_free( $U_2$ )`).

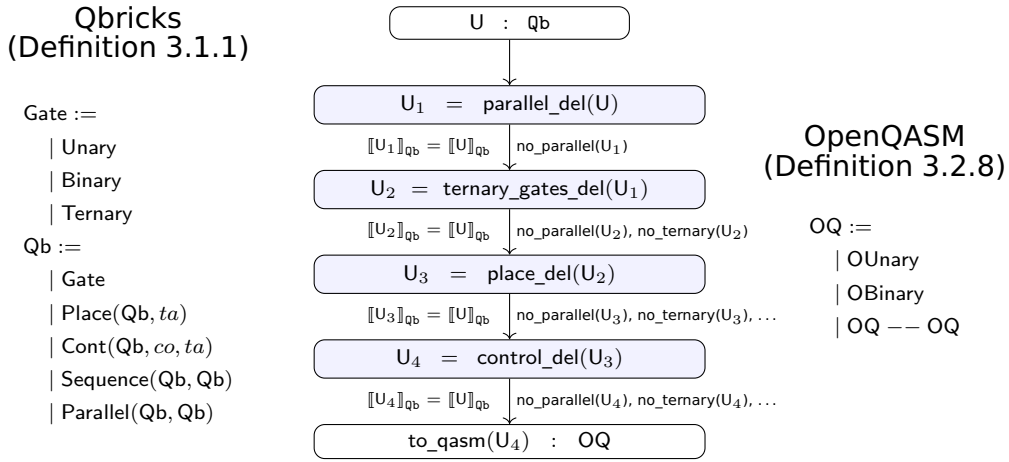


Figure 3.3: `to_qasm` Overview.

We outline the general idea and the most significant proofs. The translation from Qbricks to OpenQASM involves the following steps: Section 3.2.3 removes parallel compositions (`parallel_del`). Section 3.2.4 eliminates ternary gates (`ternary_gates_del`). Section 3.2.5 removes sub-circuit placement (`place_del`). Section 3.2.6 eliminates sub-circuit control (`control_del`). Finally, Section 3.2.7 outlines the full transformation process (`to_qasm`).

The objectives of this transformation are threefold:

1. Preserve well-formedness conditions³.
2. **Preserve the semantics** of the original circuit.
3. Ensure the transformed circuit is **compatible with OpenQASM**.

In this document, we focus on achieving objectives 2 and 3. While objective 1 is crucial, most proofs of this nature are automatically discharged by SMT solvers and have been verified in the source code [59]. Readers may refer to the source code for detailed proofs. Additionally, we omit circuit width details when not directly relevant to these primary objectives, as with `Place` and `Cont`, to streamline the presentation.

³Ensure a gate is not applied outside a circuit and a qubit is not duplicated.

3.2.3 Parallel Compositions

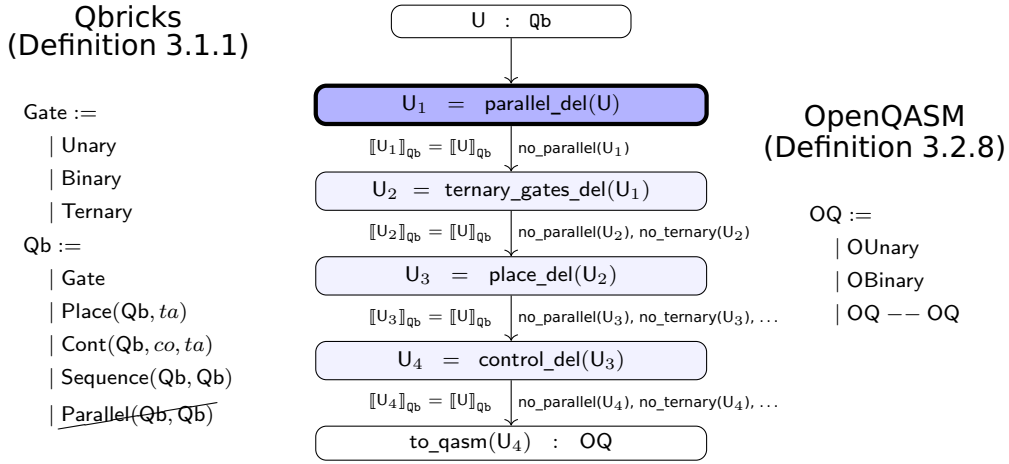


Figure 3.4: Removing Parallel Composition (parallel_del).

OQ does not include parallel composition. Therefore, we must eliminate parallel compositions from Qb while maintaining semantic equivalence. Figure 3.4 emphasizes this step in blue ($U_1 = \text{parallel_del}(U)$). This step preserves the semantics ($\llbracket U_1 \rrbracket_{\text{qb}} = \llbracket U \rrbracket_{\text{qb}}$) and removes spatial compositions ($\text{no_parallel}(U_1)$). Figure 3.5 shows

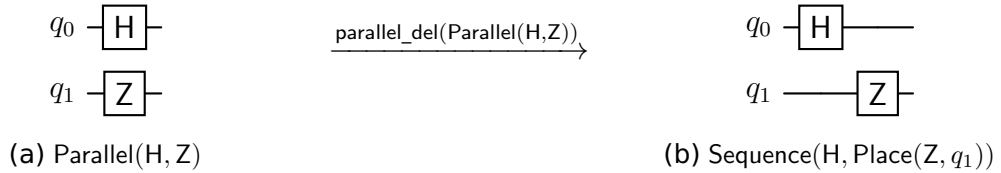


Figure 3.5: Example of Removing Parallel Composition

an example of this process. The spatial composition of the two gates, H and Z (Figure 3.5a), is transformed into a temporal composition of H and Z, as depicted in Figure 3.5b. We begin with the Definition 3.2.9, which outlines this transformation. Then Lemma 3.2.10 guarantees the preservation of semantics. Finally, Lemma 3.2.12 confirms the absence of parallel compositions in the resulting circuit.

Definition 3.2.9 (Removing Parallel). Let U , U_1 , and U_2 be three unit circuits of type Qb with widths n , n_1 , and n_2 respectively, of type `int`. The transformation $\text{parallel_del}(U)$

is defined as follows:

$$\text{parallel_del}(U) := \begin{cases} \text{Sequence}(\text{Place}(\text{parallel_del}(U_1), 0, n), \text{Place}(\text{parallel_del}(U_2), n_1, n)) & \text{if } U = \text{Parallel}(U_1, U_2) \\ & \text{where } n_1 + n_2 = n \\ \text{Sequence}(\text{parallel_del}(U_1), \text{parallel_del}(U_2)) & \text{if } U = \text{Sequence}(U_1, U_2) \\ \text{Place}(\text{parallel_del}(U_1), ta) & \text{if } U = \text{Place}(U_1, ta) \\ \text{Cont}(\text{parallel_del}(U_1), co, ta) & \text{if } U = \text{Cont}(U_1, co, ta) \\ U & \text{else} \end{cases}$$

Lemme 3.2.10 (Preservation of Semantics under Removing Parallel). Let U be a unitary circuit defined in the type Qb . Then, the transformation parallel_del preserves the semantics of U : $\llbracket \text{parallel_del}(U) \rrbracket_{Qb} = \llbracket U \rrbracket_{Qb}$.

Proof. Proof by structural induction on U :

1. If $U = \text{Parallel}(U_1, U_2)$, then:

$$\begin{aligned} \llbracket U_1 \rrbracket_{Qb} &= \llbracket \text{parallel_del}(U_1) \rrbracket_{Qb} && \text{(Induction hypothesis)} \\ \llbracket U_2 \rrbracket_{Qb} &= \llbracket \text{parallel_del}(U_2) \rrbracket_{Qb} && \text{(Induction hypothesis)} \\ \llbracket \text{parallel_del}(U) \rrbracket_{Qb} &= \llbracket \text{Place}(\text{parallel_del}(U_1), 0, (n_1 + n_2)) \rrbracket_{Qb} && \text{(Definition 3.2.9)} \\ &\quad \text{---} \llbracket \text{Place}(\text{parallel_del}(U_2), n_1, (n_1 + n_2)) \rrbracket_{Qb} \\ &= \llbracket \text{Place}(U_1, 0, (n_1 + n_2)) \rrbracket_{Qb} && \text{(By Induction Hypothesis and Lemma 6)} \\ &\quad \text{---} \llbracket \text{Place}(U_2, n_1, (n_1 + n_2)) \rrbracket_{Qb} \\ &= \llbracket \text{Place}(\text{Parallel}(U_1, U_2), 0, (n_1 + n_2)) \rrbracket_{Qb} && \text{(Lemma 3, with } ta = 0 \text{ and } n = n_1 + n_2) \\ &= \llbracket U \rrbracket_{Qb} \end{aligned}$$

2. If $U = \text{Sequence}(U_1, U_2)$, then:

$$\begin{aligned} \llbracket U_1 \rrbracket_{Qb} &= \llbracket \text{parallel_del}(U_1) \rrbracket_{Qb} && \text{(Induction hypothesis)} \\ \llbracket U_2 \rrbracket_{Qb} &= \llbracket \text{parallel_del}(U_2) \rrbracket_{Qb} && \text{(Induction hypothesis)} \\ \llbracket \text{parallel_del}(U) \rrbracket_{Qb} &= \llbracket \text{Sequence}(\text{parallel_del}(U_1), \text{parallel_del}(U_2)) \rrbracket_{Qb} && \text{(Definition 3.2.9)} \\ &= \llbracket \text{parallel_del}(U_2) \rrbracket_{Qb} \circ \llbracket \text{parallel_del}(U_1) \rrbracket_{Qb} && \text{(Definition 3.1.3)} \\ &= \llbracket U_2 \rrbracket_{Qb} \circ \llbracket U_1 \rrbracket_{Qb} && \text{(By Induction Hypothesis)} \\ &= \llbracket \text{Sequence}(U_1, U_2) \rrbracket_{Qb} && \text{(Definition 3.1.3)} \\ &= \llbracket U \rrbracket_{Qb} \end{aligned}$$

3. If $U = \text{Place}(ta, U')$, then:

$$\begin{aligned}
\llbracket U' \rrbracket_{\text{qb}} &= \llbracket \text{parallel_del}(U') \rrbracket_{\text{qb}} && \text{(Induction hypothesis)} \\
\llbracket \text{parallel_del}(U) \rrbracket_{\text{qb}} &= \llbracket \text{Place}(\text{parallel_del}(U'), ta) \rrbracket_{\text{qb}} && \text{(Definition 3.2.9)} \\
&= \llbracket \text{Place}(U', ta) \rrbracket_{\text{qb}} && \text{(By Induction Hypothesis and lemma 6)} \\
&= \llbracket U \rrbracket_{\text{qb}}
\end{aligned}$$

4. If $U = \text{Cont}(U', co, ta)$, then:

$$\begin{aligned}
\llbracket U' \rrbracket_{\text{qb}} &= \llbracket \text{parallel_del}(U') \rrbracket_{\text{qb}} && \text{(Induction hypothesis)} \\
\llbracket \text{parallel_del}(U) \rrbracket_{\text{qb}} &= \llbracket \text{Cont}(\text{parallel_del}(U'), co, ta) \rrbracket_{\text{qb}} && \text{(Definition 3.2.9)} \\
&= \llbracket \text{Cont}(U', co, ta) \rrbracket_{\text{qb}} && \text{(By Induction Hypothesis and lemma 9)} \\
&= \llbracket U \rrbracket_{\text{qb}}
\end{aligned}$$

5. Else $\llbracket \text{parallel_del}(U) \rrbracket_{\text{qb}} = \llbracket U \rrbracket_{\text{qb}}$. □

Predicate 3.2.11 (No Parallel). The predicate $\text{parallel_free}(U)$ checks for the absence of parallel composition in a given unitary circuit U . It is defined as follows:

$$\text{parallel_free}(U) := \begin{cases} \text{parallel_free}(U_1) \wedge \text{parallel_free}(U_2) & \text{if } U = \text{Sequence}(U_1, U_2), \\ \text{parallel_free}(U_1) & \text{if } U = \text{Place}(U_1, _), \\ \text{parallel_free}(U_1) & \text{if } U = \text{Cont}(U_1, _, _), \\ \text{false} & \text{if } U = \text{Parallel}(_, _), \\ \text{true} & \text{else} \end{cases}$$

Lemme 3.2.12 (No Parallel). For any unitary circuit U , the transformation $\text{parallel_del}(U)$ ensures that the resulting quantum circuit has no parallel composition, denoted by $\text{parallel_free}(\text{parallel_del}(U)) = \text{true}$.

Proof. Proof by structural induction on U :

1. If $U = \text{Parallel}(U_1, U_2)$, then

$$\begin{aligned}
& \text{parallel_free}(\text{parallel_del}(U_1)) = \text{true} && \text{(Induction Hypothesis)} \\
& \text{parallel_free}(\text{parallel_del}(U_2)) = \text{true} && \text{(Induction Hypothesis)} \\
& \text{parallel_free}(\text{parallel_del}(U)) \\
& \quad = \text{parallel_free}(\text{Sequence}(\text{Place}(\text{parallel_del}(U_1), 0, (n_1 + n_2)), \\
& \quad \quad \text{Place}(\text{parallel_del}(U_2), n_1, (n_1 + n_2)))) && \text{(Definition 3.2.9)} \\
& \quad = \text{parallel_free}(\text{parallel_del}(U_1)) && \text{(Predicate 3.2.11)} \\
& \quad \quad \wedge \text{parallel_free}(\text{parallel_del}(U_2)) \\
& \quad = \text{true} \wedge \text{true} && \text{(By Induction hypothesis)} \\
& \quad = \text{true}.
\end{aligned}$$

2. If $U = \text{Sequence}(U_1, U_2)$, then

$$\begin{aligned}
& \text{parallel_free}(\text{parallel_del}(U_1)) = \text{true} && \text{(Induction Hypothesis)} \\
& \text{parallel_free}(\text{parallel_del}(U_2)) = \text{true} && \text{(Induction Hypothesis)} \\
& \text{parallel_free}(\text{parallel_del}(U)) \\
& \quad = \text{parallel_free}(\text{Sequence}(\text{parallel_del}(U_1), \\
& \quad \quad \text{parallel_del}(U_2))) && \text{(Definition 3.2.9)} \\
& \quad = \text{parallel_free}(\text{parallel_del}(U_1)) && \text{(Predicate 3.2.11)} \\
& \quad \quad \wedge \text{parallel_free}(\text{parallel_del}(U_2)) \\
& \quad = \text{true} \wedge \text{true} && \text{(By Induction hypothesis)} \\
& \quad = \text{true}.
\end{aligned}$$

3. If $U = \text{Place}(U', ta)$, then

$$\begin{aligned}
& \text{parallel_free}(\text{parallel_del}(U')) = \text{true} && \text{(Induction Hypothesis)} \\
& \text{parallel_free}(\text{parallel_del}(U)) \\
& \quad = \text{parallel_free}(\text{Place}(\text{parallel_del}(U'), ta)), && \text{(Definition 3.2.9)} \\
& \quad = \text{parallel_free}(\text{parallel_del}(U')) && \text{(Predicate 3.2.11)} \\
& \quad = \text{true}. && \text{(By Induction hypothesis)}
\end{aligned}$$

4. If $U = \text{Cont}(U', co, ta)$, then

$$\begin{aligned}
& \text{parallel_free}(\text{parallel_del}(U')) = \text{true} && \text{(Induction Hypothesis)} \\
& \text{parallel_free}(\text{parallel_del}(U)) \\
& \quad = \text{parallel_free}(\text{Cont}(\text{parallel_del}(U'), co, ta)), && \text{(Definition 3.2.9)} \\
& \quad = \text{parallel_free}(\text{parallel_del}(U')) && \text{(Predicate 3.2.11)} \\
& \quad = \text{true}. && \text{(By Induction hypothesis)}
\end{aligned}$$

5. Else $\text{parallel_free}(\text{parallel_del}(U)) = \text{true}$. □

3.2.4 Ternary Gates

OQ does not include ternary gates. Therefore, we must eliminate ternary gates from Qb while maintaining semantic equivalence. Figure 3.6 emphasizes this step in blue ($U_2 = \text{ternary_gates_del}(U_1)$). This step preserves the semantics ($\llbracket U_2 \rrbracket_{\text{Qb}} = \llbracket U \rrbracket_{\text{Qb}}$). It removes ternary gates, ensured by $\text{no_ternary}(U_2)$. It also preserves $\text{no_parallel}(U_2)$. The last property, preservation of no_parallel , has been proven in the code but is not detailed here. Here, we assume this property holds so that we can focus on more critical proofs. More generally, although the preservation of all previous properties is not proved in this document, it is proved in the code. Figure 3.7 shows an example of this process. The Toffoli gate (Figure 3.7a) is decomposed into OQ type (Figure 3.7b). We

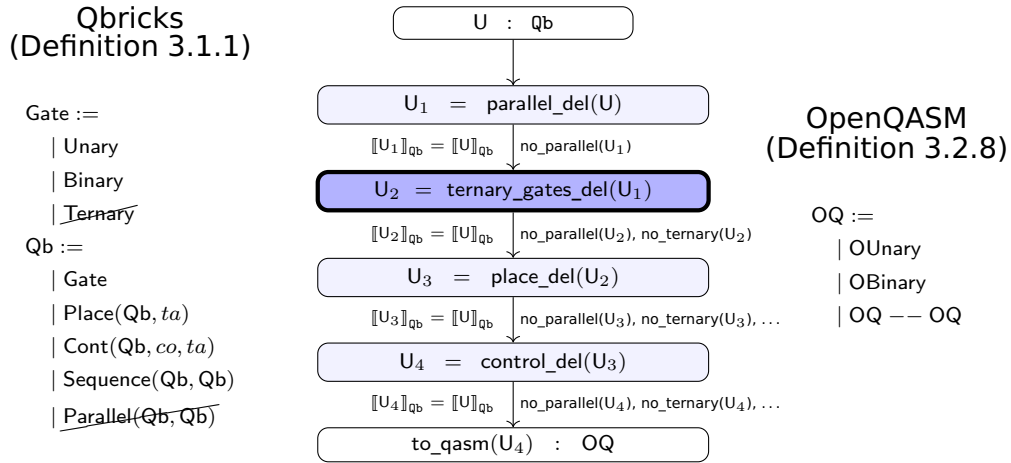


Figure 3.6: Removing Ternary Gates (ternary_gates_del).

begin by providing prerequisites about gate decompositions with their proofs of correctness, which are useful for decomposing Toffoli and Fredkin (Section 3.2.4.1). We give the Toffoli and Fredkin decomposition with their proofs of correctness (Section 3.2.4.2), and we finish with the recursive function that removes ternary gates by applying the Fredkin and Toffoli decompositions (Section 3.2.4.3) on the leaves.

3.2.4.1 Prerequisites

This section deals with the decomposition of the Control-Control-Z gate, denoted ccz , which applies a Z gate if the two controls are at $|1\rangle$, which is at the heart of our Toffoli gate decomposition. We start by giving the required intermediate decompositions, and end with the definition of the decomposition itself (Definition 3.2.17). For each decomposition, we give a lemma that ensures the preservation of semantics equivalence.

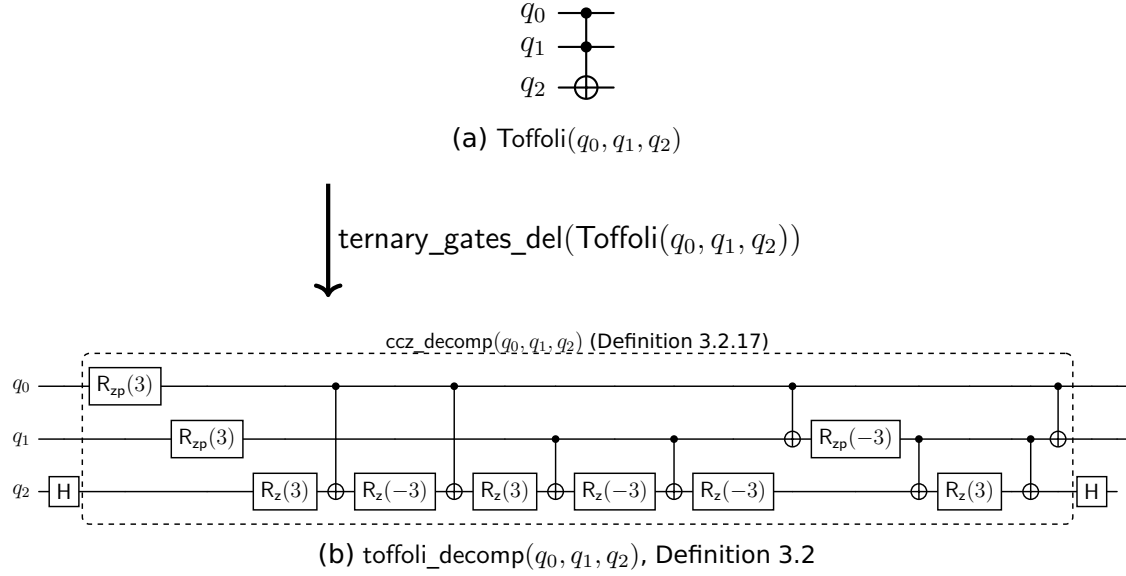


Figure 3.7: Example of Removing Ternary Gate with Toffoli

Definition 3.2.13 (crz Decomposition).

$$\text{crz_decomp}(k, co, ta) := R_z(k+1, ta) \text{---} \text{Cnot}(co, ta) \text{---} R_z(-(k+1), ta) \text{---} \text{Cnot}(co, ta).$$

Lemme 3.2.14 (Semantics Preservation under crz_decomp Decomposition).

$$\llbracket \text{crz_decomp}(k, co, ta) \rrbracket_{\text{qb}} = \llbracket \text{crz}(k, co, ta) \rrbracket_{\text{qb}}$$

Proof. Let $|x\rangle = |x_0, x_1, \dots, x_{n-1}\rangle$ be a basis state over n qubits, where $\forall i \in [0, n-1], x_i \in \{0, 1\}$, then

$$\begin{aligned}
 & \llbracket \text{crz_decomp}(k, co, ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x\rangle \rangle) \\
 &= \llbracket R_z(k+1, ta) \text{---} \text{Cnot}(co, ta) \text{---} R_z(-(k+1), ta) \text{---} \text{Cnot}(co, ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x\rangle \rangle) \\
 &= \left\langle \emptyset, \frac{2x_{ta} - 1}{2^{k+1}} - \frac{2(x_{co} + x_{ta} - 2x_{co} \cdot x_{ta}) - 1}{2^{k+1}}, |x\rangle \right\rangle \quad (\text{Definition 3.1.3, and 3.1.4}) \\
 &= \left\langle \emptyset, -\frac{2x_{co} - 4x_{co} \cdot x_{ta}}{2^{k+1}}, |x\rangle \right\rangle = \left\langle \emptyset, \frac{2x_{co} \cdot x_{ta} - x_{co}}{2^k}, |x\rangle \right\rangle \\
 &= \llbracket \text{crz}(k, co, ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x\rangle \rangle). \quad \square
 \end{aligned}$$

Definition 3.2.15 (crzp Decomposition). crz_decomp is defined in Definition 3.2.13.

$$\text{crzp_decomp}(k, co, ta) := R_{zp}(k+1, co) \text{---} \text{crz_decomp}(k+1, co, ta).$$

Lemme 3.2.16 (Semantics Preservation under crzp_decomp Decomposition).

$$\llbracket \text{crzp_decomp}(k, co, ta) \rrbracket_{\text{qb}} = \llbracket \text{crzp}(k, co, ta) \rrbracket_{\text{qb}}$$

Proof. Let $|x\rangle = |x_0, x_1, \dots, x_{n-1}\rangle$ be a basis state over n qubits, where $\forall i \in [0, n - 1], x_i \in \{0, 1\}$, then

$$\begin{aligned}
& \llbracket \text{crzp_decomp}(k, co, ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x\rangle \rangle) \\
&= \llbracket R_{zp}(k + 1, co) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x\rangle \rangle) \\
&= \left\langle \emptyset, \frac{x_{co}}{2^{k+1}} + \frac{2x_{co} \cdot x_{ta} - x_{co}}{2^{k+1}}, |x\rangle \right\rangle = \left\langle \emptyset, \frac{x_{co} \cdot x_{ta}}{2^k}, |x\rangle \right\rangle \\
&\quad \text{(Definition 3.1.3, and 3.2.13)} \\
&= \llbracket \text{crzp}(k, co, ta) \rrbracket_{\text{qb}}. \quad \square
\end{aligned}$$

Definition 3.2.17 (ccz Decomposition). crzp_decomp is defined in Definition 3.2.15.

$$\begin{aligned}
\text{ccz_decomp}(co_1, co_2, ta) &:= \text{crzp_decomp}(2, co_1, ta) \\
&\quad \text{--- crzp_decomp}(2, co_2, ta) \text{--- Cnot}(co_1, co_2) \\
&\quad \text{--- crzp_decomp}(-2, co_2, ta) \text{--- Cnot}(co_1, co_2).
\end{aligned}$$

Lemme 3.2.18 (Semantics Preservation under ccz_decomp Decomposition).

$$\llbracket \text{ccz_decomp}(co_1, co_2, ta) \rrbracket_{\text{qb}} = \llbracket \text{ccz}(co_1, co_2, ta) \rrbracket_{\text{qb}}$$

Proof. Let $|x\rangle = |x_0, x_1, \dots, x_{n-1}\rangle$ be a basis state over n qubits, where $\forall i \in [0, n - 1], x_i \in \{0, 1\}$, then

$$\begin{aligned}
& \llbracket \text{ccz_decomp}(co_1, co_2, ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x\rangle \rangle) \\
&= \llbracket \text{crzp_decomp}(2, co_1, ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x\rangle \rangle) \text{--- Cnot}(co_1, co_2) \\
&\quad \text{--- crzp_decomp}(-2, co_2, ta) \text{--- Cnot}(co_1, co_2) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x\rangle \rangle) \\
&= \left\langle \emptyset, \frac{x_{co_1} \cdot x_{ta}}{2^2} + \frac{x_{co_2} \cdot x_{ta}}{2^2} - \frac{(x_{co_1} + x_{co_2} + 2x_{co_1} \cdot x_{co_2}) \cdot x_{ta}}{2^2}, |x\rangle \right\rangle \\
&\quad \text{(Definition 3.1.4, and 3.2.15)} \\
&= \left\langle \emptyset, \frac{x_{co_1} \cdot x_{co_2} \cdot x_{ta}}{2}, |x\rangle \right\rangle = \llbracket \text{ccz}(co_1, co_2, ta) \rrbracket_{\text{qb}}. \quad \square
\end{aligned}$$

3.2.4.2 Toffoli and Fredkin Decomposition

This section defines the decompositions of the Toffoli (Definition, Equation 3.1, and 3.2) and Fredkin (Definition, Equation 3.5, and 3.6) gates. For each definition, we give the equivalence preservation Lemma, respectively Lemma 3.3, 3.4, 3.7, and 3.8.

Definition 3.2.19 (Toffoli Decomposition). crz_decomp is defined in Definition 3.2.13.

$$\text{toffoli_decomp}(co_1, co_2, ta) := H(ta) \text{--- ccz_decomp}(co_1, co_2, ta) \text{--- } H(ta). \quad (3.1)$$

$$\text{bricks_toffoli_decomp} := \text{toffoli_decomp}(0, 1, 2) \quad (3.2)$$

Lemme 3.2.20 (Toffoli Decomposition Semantics Preservation).

$$\llbracket \text{Toffoli}(co_1, co_2, ta) \rrbracket_{\text{qb}} = \llbracket \text{toffoli_decomp}(co_1, co_2, ta) \rrbracket_{\text{qb}} \quad (3.3)$$

$$\llbracket \text{Bricks_Toffoli} \rrbracket_{\text{qb}} = \llbracket \text{bricks_toffoli_decomp} \rrbracket_{\text{qb}} \quad (3.4)$$

Proof. Let $|x\rangle = |x_0x_1 \dots x_{n-1}\rangle \in \mathcal{H}^{\otimes n}$, where $\forall i \in [0, n-1], x_i \in \{0, 1\}$, then

$$\begin{aligned}
& \llbracket \text{toffoli_decomp}(co_1, co_2, ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0 \dots x_{ta-1} x_{ta} x_{ta+1} \dots x_{n-1}\rangle \rangle) \\
&= \llbracket H(ta) - \text{ccz_decomp}(co_1, co_2, ta) - H(ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0 \dots x_{ta-1} x_{ta} x_{ta+1} \dots x_{n-1}\rangle \rangle) \\
&= \left\langle \{y_0, y_1\}, \frac{x_{ta} \cdot y_0}{2} + \frac{x_{co_1} \cdot x_{co_2} \cdot y_0}{2} + \frac{y_0 \cdot y_1}{2}, |x_0 \dots x_{ta-1} y_1 x_{ta+1} \dots x_{n-1}\rangle \right\rangle.
\end{aligned}$$

(Definition 3.1.3, and 3.2.17)

Let's reason by cases:

1. If $x_{co_1} \cdot x_{co_2} = x_{ta} = 0$, then

$$\begin{aligned}
& \llbracket \text{toffoli_decomp}(co_1, co_2, ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0 \dots x_{ta-1} 0, x_{ta+1} \dots x_{n-1}\rangle \rangle) \\
&= \left\langle y_0 y_1, \frac{y_0 \cdot y_1}{2}, |x_0 \dots x_{ta-1} y_1 x_{ta+1} \dots x_{n-1}\rangle \right\rangle \\
&= \frac{1}{2} \left(\left\langle 00, 0, |x_0 \dots x_{ta-1} 0 x_{ta+1} \dots x_{n-1}\rangle \right\rangle \right. \\
&\quad + \left\langle 01, 0, |x_0 \dots x_{ta-1} 0 x_{ta+1} \dots x_{n-1}\rangle \right\rangle \\
&\quad + \left\langle 10, 0, |x_0 \dots x_{ta-1} 1 x_{ta+1} \dots x_{n-1}\rangle \right\rangle \\
&\quad \left. + \left\langle 11, \frac{1}{2}, |x_0 \dots x_{ta-1} 1 x_{ta+1} \dots x_{n-1}\rangle \right\rangle \right) \\
&= \left\langle *, 0, |x_0 \dots x_{ta-1} 0 x_{ta+1} \dots x_{n-1}\rangle \right\rangle \\
&= \llbracket \text{Toffoli}(co_1, co_2, ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |0\rangle \rangle).
\end{aligned}$$

2. If $x_{co_1} \cdot x_{co_2} = 0$ and $x_{ta} = 1$, then

$$\begin{aligned}
& \llbracket \text{toffoli_decomp}(co_1, co_2, ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0 \dots x_{ta-1} 1 x_{ta+1} \dots x_{n-1}\rangle \rangle) \\
&= \left\langle y_0 y_1, \frac{y_0}{2} + \frac{y_0 \cdot y_1}{2}, |x_0 \dots x_{ta-1} y_1 x_{ta+1} \dots x_{n-1}\rangle \right\rangle \\
&= \frac{1}{2} \left(\left\langle 00, 0, |x_0 \dots x_{ta-1} 0 x_{ta+1} \dots x_{n-1}\rangle \right\rangle \right. \\
&\quad + \left\langle 01, \frac{1}{2}, |x_0 \dots x_{ta-1} 0 x_{ta+1} \dots x_{n-1}\rangle \right\rangle \\
&\quad + \left\langle 10, 0, |x_0 \dots x_{ta-1} 1 x_{ta+1} \dots x_{n-1}\rangle \right\rangle \\
&\quad \left. + \left\langle 11, 1, |x_0 \dots x_{ta-1} 1 x_{ta+1} \dots x_{n-1}\rangle \right\rangle \right) \\
&= \left\langle *, 0, |x_0 \dots x_{ta-1} 1 x_{ta+1} \dots x_{n-1}\rangle \right\rangle \\
&= \llbracket \text{Toffoli}(co_1, co_2, ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x\rangle \rangle).
\end{aligned}$$

3. If $x_{co_1} \cdot x_{co_2} = 1$ and $x_{ta} = 0$, then

$$\begin{aligned}
& \llbracket \text{toffoli_decomp}(co_1, co_2, ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0 \dots x_{ta-1} 0 x_{ta+1} \dots x_{n-1} \rangle \rangle) \\
&= \left\langle y_0 y_1, \frac{y_0}{2} + \frac{y_0 \cdot y_1}{2}, |x_0 \dots x_{ta-1} y_1 x_{ta+1} \dots x_{n-1} \rangle \right\rangle \\
&= \left\langle \emptyset, 0, |x_0 \dots x_{ta-1} 1 x_{ta+1} \dots x_{n-1} \rangle \right\rangle \\
&= \llbracket \text{Toffoli}(co_1, co_2, ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x \rangle \rangle).
\end{aligned}$$

4. If $x_{co_1} \cdot x_{co_2} = 1$ and $x_{ta} = 1$, then

$$\begin{aligned}
& \llbracket \text{toffoli_decomp}(co_1, co_2, ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x_0 \dots x_{ta-1} 1 x_{ta+1} \dots x_{n-1} \rangle \rangle) \\
&= \left\langle y_0 y_1, \frac{y_0 \cdot y_1}{2}, |x_0 \dots x_{ta-1} y_1 x_{ta+1} \dots x_{n-1} \rangle \right\rangle \\
&= \left\langle \emptyset, 0, |x_0 \dots x_{ta-1} 0 x_{ta+1} \dots x_{n-1} \rangle \right\rangle \\
&= \llbracket \text{Toffoli}(co_1, co_2, ta) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x \rangle \rangle). \quad \square
\end{aligned}$$

The Bricks_Toffoli gate is a specific case of the Toffoli gate. Consequently, the proof of this case directly follows from the general case.

Definition 3.2.21 (Fredkin Decomposition). toffoli_decomp is defined in Definition 3.2.

$$\begin{aligned}
\text{fredkin_decomp}(co, ta_1, ta_2) &:= \text{Cnot}(ta_2, ta_1) \text{---} \text{toffoli_decomp}(co, ta_1, ta_2) \\
&\quad \text{---} \text{Cnot}(ta_2, ta_1) \tag{3.5}
\end{aligned}$$

$$\text{bricks_fredkin_decomp} := \text{fredkin_decomp}(0, 1, 2). \tag{3.6}$$

Lemme 3.2.22 (Fredkin Decomposition Semantics Preservation).

$$\llbracket \text{Fredkin}(co, ta_1, ta_2) \rrbracket_{\text{qb}} = \llbracket \text{fredkin_decomp}(co, ta_1, ta_2) \rrbracket_{\text{qb}} \tag{3.7}$$

$$\llbracket \text{bricks_fredkin_decomp} \rrbracket_{\text{qb}} = \llbracket \text{Bricks_Fredkin} \rrbracket_{\text{qb}}. \tag{3.8}$$

Proof. Let $|x\rangle = |x_0 x_1 \dots x_{n-1}\rangle$ be a basis state over n qubits, where $\forall i \in [0, n-1], x_i \in \{0, 1\}$, then

$$\begin{aligned}
& \llbracket \text{fredkin_decomp}(co, ta_1, ta_2) \rrbracket_{\text{qb}} \\
&= \left(\langle \emptyset, 0, |x_0 \dots x_{ta_1-1} x_{ta_1} x_{ta_1+1} \dots x_{ta_2-1} x_{ta_2}, x_{ta_2+1} \dots x_{n-1} \rangle \right) \\
&= \llbracket \text{Cnot}(ta_2, ta_1) \text{---} \text{toffoli_decomp}(co, ta_1, ta_2) \text{---} \text{Cnot}(ta_2, ta_1) \rrbracket_{\text{qb}} \\
&= \left(\langle \emptyset, 0, |x_0 \dots x_{ta_1-1} x_{ta_1} x_{ta_1+1} \dots x_{ta_2-1} x_{ta_2} x_{ta_2+1} \dots x_{n-1} \rangle \right) \\
&= \langle \emptyset, 0, |x_0 \dots x_{ta_1-1} (x_{ta_1} \oplus x_{ta_2} \oplus x_{co} \cdot (x_{ta_1} \oplus x_{ta_2}) \oplus x_{ta_2}) x_{ta_1+1} \\
&\quad \dots x_{ta_2-1} (x_{co} \cdot (x_{ta_1} \oplus x_{ta_2}) \oplus x_{ta_2}) x_{ta_2+1} \dots x_{n-1} \rangle \rangle \quad (\text{Definition 3.1.4, and 3.3}) \\
&= \langle \emptyset, 0, |x_0 \dots x_{ta_1-1} (\neg x_{co} \cdot x_{ta_1} \oplus x_{co} \cdot x_{ta_2}) x_{ta_1+1} \\
&\quad \dots x_{ta_2-1} (x_{co} \cdot x_{ta_1} \oplus \neg x_{co} \cdot x_{ta_2}) x_{ta_2+1} \dots x_{n-1} \rangle \rangle \\
&= \llbracket \text{Fredkin}(co, ta_1, ta_2) \rrbracket_{\text{qb}} (\langle \emptyset, 0, |x \rangle \rangle). \quad \square
\end{aligned}$$

The Bricks_Fredkin gate is a specific case of the Fredkin gate. Therefore, the proof of this case directly follows from the general case.

3.2.4.3 Ternary Delete

This section presents in the Definition 3.2.23 the function for removing ternary gates, the Lemma 3.2.24 ensures the preservation of semantics, and the Lemma 3.2.26 ensures that the transformation has indeed removed all ternary gates.

Definition 3.2.23 (Ternary Gates Delete). The function $\text{ternary_gates_del}(U)$ systematically decomposes ternary gates into unary and binary gates. Let U be a unitary circuit of Qb type, then $\text{ternary_gates_del}(U)$ is defined as follows:

$$\text{ternary_gates_del}(U) := \begin{cases} \text{bricks_toffoli_decomp} & \text{if } U = \text{Bricks_Toffoli}, \\ \quad (\text{Definition 3.2}) & \\ \text{toffoli_decomp}(co_1, co_2, ta) & \text{if } U = \text{Toffoli}(co_1, co_2, ta), \\ \quad (\text{Definition 3.1}) & \\ \text{bricks_fredkin_decomp} & \text{if } U = \text{Bricks_Fredkin}, \\ \quad (\text{Definition 3.5}) & \\ \text{fredkin_decomp}(co, ta_1, ta_2) & \text{if } U = \text{Fredkin}(co, ta_1, ta_2), \\ \quad (\text{Definition 3.6}) & \\ \text{ternary_gates_del}(U_1) \text{ --- } & \text{if } U = U_1 \text{ --- } U_2, \\ \quad \text{ternary_gates_del}(U_2) & \\ \text{Place}(\text{ternary_gates_del}(U'), ta) & \text{if } U = \text{Place}(U', ta), \\ \text{Cont}(\text{ternary_gates_del}(U'), co, ta) & \text{if } U = \text{Cont}(U', co, ta), \\ U & \text{else.} \end{cases}$$

Lemme 3.2.24 (Semantics Preservation by Decomposition of Ternary Gate). Let U be a unitary circuit of type Qb , such that $\text{parallel_free}(U) = \text{true}$, then

$$\llbracket \text{ternary_gates_del}(U) \rrbracket_{Qb} = \llbracket U \rrbracket_{Qb}.$$

Proof. Proof by structural induction on U .

1. If $U = \text{Toffoli}(co_1, co_2, ta)$, then

$$\begin{aligned} \llbracket U \rrbracket_{Qb} &= \llbracket \text{toffoli_decomp}(co_1, co_2, ta) \rrbracket_{Qb} && (\text{Definition 3.2.23}) \\ &= \llbracket \text{Toffoli}(co_1, co_2, ta) \rrbracket_{Qb}. && (\text{Lemma 3.3}) \end{aligned}$$

2. If $U = \text{Bricks_Toffoli}$, then

$$\begin{aligned} \llbracket U \rrbracket_{Qb} &= \llbracket \text{bricks_toffoli_decomp} \rrbracket_{Qb} && (\text{Definition 3.2.23}) \\ &= \llbracket \text{Bricks_Toffoli} \rrbracket_{Qb}. && (\text{Lemma 3.4}) \end{aligned}$$

3. If $U = \text{Fredkin}(co, ta_1, ta_2)$, then

$$\begin{aligned} \llbracket U \rrbracket_{Qb} &= \llbracket \text{fredkin_decomp}(co, ta_1, ta_2) \rrbracket_{Qb} && (\text{Definition 3.2.23}) \\ &= \llbracket \text{Fredkin}(co, ta_1, ta_2) \rrbracket_{Qb}. && (\text{Lemma 3.7}) \end{aligned}$$

4. If $U = \text{Bricks_Fredkin}$, then

$$\begin{aligned} \llbracket U \rrbracket_{\text{qb}} &= \llbracket \text{bricks_fredkin_decomp} \rrbracket_{\text{qb}} && \text{(Definition 3.2.23)} \\ &= \llbracket \text{Bricks_Fredkin} \rrbracket_{\text{qb}}. && \text{(Lemma 3.8)} \end{aligned}$$

5. If $U = \text{Place}(U', ta)$, then

$$\begin{aligned} \llbracket U' \rrbracket_{\text{qb}} &= \llbracket \text{ternary_gates_del}(U') \rrbracket_{\text{qb}} && \text{(Induction Hypothesis)} \\ \llbracket \text{ternary_gates_del}(U) \rrbracket_{\text{qb}} &= \llbracket \text{Place}(\text{ternary_gates_del}(U'), ta) \rrbracket_{\text{qb}} && \text{(Definition 3.2.23)} \\ &= \llbracket \text{Place}(U', ta) \rrbracket_{\text{qb}} \\ &\quad \text{(By Induction Hypothesis and Lemma 6)} \\ &= \llbracket U \rrbracket_{\text{qb}}. \end{aligned}$$

6. If $U = \text{Cont}(U', co, ta)$, then

$$\begin{aligned} \llbracket U' \rrbracket_{\text{qb}} &= \llbracket \text{ternary_gates_del}(U') \rrbracket_{\text{qb}} && \text{(Induction Hypothesis)} \\ \llbracket \text{ternary_gates_del}(U) \rrbracket_{\text{qb}} &= \llbracket \text{Cont}(\text{ternary_gates_del}(U'), co, ta) \rrbracket_{\text{qb}} && \text{(Definition 3.2.23)} \\ &= \llbracket \text{Cont}(U', co, ta) \rrbracket_{\text{qb}} = \llbracket U \rrbracket_{\text{qb}}. && \text{(By Induction Hypothesis and Lemma 9)} \end{aligned}$$

7. If $U = U_1 \text{---} U_2$, then

$$\begin{aligned} \llbracket U_1 \rrbracket_{\text{qb}} &= \llbracket \text{ternary_gates_del}(U_1) \rrbracket_{\text{qb}} && \text{(Induction Hypothesis)} \\ \llbracket U_2 \rrbracket_{\text{qb}} &= \llbracket \text{ternary_gates_del}(U_2) \rrbracket_{\text{qb}} && \text{(Induction Hypothesis)} \\ \llbracket \text{ternary_gates_del}(U) \rrbracket_{\text{qb}} &= \llbracket \text{ternary_gates_del}(U_1) \text{---} \text{ternary_gates_del}(U_2) \rrbracket_{\text{qb}} && \text{(Definition 3.2.23)} \\ &= \llbracket \text{ternary_gates_del}(U_2) \rrbracket_{\text{qb}} \circ \llbracket \text{ternary_gates_del}(U_1) \rrbracket_{\text{qb}} && \text{(Definition 3.1.3)} \\ &= \llbracket U_2 \rrbracket_{\text{qb}} \circ \llbracket U_1 \rrbracket_{\text{qb}} && \text{(By Induction Hypothesis)} \\ &= \llbracket U_1 \text{---} U_2 \rrbracket_{\text{qb}} && \text{(Definition 3.1.3)} \\ &= \llbracket U \rrbracket_{\text{qb}}. && \square \end{aligned}$$

Predicate 3.2.25 (No Ternary Gates). Let U be a unitary circuit of type Qb such that $\text{parallel_free}(U) = \text{true}$. The predicate $\text{ternary_gates_free}(U)$ returns true if and only if there are no ternary gates in U . Formally, $\text{ternary_gates_free}(U)$ is defined recursively as follows:

$$\text{ternary_gates_free}(U) := \begin{cases} \text{ternary_gates_free}(U_1) \wedge \text{ternary_gates_free}(U_2) & \text{if } U = \text{Sequence}(U_1, U_2), \\ \text{ternary_gates_free}(U_1) & \text{if } U = \text{Place}(U_1, _), \\ \text{ternary_gates_free}(U_1) & \text{if } U = \text{Cont}(U_1, _, _), \\ \text{false} & \text{if } U \text{ is a ternary gate,} \\ \text{true} & \text{else.} \end{cases}$$

Lemme 3.2.26 (No Ternary Gates). Let U be a unitary circuit of type Qb, such that $\text{parallel_free}(U) = \text{true}$, then $\text{ternary_gates_free}(\text{ternary_gates_del}(U)) = \text{true}$.

Proof. Proof by structural induction on U .

1. If $U = \text{Toffoli}(co_1, co_2, ta)$, then

$$\begin{aligned} & \text{ternary_gates_free}(\text{ternary_gates_del}(U)) \\ &= \text{ternary_gates_free}(\text{toffoli_decomp}(co_1, co_2, ta)) && \text{(Definition 3.2.23)} \\ &= \text{true}. && \text{(Definition 3.2)} \end{aligned}$$

2. If $U = \text{Bricks_Toffoli}$, then

$$\begin{aligned} & \text{ternary_gates_free}(\text{ternary_gates_del}(U)) \\ &= \text{ternary_gates_free}(\text{bricks_toffoli_decomp}) && \text{(Definition 3.2.23)} \\ &= \text{true}. && \text{(Definition 3.1)} \end{aligned}$$

3. If $U = \text{Fredkin}(co, ta_1, ta_2)$, then

$$\begin{aligned} & \text{ternary_gates_free}(\text{ternary_gates_del}(U)) \\ &= \text{ternary_gates_free}(\text{fredkin_decomp}(co, ta_1, ta_2)) && \text{(Definition 3.2.23)} \\ &= \text{true}. && \text{(Definition 3.5)} \end{aligned}$$

4. If $U = \text{Bricks_Fredkin}$, then

$$\begin{aligned} & \text{ternary_gates_free}(\text{ternary_gates_del}(U)) \\ &= \text{ternary_gates_free}(\text{bricks_fredkin_decomp}) && \text{(Definition 3.2.23)} \\ &= \text{true}. && \text{(Definition 3.6)} \end{aligned}$$

5. If $U = \text{Place}(U', ta)$, then

$$\begin{aligned} & \text{ternary_gates_free}(\text{ternary_gates_del}(U')) = \text{true} && \text{(Induction Hypothesis)} \\ & \text{ternary_gates_free}(\text{ternary_gates_del}(U)) \\ &= \text{ternary_gates_free}(\text{ternary_gates_del}(U')) && \text{(Definition 3.2.23)} \\ &= \text{true}. && \text{(By Induction Hypothesis)} \end{aligned}$$

6. If $U = \text{Cont}(U', co, ta)$, then

$$\begin{aligned} & \text{ternary_gates_free}(\text{ternary_gates_del}(U')) = \text{true} && \text{(Induction Hypothesis)} \\ & \text{ternary_gates_free}(\text{ternary_gates_del}(U)) \\ &= \text{ternary_gates_free}(\text{ternary_gates_del}(U')) && \text{(Definition 3.2.23)} \\ &= \text{true}. && \text{(By Induction Hypothesis)} \end{aligned}$$

7. If $U = U_1 \text{---} U_2$, then

$$\begin{aligned}
& \text{ternary_gates_free}(\text{ternary_gates_del}(U_1)) = \text{true} && \text{(Induction Hypothesis)} \\
& \text{ternary_gates_free}(\text{ternary_gates_del}(U_2)) = \text{true} && \text{(Induction Hypothesis)} \\
& \text{ternary_gates_free}(\text{ternary_gates_del}(U)) \\
& \quad = \text{ternary_gates_free}(\text{ternary_gates_del}(U_1) && \text{(Definition 3.2.23)} \\
& \quad \quad \text{--- ternary_gates_free}(\text{ternary_gates_del}(U_2))) \\
& \quad = \text{ternary_gates_free}(\text{ternary_gates_del}(U_1)) && \text{(Definition 3.2.25)} \\
& \quad \quad \wedge \text{ternary_gates_free}(\text{ternary_gates_del}(U_2)) \\
& \quad = \text{true} \wedge \text{true}. && \text{(By Induction Hypothesis)}
\end{aligned}$$

□

Lemme 3.2.27 (No Spatial Composition - Preservation). Let U be a circuit of type Qb , such that $\text{parallel_free}(U) = \text{true}$, then $\text{parallel_free}(\text{ternary_gates_del}(U)) = \text{true}$.

Sketch. The proof is trivial by induction on U , and it suffices to verify that ternary_gates_del does not introduce spatial composition. For readability, the details are omitted here. □

3.2.5 Sub-Circuit Placement

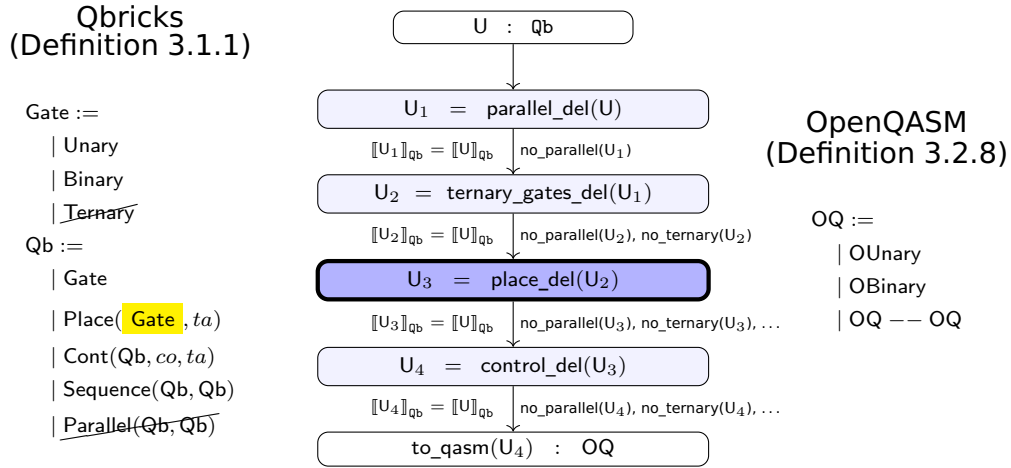


Figure 3.8: Removing Sub-Circuit Placement.

OQ does not include sub-circuit placement. Therefore, we must eliminate it from Qb while maintaining semantic equivalence. Figure 3.8 emphasizes this step in blue ($U_3 = \text{place_del}(U_2)$). This step preserves the semantics ($[U_3]_{qb} = [U]_{qb}$). It removes sub-circuit placement, ensured by $\text{atomic_place}(U_3)$. It also preserves $\text{no_parallel}(U_3)$ and $\text{no_ternary}(U_3)$. Figure 3.9 illustrates the removal of the sub-circuit placement by an example. On the left, we show sub-circuit placements. In the middle, we show the

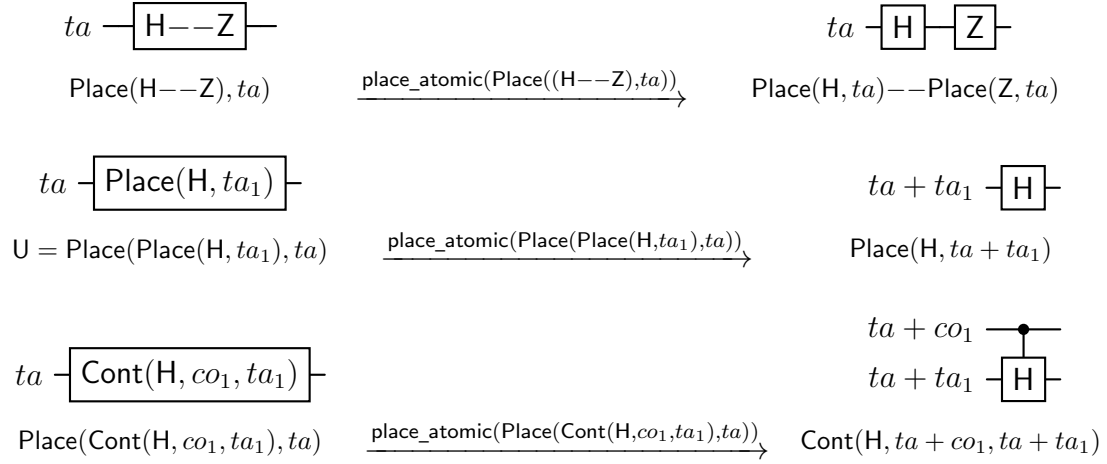


Figure 3.9: Examples of Removing Sub-Circuit Placement.

application of the removal function. On the right, we show the resulting circuit without sub-circuit placement. First, at the top, the sub-circuit is a temporal composition of H and Z. The placement is pushed on the leaves. Then, in the centre, the sub-circuit is another placement. The imbricate placement is removed, and the indices of the target qubit are updated ($ta + ta_1$). Finally, at the bottom, the sub-circuit is a sub-circuit control. The principle is the same as the imbricate placement. We update the target indices and remove the placement.

The removal of sub-circuit placements occurs in two stages. First, all nested placements are recursively removed (Section 3.2.5.1). Then, all placement controls are recursively removed (Section 3.2.5.2). This process is carried out in two stages to simplify the proof, particularly in cases involving nested controls and placements of sub-circuit.

Remark 3.2.28 (Specialized Sub-Circuit Definition). In this section, we specialize the notion of sub-circuit (Definition 4.1.1) by defining that a valid sub-circuit must contain at least two gates.

3.2.5.1 Removing Parametrised Sub-Circuit Placement

We begin by defining the function that removes all nested placements recursively (Definition 3.2.29), **without handling the case of imbrications between control and placement**. Then we ensure that this transformation preserves the semantics in Lemma 3.2.30. Next, we ensure and prove in Lemma 3.2.32 that all the nested placements are removed. Finally, we ensure that this transformation does not introduce spatial composition or ternary gates (Lemma 3.2.33).

Definition 3.2.29 (Removing Parametrised Sub-Circuit Placement). The following function recursively removes sub-circuit (Remark 3.2.28) placements:

$$\text{placed_atomic}(U, ta) := \begin{cases} \text{placed_atomic}(U', ta + ta') & \text{if } U = \text{Place}(U', ta'), \\ \text{Cont}(\text{placed_atomic}(U', 0), co' + ta, ta' + ta) & \text{if } U = \text{Cont}(U', co', ta'), \\ \text{placed_atomic}(U_1) & \text{if } U = U_1 \text{---} U_2, \\ \text{---placed_atomic}(U_2) & \\ \text{Place}(U, ta) & \text{else.} \end{cases}$$

Lemme 3.2.30 (Semantics Preservation under Removing Parametrised Sub-Circuit Placement). Let U be a circuit of type Qb , such that

$$\text{parallel_free}(U) = \text{ternary_gates_free}(U) = \text{true},$$

then $\llbracket \text{placed_atomic}(U, ta) \rrbracket_{Qb} = \llbracket \text{Place}(U, ta) \rrbracket_{Qb}$.

Proof. Proof by structural induction on U :

1. If $U = \text{Place}(U', ta')$, then

$$\begin{aligned} \llbracket \text{placed_atomic}(U', ta + ta') \rrbracket_{Qb} &= \llbracket \text{Place}(U', ta + ta') \rrbracket_{Qb} && \text{(Induction Hypothesis)} \\ \llbracket \text{placed_atomic}(U, ta) \rrbracket_{Qb} &= \llbracket \text{placed_atomic}(U', ta + ta') \rrbracket_{Qb} && \text{(Definition 3.2.29)} \\ &= \llbracket \text{Place}(U', ta + ta') \rrbracket_{Qb} && \text{(By Induction Hypothesis)} \\ &= \llbracket \text{Place}(U, ta) \rrbracket_{Qb} && \text{(Lemma 4)} \end{aligned}$$

2. If $U = \text{Cont}(U', co', ta')$, then

$$\begin{aligned} \llbracket \text{placed_atomic}(U', 0) \rrbracket_{Qb} &= \llbracket \text{Place}(U', 0) \rrbracket_{Qb} && \text{(Induction Hypothesis)} \\ \llbracket \text{placed_atomic}(U, ta) \rrbracket_{Qb} &= \llbracket \text{Cont}(\text{placed_atomic}(U', 0), co' + ta, ta' + ta) \rrbracket_{Qb} && \text{(Definition 3.2.29)} \\ &= \llbracket \text{Cont}(\text{Place}(U', 0), co' + ta, ta' + ta) \rrbracket_{Qb} && \text{(By Induction Hypothesis and Lemma 9)} \\ &= \llbracket \text{Cont}(U', co' + ta, ta' + ta) \rrbracket_{Qb} && \text{(Definition 3.1.6)} \\ &= \llbracket \text{Place}(\text{Cont}(U', co', ta'), ta) \rrbracket_{Qb} && \text{(Lemma 5)} \\ &= \llbracket \text{Place}(U, ta) \rrbracket_{Qb} \end{aligned}$$

3. If $U = U_1 \dashv\dashv U_2$, then

$$\begin{aligned}
\llbracket \text{placed_atomic}(U_1, ta) \rrbracket_{\text{qb}} &= \llbracket \text{Place}(U_1, ta) \rrbracket_{\text{qb}} && \text{(Induction Hypothesis)} \\
\llbracket \text{placed_atomic}(U_2, ta) \rrbracket_{\text{qb}} &= \llbracket \text{Place}(U_2, ta) \rrbracket_{\text{qb}} && \text{(Induction Hypothesis)} \\
\llbracket \text{placed_atomic}(U, ta) \rrbracket_{\text{qb}} &= \llbracket \text{placed_atomic}(U_1, ta) \dashv\dashv \text{placed_atomic}(U_2, ta) \rrbracket_{\text{qb}} && \text{(Definition 3.2.29)} \\
&= \llbracket \text{placed_atomic}(U_2, ta) \rrbracket_{\text{qb}} && \text{(Definition 3.1.3)} \\
&\quad \circ \llbracket \text{placed_atomic}(U_1, ta) \rrbracket_{\text{qb}} \\
&= \llbracket \text{Place}(U_2, ta) \rrbracket_{\text{qb}} \circ \llbracket \text{Place}(U_1, ta) \rrbracket_{\text{qb}} && \text{(By Induction Hypothesis)} \\
&= \llbracket \text{Place}(U_1 \dashv\dashv U_2, ta) \rrbracket_{\text{qb}} && \text{(Definition 3.1.3)} \\
&= \llbracket \text{Place}(U, ta) \rrbracket_{\text{qb}}
\end{aligned}$$

4. Else $\llbracket \text{placed_atomic}(U, ta) \rrbracket_{\text{qb}} = \llbracket \text{Place}(U, ta) \rrbracket_{\text{qb}}$. □

Predicate 3.2.31 (No Sub-Circuit Placement). This predicate returns `true` if there is no sub-circuit placement and `false` otherwise.

$$\text{atomic_place}(U) := \begin{cases} \text{true} & \text{if } U = \text{Place}(U', _) \text{ and } U' \text{ is a gate,} \\ \text{false} & \text{if } U = \text{Place}(U', _), \\ \text{atomic_place}(U') & \text{if } U = \text{Cont}(U', _), \\ \text{atomic_place}(U_1) & \text{if } U = U_1 \dashv\dashv U_2, \\ \quad \wedge \text{atomic_place}(U_2). \end{cases}$$

Lemme 3.2.32 (No Parametrised Sub-Circuit Placement). Let U be a circuit of type Qb , such that $\text{parallel_free}(U) = \text{ternary_gates_free}(U) = \text{true}$, and let ta be a placement parameter. The transformation placed_atomic guarantees that there are no sub-circuit (Remark 3.2.28) placements. Formally, $\text{atomic_place}(\text{placed_atomic}(U, ta)) = \text{true}$.

Proof. Proof by structural induction on U :

1. If $U = \text{Place}(U', ta')$, then

$$\begin{aligned}
\text{atomic_place}(\text{placed_atomic}(U', ta + ta')) &= \text{true} && \text{(Induction Hypothesis)} \\
\text{atomic_place}(\text{placed_atomic}(U, ta)) &= \text{atomic_place}(\text{placed_atomic}(U', ta + ta')) && \text{(Definition 3.2.29)} \\
&= \text{true} && \text{(By Induction Hypothesis)}
\end{aligned}$$

2. If $U = \text{Cont}(U', co, ta')$, then

$$\begin{aligned}
\text{atomic_place}(\text{placed_atomic}(U', 0)) &= \text{true} && \text{(Induction Hypothesis)} \\
\text{atomic_place}(\text{placed_atomic}(U, ta)) &= \text{atomic_place}(\text{Cont}(\text{placed_atomic}(U', 0), co + ta, ta' + ta)) && \text{(Definition 3.2.29)} \\
&= \text{true} && \text{(By Induction Hypothesis)}
\end{aligned}$$

3. If $U = U_1 \text{---} U_2$, then

$$\begin{aligned}
& \text{atomic_place}(\text{placed_atomic}(U_1, ta)) = \text{true} && \text{(Induction Hypothesis)} \\
& \text{atomic_place}(\text{placed_atomic}(U_2, ta)) = \text{true} && \text{(Induction Hypothesis)} \\
& \text{atomic_place}(\text{placed_atomic}(U, ta)) \\
& \quad = \text{atomic_place}(\text{placed_atomic}(U_1, ta) \text{---} \text{placed_atomic}(U_2, ta)) && \text{(Definition 3.2.29)} \\
& \quad = \text{atomic_place}(\text{placed_atomic}(U_1, ta) \wedge \text{placed_atomic}(U_2, ta)) && \text{(Predicate 3.2.31)} \\
& \quad = \text{true} && \text{(By Induction Hypothesis)}
\end{aligned}$$

4. Else: $\text{atomic_place}(\text{placed_atomic}(U)) = \text{true}$. $\square$

Lemme 3.2.33 (No Spatial Composition and Ternary Gates - Preservation). Let U be of type Qb , such that $\text{parallel_free}(U) = \text{true}$ and $\text{ternary_gates_free}(U) = \text{true}$, then $\text{parallel_free}(\text{placed_atomic}(U)) = \text{true}$ and $\text{ternary_gates_free}(\text{placed_atomic}(U)) = \text{true}$.

Sketch Proof. The proof is trivial by induction on U and verifying that placed_atomic does not introduce spatial composition or ternary gates. $\square$

3.2.5.2 Sub-Circuit Placement

We begin by defining the function that removes all nested placements recursively (Definition 3.2.34), **with handling the case of imbrications between control and placement** of sub-circuit. Then we ensure that this transformation preserves the semantics in Lemma 3.2.35. Next, we ensure and prove in Lemma 3.2.36 that all the nested placements are removed. Finally, we ensure that this transformation does not introduce spatial composition or ternary gates (Lemma 3.2.37).

Definition 3.2.34 (Removing Sub-Circuit Placement). The following function recursively removes all sub-circuit (Remark 3.2.28) placements in a circuit:

$$\text{place_atomic}(U) := \begin{cases} \text{placed_atomic}(U', ta) & \text{(Definition 3.2.29) if } U = \text{Place}(U', ta), \\ \text{Cont}(\text{place_atomic}(U'), co, ta) & \text{if } U = \text{Cont}(U', co, ta), \\ \text{place_atomic}(U_1) & \text{if } U = U_1 \text{---} U_2, \\ \text{---} \text{place_atomic}(U_2) & \\ U & \text{else.} \end{cases}$$

Lemme 3.2.35 (Semantics Preservation under Removing Sub Circuit Placement). Let U be a unitary circuit of type Qb , such that $\text{parallel_free}(U) = \text{ternary_gates_free}(U) = \text{true}$, then $\llbracket \text{place_atomic}(U) \rrbracket_{Qb} = \llbracket U \rrbracket_{Qb}$.

Proof. Proof by structural induction on U .

1. If $U = \text{Place}(U', ta)$, then

$$\begin{aligned}
& \llbracket \text{place_atomic}(U) \rrbracket_{\text{Qb}} \\
&= \llbracket \text{placed_atomic}(U', ta) \rrbracket_{\text{Qb}} && \text{(Definition 3.2.34)} \\
&= \llbracket \text{Place}(U', ta) \rrbracket_{\text{Qb}} && \text{(Lemma 3.2.30)} \\
&= \llbracket U \rrbracket_{\text{Qb}}
\end{aligned}$$

2. If $U = \text{Cont}(U', co, ta)$

$$\begin{aligned}
& \llbracket \text{place_atomic}(U') \rrbracket_{\text{Qb}} = \llbracket U' \rrbracket_{\text{Qb}} && \text{(Induction Hypothesis)} \\
& \llbracket \text{place_atomic}(U) \rrbracket_{\text{Qb}} \\
&= \llbracket \text{Cont}(\text{place_atomic}(U'), co, ta) \rrbracket_{\text{Qb}} && \text{(Definition 3.2.34)} \\
&= \llbracket \text{Cont}(U', co, ta) \rrbracket_{\text{Qb}} && \text{(By Induction Hypothesis and Lemma 8)} \\
&= \llbracket U \rrbracket_{\text{Qb}} .
\end{aligned}$$

3. $U = U_1 \text{--} U_2$

$$\begin{aligned}
& \llbracket \text{place_atomic}(U_1) \rrbracket_{\text{Qb}} = \llbracket U_1 \rrbracket_{\text{Qb}} && \text{(Induction Hypothesis)} \\
& \llbracket \text{place_atomic}(U_2) \rrbracket_{\text{Qb}} = \llbracket U_2 \rrbracket_{\text{Qb}} && \text{(Induction Hypothesis)} \\
& \llbracket \text{place_atomic}(U) \rrbracket_{\text{Qb}} \\
&= \llbracket \text{place_atomic}(U_1) \text{--} \text{place_atomic}(U_2) \rrbracket_{\text{Qb}} && \text{(Definition 3.2.34)} \\
&= \llbracket \text{place_atomic}(U_2) \rrbracket_{\text{Qb}} \circ \llbracket \text{place_atomic}(U_1) \rrbracket_{\text{Qb}} && \text{(Definition 3.1.3)} \\
&= \llbracket U_2 \rrbracket_{\text{Qb}} \circ \llbracket U_1 \rrbracket_{\text{Qb}} && \text{(By Induction Hypothesis)} \\
&= \llbracket U_1 \text{--} U_2 \rrbracket_{\text{Qb}} && \text{(Definition 3.1.3)} \\
&= \llbracket U \rrbracket_{\text{Qb}} . && \square
\end{aligned}$$

Lemme 3.2.36 (No Sub-Circuit Placement). The transformation `place_atomic` guarantees the absence of sub-circuit (Remark 3.2.28) placements. Formally, let U be a circuit of type Qb such that $\text{parallel_free}(U) = \text{ternary_gates_free}(U) = \text{true}$, then

$$\text{atomic_place}(\text{place_atomic}(U)) = \text{true}.$$

Proof. Proof by structural induction on U :

1. If $U = \text{Place}(U', ta)$, then

$$\begin{aligned}
& \text{atomic_place}(\text{place_atomic}(U)) \\
&= \text{atomic_place}(\text{placed_atomic}(U', ta)) && \text{(Definition 3.2.34)} \\
&= \text{true} && \text{(Lemma 3.2.32)}
\end{aligned}$$

2. If $U = \text{Cont}(U', co, ta)$, then

$$\begin{aligned}
& \text{atomic_place}(\text{place_atomic}(U')) = \text{true} && \text{(Induction Hypothesis)} \\
& \text{atomic_place}(\text{place_atomic}(U)) \\
& \quad = \text{atomic_place}(\text{Cont}(\text{place_atomic}(U'), co, ta)) && \text{(Definition 3.2.34)} \\
& \quad = \text{true} && \text{(By Induction Hypothesis)}
\end{aligned}$$

3. If $U = U_1 \text{---} U_2$, then

$$\begin{aligned}
& \text{atomic_place}(\text{place_atomic}(U_1)) = \text{true} && \text{(Induction Hypothesis)} \\
& \text{atomic_place}(\text{place_atomic}(U_2)) = \text{true} && \text{(Induction Hypothesis)} \\
& \text{atomic_place}(\text{place_atomic}(U)) \\
& \quad = \text{atomic_place}(\text{place_atomic}(U_1) \\
& \quad \quad \text{--- place_atomic}(U_2)) && \text{(Definition 3.2.34)} \\
& \quad = \text{atomic_place}(\text{place_atomic}(U_1)) && \text{(Predicate 3.2.31)} \\
& \quad \quad \wedge \text{atomic_place}(\text{place_atomic}(U_2)) \\
& \quad = \text{true} && \text{(Induction Hypothesis)}
\end{aligned}$$

4. Else: $\text{atomic_place}(\text{place_atomic}(U)) = \text{true}$. □

Lemme 3.2.37 (No Spatial Composition and Ternary Gates - Preservation). Let U be of type Qb , such that $\text{parallel_free}(U) = \text{true}$ and $\text{ternary_gates_free}(U) = \text{true}$, then $\text{parallel_free}(\text{place_atomic}(U)) = \text{true}$ and $\text{ternary_gates_free}(\text{place_atomic}(U)) = \text{true}$.

Proof Sketch. The proof is trivial by induction on U and verifying that placed_atomic does not introduce spatial composition or ternary gates. □

3.2.6 Sub-Circuit Control

OQ does not include sub-circuit control. Therefore, we must eliminate it from Qb while maintaining semantic equivalence. Figure 3.10 emphasizes this step in blue ($U_4 = \text{control_del}(U_3)$). This step preserves the semantics ($\llbracket U_4 \rrbracket_{Qb} = \llbracket U \rrbracket_{Qb}$). It removes sub-circuit control, ensured by $\text{cont_depth}(\text{control_del}(U)) = 0$. It also preserves $\text{no_parallel}(U_3)$, $\text{no_ternary}(U_3)$, and $\text{atomic_place}(U_3)$. Figure 3.11 illustrates the removal of the controlled temporal composition by an example. On the first row, on the left, we show controlled temporal composition. In the middle, we show the application of the removal function. On the right, we show the resulting circuit without controlled temporal composition, but with controlled gates. The only control gate present in OQ is a control not, so we need to decompose it. The decomposition is on the second row. On the left, we show the recursive application of the function to remove the control. On the right, we show the decomposition result. Figure 3.12 illustrates the removal of nested control by an example. On the first row, on the left, we show a nested control.

Qbricks
(Definition 3.1.1)

Gate :=

- | Unary
- | Binary
- | ~~Ternary~~

Qb :=

- | Gate
- | Place(**Gate**, *ta*)
- | ~~Cont(Qb, *co*, *ta*)~~
- | ~~Sequence(Qb, Qb)~~
- | ~~Parallel(Qb, Qb)~~

OpenQASM
(Definition 3.2.8)

OQ :=

- | OUnary
- | OBinary
- | OQ -- OQ

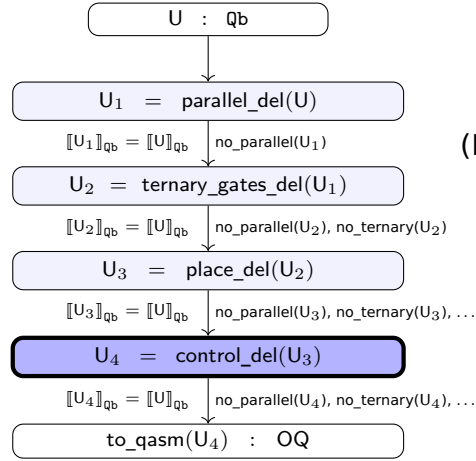


Figure 3.10: Removing Sub-Circuit Control.

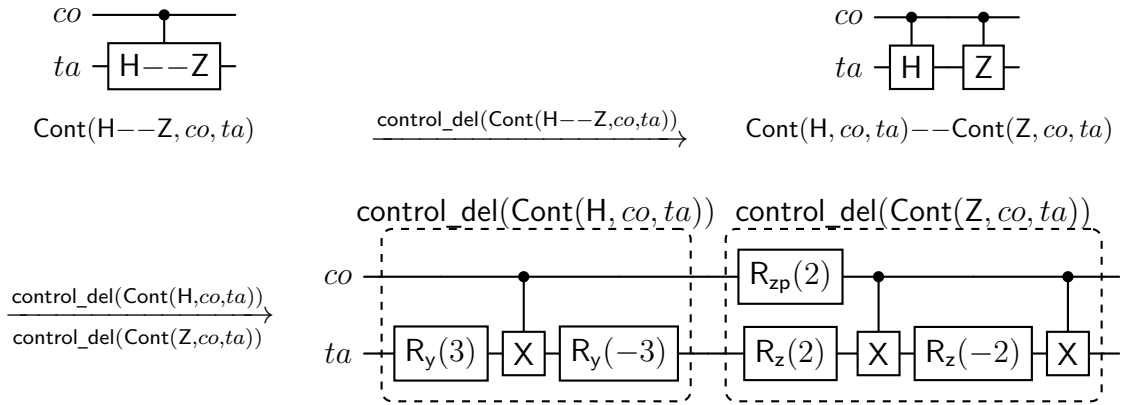


Figure 3.11: Example Illustrating the Removal of Controlled Temporal Composition.

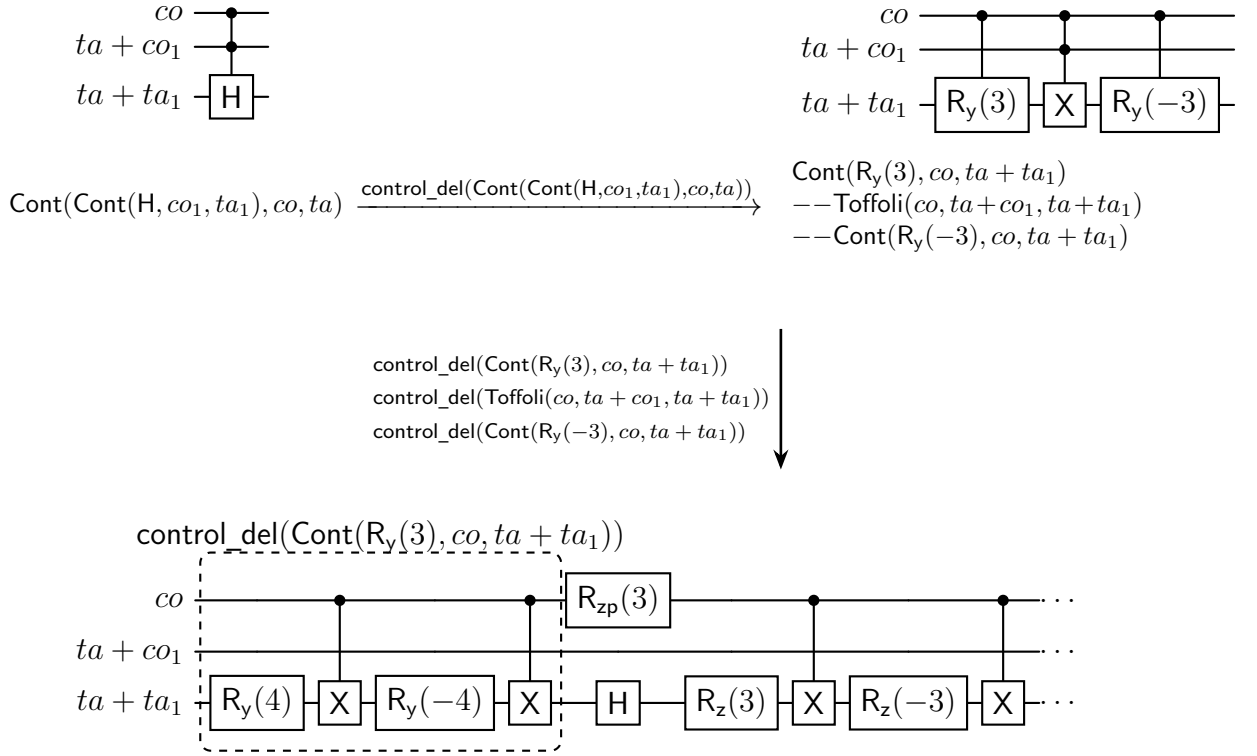


Figure 3.12: Example Illustrating the Removal of Nested Control.

In the middle, we show the application of the removal function. On the right, we show the resulting circuit without nested control, but with controlled gates not in OQ, so we need to decompose them. The decomposition is on the second row. On the left, we show the recursive application of the function to remove the control. On the right, we show the partial result of the decomposition.

The removal of sub-circuit control involves two functions. The first function decomposes the control gates (Section 3.2.6.1). The second function recursively calls the first to decompose all control gates, except for the Control Not gate, which is a primitive of OQ (Section 3.2.6.2). The resulting circuit is of Qb type, devoid of spatial composition, ternary gates, sub-circuit placement, and control operations. Thus, the result matches the desired OQ type.

Definition 3.2.38 (Control Nesting Depth). Let U be a circuit of type Qb. We define the function cont_depth that calculates the nesting depth of the Cont instruction in the circuit U . Formally, with $\max(i, j)$ defined as $\max(i, j) = \begin{cases} j & \text{if } i < j, \\ i & \text{else} \end{cases}$, the function

$\text{cont_depth}(U)$ is defined recursively as follows:

$$\text{cont_depth}(U) := \begin{cases} \text{cont_depth}(U_1) + 1 & \text{if } U = \text{Cont}(U_1, _, _), \\ \text{cont_depth}(U_1) & \text{if } U = \text{Place}(U_1, _), \\ \max(\text{cont_depth}(U_1), \text{cont_depth}(U_2)) & \text{if } U = U_1 - U_2, \\ 0 & \text{else.} \end{cases}$$

3.2.6.1 Controls Decomposition

In this section, we define the function control_del_step (Definition 3.2.39) that suppresses a sub-circuit control by decomposing it into Unary gates and Binary gates (Def. 3.1.1). We give the lemma 3.2.41, which ensures the preservation of the semantics by its application, and we report the intermediate lemmas, without proving them (they were proven in Qbricks [59]). We then give the lemma 3.2.42, which guarantees that the sub-circuit control has been suppressed.

Definition 3.2.39 (Controls Decomposition). We do not reproduce definitions of the functions prefixed by cont here; they are available in the file: https://github.com/Qbricks/qbricks.github.io/blob/main/Qbricks/circuits_equiv.mlw. Let U be of type Qb such that: $\text{parallel_free}(U) = \text{ternary_gates_free}(U) = \text{atomic_place}(U) = \text{true}$, and $\text{cont_depth}(U) = 0$, we define control_del_step in Figure 3.13.

The following lemmas, extracted from https://github.com/Qbricks/qbricks.github.io/blob/main/Qbricks/circuits_equiv.mlw, are required for subsequent developments. All have been formally verified in Qbricks.

Lemme 3.2.40 (Sub-Circuit Control Equivalence Lemmas).

$$\llbracket \text{control_del_step}(\text{Phase}(k), co, ta) \rrbracket_{\text{Qb}} = \llbracket \text{Cont}(\text{Phase}(k), co, ta) \rrbracket_{\text{Qb}} \quad (3.9)$$

$$\llbracket \text{control_del_step}(\text{R}_x(k), co, ta) \rrbracket_{\text{Qb}} = \llbracket \text{Cont}(\text{R}_x(k), co, ta) \rrbracket_{\text{Qb}} \quad (3.10)$$

$$\llbracket \text{control_del_step}(\text{R}_y(k), co, ta) \rrbracket_{\text{Qb}} = \llbracket \text{Cont}(\text{R}_y(k), co, ta) \rrbracket_{\text{Qb}} \quad (3.11)$$

$$\llbracket \text{control_del_step}(\text{R}_z(k), co, ta) \rrbracket_{\text{Qb}} = \llbracket \text{Cont}(\text{R}_z(k), co, ta) \rrbracket_{\text{Qb}} \quad (3.12)$$

$$\llbracket \text{control_del_step}(\text{R}_{zp}(k), co, ta) \rrbracket_{\text{Qb}} = \llbracket \text{Cont}(\text{R}_{zp}(k), co, ta) \rrbracket_{\text{Qb}} \quad (3.13)$$

$$\llbracket \text{control_del_step}(\text{H}, co, ta) \rrbracket_{\text{Qb}} = \llbracket \text{Cont}(\text{H}, co, ta) \rrbracket_{\text{Qb}} \quad (3.14)$$

$$\llbracket \text{control_del_step}(\text{S}, co, ta) \rrbracket_{\text{Qb}} = \llbracket \text{Cont}(\text{S}, co, ta) \rrbracket_{\text{Qb}} \quad (3.15)$$

$$\llbracket \text{control_del_step}(\text{T}, co, ta) \rrbracket_{\text{Qb}} = \llbracket \text{Cont}(\text{T}, co, ta) \rrbracket_{\text{Qb}} \quad (3.16)$$

$$\llbracket \text{control_del_step}(\text{Y}, co, ta) \rrbracket_{\text{Qb}} = \llbracket \text{Cont}(\text{Y}, co, ta) \rrbracket_{\text{Qb}} \quad (3.17)$$

$$\llbracket \text{control_del_step}(\text{Z}, co, ta) \rrbracket_{\text{Qb}} = \llbracket \text{Cont}(\text{Z}, co, ta) \rrbracket_{\text{Qb}} \quad (3.18)$$

Proof. The proofs of Lemmas 3.9, 3.10, 3.11, 3.12, 3.13, 3.14, 3.15, 3.16, 3.17, and 3.18 are not included in this work. The proofs were conducted using Qbricks, and they can be found with the file https://github.com/Qbricks/qbricks.github.io/blob/main/Qbricks/circuits_equiv.mlw. $\square$

control_del_step(U, co, ta) :=	control_del_step(U ₁ , co, ta)	if U = U ₁ -- U ₂
	--control_del_step(U ₂ , co, ta)	
	control_del_step(U', co, ta + ta')	if U = Place(U', ta')
	R _{zp} (k, co)	if U = Phase(k)
	control_del_step(H, co, ta) -- crz_decomp(k, co, ta)	if U = R _x (k)
	--crz_decomp(k, co, ta) (Def. 3.2.13)	
	--control_del_step(H, co, ta)	
	Place(R _y (k + 1), ta) -- Cnot(co, t)	if U = R _y (k)
	--Place(R _y (-(k + 1)), ta) -- Cnot(co, t)	and k ≥ 0
	Place(R _y (k - 1), ta) -- Cnot(co, t)	if U = R _y (k)
	--Place(R _y (-(k - 1)), ta) -- Cnot(co, ta)	and k < 0
	crz_decomp(k, co, ta) (Def. 3.2.13)	if U = R _z (k)
	crzp_decomp(k, co, ta) (Def. 3.2.15)	if U = R _{zp} (k)
	crzp_decomp(2, co, ta) (Def. 3.2.15)	if U = S
	crzp_decomp(3, co, ta) (Def. 3.2.15)	if U = T
	Place(R _y (3), ta) -- Bricks_Cnot(co, ta)	if U = H
	--Place(R _y (-3), ta)	
	Phase(2) -- control_del_step(H, co, ta)	if U = X
	--crz_decomp(1, co, ta) (Def. 3.2.13)	
	--crz_decomp(k, co, ta)	
	--control_del_step(H, co, ta)	
	Phase(2) -- Place(R _y (2), ta) -- Cnot(co, t)	if U = Y
	--Place(R _y (-2), ta) -- Cnot(co, t)	and k ≥ 0
	crzp_decomp(1, co, ta) (Def. 3.2.15)	if U = Z
	toffoli_decomp(co, co' + ta, ta' + ta)	if U = Cnot(co', ta')
	fredkin_decomp(co, ta1 + ta, ta2 + ta)	if U = Swap(ta1, ta2)
	bricks_toffoli_decomp(co, ta, ta + 1)	if U = Bricks_Cnot
	bricks_fredkin_decomp(co, ta, ta + 1)	if U = Bricks_Swap
	U	else

Figure 3.13: Definition of Controls Decomposition

Lemme 3.2.41 (Semantics Preservation under Control Decomposition). Let U be a circuit of type Qb , such that $\text{parallel_free}(U) = \text{ternary_gates_free}(U) = \text{atomic_place}(U) = \text{true}$, and $\text{cont_depth}(U) = 0$, then $\llbracket \text{control_del_step}(U, co, ta) \rrbracket_{Qb} = \llbracket \text{Cont}(U, co, ta) \rrbracket_{Qb}$.

Proof. Proof by structural induction on U :

1. If $U = \text{Place}(U', ta')$:

$$\begin{aligned}
 & \llbracket \text{control_del_step}(U, co, ta) \rrbracket_{Qb} \\
 &= \llbracket \text{control_del_step}(U', co, ta + ta') \rrbracket_{Qb} \\
 &= \llbracket \text{Cont}(U', co, ta + ta') \rrbracket_{Qb} && \text{(Induction Hypothesis)} \\
 &= \llbracket \text{Cont}(\text{Place}(U', ta'), co, ta) \rrbracket_{Qb} && \text{(Lemma 8)} \\
 &= \llbracket \text{Cont}(U, co, ta) \rrbracket_{Qb}
 \end{aligned}$$

2. If $U = U_1 \text{---} U_2$:

$$\begin{aligned}
 & \llbracket \text{control_del_step}(U_1, co, ta) \rrbracket_{Qb} = \llbracket \text{Cont}(U_1, co, ta) \rrbracket_{Qb} && \text{(Induction Hypothesis)} \\
 & \llbracket \text{control_del_step}(U, co, ta) \rrbracket_{Qb} \\
 &= \llbracket \text{control_del_step}(U_1, co, ta) \rrbracket_{Qb} \\
 & \quad \text{---} \llbracket \text{control_del_step}(U_2, co, ta) \rrbracket_{Qb} && \text{(Definition 3.2.39)} \\
 &= \llbracket \text{control_del_step}(U_2, co, ta) \rrbracket_{Qb} && \text{(Definition 3.1.3)} \\
 & \quad \circ \llbracket \text{control_del_step}(U_1, co, ta) \rrbracket_{Qb} \\
 &= \llbracket \text{Cont}(U_2, co, ta) \rrbracket_{Qb} \circ \llbracket \text{Cont}(U_1, co, ta) \rrbracket_{Qb} && \text{(By Induction Hypothesis)} \\
 &= \llbracket \text{Cont}(U_1, co, ta) \text{---} \text{Cont}(U_2, co, ta) \rrbracket_{Qb} && \text{(Definition 3.1.3)} \\
 &= \llbracket \text{Cont}(U, co, ta) \rrbracket_{Qb}. && \text{(Lemma 7)}
 \end{aligned}$$

3. Else, from lemmas 3.3, 3.4, 3.7, 3.8, 3.9, 3.10, 3.11, 3.12, 3.13, 3.14, 3.15, 3.16, 3.17, and 3.18, all basis cases are proven, thus

$$\llbracket \text{control_del_step}(U, co, ta) \rrbracket_{Qb} = \llbracket \text{Cont}(U, co, ta) \rrbracket_{Qb}. \quad \square$$

Lemme 3.2.42 (No Control under Controls Decomposition). Let U be a circuit of type Qb , such that $\text{parallel_free}(U) = \text{ternary_gates_free}(U) = \text{atomic_place}(U) = \text{true}$, and $\text{cont_depth}(U) = 0$, then $\text{cont_depth}(\text{control_del_step}(U, co, ta)) = 0$.

Proof. Proof by structural induction on U :

1. If $U = \text{Place}(U', ta)$, then

$$\begin{aligned}
 & \text{cont_depth}(\text{control_del_step}(U, co, ta)) \\
 &= \text{cont_depth}(\text{control_del_step}(U', co, ta + ta')) \\
 &= 0. && \text{(By Induction Hypothesis)}
 \end{aligned}$$

2. If $U = U_1 \dashv\dashv U_2$, then

$$\begin{aligned}
& \text{cont_depth}(\text{control_del_step}(U_1, co, ta)) = 0 && \text{(Induction Hypothesis)} \\
& \text{cont_depth}(\text{control_del_step}(U_2, co, ta)) = 0 && \text{(Induction Hypothesis)} \\
& \text{cont_depth}(\text{control_del_step}(U, co, ta)) \\
& \quad = \text{cont_depth}(\text{control_del_step}(U_1, co, ta) \dashv\dashv \text{control_del_step}(U_2, co, ta)) \\
& \quad = \max(\text{cont_depth}(\text{control_del_step}(U_1, co, ta)), \text{cont_depth}(\text{control_del_step}(U_2, co, ta))) && \text{(Definition 3.2.38)} \\
& \quad = 0. && \text{(By Induction Hypothesis)}
\end{aligned}$$

3. Else: $\text{cont_depth}(\text{control_del_step}(U, co, ta)) = 0$. □

3.2.6.2 Removing Controls

In this section, we define the function `control_del` that suppresses sub-circuit controls (Definition 3.2.43). We give the Lemma 3.2.44, which ensures the preservation of semantics, and the Lemma 3.2.45, which guarantees the suppression of all controls.

Definition 3.2.43 (Removing Controls). Let U be a Qb such that: $\text{parallel_free}(U) = \text{ternary_gates_free}(U) = \text{atomic_place}(U) = \text{true}$. We define `control_del` as follows:

$$\text{control_del}(U) := \begin{cases} \text{control_del_step}(U', co, ta) & \text{if } U = \text{Cont}(U', co, ta) \\ & \text{and } \text{cont_depth}(U') = 0 \\ \text{control_del_step}(\text{control_del}(U'), co, ta) & \text{if } U = \text{Cont}(U', co, ta) \\ \text{control_del}(U_1) \dashv\dashv \text{control_del}(U_2) & \text{if } U = U_1 \dashv\dashv U_2 \\ U & \text{else} \end{cases}$$

Lemme 3.2.44 (Semantics Preservation under Removing Controls). Let U be a circuit of type Qb, such that $\text{parallel_free}(U) = \text{ternary_gates_free}(U) = \text{atomic_place}(U) = \text{true}$, then $\llbracket \text{control_del}(U) \rrbracket_{\text{qb}} = \llbracket U \rrbracket_{\text{qb}}$.

Proof. We proceed by structural induction on U .

1. If $U = \text{Cont}(U', co, ta)$ and $\text{cont_depth}(U') > 0$, then

$$\begin{aligned}
& \llbracket \text{control_del}(U') \rrbracket_{\text{qb}} = \llbracket U' \rrbracket_{\text{qb}} && \text{(Induction Hypothesis)} \\
& \llbracket \text{control_del}(U) \rrbracket_{\text{qb}} \\
& \quad = \llbracket \text{control_del_step}(\text{control_del}(U'), co, ta) \rrbracket_{\text{qb}} && \text{(Definition 3.2.43)} \\
& \quad = \llbracket \text{Cont}(\text{control_del}(U'), co, ta) \rrbracket_{\text{qb}} && \text{(Lemma 3.2.41)} \\
& \quad = \llbracket \text{Cont}(U', co, ta) \rrbracket_{\text{qb}} && \text{(By Induction Hypothesis and Lemma 9)} \\
& \quad = \llbracket U \rrbracket_{\text{qb}}
\end{aligned}$$

2. If $U = \text{Cont}(U', co, ta)$ and $\text{cont_depth}(U') = 0$:

$$\begin{aligned}
& \llbracket \text{control_del}(U) \rrbracket_{Qb} \\
&= \llbracket \text{control_del_step}(U', co, ta) \rrbracket_{Qb} && \text{(Definition 3.2.43)} \\
&= \llbracket \text{Cont}(U', co, ta) \rrbracket_{Qb} && \text{(Lemma 3.2.41)} \\
&= \llbracket U \rrbracket_{Qb}
\end{aligned}$$

3. If $U = U_1 - U_2$:

$$\begin{aligned}
& \llbracket \text{control_del}(U_1) \rrbracket_{Qb} = \llbracket U_1 \rrbracket_{Qb} && \text{(Induction Hypothesis)} \\
& \llbracket \text{control_del}(U_2) \rrbracket_{Qb} = \llbracket U_2 \rrbracket_{Qb} && \text{(Induction Hypothesis)} \\
& \llbracket \text{control_del}(U) \rrbracket_{Qb} \\
&= \llbracket \text{control_del}(U_1) - \text{control_del}(U_2) \rrbracket_{Qb} && \text{(Definition 3.2.43)} \\
&= \llbracket \text{control_del}(U_2) \rrbracket_{Qb} \circ \llbracket \text{control_del}(U_1) \rrbracket_{Qb} && \text{(Definition 3.1.3)} \\
&= \llbracket U_2 \rrbracket_{Qb} \circ \llbracket U_1 \rrbracket_{Qb} && \text{(By Induction Hypothesis)} \\
&= \llbracket U_1 - U_2 \rrbracket_{Qb} && \text{(Definition 3.1.3)} \\
&= \llbracket U \rrbracket_{Qb}
\end{aligned}$$

4. Else: $\text{control_del}(U) = U$, thus $\llbracket \text{control_del}(U) \rrbracket_{Qb} = \llbracket U \rrbracket_{Qb}$. □

Lemme 3.2.45 (No Control under Removing Controls). Let U be a circuit of type Qb , such that $\text{parallel_free}(U) = \text{ternary_gates_free}(U) = \text{atomic_place}(U) = \text{true}$, then $\text{cont_depth}(\text{control_del}(U)) = 0$.

Proof. Proof by structural induction on U .

1. If $U = \text{Cont}(U', co, ta)$ and $\text{cont_depth}(U') = 0$, then

$$\begin{aligned}
& \text{cont_depth}(\text{control_del}(U)) \\
&= \text{cont_depth}(\text{control_del_step}(U', co, ta)) && \text{(Definition 3.2.43)} \\
&= 0 && \text{(Definition 3.2.39)}
\end{aligned}$$

2. If $U = \text{Cont}(U', co, ta)$ and $\text{cont_depth}(U') > 0$, then

$$\begin{aligned}
& \text{cont_depth}(\text{control_del}(U')) = 0 && \text{(Induction Hypothesis)} \\
& \text{cont_depth}(\text{control_del}(U)) \\
&= \text{cont_depth}(\text{control_del_step}(\text{control_del}(U'), co, ta)) && \text{(Definition 3.2.43)} \\
&= 0 && \text{(By Lemma 3.2.42 and Induction Hypothesis)}
\end{aligned}$$

3. If $U = U_1 \text{---} U_2$, then

$$\begin{aligned}
& \text{cont_depth}(\text{control_del}(U_1)) = 0 && \text{(Induction Hypothesis)} \\
& \text{cont_depth}(\text{control_del}(U_2)) = 0 && \text{(Induction Hypothesis)} \\
& \text{cont_depth}(\text{control_del}(U)) \\
& \quad = \text{cont_depth}(\text{control_del}(U_1)) && \text{(Definition 3.2.43)} \\
& \quad \quad \text{--- control_del}(U_2)) \\
& \quad = \max(\text{cont_depth}(\text{control_del}(U_1)), && \text{(Definition 3.2.38)} \\
& \quad \quad \text{cont_depth}(\text{control_del}(U_2))) \\
& \quad = 0 && \text{(By Induction Hypothesis)}
\end{aligned}$$

4. Else: $\text{cont_depth}(U) = 0$, thus $\text{cont_depth}(\text{control_del}(U)) = 0$. $\square$

3.2.7 Complete Conversion

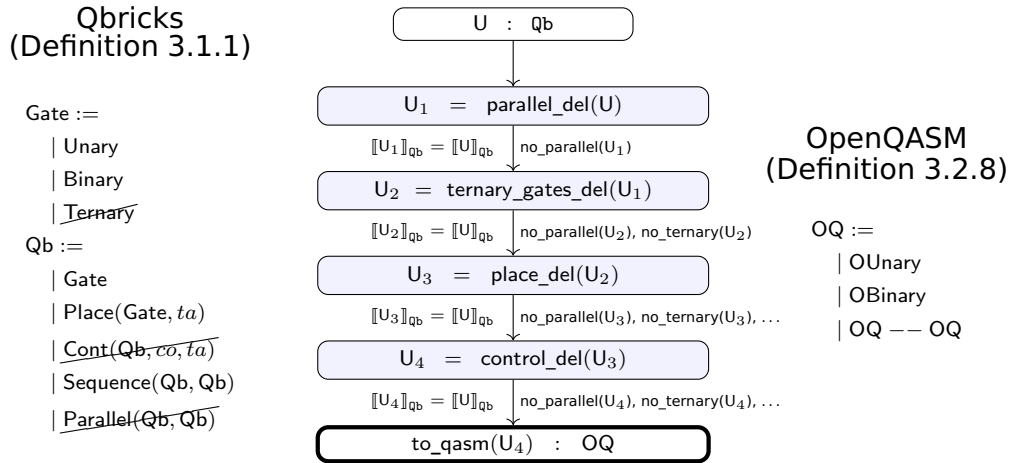


Figure 3.14: Qb into OQ.

In the following, we present the final step: the conversion of type Qb without spatial composition, ternary gate, placement of sub-circuit and generic control in type OQ. This is represented in Figure 3.14 by the step $\text{to_qasm}(U_4) : \text{OQ}$.

We begin by defining our complete conversion (Definition 3.2.46), then we give Theorem 3.2.47, which ensures that complete conversion preserves semantics. Next, we give Theorem 3.2.48, which ensures that Qb is reduced to a level that allows direct translation into OQ. Furthermore, we note that Qbricks uses integers to represent angles; whereas, OpenQASM uses reals⁴. Then we define a `conv_rot` function to convert

⁴In the path-sum formalism (Section 2.3), rotation angles are expressed by the parameter k . In Qbricks, this parameter determines the sign and value of a dyadic. If $k < 0$, the angle is given by $-\frac{2\pi i}{2^{-k}}$. Otherwise, it is given by $\frac{2\pi i}{2^k}$, where $k \in \mathbb{Z}$.

the angle representation (Definition 3.2.49). Finally, we define a function to convert the Qb type to the OQ type (Definition 3.2.50).

Definition 3.2.46 (Complete Conversion (into_qasm)). Let U be a circuit of type Qb, then we define into_qasm as:

$$\text{into_qasm}(U) := \text{control_del}(\text{place_atomic}(\text{ternary_gates_del}(\text{parallel_del}(U)))).$$

Theorem 3.2.47 (Semantics Preservation by Complete Transformation). Let U be a circuit of type Qb. The complete transformation into_qasm preserves the semantics of the circuit. Formally, $\llbracket \text{into_qasm}(U) \rrbracket_{\text{qb}} = \llbracket U \rrbracket_{\text{qb}}$.

Proof.

$$\begin{aligned} & \llbracket \text{into_qasm}(U) \rrbracket_{\text{qb}} \\ &= \llbracket \text{control_del}(\text{place_atomic}(\text{ternary_gates_del}(\text{parallel_del}(U)))) \rrbracket_{\text{qb}} \\ &= \llbracket \text{place_atomic}(\text{ternary_gates_del}(\text{parallel_del}(U))) \rrbracket_{\text{qb}} && \text{(By Lemma 3.2.44)} \\ &= \llbracket \text{ternary_gates_del}(\text{parallel_del}(U)) \rrbracket_{\text{qb}} && \text{(By Lemma 3.2.35)} \\ &= \llbracket \text{parallel_del}(U) \rrbracket_{\text{qb}} && \text{(By Lemma 3.2.24)} \\ &= \llbracket U \rrbracket_{\text{qb}}. && \text{(By Lemma 3.2.10)} \end{aligned}$$

□

Theorem 3.2.48 (Properties of the complete transformation). Let U be a circuit of type Qb. The complete transformation into_qasm guarantees the four following properties:

$$\begin{aligned} \text{parallel_free}(\text{into_qasm}(U)) &= \text{true}, \\ \text{ternary_gates_free}(\text{into_qasm}(U)) &= \text{true}, \\ \text{atomic_place}(\text{into_qasm}(U)) &= \text{true}, \\ \text{atomic_control}(\text{into_qasm}(U)) &= \text{true}. \end{aligned}$$

Proof.

$$\begin{aligned} \text{parallel_free}(\text{into_qasm}(U)) &= \text{true}, && \text{(By Lemma 3.2.12)} \\ \text{ternary_gates_free}(\text{into_qasm}(U)) &= \text{true}, && \text{(By Lemma 3.2.26)} \\ \text{atomic_place}(\text{into_qasm}(U)) &= \text{true}, && \text{(By Lemma 3.2.36)} \\ \text{cont_depth}(\text{into_qasm}(U)) &= 0. && \text{(By Lemma 3.2.45)} \end{aligned}$$

□

Definition 3.2.49 (Angle conversion). Let k be an angle defined in Qbricks and θ be an angle defined in OpenQASM, then we define conv_rot such that conv_rot :

$$\text{conv_rot}(k) := \begin{cases} -\frac{2\pi i}{2^{-k}} & \text{if } k < 0 \\ \frac{2\pi i}{2^k} & \text{else} \end{cases}.$$

Definition 3.2.50 (From Qb to OQ). Let U be a circuit of type Qb (Definition 3.1.1), and let OQ be the OpenQASM AST (Definition 3.2.8).

The translation from Qb to OQ is defined in two steps:

1. The preprocessing function `into_qasm` (Definition 3.2.46) rewrites U into a circuit $U' = \text{into_qasm}(U)$ whose structure is restricted to temporal composition of Unary gates and Binary gates (Definition 3.1.1).
2. The function `to_qasm` translates such restricted Qb circuits into OQ.

For readability, we make the following conventions:

- Angle conversions (Definition 3.2.49) for rotation gates are omitted.
- Qubit indices are omitted when unambiguous (e.g. $\text{to_qasm}(\text{Binary}) = \text{OBinary}$, where $\text{Binary} = \text{Cnot}(co, ta)$ and $\text{OBinary} = \text{Cnot } co \ ta$).
- We use the notation $\text{OUnary } ta$ to denote the shift of OUnary by ta (e.g., if $\text{OUnary} = \text{H } ta'$, then $\text{OUnary } ta = \text{H } (ta + ta')$).

$$\begin{aligned}
 U' &= \text{into_qasm}(U) && \text{(Definition 3.2.46)} \\
 \text{to_qasm}(U') &:= \begin{cases} \text{OUnary} & \text{if } U = \text{Unary} \\ \text{OBinary} & \text{if } U = \text{Binary} \\ \text{OUnary } ta & \text{if } U = \text{Place}(\text{Unary}, ta) \\ \text{OBinary } ta & \text{if } U = \text{Place}(\text{Binary}, ta) \\ \text{into_qasm}(U_1) \text{---} \text{into_qasm}(U_2) & \text{if } U = U_1 \text{---} U_2 \end{cases}
 \end{aligned}$$

3.3 Experience

This chapter focuses on the practical aspects of our work. Section 3.3.1 explains how we certified this transformation. Finally, examples of usage are provided in Section 3.3.2. Figure 3.15 gives an overview of the functioning of the conversion from Qbricks to OpenQASM. First, a certified unitary circuit and its converter, including the functions `into_qasm` and `to_qasm` (Definition 3.2.46 and 3.2.50 respectively), are extracted in OCaml. Second, the unitary circuit extracted in OCaml is converted, thanks to the converter extracted in OCaml⁵, into an OpenQASM unitary circuit. Finally, it is possible to execute the OpenQASM unitary circuit with a quantum computer/simulator. To use the conversion, please refer to the README⁶.

⁵OCaml: <https://ocaml.org/>.

⁶Qbricks README: <https://github.com/Qbricks/qbricks.github.io/blob/main/README.md>.

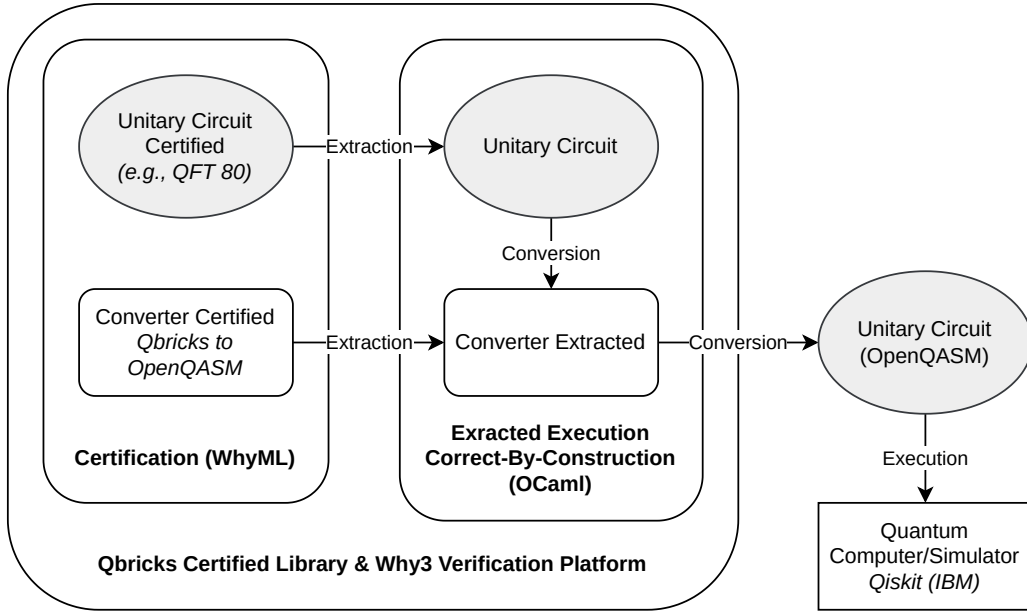


Figure 3.15: Qbricks to OpenQASM - Overview

3.3.1 Practical Implementation

We focus on verifying quantum circuit transformations. In this chapter, we concentrate on certifying a quantum circuit compilation pass. This pass translates high-level Qbricks instructions to low-level OpenQASM 2 code. In Section 3.2, we theoretically describe this pass and its certification. Here, we explain how we implemented and proved it. For this, we utilised the Qbricks library and the Why3 deductive verification platform [35], which enables us to define and certify parametrised unitary quantum circuits. With Why3, we can write code and specifications in the same file.

The general principle to prove a program with Why3 is that for each function, we provide preconditions (which the input data must satisfy) and postconditions (which the output data must satisfy, Hoare logic [41]). For example, a function f might have a precondition $x > 0$, and a postcondition $f(x) > 1$. After the call, the caller may assume the callees postcondition. Specifically, if function f calls function g , then f must ensure that precondition of g holds before the call, and f may rely on postcondition of g after the call. Additionally, when functions are recursive or contain loops, we must prove termination (using a variant). For example, a loop might use a variant $n \geq 0$ that decreases with each iteration until it reaches 0, ensuring termination. We must also prove that the loop or recursion preserves correctness (i.e., the loop invariant). For example, an invariant might be that a variable i remains within the bounds of an array throughout the loop. We can also add intermediate assertions and lemmas to aid the proof. From the code and specifications, Why3 generates proof obligations. These

obligations must be proven either manually using transformations (tactics in Rocq [25]) or automatically with SMT solvers (which Why3 allows to integrate).

In our case, the specifications will focus on the state of the circuit (formalised by a path-sum, Section 3.1.2), the well-formedness conditions of the circuits (conditions on the indices, which must not go out of the circuit and must respect the no-cloning theorem), and conditions on the structure of the circuits which must respect the constraints of our OpenQASM AST. For example, consider the first step of our transformation: removing parallel compositions (Section 3.2.3). The corresponding function in Why3⁷ is:

```

1  let rec function parallel_del (c: Qb) : Qb
2    requires{correct c}
3    requires{ancilla_free c}
4    ensures{parallel_free result}
5    ensures{result ≤> c}
6    ensures{correct result}
7    ensures{ancilla_free result}
8    variant{c}
9    = match c with
10     | Place c1 ta n → Place (parallel_del c1) ta n
11     | Cont c1 co ta n → Cont (parallel_del c1) co ta n
12     | Sequence d e → Sequence (parallel_del d) (parallel_del e)
13     | Parallel d e →
14       remove_parallel (parallel_del d) (parallel_del e)
15     | _ → c
16  end

```

The function signature (line 1) defines a recursive function `parallel_del` that takes a circuit `c` of type `Qb` and returns a circuit `result` of type `Qb`. The preconditions (lines 2 and 3) state that `c` must be correct (i.e., indices must satisfy well-formed conditions of Definition 3.1.1) and must not contain any ancilla (Definition 2.2.45), which are not handled by our transformation. The postconditions (lines 4 to 7) ensure that the resulting circuit will not contain any spatial composition (line 4), will be equivalent to `c`⁸ (line 5), and will preserve both correctness (line 6) and the absence of ancilla (line 7). The termination of the function is guaranteed by the variant `c`, with the termination condition being the absence of instructions in `c`. In recursive functions, the invariant is implicit and consists of the preconditions that must be met. The code that will be extracted to OCaml consists of lines 1 and 9-16. Figure 3.16 shows a screenshot of the Why3 IDE. The function being processed is `parallel_del`. The highlighted yellow text on the right represents the goals that need to be proven. To facilitate this, Why3 generates proof obligations, which are displayed on the left (using the transformation `split_goal_right`). The green checkmarks on the far left indicate that these obligations have been successfully proven. Figure 3.17 shows an example of a goal to be proven. Here, we aim to show that the

⁷`parallel_del` in Qbricks: https://github.com/Qbricks/qbricks.github.io/blob/main/Qbricks_to_oqasm/parallel_delete.mlw. We replace `wired_circuit` by `Qb` for readability.

⁸`circuit_equiv`: https://github.com/Qbricks/qbricks.github.io/blob/main/Qbricks/circuits_equiv.mlw.

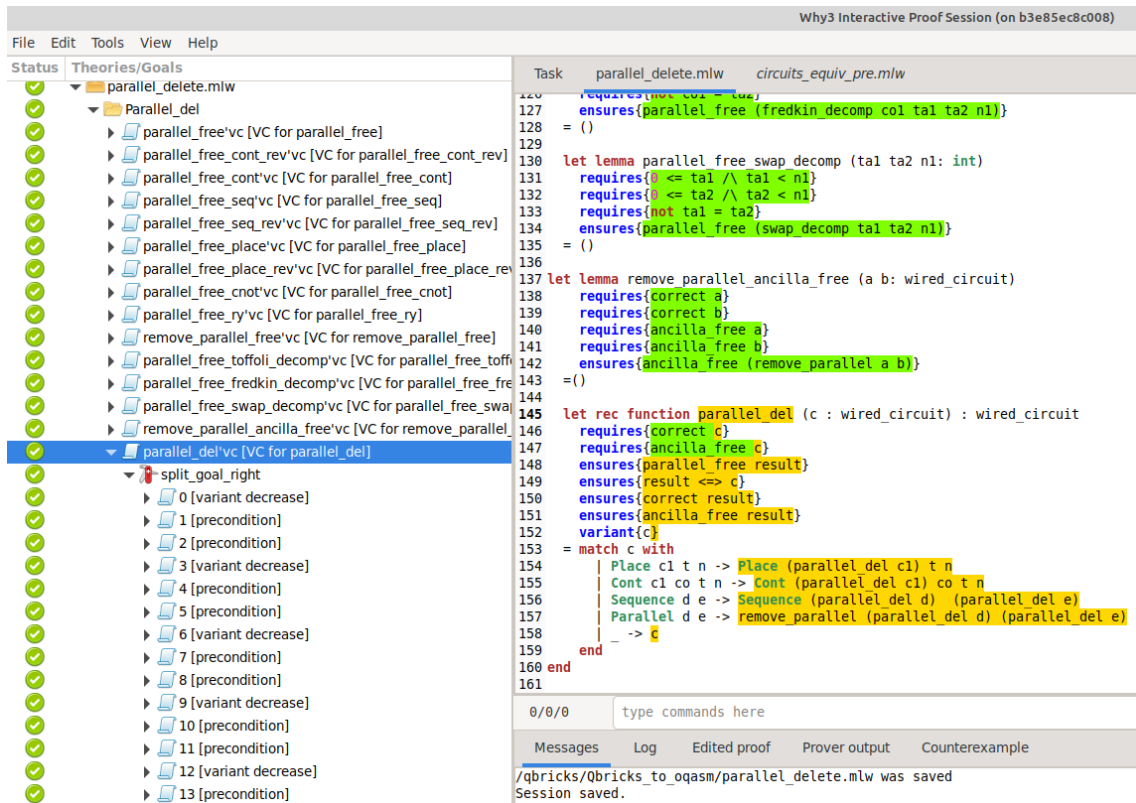


Figure 3.16: Proof Environment in Why3 IDE for parallel_del

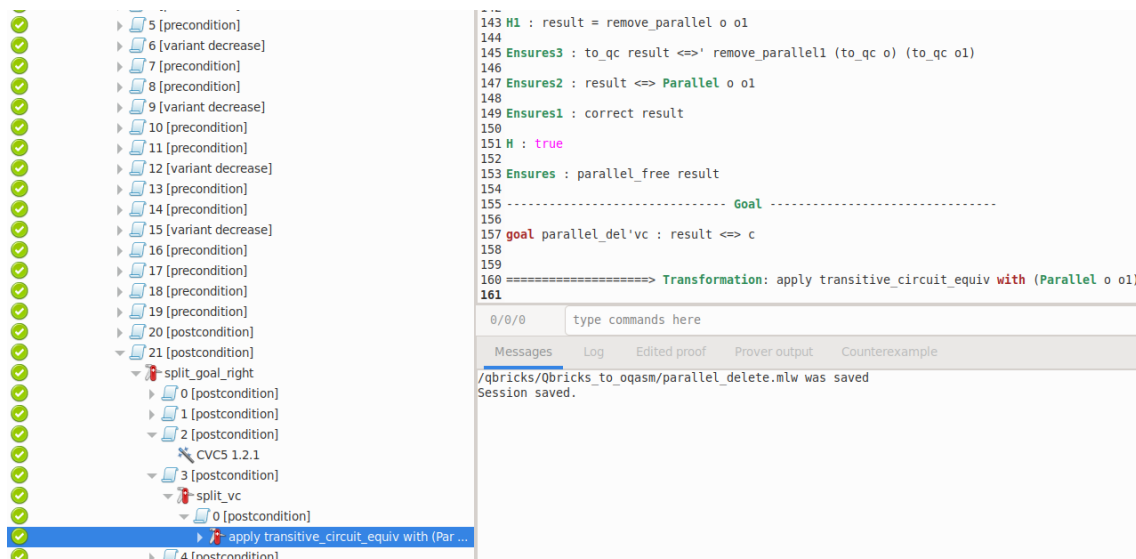


Figure 3.17: Example of Equivalence Proof in Why3 IDE

result is equivalent to the input circuit (line 157), given that c is a spatial composition and thus needs to be removed (line 143). On the left, we see that sub-postcondition 2 of postcondition 21 has been automatically proven by an SMT solver (in this case, CVC5⁹). However, for sub-postcondition 3, it was necessary to apply a lemma on the transitivity of the equivalence relation.

The Qbricks to OpenQASM implementation in WhyML comprises approximately 1800 lines of code, including 103 lemmas, 11 predicates, and 42 functions. For usage and tutorial of Qbricks, please refer to the README¹⁰.

3.3.2 Compilation Usage Examples

Table 3.1: From Qbricks to Qiskit simulation.

Program Quantum Reg. Size	Quantum Fourier Transfom					Shor ¹¹		
	5	24	80	1000	2000	12	16	20
Qbricks LoC	≈ 20	≈ 20	≈ 20	≈ 20	≈ 20	≈ 60	≈ 60	≈ 60
OpenQASM LoC	58	1.5k	15k	2.5M	10M	2M	8M	23M
OpenQASM Extraction (s)	0.0	0.1	0.1	40	326	4	41	52
Qiskit (IBM) Simulation (s)	0.0	2	TO	TO	TO	57	TO	TO

k: 10^3 , M: 10^6 , TO: Aer Qiskit simulator timeout.

Program: the quantum circuits to simulate (QFT and Shor),

Quantum Reg. Size: the number of qubits for each quantum circuit (from 5 to 2000),

Qbricks LoC: the number of lines of Qbricks code for parametrised unitary circuits,

OpenQASM LoC: the number of lines of OpenQASM code for unitary circuit instances,

OpenQASM Extraction (s): the time for extracting from Qbricks to OpenQASM,

Qiskit (IBM) Simulation (s): the time to simulate on the Aer Qiskit simulator.

We exemplify our conversion process (`into_qasm`) by transforming several parametrised circuits, including the quantum Fourier transform (QFT) and Shor’s algorithm (Table 3.1). The parametric aspect of Qbricks allows describing unitary circuits with millions of gates using a few lines of code (LoC): the QFT requires around 20 LoC and Shor’s algorithm about 60 LoC. This enables extracting instances with 10 million gates for QFT (2000 qubits) and 23 million gates for Shor (20 qubits). Extraction is fast: generating the 23 million OpenQASM lines for the latter takes just 52 seconds.

Our certified transformation, `into_qasm`, enables scalable simulation on IBM hardware without sacrificing correctness. Using Qiskit (Aer), we simulated a certified QFT instance (24 qubits) in 2 seconds and Shor’s algorithm (12 qubits) in 57 seconds.

⁹CVC5: <https://cvc5.github.io/>.

¹⁰Qbricks README: <https://github.com/Qbricks/qbricks.github.io/blob/main/README.md>

Chapter 4

Circuit Equivalences

Our goal is to verify that a circuit transformation does not alter its behaviour. In Chapter 3, we demonstrated how to do this by proving the correctness of the transformation itself, but we limited ourselves to a simple transformation that preserves the circuit type, from unitary circuits to unitary circuits (Section 2.2.4), and does not add auxiliary wires. We saw that, even with a semi-automated proof tool, proving a trivial transformation is long and complex. In this chapter, we will lay the theoretical groundwork for developing an automatic hybrid-circuit (Definition 2.2.47) verification tool that can handle discard (Definition 2.2.55). This tool will allow us to verify that a transformation does not modify the behaviour of a given circuit, even for non-trivial transformations such as One-Way Measurement (Section 2.5.3) or Teleportation Insertion (Section 2.5.4).

Figure 4.1 shows the dependency graphs of our equivalence relations. Most equivalence relations are given with two approaches, we called *Parallel* (e.g., $U_1 \equiv U_2$) and the other called *Sequential* (e.g., $U_1 - U_2^\dagger \equiv I$). Partial equivalences are shown with dashed borders, full equivalences have solid borders, unitary equivalences are shown with a blue background, and hybrid equivalences are shown with a grey background. The main point is that **the easily implementable partial equivalences defined with path-sums (in yellow) imply our goal: partial equivalence relations of hybrid circuits (in green)**. We then define a general notation for circuit equivalence (Notation 4.0.1) and outline the classification of our equivalence relations (Remark 4.0.2), to be expanded in Section 4.1.

Notation 4.0.1 (Equivalent and Not Equivalent). In this document, we use the symbol ($\equiv$) to represent an equivalence relation. To distinguish between different types of equivalence relations, we add a subscript to the symbol (e.g., $(\equiv_s)$) or use a dot to indicate partial equivalence ($\doteq$). We also use the notation ($\not\equiv$) to indicate that an equivalence relation is not satisfied.

Remark 4.0.2 (Classification of Equivalence Relations). To classify what is currently state-of-the-art, we aim to be as exhaustive as possible in the equivalence relations we provide to obtain a complete overview. In this context, we distinguish between three

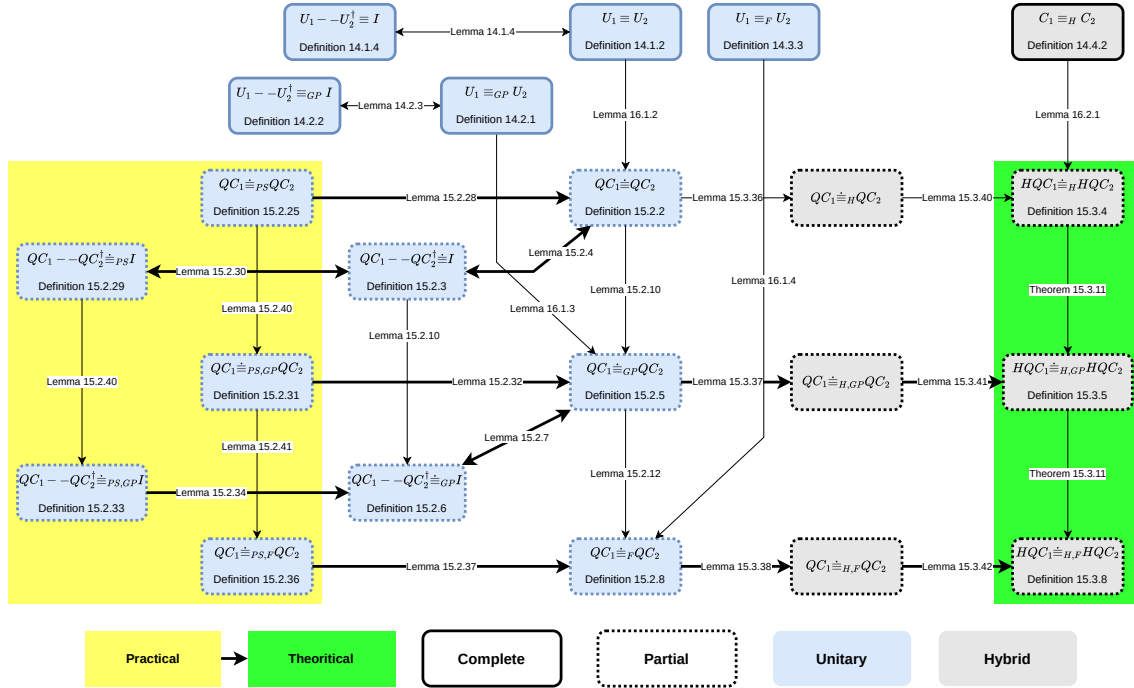


Figure 4.1: Equivalence Relations Overview

families of relations: *sub-circuit-equivalence*, which is based on vector equality (Definition 2.2.10). *full-circuit-equivalence*, which is based on conditional phase equivalence (Definition 2.2.14). In this regard, we consider the circuit as a whole and measure all output wires. *global-phase equivalence*, which is based on global phase equivalence (Definition 2.2.12). It serves as an intermediate concept between sub-circuit and full-circuit and is frequently used in equivalence checkers such as Feynman, QCEC, and PyZX. While full-circuit equivalence is implemented by tools such as QCEC, we will not implement it because it does not scale (see Remark 4.3.38). However, our experimental evaluation shows that the lack of this relation does not prevent us from achieving our objective.

In Section 4.1, we classify circuit transformations, define equivalence classes, and present two ways of viewing a circuit: as a whole or as part of a larger circuit. Once we have established our classification, we begin the definitions of our equivalence relations with those we call *complete* in the sense that all wires are observed (Section 4.2). Then, we move on to the family of *partial* relations, where only some of the wires are observed (Section 4.3). Finally, we give the relations between the two families complete and partial in Section 4.4.

4.1 Classifications

The current computational model is a hybrid one that combines classical and quantum instructions. Section 2.2.5 defines hybrid circuits and fixes the underlying circuit model. This notion plays a central role in practical implementations. We can refine this classification (unitary/hybrid), particularly in our goal of verifying the correctness of quantum circuit transformations.

We begin by defining kinds of circuit transformations in Section 4.1.1. This leads us to determine classes of circuit equivalence based on whether they contain discards. We will also relate these to our types of transformations, all in Section 4.1.2. We add another point to our classifications that we consider important: whether the circuits we want to test are full (complete measurement of the register at the output, loss of phase factors) or subcircuits (important to keep the phase). We address this in Section 4.1.3. Next, we present two standard approaches for checking circuit equivalence in Section 4.1.4.

4.1.1 Circuit Transformations

We define three kinds of circuit transformations:

1. Some circuit transformations preserve the circuit type, mapping unitary circuits (Section 2.2.4) to unitary circuits or hybrid circuits to hybrid circuits. Examples include compilation from Qbricks to OpenQASM (Chapter 3) and transpilation with Qiskit (Section 2.5.2). These transformations preserve the number of qubits.
2. Other transformations change the circuit type, transforming unitary circuits into hybrid circuits. A simple example is hybridisation, described in Section 2.5.5. Hybridisation is relevant for practical implementations because it limits the coherence time required for qubits. This type includes transformations of type 1.
3. More advanced transformations include quantum teleportation (Section 2.5.4), Measurement Based Quantum Computing (MBQC) (Section 2.5.3), error correction. These transformations increase the number of qubits by introducing discards (Definition 2.2.55). However, error correction is not treated here. This type includes transformations of type 1 and 2.

Most current tools, such as AutoQ (Section 2.4.2.1 and 2.4.2.2), Feynman (Section 2.4.2.3), PyZX (Section 2.4.2.4), and SliQEC (Section 2.4.2.6), are limited to unitary circuits (Section 2.4), and thus to transformations of type 1. While some tools, such as QCEC and VeriQC, can verify hybrid circuits, they do not support discard. This limits them to transformations of type 1 and 2. Notably, the authors of [42] identified the importance of discards in equivalence checking and provided an equivalence relation, but did not implement it. Therefore, we believe that the presence or absence of discards is the main challenge in verifying circuit transformations.

4.1.2 Equivalence Classes

The presence or absence of discards defines three distinct equivalence classes:

- **DisFree:** Both circuits contain no discards. This class is limited to checking equivalence between circuits transformed using transformations of type 1 and 2.
- **Mix:** One of the two circuits contains discards. In this class, we can check the equivalence between circuits transformed by transformations of type 1 and 2 and circuits transformed by transformations of type 3.
- **Dis:** Both circuits contain discards. In this class, we can check the equivalence between circuits transformed by transformations of type 3.

Table 4.1: Automatic QC Equivalence Tools.

Name	Techno.	Equivalence Checking Abilities				Prerequisites†
		Unitary	Hybrid			
			DisFree	Mix	Dis	
AutoQ [21, 1]	Tree Automata	✓	×	×	×	×
AutoQ-2 [20]	Tree Automata	✓	×	×	×	×
Feynman [2, 3, 4]	Path-Sum	✓	×	×	×	×
PyZX [47]	ZX Calculus	✓	×	×	×	×
SliQEC [81, 19]	Decision Diagram (DD)	✓	×	×	×	×
QCEC [70, 54, 14]	ZX, DD, Simulation	✓	✓	×	×	×
VeriQC [43, 42]	Tensor DD	✓	✓	**	**	×
SV	Path-Sum	✓	×	×	×	✓
SV + SL	Path-Sum	✓	✓	✓	✓	✓
(*) + SL		✓	✓	×	×	×

✓: Valid, ×: Invalid, SV: SQbricks-Verif, SL: SQbricks-Lift,

(*): Any unitary equivalence checker considered here,

** : The approach is unclear on these points (VeriQC),

†: Meet our prerequisites (featuring separation tests and projections) to be able to verify lifted problems from Mix and Dis.

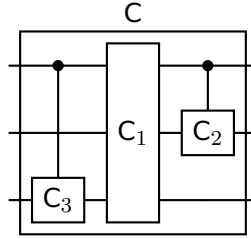
These classes allow us to categorise existing automatic quantum circuit equivalence tools, as shown in Table 4.1. Tools such as QCEC and VeriQC can verify circuits within the DisFree class. What is currently missing is partial equivalence, which allows considering only a part of the register.

4.1.3 Circuit Views

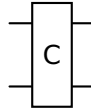
The equivalence relations found in the state of the art are of three kinds. The first, which is the most common, is based on vector equality (Definition 2.2.10). The second is based on global phase equivalence (Definition 2.2.12), which is often implemented.

The third, less often implemented due to its poor scalability (Remark 4.3.38), is based on conditional phase equivalence (Definition 2.2.14). We denote the first as *sub-circuit equivalence*, the second as *global-phase equivalence*, and the third as *full-circuit equivalence*. In this paragraph, we define what we mean by sub-circuit in Definition 4.1.1 and what we mean by full-circuit in Definition 4.1.2. Finally, we discuss the presence of the full-circuit-equivalence in Remark 4.1.3.

Definition 4.1.1 (Sub-Circuit (SubCir)). We define a *sub-circuit* (SubCir) as a quantum circuit integrated within a larger quantum circuit. To illustrate this definition, consider the following quantum circuit C where C_1 , C_2 , and C_3 are sub-circuit of C:



Definition 4.1.2 (Full-Circuit (FullCir)). We define a *full-circuit* (FullCir) as a standalone quantum circuit that operates independently and is not part of a larger one. To illustrate this definition, consider the following quantum circuit C applied accross all qubits:



In this example, the entire circuit is considered as a full-circuit because it operates independently and is not integrated as a part of a larger quantum circuit.

Remark 4.1.3 (Full-Circuit-Equivalence). This chapter aims to provide a landscape of the equivalence relations. Full-Circuit is presented here because specific tools, such as QCEC use it. In practice, however, we do not use full-circuit-equivalence due to its poor scalability, as highlighted in Remark 4.3.38. This limitation poses no problem, as our evaluation confirms that we can achieve the desired results without depending on it.

4.1.4 Equivalence Approaches

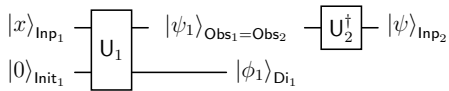


Figure 4.2: Sequential Verification (—)

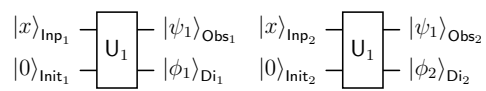


Figure 4.3: Parallel Verification (//)

Most equivalence relations considered in this work are characterised using two verification approaches: parallel and sequential. The *parallel* approach compares the outputs of the two circuits (Figure 4.3), while the *sequential* approach composes a circuit

with the inverse of another and checks whether the resulting circuit acts as the identity (Figure 4.2). Each of these approaches has pros and cons. The sequential approach cannot be used to characterise full-circuit equivalence (Lemma 4.2.16). The parallel approach requires a normal form, hence a complete equational theory, which is not the case with our extension of path-sums (Section 5.2.1).

4.2 Complete Equivalence (Comp)

This section introduces and classifies *complete equivalence* relations that treat all registers as outputs. These relations are widely used in the literature and are implemented by tools such as Feynman, QCEC, PyZX, and AutoQ. We classify these relations within our scheme of sub/full-circuit equivalence and unitary/hybrid equivalence. For hybrid equivalence, we define only the sub-circuit hybrid equivalence, as it is the only relation implemented in prior work (QCEC and VeriQC). This relation is implemented by QCEC using deferred measurement and is defined in [14].

For each unitary equivalence, we define both a parallel equivalence relation, where we compare circuit outputs, and a sequential equivalence relation, where we use the unitarity of quantum circuits to compose one circuit with the inverse of another and verify that the resulting circuit acts as the identity (Section 4.1.4). We show that the parallel and sequential equivalence relations coincide. Figure 4.4 provides an overview of our complete equivalence relations (the complete register is observed). Unitary

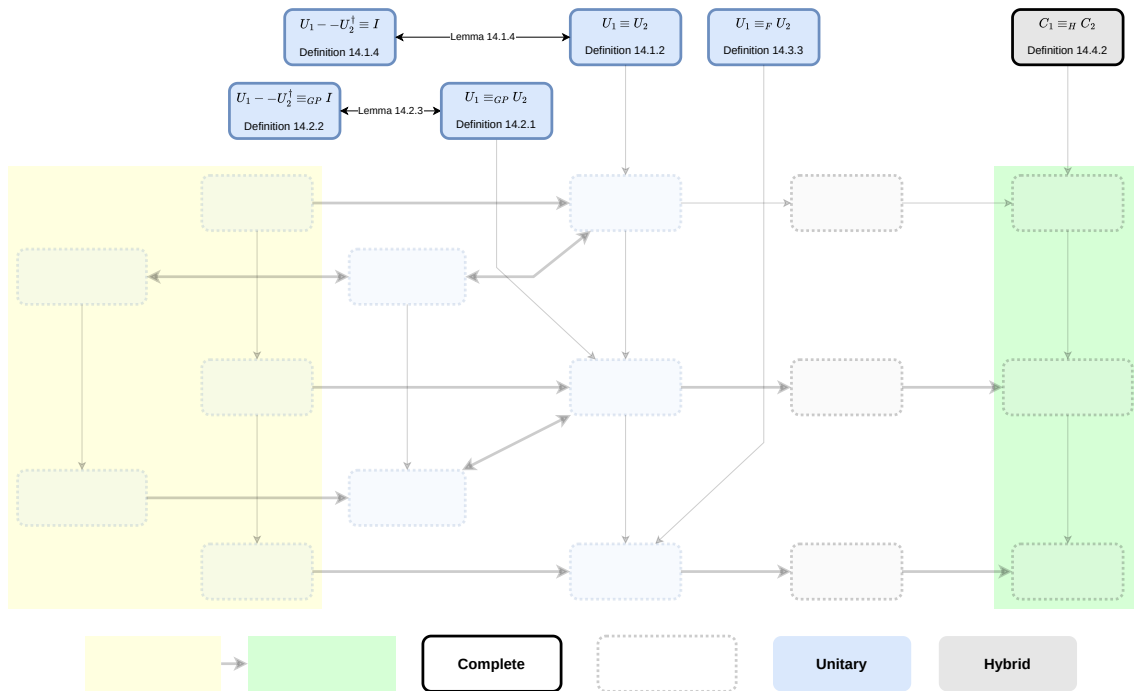


Figure 4.4: Complete Equivalence Relations Overview

equivalences are shown with a white background, and hybrid equivalences with a grey background. We introduce $(\equiv_{\mathbb{H}})$ here because it is used in prior work QCEC (Section 2.4.2.5) and VeriQC (Section 2.4.2.8). **sub-circuit and global-phase are each defined in two forms: a parallel form (e.g., $U_1 \equiv U_2$) and a sequential form (e.g., $U_1 \equiv_{(-)} U_2$).** However, no sequential relation can be defined for full-circuit (Lemma 4.2.16).

The chapter is structured as follows: We begin with unitary complete equivalence in Section 4.2.1, followed by hybrid complete equivalence in Section 4.2.2.

4.2.1 Unitary-Complete-Equivalences (Unit-Comp)

This section presents our three kinds of unitary complete equivalence relations. The first is sub-circuit-equivalence (Section 4.2.1.1). The second is global-phase-equivalence (Section 4.3.2.2). The third is full-circuit-equivalence (Section 4.2.1.3).

4.2.1.1 Sub-Circuit Unitary Complete Equivalence (SubCir-Unit-Comp)

In this section, we begin by emphasising the importance of sub-circuit unitary complete equivalence using Example 4.2.1. We then present two equivalence relations for sub-circuits. The first relation compares the output states obtained from identical inputs (Definition 4.2.2). The second relation requires that the composition of one circuit with the inverse of another behaves as the identity (Definition 4.2.3). Lemma 4.2.4 shows that the two relations coincide.

Various tools and studies in quantum computing use these verification approaches, notably PyZX [47], QCEC [12], Feynman [2], and AutoQ [21].

Example 4.2.1 (The Challenge of Global-Phase in Sub-Circuit Context). Consider the following two circuits:

$$|0\rangle \xrightarrow{\boxed{I}} |0\rangle \qquad |0\rangle \xrightarrow{\boxed{\text{GP}(\pi)}} -|0\rangle$$

These circuits are observationally indistinguishable when applied to isolated quantum states because the global-phase has no measurable effect on the outcomes of quantum measurements. However, when these unitaries are embedded within a larger controlled circuit (Definition 4.1.1), the global-phase can become observable:

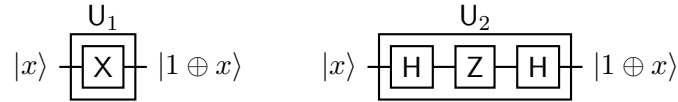
$$\begin{array}{cc} |0\rangle \xrightarrow{\boxed{H}} \bullet \xrightarrow{\boxed{H}} |0\rangle & |0\rangle \xrightarrow{\boxed{H}} \bullet \xrightarrow{\boxed{H}} |1\rangle \\ |0\rangle \xrightarrow{\quad} \boxed{I} \xrightarrow{\quad} |0\rangle & |0\rangle \xrightarrow{\quad} \boxed{\text{GP}(\pi)} \xrightarrow{\quad} |0\rangle \end{array}$$

Thus, I and $\text{GP}(\pi)$ are not equivalent when considered as components of a controlled sub-circuit. This demonstrates the importance of considering global-phase in sub-circuit equivalence verification.

Definition 4.2.2 (SubCir-Unit-Comp-Par). Let $\mathcal{H}^{\otimes n}$ be a Hilbert space of n qubits. We define the *sub-circuit unitary complete parallel equivalence* (SubCir-Unit-Comp-Par) on unitary circuits acting on $\mathcal{H}^{\otimes n}$. For two unitary circuits U_1 and U_2 , we write $U_1 \equiv U_2$ if and only if

$$\forall |\xi\rangle \in \mathcal{H}^{\otimes n}, \llbracket U_1 \rrbracket(|\xi\rangle) = \llbracket U_2 \rrbracket(|\xi\rangle).$$

For example, consider two unitary circuits, U_1 and U_2 , which act on a computational basis state $|x\rangle$ (Definition 2.2.9) as follows:

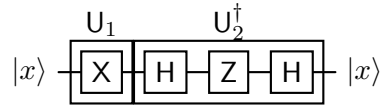


For all $|x\rangle \in \mathcal{H}$, both circuits produce the same output state $|1 \oplus x\rangle$. Therefore, $U_1 \equiv U_2$.

Definition 4.2.3 (SubCir-Unit-Comp-Seq). Let $\mathcal{H}^{\otimes n}$ be a Hilbert space of n qubits. We define the *sub-circuit unitary complete sequence equivalence* (SubCir-Unit-Comp-Seq) on unitary circuits acting on $\mathcal{H}^{\otimes n}$. For two unitary circuits U_1 and U_2 , we write $U_1 \equiv_{(-)} U_2$ if and only if

$$\forall |\xi\rangle \in \mathcal{H}^{\otimes n}, \llbracket U_1 - U_2^\dagger \rrbracket(|\xi\rangle) = |\xi\rangle.$$

For example, we revisit the example from Definition 4.2.2:



Therefore, $U_1 \equiv_{(-)} U_2$.

Lemme 4.2.4 (Equivalence between SubCir-Unit-Comp-Par and SubCir-Unit-Comp-Seq). Let U_1 and U_2 be two unitary circuits. Then

$$U_1 \equiv U_2 \quad \text{if and only if} \quad U_1 \equiv_{(-)} U_2,$$

where $\equiv$ and $\equiv_{(-)}$ are defined in Definition 4.2.2 and Definition 4.2.3, respectively.

Proof. We prove the two implications. Let $\mathcal{H}^{\otimes n}$ be a Hilbert space of n qubits.

1. Assume $(U_1 \equiv U_2)$. By Definition 4.2.2, $\forall |\xi\rangle \in \mathcal{H}^{\otimes n}, \llbracket U_1 \rrbracket(|\xi\rangle) = \llbracket U_2 \rrbracket(|\xi\rangle)$. Hence,

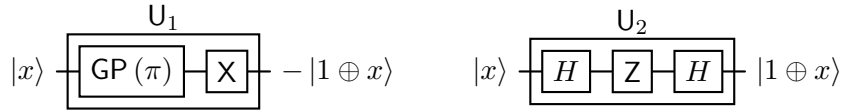
$$\begin{aligned} \llbracket U_1 - U_2^\dagger \rrbracket(|\xi\rangle) &= \llbracket U_2^\dagger \rrbracket \circ \llbracket U_1 \rrbracket(|\xi\rangle) \\ &= \llbracket U_2^\dagger \rrbracket \circ \llbracket U_2 \rrbracket(|\xi\rangle) \\ &= |\xi\rangle, \end{aligned}$$

by unitarity. Therefore $U_1 \equiv_{(-)} U_2$ by Definition 4.2.3.

2. Assume $U_1 \equiv_{(-)} U_2$. By Definition 4.2.3, $\forall |\xi\rangle \in \mathcal{H}^{\otimes n}$, $\llbracket U_1 - U_2^\dagger \rrbracket(|\xi\rangle) = |\xi\rangle$.

Composing on the right with $\llbracket U_2 \rrbracket$, we obtain $\llbracket U_2 \rrbracket \circ \llbracket U_1 - U_2^\dagger \rrbracket = \llbracket U_1 \rrbracket$. Hence, $\forall |\xi\rangle$, $\llbracket U_1 \rrbracket(|\xi\rangle) = \llbracket U_2 \rrbracket(|\xi\rangle)$, and therefore $U_1 \equiv U_2$ by Definition 4.2.2. $\square$

Example 4.2.5 (Limit of SC Equivalences in the Context of FC). We revisit the circuit from example of Definition 4.2.2 by adding a global-phase factor at U_1 . This serves to illustrate a limitation of SubCir-Unit-Comp-Par (Definition 4.2.2) and, by extension, of SubCir-Unit-Comp-Seq (Definition 4.2.3) in the context of the full-circuit.



By Definition 4.2.2, we have $U_1 \not\equiv U_2$, and $U_1 \not\equiv_{(-)} U_2$ (Notation 4.0.1). This result is relevant in the context of sub-circuit, but not in the context of full-circuit, where the phase factor is not considered relevant. Therefore, we also need an equivalence that confirms the equivalence despite the global-phase difference.

4.2.1.2 Global Phase Unitary Complete Equivalence (GP-Unit-Comp)

We start this section with global-phase unitary complete parallel equivalence (Definition 4.2.6), then introduce global-phase unitary complete sequence equivalence (Definition 4.2.7). We then prove that these two equivalence relations coincide (Lemma 4.2.8), and finally illustrate a limitation of global-phase unitary complete equivalence in the context of full-circuit (Example 4.2.9).

Definition 4.2.6 (GP-Unit-Comp-Par). Let $\mathcal{H}^{\otimes n}$ be a Hilbert space of n qubits. We define the *global-phase unitary complete parallel equivalence* (GP-Unit-Comp-Par) on unitary circuits acting on $\mathcal{H}^{\otimes n}$. For two unitary circuits U_1 and U_2 , we write $U_1 \equiv_{\text{gp}} U_2$ if and only if

$$\forall |\xi\rangle \in \mathcal{H}^{\otimes n}, \llbracket U_1 \rrbracket(|\xi\rangle) \approx_{\text{gp}} \llbracket U_2 \rrbracket(|\xi\rangle).$$

We revisit Example 4.2.5 with global-phase unitary complete parallel equivalence; in this case, $U_1 \equiv_{\text{gp}} U_2$.

Definition 4.2.7 (GP-Unit-Comp-Seq). Let $\mathcal{H}^{\otimes n}$ be a Hilbert space of n qubits. We define the *global-phase unitary complete sequence equivalence* (GP-Unit-Comp-Seq) on unitary circuits acting on $\mathcal{H}^{\otimes n}$. For two unitary circuits U_1 and U_2 , we write $U_1 \equiv_{(-, \text{gp})} U_2$ if and only if

$$\forall |\xi\rangle \in \mathcal{H}^{\otimes n}, \llbracket U_1 - U_2^\dagger \rrbracket(|\xi\rangle) \approx_{\text{gp}} |\xi\rangle.$$

Lemme 4.2.8 (Equivalence between GP-Unit-Comp-Par and GP-Unit-Comp-Seq). Let U_1 and U_2 be two unitary circuits. Then $U_1 \equiv_{\text{gp}} U_2$ if and only if $U_1 \equiv_{(-, \text{gp})} U_2$, where $\equiv_{\text{gp}}$ and $\equiv_{(-, \text{gp})}$ are defined in Definition 4.2.6 and Definition 4.2.7, respectively.

Proof. The proof follows the same structure as Lemma 4.2.4, replacing $=$ with $\approx_{\text{gp}}$, $\equiv$ with $\equiv_{\text{gp}}$, and $\equiv_{(-)}$ with $\equiv_{(-, \text{gp})}$. $\square$

Example 4.2.9 (Limit of GP-Unit-Comp in the Context of FullCir). Consider the unitary circuits U_1 and U_2 such that:

$$|0\rangle \xrightarrow{U_1} \boxed{\text{H}} \xrightarrow{\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)} \quad |0\rangle \xrightarrow{U_2} \boxed{\text{X} \text{---} \text{H}} \xrightarrow{\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)}$$

By Definition 4.2.6, $U_1 \not\equiv_{\text{gp}} U_2$ (Notation 4.0.1). However, both circuits yield the same measurement probabilities for $|0\rangle$ and $|1\rangle$, namely $p^0 = p^1 = \frac{1}{2}$. This example shows that global-phase-unitary-equivalence does not capture equivalence at the level of observable measurement statistics in the context of full-circuit, as it distinguishes circuits that differ by relative phase while inducing identical probability distributions. An alternative equivalence relation is therefore required to characterise equality of observable behaviours.

4.2.1.3 Full-Circuit Unitary Complete Equivalence (FullCir-Unit)

This section introduces a unitary equivalence relation in the context of full-circuit (Definition 4.1.2), addressing the limitations of sub-circuit unitary complete equivalence and global-phase unitary complete equivalence in handling conditional phase differences (Definition 2.2.14). We begin by defining full-circuit unitary complete parallel equivalence (Definition 4.2.12). For full-circuit, only one equivalence relation is considered due to Corollary 4.2.16, which states that no sequential equivalence relation can be defined in this context. Thus, we often omit the term “parallel” and refer to it simply as full-circuit unitary complete equivalence. Finally, we provide an example (Example 4.2.17) where a partial equivalence relation is necessary. Tools such as QCEC [12] perform full-circuit unitary partial equivalence using this relation¹. We explore partial equivalence in Chapter 4.3.

Definition 4.2.10 (Equality of Probability Distributions of Quantum States). Let $\mathcal{H}^{\otimes n}$ be a Hilbert space of n qubits, let $|\psi^1\rangle, |\psi^2\rangle \in \mathcal{H}^{\otimes n}$. Assume $\forall i \in \{1, 2\}, d(\psi^i) = \{p_c^i, |c\rangle, c\}_{c \in \{0,1\}^n}$ (Definition 2.2.24). The probability distributions $d(\psi^1)$ and $d(\psi^2)$ are equal, denoted by $d(\psi^1) = d(\psi^2)$ if and only if $\forall c \in \{0, 1\}^n, p_c^1 = p_c^2$.

Notation 4.2.11 (Functional Vector). Let $|\xi\rangle \in \mathcal{H}^{\otimes n}$. We introduce the notation $|\psi(\xi)\rangle$ to denote the image of $|\xi\rangle$ under the map $\psi : \mathcal{H}^{\otimes n} \rightarrow \mathcal{H}^{\otimes n}$.

Definition 4.2.12 (FullCir-Unit-Comp-Par). Let $\mathcal{H}^{\otimes n}$ be a Hilbert space of n qubits. We define *full-circuit-unitary-equivalence* (FullCir-Unit-Comp-Par) on unitary circuits acting on $\mathcal{H}^{\otimes n}$. For two unitary circuits U_1 and U_2 , we write $U_1 \equiv_{\text{f}} U_2$ if and only if

¹QCEC with partial unitary equivalence option: https://mqt.readthedocs.io/projects/qcec/en/latest/partial_equivalence.html

let $\llbracket U_i \rrbracket(|\xi\rangle) = |\psi_i(\xi)\rangle$ for $i \in \{1, 2\}$. Then

$$\forall |\xi\rangle \in \mathcal{H}^{\otimes n}, \mathbf{d}(\psi_1(\xi)) = \mathbf{d}(\psi_2(\xi)),$$

where $\mathbf{d}(\psi_i(\xi))$ denotes the probability distribution obtained by measuring the entire register (Definition 2.2.24).

We revisit Example 4.2.9 with full-circuit-unitary-equivalence:

$$\begin{aligned} \mathbf{d}\left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\right) &= \left\{\left(\frac{1}{2}, |0\rangle, 0\right), \left(\frac{1}{2}, |1\rangle, 1\right)\right\}, \\ \mathbf{d}\left(\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)\right) &= \left\{\left(\frac{1}{2}, |0\rangle, 0\right), \left(\frac{1}{2}, |1\rangle, 1\right)\right\}. \end{aligned}$$

Therefore, $U_1 \equiv_{\mathbf{f}} U_2$.

In the following, we show that a characterisation based on sequential composition, as in Definitions 4.2.3 and 4.2.7, cannot be defined in the case of full-circuit unitary complete equivalence.

Lemme 4.2.13 (FullCir-Unit-Comp-Par Not Preserved under Right Composition). Let U_1 , U_2 , and U be unitary circuits. If $U_1 \equiv_{\mathbf{f}} U_2$ (Definition 4.2.12), it does not necessarily follow that $U_1 \text{---} U \equiv_{\mathbf{f}} U_2 \text{---} U$.

Proof. Consider the unitary circuits $U_1 = X \text{---} H$, $U_2 = H$, and $U = H$.

1. By Definition 2.2.40,

$$\llbracket U_1 \rrbracket(|0\rangle) = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |\psi_1\rangle, \quad \llbracket U_2 \rrbracket(|0\rangle) = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |\psi_2\rangle.$$

By Definition 2.2.24,

$$\forall i \in \{1, 2\}, \mathbf{d}(\psi_i) = \left\{\left(\frac{1}{2}, |0\rangle, 0\right), \left(\frac{1}{2}, |1\rangle, 1\right)\right\}.$$

Hence, by Definition 4.2.12, $U_1 \equiv_{\mathbf{f}} U_2$.

2. By Definition 2.2.40,

$$\llbracket U_1 \text{---} H \rrbracket(|0\rangle) = |1\rangle = |\psi'_1\rangle, \quad \llbracket U_2 \text{---} H \rrbracket(|0\rangle) = |0\rangle = |\psi'_2\rangle.$$

By Definition 2.2.24,

$$\mathbf{d}(\psi'_1) = \{(1, |1\rangle, 1)\}, \quad \mathbf{d}(\psi'_2) = \{(1, |0\rangle, 0)\}.$$

Therefore, by Definition 4.2.12, $U_1 \text{---} H \not\equiv_{\mathbf{f}} U_2 \text{---} H$.

Thus, $U_1 \equiv_{\mathbf{f}} U_2$ does not imply $U_1 \text{---} H \equiv_{\mathbf{f}} U_2 \text{---} H$. □

Lemme 4.2.14 (FullCir-Unit-Comp-Par Not Preserved under Left Composition). Let U_1 , U_2 , and U be unitary circuits. If $U_1 \equiv_f U_2$, it does not necessarily follow that $U \dashv\vdash U_1 \equiv_f U \dashv\vdash U_2$.

Proof. We adapt the proof of Lemma 4.2.13 for left composition. Let $U_1 := H \dashv\vdash X$, $U_2 := H$, and consider left composition with H . The remaining steps are analogous and are omitted. $\square$

Theorem 4.2.15 (FullCir-Unit-Comp-Par Not Preserved under Composition). Let U_1 , U_2 , and U be unitary circuits. If $U_1 \equiv_f U_2$ (Definition 4.2.12), it does not necessarily follow that $U_1 \dashv\vdash U \equiv_f U_2 \dashv\vdash U$, nor that $U \dashv\vdash U_1 \equiv_f U \dashv\vdash U_2$.

Proof. The result follows directly from Lemmas 4.2.13 and 4.2.14.

By Lemma 4.2.13, $U_1 \equiv_f U_2$ does not imply $U_1 \dashv\vdash U \equiv_f U_2 \dashv\vdash U$.

By Lemma 4.2.14, $U_1 \equiv_f U_2$ does not imply $U \dashv\vdash U_1 \equiv_f U \dashv\vdash U_2$. $\square$

Corollary 4.2.16 (FullCir-Unit-Comp-Seq Not Equivalent to FullCir-Unit-Comp-Par). Let U_1 and U_2 be unitary circuits. Then $U_1 \equiv_f U_2$ (Def. 4.2.12) does not imply $U_1 \dashv\vdash U_2^\dagger \equiv_f I$, and conversely, $U_1 \dashv\vdash U_2^\dagger \equiv_f I$ does not imply $U_1 \equiv_f U_2$.

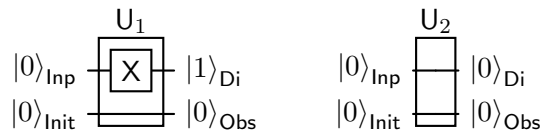
Proof. We prove the two non-implications.

First, by Lemma 4.2.13 with $U = U_2^\dagger$, $U_1 \equiv_f U_2$ does not imply $U_1 \dashv\vdash U_2^\dagger \equiv_f I$.

Second, by Lemma 4.2.14 with $U = U_2$, $U_1 \dashv\vdash U_2^\dagger \equiv_f I$ does not imply $U_1 \equiv_f U_2$. $\square$

Thus, by Corollary 4.2.16, we have shown that defining full-circuit unitary complete equivalence sequential is not sufficient to guarantee that circuits have the same behaviour.

Example 4.2.17 (Need for Partial-Equivalence). Definition 2.2.56 defines the registers used in this example and in the following. Consider the unitary circuits U_1 and U_2 defined by the following quantum circuits:



We aim to verify equivalence by considering only the observed qubits Obs. However, none of the previously defined equivalence relations captures this situation, as they require equality on the full register. An equivalence relation restricted to the observed subsystem Obs is therefore required. We introduce such partial equivalence relations in Chapter 4.3.

In the following Section 4.2.1, we introduce sub-circuit-hybrid-equivalence, as it is the only one implemented (by QCEC [14], and VeriQC [42]). More general cases of hybrid circuit equivalence will be discussed in the section on partial equivalence (Chapter 4.3).

$$\{p_c, |\xi_c\rangle, c\}_{c \in \{0,1\}^n} \xrightarrow{C_i} \{p_{c_i}^i, |\psi_{c_i}^i(\xi_c)\rangle, c_i\}_{c_i \in \{0,1\}^n}$$

$$C_1 \equiv_H C_2 \quad (\text{Definition 4.2.18})$$

Figure 4.5: Sub-Circuit Hybrid Equivalence.

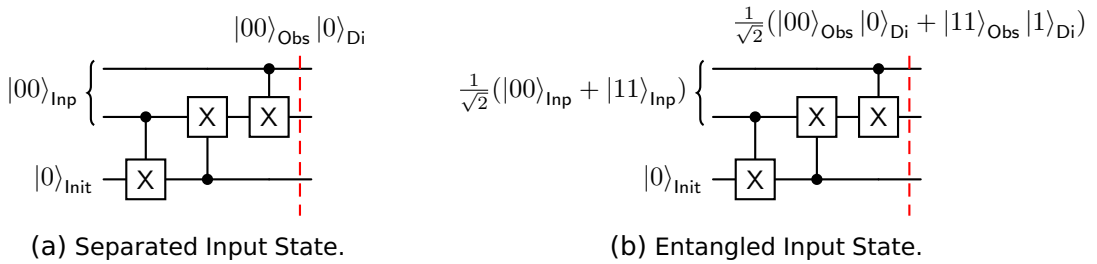
4.2.2 Hybrid-Complete-Equivalences (Hyb-Comp)

In previous sections, we discussed equivalence relations for unitary circuits, which typically only handle final measurements and do not cover intermediate measurements or classical control. Tools like VeriQC [42, 80] and QCEC [14] use these types of equivalence relations. However, current methods are limited because they lack relations that can verify partial equivalence in hybrid circuits. For instance, although VeriQC attempted to tackle this, it only reached sub-circuit unitary complete equivalence (Definition 4.2.18). To address this gap, we introduce the concept of partial equivalence in Chapter 4.3. Figure 4.5 illustrates this concept.

Definition 4.2.18 (SubCir-Hyb-Comp-Par). Let $\rho = \{p_c, |\xi_c\rangle, c\}_c$ be an extended mixed state (Definition 2.2.20). We define *sub-circuit-hybrid-equivalence* (SubCir-Hyb-Comp-Par) on hybrid circuits. For two hybrid circuits C_1 and C_2 , we write $C_1 \equiv_H C_2$ if and only if let $\llbracket C_i \rrbracket_H(\rho) = \rho_i$ for $i \in \{1, 2\}$; $\rho_1 = \rho_2$, where equality is defined in Definition 2.2.22.

4.3 Partial Equivalences (Part)

Partial equivalence verification involves checking the equivalence between two circuits by considering only the observed qubits (denoted by Obs). However, adopting a naive approach without ensuring separation between output qubits and others (denoted by Di) can lead to unexpected behaviour. For example, in Figure 4.6a, the two input qubits,



denoted by Inp, are separated (Definition 2.2.16). In this case, Obs qubits are also separated from Di qubit. In contrast, Figure 4.6b uses the same circuit but with an entangled input state where the two input qubits in Inp are entangled. Consequently, Obs is entangled with Di. This example illustrates that testing entanglement is not a

trivial task. It cannot be determined naively by testing only computational basis states such as $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$. Thus, we need to verify that the Obs is not implicitly entangled with Di.

Definition 4.3.1 (Separation Test). To address the issue seen in the previous example, we need to ensure that the state on Obs and Di are separated and that the state on Di is independent of the input state. Formally, for $i \in \{1, 2\}$, let U_i be two unitary circuits (Section 2.2.4) such that $\llbracket U_i \rrbracket(|\xi\rangle) = |\psi(\xi)\rangle_{\text{Obs}, \text{Di}}$. We say that the state on Obs is *separated* from the state on Di if we can write $|\psi(\xi)\rangle_{\text{Obs}, \text{Di}} = |\psi'(\xi)\rangle_{\text{Obs}} \otimes |\phi\rangle_{\text{Di}}$.

We propose defining partial equivalence relations for hybrid circuits (with discards) that are easily implementable and scalable. To achieve this, we utilise the path-sums formalism, which is easily encoded (as a standard data structure) and scalable (via symbolic representation). Therefore, we define partial and standard equivalence relations for unitary circuits, define partial equivalence relations for path-sums, and verify that the relations based on path-sums are correct with respect to the standards. However, path-sums are limited to unitary circuits. So, once we have these easily encodable partial unitary relations, we lift them to hybrid cases using deferred measurement (Section 2.2.6). This extension allows us to reach the Mix and Dis classes. As a side effect, this lifting also extends all unitary equivalence tools to DisFree, aligning with the state of the art. Figure 4.7 shows our partial equivalence relations. Unitary equiv-

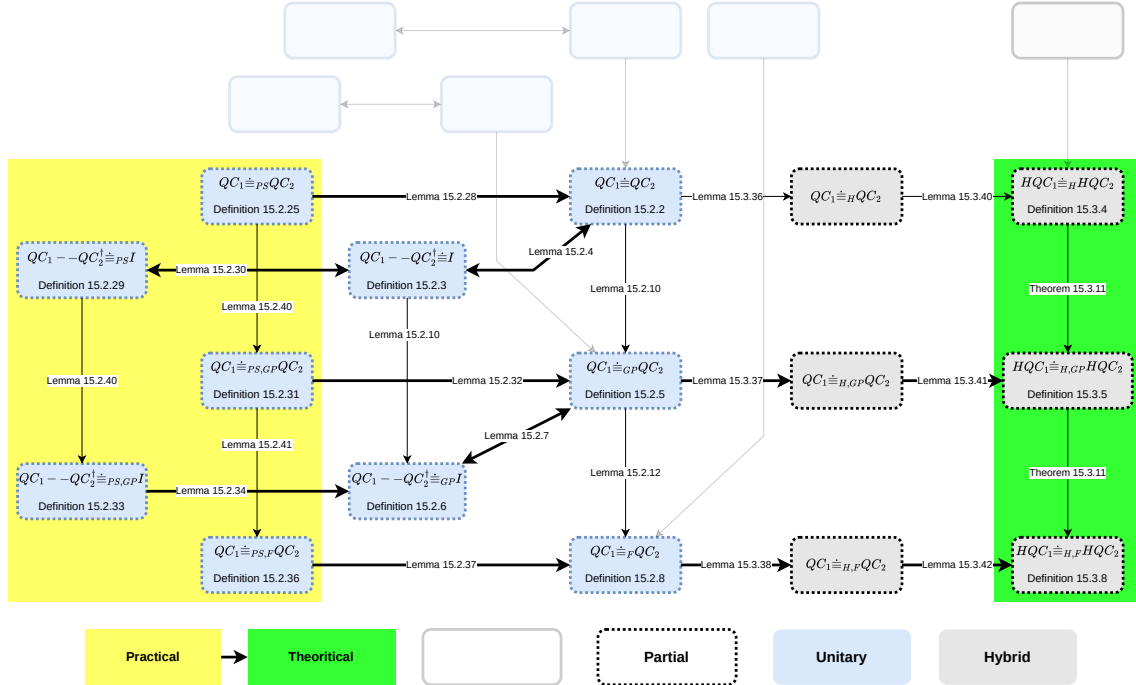


Figure 4.7: Partial Equivalence Relations Overview

alence relations feature a blue background, and hybrid equivalence relations a grey

background. The main point is that **practical unitary equivalence with path-sum in yellow implies theoretical partial equivalence of hybrid circuit in green**².

To simplify the proofs, we introduce SV, a data type based on our QASM type (Definition 2.2.41) but with a gate set limited to $\{GP(\theta), U_1(\theta), X, H\}$. Our SV type retains quantum multi-control and the ability to represent any quantum computation.

Definition 4.3.2 (SV Syntax). We define the SV algebraic data type, which uses our unitary QASM representation with a limited gate set. The angle θ is given either as a real number in the standard semantics or with integers d and k to facilitate the encoding and verification of angles. These parameters define dyadic angles such that $\theta = \frac{2\pi d}{2^k}$. The lists *co* and *ta* represent the indices of the control and target qubits, respectively. The syntax of SV is as follows:

$$\begin{aligned} \text{Unary} &:= GP(\theta) \mid U_1(\theta) \mid X \mid H \\ \text{GA} &:= \text{Apply}(\text{Unary}, \text{co}, \text{ta}) \\ \text{Ins} &:= E \mid \text{GA} \\ \text{SV} &:= \text{Ins} \text{---} \text{SV} \mid \text{Ins} \end{aligned}$$

The *well-formedness conditions* are the well-formed conditions of our QASM representation of unitary circuits (Definition 2.2.41).

We begin by outlining the prerequisites for partial equivalence in Section 4.3.1. Next, we introduce unitary partial equivalence in Section 4.3.2. We then present our path-sum partial equivalence, including proofs of correctness, in Section 4.3.3. Following this, Section 4.3.4 describes hybrid partial equivalence relations. Finally, Section 4.3.5 details our method for verifying hybrid partial equivalence using unitary partial equivalence and deferred measurement.

4.3.1 Prerequisites

This section introduces the definitions and notations necessary for understanding partial equivalence. We begin by defining the concept of observed (Definition 4.3.3), which is central to partial equivalence as it represents the output of the circuit. Definitions 4.3.4, 4.3.5, and 4.3.6 discuss partial final measurements used in full-circuit-partial-equivalence. The concept of circuit tuples, crucial in all partial-equivalence relations, is introduced by Definition 4.3.7 and Notation 4.3.8. Finally, Definitions 4.3.9 and 4.3.10 explain the compositions of tuples, and Definition 4.3.11 introduces the notion of the inverse of a tuple for unitary circuits.

Definition 4.3.3 (Observed). The *observed* part of a quantum register, denoted as *Obs*, is the output of the circuit. The discarded part, denoted as *Di* (Definition 2.2.55),

²We recall that equivalence relations are typically defined using two approaches: the parallel approach (e.g., $U_1 \equiv U_2$) and the sequential approach (e.g., $U_1 \text{---} U_2^\dagger \equiv I$).

represents the portion of the register that is forgotten after measurement (Section 2.2.2). Thus, the observed part is the complement of the discarded part. Formally, for a quantum register Q of size n , the observed and discarded parts are defined such that:

$$Q = \text{Obs} \cup \text{Di} = \{0, 1, \dots, n-1\}, \quad \text{and} \quad \text{Obs} \cap \text{Di} = \emptyset. \quad (\text{Notation 2.1.2})$$

Definition 4.3.4 (Subset Projection Probability). Consider a quantum state $|\psi\rangle$ on n qubits and a set of qubit indices S , such that $\forall i \in S, i \in [0, n-1]$. Let $x \in \{0, 1\}^n$ be a bit-string, let T be the complement of S such that $S \cup T = \{0, \dots, n-1\}$ and $S \cap T = \emptyset$. The sum is taken over all possible states $x|_T \in \{0, 1\}^T$ of the qubits over T (Notation 2.1.3), and $\langle x|_S x|_T | \psi \rangle$ is the inner product (Definition 2.2.4) between $|x|_S x|_T\rangle \in \mathcal{H}^{\otimes n}$ and $|\psi\rangle$. We define the *subset projection probability* of the state $|\psi\rangle$ by:

$$p(\psi, S, x) := \sum_{x|_T \in \{0,1\}^T} |\langle x|_S x|_T | \psi \rangle|^2. \quad (\text{Definition 2.2.4})$$

For example, let $|\psi\rangle = \frac{1}{\sqrt{2}}|00\rangle + \frac{1}{2}|01\rangle + \frac{1}{2}|10\rangle$, $S = \{0\}$, and $x_0 = 0$, $T = \{1\}$ and $x|_T$ represents the possible states of the qubit in T . By Definition 4.3.4:

$$p(\psi, S, x) = \sum_{x|_T} |\langle x|_S x|_T | \psi \rangle|^2 = |\langle 00 | \psi \rangle|^2 + |\langle 01 | \psi \rangle|^2 = \frac{3}{4}.$$

Definition 4.3.5 (Complete Register Measurement with Partial Observation). Recall the notation for probability distributions as defined in Notation 2.2.21. Let $\mathcal{H}^{\otimes n}$ be a Hilbert space of n qubits. Let $\text{Obs} \subseteq \{0, \dots, n-1\}$ be an observed set of qubit indices (Definition 4.3.3). We define a *complete register measurement with partial observation* as the probability distribution of set projection on Obs , as given by Definition 4.3.4. Formally, for a quantum state $|\psi\rangle \in \mathcal{H}^{\otimes n}$, the probability of observing the state $|x|_{\text{Obs}}\rangle$ in the subsystem Obs is denoted by $p(\psi, \text{Obs}, x)$, as defined in Definition 4.3.4. The resulting probability distribution is given by:

$$d(\psi, \text{Obs}) := \{p(\psi, \text{Obs}, x), |x\rangle, x|_{\text{Obs}}\}_{x \in \{0,1\}^{|\text{Obs}|}}, \quad (\text{Definition 4.3.4})$$

For example, let $|\psi\rangle = \frac{1}{\sqrt{2}}|00\rangle_{q_0, q_1} + \frac{1}{2}|01\rangle_{q_0, q_1} + \frac{1}{2}|10\rangle_{q_0, q_1}$, $\text{Obs} = \{q_0\}$, and $T = \{q_1\}$. We calculate the probabilities for each possible outcome of $x|_{\text{Obs}}$: If $x_{q_0} = 0$, then $p(\psi, \text{Obs}, x|_{\text{Obs}}) = \frac{3}{4}$, else, $p(\psi, \text{Obs}, x|_{\text{Obs}}) = \frac{1}{4}$ (Definition 4.3.4). Thus, the probability distribution is: $d(\psi, \text{Obs}) = \left\{ \left(\frac{3}{4}, |0\rangle_{q_0}, 0 \right), \left(\frac{1}{4}, |1\rangle_{q_0}, 1 \right) \right\}$.

Definition 4.3.6 (Equivalence of Complete Register Measurement with Partial Observation). Let $\mathcal{H}^{\otimes n}$ be the Hilbert space of n qubits. For $i \in \{1, 2\}$, let $|\psi^i\rangle \in \mathcal{H}^{\otimes n}$ be a quantum state and $\text{Obs}_i \subseteq \{0, \dots, n-1\}$ a set of qubit indices. Let $d(\psi^i, \text{Obs}_i)$ denote the complete register measurement with partial observation (Definition 4.3.5).

We define equality of complete register measurements with partial observation by $d(\psi^1, \text{Obs}_1) = d(\psi^2, \text{Obs}_2)$ if and only if

$$\text{Obs}_1 \simeq \text{Obs}_2, \quad (4.1)$$

$$\forall x_{|\text{Obs}_1} : \text{Obs}_1 \mapsto \{0, 1\}^{|\text{Obs}_1|}, \quad p(\psi^1, \text{Obs}_1, x_{|\text{Obs}_1}) = p(\psi^2, \text{Obs}_2, x_{|\text{Obs}_1}), \quad (4.2)$$

where $\simeq$ is defined in Notation 2.1.2.

In the context of partial equivalence, our objective is to verify the equivalence of circuits based on a subset of registers. This method enables us to compare circuits with differing numbers of qubits. To accomplish this, managing each circuit's inputs and outputs is essential. We introduce the concept of a “tuple” for circuits (Definition 4.3.7), which links a circuit to specific sets of qubit indices. We introduce compositions in Definitions 4.3.9 and 4.3.10. Finally, we define the inverse of a unitary circuit tuple in Definition 4.3.11.

Definition 4.3.7 (Circuit Tuple). Let $\mathcal{H}^{\otimes n}$ be the Hilbert space of n qubits, equipped with a quantum register $Q = \{0, 1, \dots, n-1\}$. Let $\text{Inp} \subseteq Q$ be the set of input qubit indices, $\text{Init} \subseteq Q$ be the set of initialised qubit indices, $\text{Obs} \subseteq Q$ be the set of output qubit indices, $\text{Di} \subseteq Q$ be the set of discarded qubit indices, and let C be a quantum circuit (unitary or hybrid) acting on Q . A *circuit tuple* is defined as:

$$\mathcal{T}(C) := (\text{Inp}, \text{Init}, \text{Obs}, \text{Di}, C).$$

This tuple must satisfy the following *well-formedness* conditions:

$$\text{Inp} \cup \text{Init} = Q, \quad (4.3)$$

$$\text{Obs} \cup \text{Di} = Q, \quad (4.4)$$

$$\text{Inp} \cap \text{Init} = \emptyset, \quad (4.5)$$

$$\text{Obs} \cap \text{Di} = \emptyset, \quad (4.6)$$

$$\forall \text{qb} \subseteq Q, \quad \text{if } \text{InitQ}(\text{qb}) \text{ is applied in } C, \text{ then } \text{qb} \subseteq \text{Init}. \quad (4.7)$$

Notation 4.3.8 (Circuit Tuple). We denote $\mathcal{T}(C)$ as $\mathcal{QC}$ when C is a unitary circuit, and as $\mathcal{HQC}$ when C is a hybrid circuit.

Given a circuit tuple $\mathcal{T}(C) := (\text{Inp}, \text{Init}, \text{Obs}, \text{Di}, C)$, and considering that Inp can be derived from Init and Obs can be derived from Di , we adopt the following simplified notation for convenience: $\mathcal{T}(C) := (\text{Init}, \text{Di}, C)$.

Definition 4.3.9 (Circuit Tuple Temporal Composition ($--$)). Let

$$\mathcal{T}_1(U_1) := (\text{Inp}_1, \text{Init}_1, \text{Obs}_1, \text{Di}_1, U_1), \text{ and, } \mathcal{T}_2(U_2) := (\text{Inp}_2, \text{Init}_2, \text{Obs}_2, \text{Di}_2, U_2)$$

be two unitary circuit tuples (Definition 4.3.7). The *temporal composition* of $\mathcal{T}_1(C_1)$ and $\mathcal{T}_2(C_2)$, denoted as $\mathcal{T}_1(C_1) -- \mathcal{T}_2(C_2)$, is defined by:

$$\mathcal{T}_1(C_1) -- \mathcal{T}_2(C_2) := (\text{Inp}_1, \text{Init}_1 \cup \text{Init}_2, \text{Obs}_2, \text{Di}_1 \cup \text{Di}_2, C_1 -- C_2),$$

where $C_1 -- C_2$ is the standard temporal composition of unitary circuits (Def. 2.2.40).

This composition is *well-formed* if it meets the following conditions, using the notations from Notation 2.1.2:

1. The output of the first circuit matches the input of the second: $\text{Obs}_1 \simeq \text{Inp}_2$.
2. No qubit is reused: $\text{Init}_2 \cap (\text{Obs}_1 \cup \text{Di}_1) = \emptyset$.

Definition 4.3.10 (Circuit Tuple Spatial Composition ($//$)). For all $i \in \{1, 2\}$, let $\mathcal{T}_i(C_i) := (\text{Init}_i, \text{Di}_i, U_i)$ be two circuit tuples (Definition 4.3.7). The *spatial composition* of $\mathcal{T}_1(C_1)$ and $\mathcal{T}_2(C_2)$, denoted as $\mathcal{T}_1(C_1) // \mathcal{T}_2(C_2)$, is defined by:

$$\mathcal{T}_1(C_1) // \mathcal{T}_2(C_2) := (\text{Inp}_1 \cup \text{Inp}_2, \text{Init}_1 \cup \text{Init}_2, \text{Obs}_1 \cup \text{Obs}_2, \text{Di}_1 \cup \text{Di}_2, C_1 // C_2),$$

where $C_1 // C_2$ is the standard spatial composition of brick-based representation of unitary circuits (Definition 2.2.40).

This composition is *well-formed* if and only if the following condition is satisfied:

$$(\text{Inp}_1 \cup \text{Init}_1 \cup \text{Obs}_1 \cup \text{Di}_1) \cap (\text{Inp}_2 \cup \text{Init}_2 \cup \text{Obs}_2 \cup \text{Di}_2) = \emptyset.$$

Definition 4.3.11 (Inverse of a Unitary Circuit Tuple). Let $\mathcal{QC} := (\text{Init}, \text{Di}, U)$ be a unitary circuit tuple (Definition 4.3.7), and let $U^\dagger$ be the adjoint of U (Definition 2.2.29), such that $UU^\dagger = I$. The *inverse* of $\mathcal{QC}$, denoted as $\mathcal{QC}^\dagger$, is defined by:

$$\mathcal{QC}^\dagger := (\text{Di}, \text{Init}, U^\dagger).$$

For example, consider the unitary circuit tuple $\mathcal{QC}$, as depicted in Figure 4.8a. Its inverse, according to Definition 4.3.11, is illustrated in Figure 4.8b.



Figure 4.8: Example: Inverse of a Unitary Circuit Tuple.

4.3.2 Unitary Partial Equivalences (Unit-Part)

This section introduces our unitary partial equivalence relations. We begin with sub-circuit unitary partial equivalence in Section 4.3.2.1. Next, we introduce global phase unitary partial equivalence in Section 4.3.2.2, followed by full-circuit unitary partial equivalence in Section 4.3.2.3. Section 4.3.2.4 presents the relations between these unitary partial equivalence relations.

4.3.2.1 Sub-Circuit Unitary Partial Equivalence (SubCir-Unit-Part)

We define sub-circuit unitary partial parallel equivalence (Definition 4.3.12) and sub-circuit unitary partial sequence equivalence (Definition 4.3.13). Lemma 4.3.14 shows that these two relations coincide.

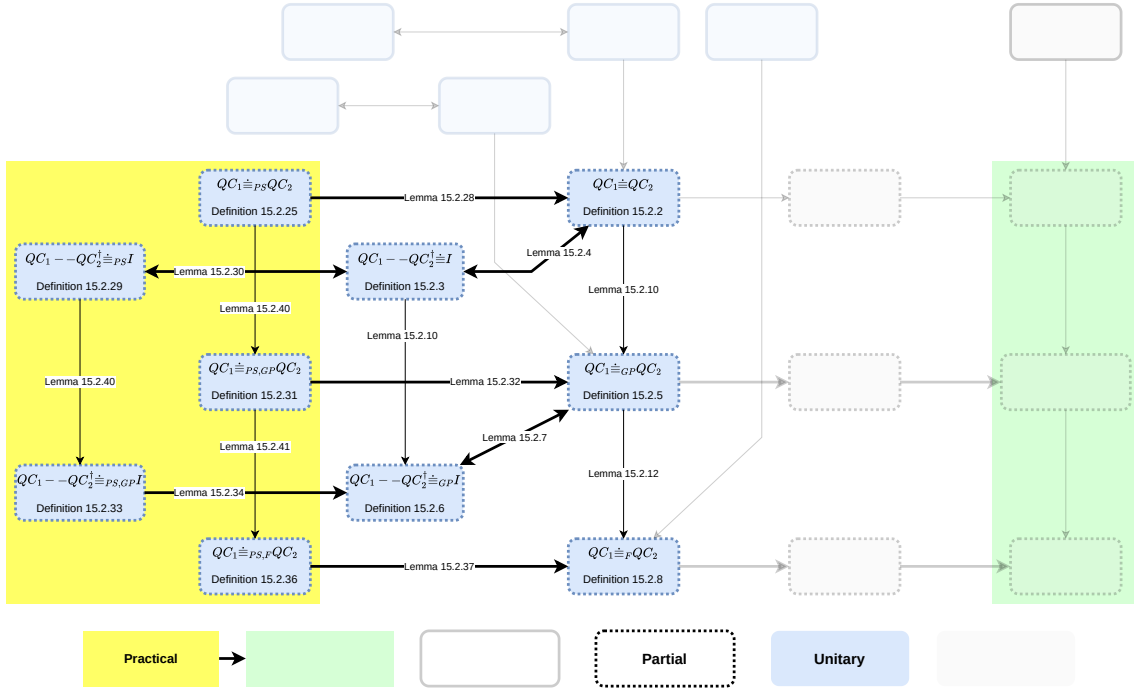


Figure 4.9: Unitary Partial Equivalence Relations

Definition 4.3.12 (SubCir-Unit-Part-Par). For $i \in \{1, 2\}$, let $QC_i = (\text{Init}_i, \text{Di}_i, U_i)$ be two unitary circuit tuples (Notation 4.3.8). For each i and for any input state $|\xi^i\rangle \in \mathcal{H}^{\text{Inp}_i}$, the circuit action is given by

$$\llbracket U_i \rrbracket \left(|\xi^i\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i} \right) = |f^i(\xi^i)\rangle_{\text{Obs}_i, \text{Di}_i}.$$

We assume that $|\xi^1\rangle_{\text{Inp}_1} \sim_{(\text{Inp}_1, \text{Inp}_2)} |\xi^2\rangle_{\text{Inp}_2}$ (Notation 2.2.23).

We define *sub-circuit unitary partial parallel equivalence* (SubCir-Unit-Part-Par) on such circuit tuples. We write $QC_1 \equiv QC_2$ if and only if the output and discard parts are separated (Definition 4.3.1), and the output states are equal up to mapping:

$$\begin{aligned} \forall i \in \{1, 2\}, \exists |\phi^i\rangle \in \mathcal{H}^{\text{Di}_i}, \forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \exists |\psi^i(\xi)\rangle \in \mathcal{H}^{\text{Obs}_i}, \\ |f^i(\xi)\rangle_{\text{Obs}_i, \text{Di}_i} = |\psi^i(\xi)\rangle_{\text{Obs}_i} \otimes |\phi^i\rangle_{\text{Di}_i} \end{aligned} \quad (4.8)$$

$$\left. \begin{aligned} \text{Obs}_1 &\simeq \text{Obs}_2 && (\text{Notation 2.1.2}) \\ |\psi^1(\xi)\rangle_{\text{Obs}_1} &\sim_{(\text{Obs}_1, \text{Obs}_2)} |\psi^2(\xi)\rangle_{\text{Obs}_2}. && (\text{Notation 2.2.23}) \end{aligned} \right\} \quad (4.9)$$

Thanks to this definition, we address the problem in Challenge (2) by ensuring Obs is separated from Di and that the state in Di is constant.

Definition 4.3.13 (SubCir-Unit-Part-Seq). Let $QC_1 = (\text{Init}_1, \text{Di}_1, U_1)$ and $QC_2 = (\emptyset, \emptyset, U_2)$ be two unitary circuit tuples (Notation 4.3.8). For any input state $|\xi\rangle \in \mathcal{H}^{\text{Inp}_1}$, the circuit

actions are given by

$$\begin{aligned} \llbracket U_1 \rrbracket \left(|\xi\rangle_{\text{Inp}_1} |0\rangle_{\text{Init}_1} \right) &= |f^1(\xi)\rangle_{\text{Obs}_1, \text{Di}_1}, \\ \llbracket U_1 - U_2^\dagger \rrbracket \left(|\xi\rangle_{\text{Inp}_1} |0\rangle_{\text{Init}_1} \right) &= |f(\xi)\rangle_{\text{Inp}_2, \text{Di}_1}. \end{aligned}$$

We assume that $|\xi^1\rangle_{\text{Inp}_1} \sim_{(\text{Inp}_1, \text{Inp}_2)} |\xi^2\rangle_{\text{Inp}_2}$ (Notation 2.2.23).

We define *sub-circuit unitary partial sequence equivalence* (SubCir-Unit-Part-Seq) on such circuit tuples. We write $\mathcal{QC}_1 \dot{=}_{(-)} \mathcal{QC}_2$ if and only if the output and discard parts are separated (Definition 4.3.1), and the sequence acts as the identity on the input up to mapping:

$$\begin{aligned} \exists |\phi^1\rangle \in \mathcal{H}^{\text{Di}_1}, \forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \exists |\psi^1(\xi)\rangle \in \mathcal{H}^{\text{Obs}_1}, \exists |\psi(\xi)\rangle \in \mathcal{H}^{\text{Inp}_2}, \\ |f^1(\xi)\rangle = |\psi^1(\xi)\rangle_{\text{Obs}_1} \otimes |\phi^1\rangle_{\text{Di}_1} \end{aligned} \quad (4.10)$$

$$\left. \begin{aligned} \text{Obs}_1 &\simeq \text{Obs}_2 && \text{(Notation 2.1.2)} \\ |f(\xi)\rangle &= |\psi(\xi)\rangle_{\text{Inp}_2} \otimes |\phi^1\rangle_{\text{Di}_1} \\ |\psi(\xi)\rangle_{\text{Inp}_1} &\sim_{(\text{Inp}_1, \text{Inp}_2)} |\xi\rangle_{\text{Inp}_2} && \text{(Notation 2.2.23)} \end{aligned} \right\} \quad (4.11)$$

Lemme 4.3.14 (Equivalence between SubCir-Unit-Part-Par and SubCir-Unit-Part-Seq). Let $\mathcal{QC}_1 = (\text{Init}_1, \text{Di}_1, U_1)$ and $\mathcal{QC}_2 = (\emptyset, \emptyset, U_2)$ be two unitary circuit tuples (Notation 4.3.8). Then $\mathcal{QC}_1 \dot{=} \mathcal{QC}_2$ if and only if $\mathcal{QC}_1 \dot{=}_{(-)} \mathcal{QC}_2$.

Proof. We assume that $|\xi^1\rangle_{\text{Inp}_1} \sim_{(\text{Inp}_1, \text{Inp}_2)} |\xi^2\rangle_{\text{Inp}_2}$ (Notation 2.2.23). We prove the two implications.

1. Assume $\mathcal{QC}_1 \dot{=} \mathcal{QC}_2$:

(a) Property 4.8 directly satisfies Property 4.10.

(b) By Property 4.9 and unitarity of U_2 :

$$\begin{aligned} \forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \quad \llbracket U_2^\dagger \rrbracket \left(|\psi^1(\xi)\rangle_{\text{Obs}_1} \right) &\sim_{(\text{Obs}_1, \text{Obs}_2)} \llbracket U_2^\dagger \rrbracket \left(|\psi^2(\xi)\rangle_{\text{Obs}_2} \right) \\ &\sim_{(\text{Obs}_2, \text{Inp})} |\xi\rangle_{\text{Inp}}. \end{aligned}$$

(c) Composing with $\llbracket U_1 \rrbracket$ gives:

$$\forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \quad \llbracket U_1 - U_2^\dagger \rrbracket \left(|\xi\rangle_{\text{Inp}} |0\rangle_{\text{Init}_1} \right) = |\xi\rangle_{\text{Inp}} \otimes |\phi^1(\xi)\rangle_{\text{Di}_1}.$$

Thus Property 4.11 is satisfied.

(d) Therefore, $\mathcal{QC}_1 \dot{=}_{(-)} \mathcal{QC}_2$.

2. Assume $\mathcal{QC}_1 \dot{=}_{(-)} \mathcal{QC}_2$:

(a) Property 4.10 directly satisfies Property 4.8.

(b) By Property 4.11: On one side,

$$\begin{aligned} \llbracket U_1 \dashv \dashv U_2^\dagger \dashv \dashv U_2 \rrbracket \left(|\xi\rangle_{\text{Inp}_1} |0\rangle_{\text{Init}_1} \right) &= \llbracket U_2 \rrbracket \circ \llbracket U_2^\dagger \rrbracket \circ \llbracket U_1 \rrbracket \left(|\xi\rangle_{\text{Inp}_1} |0\rangle_{\text{Init}_1} \right) \\ &= |\psi^1(\xi)\rangle_{\text{Obs}_1} \otimes |\phi^1(\xi)\rangle_{\text{Di}_1}. \end{aligned}$$

On the other side,

$$\begin{aligned} \llbracket U_2 \rrbracket \circ \llbracket U_1 \dashv \dashv U_2^\dagger \rrbracket \left(|\xi\rangle_{\text{Inp}} |0\rangle_{\text{Init}_1} \right) &= \llbracket U_2 \rrbracket \left(|\xi\rangle_{\text{Inp}} \otimes |\phi^1(\xi)\rangle_{\text{Di}_1} \right) \\ &= |\psi^2(\xi)\rangle_{\text{Obs}_2} \otimes |\phi^1(\xi)\rangle_{\text{Di}_1}. \end{aligned}$$

Thus, $\forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, |\psi^1(\xi)\rangle_{\text{Obs}_1} \sim_{(\text{Obs}_1, \text{Obs}_2)} |\psi^2(\xi)\rangle_{\text{Obs}_2}$. Hence Property 4.9 holds.

(c) Therefore, $\mathcal{QC}_1 \doteq \mathcal{QC}_2$. □

4.3.2.2 Global Phase Unitary Partial Equivalence (GP-Unit-Part)

In this section, we define two partial equivalence relations: global-phase unitary partial parallel equivalence (Definition 4.3.15) and global-phase unitary partial sequence equivalence (Definition 4.3.16). We also state Lemma 4.3.17, which shows that these two relations coincide. As before, the definition for global-phase is almost identical to that for sub-circuit in Section 4.3.2.1; the only difference is that vector equality = (Definition 2.2.10) is replaced by global phase equivalence $\approx_{\text{gp}}$ (Definition 2.2.12).

Definition 4.3.15 (GP-Unit-Part-Par). For $i \in \{1, 2\}$, let $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, U_i)$ be two unitary circuit tuples (Notation 4.3.8). For each i and for any input state $|\xi^i\rangle \in \mathcal{H}^{\text{Inp}_i}$, the circuit action is given by

$$\llbracket U_i \rrbracket \left(|\xi^i\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i} \right) = |f^i(\xi^i)\rangle_{\text{Obs}_i, \text{Di}_i}.$$

We assume that $|\xi^1\rangle_{\text{Inp}_1} \sim_{(\text{Inp}_1, \text{Inp}_2)} |\xi^2\rangle_{\text{Inp}_2}$ (Notation 2.2.23).

We define *global-phase unitary partial parallel equivalence* (GP-Unit-Part-Par) on such circuit tuples. We write $\mathcal{QC}_1 \doteq_{\text{gp}} \mathcal{QC}_2$ if and only if the output and discard parts are separated (Definition 4.3.1), and the output states are equal up to global phase and qubit mapping:

$$\begin{aligned} \forall i \in \{1, 2\}, \exists |\phi^i\rangle \in \mathcal{H}^{\text{Di}_i}, \forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \exists |\psi^i(\xi)\rangle \in \mathcal{H}^{\text{Obs}_i}, \\ |f^i(\xi)\rangle_{\text{Obs}_i, \text{Di}_i} = |\psi^i(\xi)\rangle_{\text{Obs}_i} \otimes |\phi^i\rangle_{\text{Di}_i} \quad \text{and} \end{aligned} \tag{4.12}$$

$$\left. \begin{aligned} \text{Obs}_1 &\simeq \text{Obs}_2 && \text{(Notation 2.1.2)} \\ |\psi^1(\xi)\rangle_{\text{Obs}_1} &\sim_{(\text{Obs}_1, \text{Obs}_2, \text{gp})} |\psi^2(\xi)\rangle_{\text{Obs}_2} && \text{(Notation 2.2.23)} \end{aligned} \right\} \tag{4.13}$$

Definition 4.3.16 (GP-Unit-Part-Seq). Let $\mathcal{QC}_1 = (\text{Init}_1, \text{Di}_1, U_1)$ and $\mathcal{QC}_2 = (\emptyset, \emptyset, U_2)$ be two unitary circuit tuples (Notation 4.3.8). For any input state $|\xi^1\rangle \in \mathcal{H}^{\text{Inp}_1}$, the circuit actions are given by

$$\begin{aligned} \llbracket U_1 \rrbracket (|\xi^1\rangle_{\text{Inp}_1} |0\rangle_{\text{Init}_1}) &= |f^1(\xi^1)\rangle_{\text{Obs}_1, \text{Di}_1}, \\ \llbracket U_1 - U_2^\dagger \rrbracket (|\xi^1\rangle_{\text{Inp}_1} |0\rangle_{\text{Init}_1}) &= |f(\xi^1)\rangle_{\text{Inp}_2, \text{Di}_1}. \end{aligned}$$

We assume that $|\xi^1\rangle_{\text{Inp}_1} \sim_{(\text{Inp}_1, \text{Inp}_2)} |\xi^2\rangle_{\text{Inp}_2}$ (Notation 2.2.23).

We define *global-phase unitary partial sequence equivalence* (GP-Unit-Part-Seq) on such circuit tuples. We write $\mathcal{QC}_1 \dot{=}_{(-, \text{gp})} \mathcal{QC}_2$ if and only if the output and discard parts are separated (Definition 4.3.1), and the sequence acts as the identity on the input up to global phase and qubit mapping:

$$\begin{aligned} \forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \exists |\psi^1(\xi)\rangle \in \mathcal{H}^{\text{Obs}_1}, \exists |\phi^1\rangle \in \mathcal{H}^{\text{Di}_1}, \exists |\psi(\xi)\rangle \in \mathcal{H}^{\text{Inp}_2}, \\ |f^1(\xi)\rangle = |\psi^1(\xi)\rangle_{\text{Obs}_1} \otimes |\phi^1\rangle_{\text{Di}_1} \end{aligned} \quad (4.14)$$

$$\left. \begin{aligned} \text{Obs}_1 &\simeq \text{Obs}_2 && (\text{Notation 2.1.2}) \\ |f(\xi)\rangle &= |\psi(\xi)\rangle_{\text{Inp}_2} \otimes |\phi^1\rangle_{\text{Di}_1} \\ |\psi(\xi)\rangle_{\text{Inp}_1} &\sim_{(\text{Inp}_1, \text{Inp}_2, \text{gp})} |\xi\rangle_{\text{Inp}_2} && (\text{Notation 2.2.23}) \end{aligned} \right\} \quad (4.15)$$

Lemme 4.3.17 (Equivalence between GP-Unit-Part-Par and GP-Unit-Part-Seq). Let $\mathcal{QC}_1 = (\text{Init}_1, \text{Di}_1, U_1)$ and $\mathcal{QC}_2 = (\emptyset, \emptyset, U_2)$ be two unitary circuit tuples (Notation 4.3.8). Then $\mathcal{QC}_1 \dot{=}_{\text{gp}} \mathcal{QC}_2$ if and only if $\mathcal{QC}_1 \dot{=}_{(-, \text{gp})} \mathcal{QC}_2$.

Proof. The proof follows the same structure as that of Lemma 4.3.14, replacing vector equality (Definition 2.2.10) with global phase equivalence (Definition 2.2.12). $\square$

4.3.2.3 Full-Circuit Unitary Partial Equivalence (FullCir-Unit-Part)

This section introduces full-circuit unitary partial equivalence, corresponding to the state-of-the-art approach to partial unitary equivalence. This method measures the entire register while restricting comparison to the observed part; the probabilities are therefore summed over all possible values of the remaining qubits. Tools such as QCEC [12]³ use this type of equivalence relation. In Section 4.3.18, we define full-circuit unitary partial equivalence. We recall that sequential equivalence cannot be defined for full-circuit equivalence, as stated in Lemma 4.2.16.

Definition 4.3.18 (Full-Circuit Unitary Partial Equivalence (FullCir-Unit-Part-Par)). For $i \in \{1, 2\}$, let $\mathcal{QC}_i := (\text{Inp}_i, \text{Init}_i, \text{Obs}_i, \text{Di}_i, U_i)$ be two unitary circuit tuples (Notation 4.3.8), and let $\mathcal{H}^{\text{Inp}_i}$ be the Hilbert spaces associated with the input qubits Inp_i . For any quantum state $|\xi^i\rangle \in \mathcal{H}^{\text{Inp}_i}$, we assume that $|\xi^1\rangle_{\text{Inp}_1} \sim_{(\text{Inp}_1, \text{Inp}_2)} |\xi^2\rangle_{\text{Inp}_2}$ (Notation 2.2.23); then

$$\llbracket U_i \rrbracket (|\xi^i\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i}) = |f^i(\xi^i)\rangle_{\text{Obs}_i, \text{Di}_i}.$$

³https://mqt.readthedocs.io/projects/qcec/en/latest/partial_equivalence.html

We define *full-circuit unitary partial equivalence* (FullCir-Unit-Part-Par) on such tuples. We write $\mathcal{QC}_1 \dot{=} \mathcal{QC}_2$ if and only if

$$\left. \begin{array}{l} \forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \\ \text{Obs}_1 \simeq \text{Obs}_2 \quad (\text{Notation 2.1.2}) \\ d(f^1(\xi), \text{Obs}_1) = d(f^2(\xi), \text{Obs}_2). \quad (\text{Definition 4.3.5}) \end{array} \right\} \quad (4.16)$$

4.3.2.4 Relations Between Unitary Partial Equivalence

In this section, we present the relationships between the partial unitary equivalence relations. Although these results are analogous to those established for complete equivalence, we include them for completeness. Lemma 4.3.19 states the relation between SubCir-Unit-Part-Par and GP-Unit-Part-Par. Lemma 4.3.21 states the relation between GP-Unit-Part-Par and FullCir-Unit-Part-Par. Finally, Theorem 4.3.22 states the relation between SubCir-Unit-Part-Par and FullCir-Unit-Part-Par. The statements are given only for the parallel relations, since whenever the sequential relation exists, it is equivalent to the parallel one.

Lemme 4.3.19 (SubCir-Unit-Part-Par implies GP-Unit-Part-Par). For $i \in \{1, 2\}$, let $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, \text{U}_i)$ be two unitary circuit tuples (Notation 4.3.8). If $\mathcal{QC}_1 \dot{=} \mathcal{QC}_2$ (Def. 4.3.12), then $\mathcal{QC}_1 \dot{=}_{\text{gp}} \mathcal{QC}_2$ (Def. 4.3.15).

Proof. Assume $\mathcal{QC}_1 \dot{=} \mathcal{QC}_2$. Property 4.8 directly implies Property 4.14. By Property 4.9 and Definition 2.2.12, Property 4.13 holds. Therefore $\mathcal{QC}_1 \dot{=}_{\text{gp}} \mathcal{QC}_2$. $\square$

Lemme 4.3.20 (Global-Phase Equivalence implies Probability Distribution Equality). Let $|\psi^1\rangle_{S_1}$ and $|\psi^2\rangle_{S_2}$ be two quantum states on the sets of qubits S_1 and S_2 , respectively. If $|\psi^1\rangle_{S_1} \sim_{(S_1, S_2, \text{gp})} |\psi^2\rangle_{S_2}$ and $S_1 \simeq S_2$ (Notation 2.2.23), then their probability distributions are identical (Definition 4.3.6):

$$d(\psi^1, S_1) = d(\psi^2, S_2).$$

Proof. Assume $|\psi^1\rangle_{S_1} \sim_{(S_1, S_2, \text{gp})} |\psi^2\rangle_{S_2}$ and $S_1 \simeq S_2$ (Notation 2.1.2). Then there exists $\alpha \in \mathbb{R}$ such that $|\psi^2\rangle_{S_2} = e^{i\alpha} |\psi^1\rangle_{S_1}$. By Definition 4.3.5:

$$\begin{aligned} d(\psi^1, S_1) &= \{p(\psi^1, S_1, x_{|S_1}), x_{|S_1}\}_{x_{|S_1}} & (\text{Notation 2.1.3}) \\ d(\psi^2, S_2) &= \{p(\psi^2, S_2, x_{|S_2}), x_{|S_2}\}_{x_{|S_2}} \\ &= \{p(\psi^1, S_1, x_{|S_1}), x_{|S_1}\}_{x_{|S_1}} & (\text{Definition 4.3.4}) \\ &= d(\psi^1, S_1). & \square \end{aligned}$$

Lemme 4.3.21 (GP-Unit-Part-Par implies FullCir-Unit-Part-Par). Let $\mathcal{QC}_1$ and $\mathcal{QC}_2$ be two unitary circuit tuples (Notation 4.3.8). If $\mathcal{QC}_1 \dot{=}_{\text{gp}} \mathcal{QC}_2$ (Definition 4.3.15), then $\mathcal{QC}_1 \dot{=} \mathcal{QC}_2$ (Definition 4.3.18).

Proof. By Property 4.13 and Lemma 4.3.20, Property 4.16 holds. Therefore, by Definition 4.3.18, $QC_1 \stackrel{\text{f}}{\equiv} QC_2$. $\square$

Theorem 4.3.22 (SubCir-Unit-Part-Par implies FullCir-Unit-Part-Par). Let QC_1 and QC_2 be two unitary circuit tuples (Notation 4.3.8). If $QC_1 \stackrel{\text{g}}{\equiv} QC_2$ (Definition 4.3.12), then $QC_1 \stackrel{\text{f}}{\equiv} QC_2$ (Definition 4.3.18).

Proof. Assume $QC_1 \stackrel{\text{g}}{\equiv} QC_2$, then by Lemma 4.3.19, $QC_1 \stackrel{\text{gp}}{\equiv} QC_2$, thus by Lemma 4.3.21, $QC_1 \stackrel{\text{f}}{\equiv} QC_2$. $\square$

4.3.3 Path-Sum Partial Equivalences (PS-Part)

We begin this section by introducing the notion of path-sum equality and equivalence up to variable renaming in Section 4.3.3.1. Next, we present our extension of path-sums to quantum multi-control, together with proofs of correctness, in Section 4.3.3.2. We then define partial equivalence relations using path-sums in Sections 4.3.3.3, 4.3.3.4, and 4.3.3.5. Finally, we state the relations between these path-sum-based partial equivalence relations in Section 4.3.3.6. Figure 4.10 illustrates the relationships between our path-sum partial unitary equivalences.

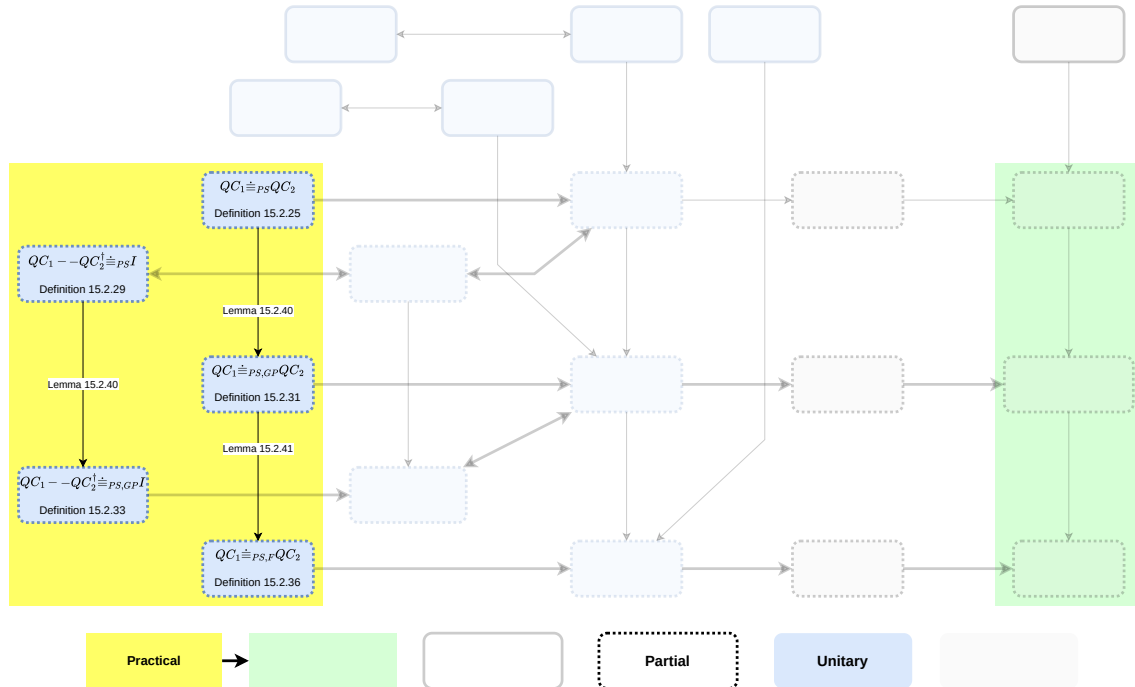


Figure 4.10: Path-Sum Partial Unitary Equivalence Relations

4.3.3.1 Path-Sum Relations

In this section, we give the notation of path-sum equality (Notation 4.3.23) and the definition of a path-sum equivalence up to renaming (Definition 4.3.24).

Notation 4.3.23 (Path-Sum Equality). Let ps_1 and ps_2 be two path-sums (Def. 2.3.3). We denote by $ps_1 = ps_2$ the syntactic equality of path-sums. For example, let $s_1 = \{0\}$, $s_2 = \{4\}$, $s'_1 = \{0, 1\}$, $s'_2 = \{4, 3\}$ be four indices sets indices, then

$$\left\langle y_0 y_1, \frac{x_0 \cdot y_0 + y_0 \cdot y_1}{2}, y_1 \right\rangle \neq \left\langle y_4 y_3, \frac{x_4 \cdot y_3 + y_3 \cdot y_4}{2}, y_3 \right\rangle$$

Definition 4.3.24 (Path-Sum Equivalence Up To Renaming). Let $ps_1 = \langle y^1, p^1(x^1, y^1), f^1(x^1, y^1) \rangle$ and $ps_2 = \langle y^2, p^2(x^2, y^2), f^2(x^2, y^2) \rangle$ be two path-sums (Definition 2.3.3). We denote by $ps_1 \sim_{\text{ps}} ps_2$ the equivalence of path-sums up to variable renaming.

$$ps_1 \sim_{\text{ps}} ps_2 \iff \begin{cases} p^2(x^2, y^2) = (\sigma, \sigma')(p^1(x^1, y^1)) \\ f^2(x^2, y^2) = (\sigma, \sigma')(f^1(x^1, y^1)), \end{cases}$$

where

1. $s_1 = \text{Indices}(x^1)$, $s_2 = \text{Indices}(x^2)$, $s'_1 = \text{Indices}(y^1)$, and $s'_2 = \text{Indices}(y^2)$ be four set of indices (Notation 2.1.3).
2. σ (resp. σ') be the unique permutation mapping the elements of s_1 (resp. s'_1) in increasing order to those of s_2 (resp. s'_2) in increasing order, assuming $|s_1| = |s_2|$ (resp. $|s'_1| = |s'_2|$).
3. $(\sigma, \sigma')(p(x, y))$ denote the expressions obtained by replacing each variable x_i by $x_{\sigma(i)}$ and each y_j by $y_{\sigma'(j)}$
4. $(\sigma, \sigma')(f(x, y))$ denote the expressions obtained by replacing each variable x_i by $x_{\sigma(i)}$ and each y_j by $y_{\sigma'(j)}$

For example, let $s_1 = \{0\}$, $s_2 = \{4\}$, $s'_1 = \{0, 1\}$, $s'_2 = \{4, 3\}$ be four sets of indices, then

$$\left\langle y_0 y_1, \frac{x_0 \cdot y_0 + y_0 \cdot y_1}{2}, y_1 \right\rangle \sim_{\text{ps}} \left\langle y_4 y_3, \frac{x_4 \cdot y_3 + y_3 \cdot y_4}{2}, y_3 \right\rangle$$

4.3.3.2 Path-Sum Extension to Multi-Control

In this chapter, our goal is to define a practical method for checking hybrid circuit equivalence with discard. We leverage the well-known deferred measurement transformation to extend unitary equivalence to the hybrid case. The path-sum semantics, as defined in Definition 2.3.7, are chosen to implement unitary equivalence practically. However, since deferred measurement transforms classical control into quantum control by introducing quantum multi-control, we must extend the path-sum semantics to

accommodate it, which is the objective of this section. Definition 4.3.25 formalises this semantics. Theorem 4.3.26 establishes its equivalence with our bricks unitary semantics (Definition 2.2.40).

Definition 4.3.25 (Extended Path-Sum Semantics (EPS)). Let n be the size of a quantum register, with $\text{co}, \text{ta} \in [0, n - 1]$ denoting the indices of control and target qubits, respectively. For a list l of qubit indices, define $\Pi(x_l) = \prod_{i \in l} x_i$. The semantics is

$$\begin{aligned}
\llbracket \text{E} \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle) &= \langle \emptyset, 0, x \rangle \\
\llbracket \text{Apply}(\text{GP}(\text{s}, \text{k}), \text{co}, \text{ta}) \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle) &= \left\langle \emptyset, \frac{\text{s}}{2^{\text{k}}} \Pi(x_{\text{co}}), x \right\rangle \\
\llbracket \text{Apply}(\text{X}, \text{co}, \text{ta}) \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle) &= \langle \emptyset, 0, x_0, \dots, x_{\text{ta}-1}, x_{\text{ta}} \oplus \Pi(x_{\text{co}}), x_{\text{ta}+1}, \dots, x_{n-1} \rangle \\
\llbracket \text{Apply}(\text{U}_1(\text{s}, \text{k}), \text{co}, \text{ta}) \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle) &= \left\langle \emptyset, \frac{\text{s}}{2^{\text{k}}} (\Pi(x_{\text{co}}) \Pi(x_{\text{ta}})), x \right\rangle \\
\llbracket \text{Apply}(\text{H}, \text{co}, \text{ta}) \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle) &= \left\langle \{y\}, \frac{1}{8} + \frac{7}{8} \Pi(x_{\text{co}}) + \frac{1}{2} \Pi(x_{\text{co}}) \Pi(x_{\text{ta}}) y + \frac{1}{4} \Pi(x_{\text{co}}) y + \frac{3}{4} y, \right. \\
&\quad \left. x_0, \dots, x_{\text{ta}-1}, \Pi(x_{\text{co}}) y + \neg \Pi(x_{\text{co}}) x_{\text{ta}}, x_{\text{ta}+1}, \dots, x_{n-1} \right\rangle. \\
\llbracket \text{Sequence}(\text{SV}_1, \text{SV}_2) \rrbracket_{\text{PS}} &:= \llbracket \text{SV}_2 \rrbracket_{\text{PS}} \circ \llbracket \text{SV}_1 \rrbracket_{\text{PS}}
\end{aligned}$$

This semantics is extended by linearity to all input states.

Theorem 4.3.26 (Equivalence of EPS Semantics and Matrix Semantics). Let $\mathcal{H}^{\otimes n}$ be a Hilbert space of n qubits, and let U be a unitary circuit of type SV . EPS (Definition 4.3.25) is equivalent to the matrix semantics (Definition 2.2.40). Specifically, for every bitstring $x \in \{0, 1\}^n$ with corresponding basis state $|x\rangle = \mathcal{V}(\langle \emptyset, 0, x \rangle) \in \mathcal{H}^{\otimes n}$,

$$\mathcal{V}(\llbracket \text{U} \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)) : x \mapsto \llbracket \text{U} \rrbracket(|x\rangle).$$

By linearity, this equivalence extends to all quantum states in $\mathcal{H}^{\otimes n}$.

Proof. We proceed by structural induction on U . Let $\mathcal{H}^{\otimes n}$ be a Hilbert space of n qubits. For every bitstring $x \in \{0, 1\}^n$, there exists a corresponding basis state $|x\rangle \in \mathcal{H}^{\otimes n}$ such that $\mathcal{V}(\langle \emptyset, 0, x \rangle) : x \mapsto |x\rangle$. We show that, for any U of type SV ,

$$\mathcal{V}(\llbracket \text{U} \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)) : x \mapsto \llbracket \text{U} \rrbracket(|x\rangle).$$

By linearity, this equivalence extends to all quantum states in $\mathcal{H}^{\otimes n}$.

1. If $\text{U} = \text{E}$, then $\mathcal{V}(\llbracket \text{E} \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)) = \mathcal{V}(\langle \emptyset, 0, x \rangle)$, thus $\mathcal{V}(\langle \emptyset, 0, x \rangle) : x \mapsto \llbracket \text{E} \rrbracket(|x\rangle)$.
2. If $\text{U} = \text{Apply}(\text{GP}(\text{s}, \text{k}), \text{co}, \text{ta})$:

- (a) If $\Pi(x_{\text{co}}) = 0$, then

$$\begin{aligned}
\mathcal{V}(\llbracket \text{Apply}(\text{GP}(\text{s}, \text{k}), \text{co}, \text{ta}) \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)) &= \mathcal{V}(\langle \emptyset, 0, x \rangle) \\
\mathcal{V}(\langle \emptyset, 0, x \rangle) : x &\mapsto \llbracket \text{I}^{\text{ta}} \rrbracket(|x\rangle).
\end{aligned}$$

(b) If $\Pi(x_{\text{co}}) = 1$, then

$$\begin{aligned}\mathcal{V}(\llbracket \text{Apply}(\text{GP}(\text{s}, \text{k}), \text{co}, \text{ta}) \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)) &= \mathcal{V}\left(\left\langle \emptyset, \frac{\text{s}}{2^{\text{k}}}, x \right\rangle\right) \\ \mathcal{V}\left(\left\langle \emptyset, \frac{\text{s}}{2^{\text{k}}}, x \right\rangle\right) : x &\mapsto \llbracket \text{GP}^{\text{ta}}\left(2\pi \frac{\text{s}}{2^{\text{k}}}\right) \rrbracket(|x\rangle).\end{aligned}$$

3. If $\text{U} = \text{Apply}(\text{X}, \text{co}, \text{ta})$:

(a) If $\Pi(x_{\text{co}}) = 0$, then

$$\begin{aligned}\mathcal{V}(\llbracket \text{Apply}(\text{X}, \text{co}, \text{ta}) \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)) &= \mathcal{V}(\langle \emptyset, 0, x_0, \dots, x_{\text{ta}-1}, x_{\text{ta}}, x_{\text{ta}+1}, \dots, x_{n-1} \rangle) \\ \mathcal{V}(\langle \emptyset, 0, x_0, \dots, x_{\text{ta}-1}, x_{\text{ta}}, x_{\text{ta}+1}, \dots, x_{n-1} \rangle) : x &\mapsto \llbracket \text{I} \rrbracket(|x\rangle).\end{aligned}$$

(b) If $\Pi(x_{\text{co}}) = 1$, then

$$\begin{aligned}\mathcal{V}(\llbracket \text{Apply}(\text{X}, \text{co}, \text{ta}) \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)) &= \mathcal{V}(\langle \emptyset, 0, x_0, \dots, x_{\text{ta}-1}, \neg x_{\text{ta}}, x_{\text{ta}+1}, \dots, x_{n-1} \rangle) \\ \mathcal{V}(\langle \emptyset, 0, x_0, \dots, x_{\text{ta}-1}, \neg x_{\text{ta}}, x_{\text{ta}+1}, \dots, x_{n-1} \rangle) : x &\mapsto \llbracket \text{X}^{\text{ta}} \rrbracket(|x\rangle).\end{aligned}$$

4. If $\text{U} = \text{Apply}(\text{U}_1(\text{s}, \text{k}), \text{co}, \text{ta})$:

(a) If $\Pi(x_{\text{co}}) = 0$, then

$$\begin{aligned}\mathcal{V}(\llbracket \text{Apply}(\text{U}_1(\text{s}, \text{k}), \text{co}, \text{ta}) \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)) &= \mathcal{V}(\langle \emptyset, 0, x \rangle) \\ \mathcal{V}(\langle \emptyset, 0, x \rangle) : x &\mapsto \llbracket \text{I} \rrbracket(|x\rangle).\end{aligned}$$

(b) If $\Pi(x_{\text{co}}) = 1$, then

$$\begin{aligned}\mathcal{V}(\llbracket \text{Apply}(\text{U}_1(\text{s}, \text{k}), \text{co}, \text{ta}) \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)) &= \mathcal{V}\left(\left\langle \emptyset, \frac{\text{s}}{2^{\text{k}}} \Pi(x_{\text{ta}}), x \right\rangle\right) \\ \mathcal{V}\left(\left\langle \emptyset, \frac{\text{s}}{2^{\text{k}}} \Pi(x_{\text{ta}}), x \right\rangle\right) : x &\mapsto \llbracket \text{U}_1^{\text{ta}}\left(2\pi \frac{\text{s}}{2^{\text{k}}}\right) \rrbracket(|x\rangle).\end{aligned}$$

5. If $\text{U} = \text{Apply}(\text{H}, \text{co}, \text{ta})$:

(a) If $\Pi(x_{\text{co}}) = 0$, then

$$\begin{aligned}\mathcal{V}(\llbracket \text{Apply}(\text{H}, \text{co}, \text{ta}) \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)) &= \mathcal{V}\left(\left\langle y, \frac{1}{8} + \frac{3}{4}y, x_0, \dots, x_{\text{ta}-1}, x_{\text{ta}}, x_{\text{ta}+1}, \dots, x_{n-1} \right\rangle\right) \\ \mathcal{V}\left(\left\langle y, \frac{1}{8} + \frac{3}{4}y, x_0, \dots, x_{\text{ta}-1}, x_{\text{ta}}, x_{\text{ta}+1}, \dots, x_{n-1} \right\rangle\right) : x &\mapsto \frac{1}{\sqrt{2}} \sum_{y \in \{0,1\}} e^{2\pi i(\frac{1}{8} + \frac{3}{4}y)} |x\rangle = \frac{1}{\sqrt{2}}(e^{\frac{\pi i}{4}} + e^{-\frac{\pi i}{4}}) |x\rangle \\ &= |x\rangle = \llbracket \text{I} \rrbracket(|x\rangle)\end{aligned}$$

(b) If $\Pi(x_{co}) = 1$, then

$$\begin{aligned}
& \mathcal{V}(\llbracket \text{Apply}(\text{H}, \text{co}, \text{ta}) \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)) \\
&= \mathcal{V}\left(\left\langle y, \frac{1}{2}\Pi(x_{\text{ta}})y, x_0, \dots, x_{\text{ta}-1}, y, x_{\text{ta}+1}, \dots, x_{n-1} \right\rangle\right) \\
& \mathcal{V}\left(\left\langle y, \frac{1}{2}\Pi(x_{\text{ta}})y, x_0, \dots, x_{\text{ta}-1}, y, x_{\text{ta}+1}, \dots, x_{n-1} \right\rangle\right) : x \\
& \mapsto \frac{1}{\sqrt{2}} \sum_{y \in \{0,1\}} e^{\Pi(x_{\text{ta}})y\pi i} |x_0, \dots, x_{\text{ta}-1}, y, x_{\text{ta}+1}, \dots, x_{n-1}\rangle \\
&= \frac{1}{\sqrt{2}}(|x_0, \dots, x_{\text{ta}-1}, 0, x_{\text{ta}+1}, \dots, x_{n-1}\rangle \\
& \quad + (-1)^{\Pi(x_{\text{ta}})} |x_0, \dots, x_{\text{ta}-1}, 1, x_{\text{ta}+1}, \dots, x_{n-1}\rangle) \\
&= \llbracket \text{H}^{\text{ta}} \rrbracket(|x\rangle).
\end{aligned}$$

6. If $U = \text{Sequence}(\text{Apply}(U_1, \text{co}_1, \text{ta}_1), \text{Apply}(U_2, \text{co}_2, \text{ta}_2))$: Let U_1 and U_2 be unitary circuits, by induction hypothesis:

$$\mathcal{V}(\llbracket U_1 \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)) : x \mapsto \llbracket U_1 \rrbracket(|x\rangle), \quad (4.17)$$

$$\mathcal{V}(\llbracket U_2 \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)) : x \mapsto \llbracket U_2 \rrbracket(|x\rangle). \quad (4.18)$$

For any basis state $\mathcal{V}(\langle \emptyset, 0, x \rangle) : x \mapsto |x\rangle \in \mathcal{H}^{\otimes n}$, we have:

$$\begin{aligned}
\mathcal{V}(\llbracket \text{Sequence}(U_1, U_2) \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)) &= \mathcal{V}(\llbracket U_2 \rrbracket_{\text{PS}} \circ \llbracket U_1 \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)) \\
&= \mathcal{V}(\llbracket U_2 \rrbracket_{\text{PS}}(\mathcal{V}(\llbracket U_1 \rrbracket_{\text{PS}}(\langle \emptyset, 0, x \rangle)))) \\
&= \mathcal{V}(\llbracket U_2 \rrbracket_{\text{PS}}(\llbracket U_1 \rrbracket(|x\rangle))) \quad (\text{By 4.17})
\end{aligned}$$

$$\begin{aligned}
\mathcal{V}(\llbracket U_2 \rrbracket_{\text{PS}}(\llbracket U_1 \rrbracket(|x\rangle))) &: x \mapsto \llbracket U_2 \rrbracket(\llbracket U_1 \rrbracket(|x\rangle)) \quad (\text{By 4.18}) \\
&= \llbracket \text{Sequence}(U_1, U_2) \rrbracket(|x\rangle). \quad \square
\end{aligned}$$

4.3.3.3 Sub-Circuit Path-Sum Partial Equivalence (SubCir-PS-Part)

In this section, we introduce sub-circuit path-sum partial parallel equivalence (Definition 4.3.27) and prove its correctness (Lemma 4.3.28). We then introduce sub-circuit path-sum partial sequential equivalence (Definition 4.3.29) and prove its correctness (Lemma 4.3.30).

Definition 4.3.27 (SubCir-PS-Part-Par). For $i \in \{1, 2\}$, let $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, U_i)$ be two unitary circuit tuples such that

$$\llbracket U_i \rrbracket_{\text{PS}}(\langle \emptyset, 0, x_{|\text{Inp}_i}|0_{|\text{Init}_i}\rangle) = \left\langle y^i, p^i(x, y^i), f^i(x, y^i)_{|\text{Obs}_i, \text{Di}_i} \right\rangle.$$

We assume that $\langle \emptyset, 0, x_{|\text{Inp}_1} \rangle \sim_{\text{ps}} \langle \emptyset, 0, x_{|\text{Inp}_2} \rangle$ (Definition 4.3.24). We define *sub-circuit path-sum partial parallel equivalence* (SubCir-PS-Part-Par) on such tuples. We write

$$\mathcal{QC}_1 \dot{=}_{(\text{ps})} \mathcal{QC}_2$$

if and only if (4.19) the observed and discard parts share no variables and the discard part contains only path variables, and (4.20) the observed parts are equal. Formally:

$$\left. \begin{aligned} \forall i \in \{1, 2\}, \\ y^i = y^{i'} y^{i''}, \\ f^i(x, y^i)_{|\text{Obs}_i, \text{Di}_i} = f^{i'}(x, y^{i'})_{|\text{Obs}_i} f^{i''}(y^{i''})_{|\text{Di}_i}, \\ p^i(x, y^i) = p^{i'}(x, y^{i'}) + p^{i''}(y^{i''}), \end{aligned} \right\} \quad (4.19)$$

$$\left. \begin{aligned} \text{Obs}_1 \simeq \text{Obs}_2 & \quad (\text{Notation 2.1.2}) \\ \langle y^{1'}, p^{1'}(x, y^{1'}), f^{1'}(x, y^{1'})_{|\text{Obs}_1} \rangle & \quad (\text{Definition 4.3.24}) \\ \sim_{\text{ps}} \langle y^{2'}, p^{2'}(x, y^{2'}), f^{2'}(x, y^{2'})_{|\text{Obs}_2} \rangle & \end{aligned} \right\} \quad (4.20)$$

Lemme 4.3.28 (Correctness of $\dot{=}_{(\text{ps})}$ for $\dot{=}$). The relation $\dot{=}_{(\text{ps})}$, as defined in Definition 4.3.27, is a correct implementation of $\dot{=}$, as defined in Definition 4.3.12. Formally, for any two unitary circuit tuples $\mathcal{QC}_1$ and $\mathcal{QC}_2$ (Notation 4.3.8),

$$\mathcal{QC}_1 \dot{=}_{(\text{ps})} \mathcal{QC}_2 \implies \mathcal{QC}_1 \dot{=} \mathcal{QC}_2.$$

Proof. Let $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, \text{U}_i)$ for $i \in \{1, 2\}$. Assume that $\text{Inp}_1 = \text{Inp}_2$ (Notation 2.1.2). Let $x \in \{0, 1\}^{\text{Inp}}$ be a bit-string such that

$$\llbracket \text{U}_i \rrbracket_{\text{PS}} (\langle \emptyset, 0, x_{|\text{Inp}_i} 0_{|\text{Init}_i} \rangle) = \langle y^i, p^i(x, y^i), f^i(x, y^i)_{|\text{Obs}_i, \text{Di}_i} \rangle.$$

Assume that $\mathcal{QC}_1 \dot{=}_{(\text{ps})} \mathcal{QC}_2$. By Property 4.19:

$$\begin{aligned} & \langle y^i, p^i(x, y^i), f^i(x, y^i)_{|\text{Obs}_i, \text{Di}_i} \rangle \\ &= \langle y^{i'} y^{i''}, p^{i'}(x, y^{i'}) + p^{i''}(y^{i''}), f^{i'}(x, y^{i'})_{|\text{Obs}_i} f^{i''}(y^{i''})_{|\text{Di}_i} \rangle \\ &= \langle y^{i'}, p^{i'}(x, y^{i'}), f^{i'}(x, y^{i'})_{|\text{Obs}_i} \rangle \otimes \langle y^{i''}, p^{i''}(y^{i''}), f^{i''}(y^{i''})_{|\text{Di}_i} \rangle. \end{aligned}$$

Then, for all $x \in \{0, 1\}^{\text{Inp}}$,

$$\begin{aligned} & \mathcal{V} \left(\langle y^i, p^i(x, y^i), f^i(x, y^i)_{|\text{Obs}_i, \text{Di}_i} \rangle \right) (x) \\ &= \left(\mathcal{V} \left(\langle y^{i'}, p^{i'}(x, y^{i'}), f^{i'}(x, y^{i'})_{|\text{Obs}_i} \rangle \right) \otimes \mathcal{V} \left(\langle y^{i''}, p^{i''}(y^{i''}), f^{i''}(y^{i''})_{|\text{Di}_i} \rangle \right) \right) (x). \end{aligned}$$

Thus Property 4.8 holds. By Property 4.20:

$$\langle y^{1'}, p^{1'}(x, y^{1'}), f^{1'}(x, y^{1'})_{|\text{Obs}_1} \rangle \sim_{\text{ps}} \langle y^{2'}, p^{2'}(x, y^{2'}), f^{2'}(x, y^{2'})_{|\text{Obs}_2} \rangle \quad (\text{Definition 4.3.24})$$

Then, for all $x \in \{0, 1\}^{\text{Inp}}$,

$$\begin{aligned} & \mathcal{V} \left(\left\langle y^{1'}, p^{1'}(x, y^{1'}), f^{1'}(x, y^{1'})_{|\text{Obs}_1} \right\rangle \right) (x) \\ & \sim_{(\text{Obs}_1, \text{Obs}_2)} \mathcal{V} \left(\left\langle y^{2'}, p^{2'}(x, y^{2'}), f^{2'}(x, y^{2'})_{|\text{Obs}_2} \right\rangle \right) (x) \end{aligned} \quad (\text{Notation 2.2.23})$$

Thus Property 4.9 holds. Therefore, by Definition 4.3.12, $\mathcal{QC}_1 \dot{=} \mathcal{QC}_2$. $\square$

Definition 4.3.29 (SubCir-PS-Part-Seq). Let $\mathcal{QC}_1 = (\text{Init}_1, \text{Di}_1, \text{U}_1)$ and $\mathcal{QC}_2 = (\emptyset, \emptyset, \text{U}_2)$ be two unitary circuit tuples such that

$$\begin{aligned} & \llbracket \text{U}_1 \rrbracket_{\text{PS}} (\langle \emptyset, 0, x_{|\text{Inp}_1} 0_{|\text{Init}_1} \rangle) = \langle y^1, p^1(x, y^1), f^1(x, y^1)_{|\text{Obs}_1, \text{Di}_1} \rangle, \\ & \llbracket \text{U}_1 - - \text{U}_2^\dagger \rrbracket_{\text{PS}} (\langle \emptyset, 0, x_{|\text{Inp}_1} 0_{|\text{Init}_1} \rangle) = \langle y, p(x, y), f(x, y)_{|\text{Inp}_2, \text{Di}_1} \rangle. \end{aligned}$$

We assume that $\langle \emptyset, 0, x_{|\text{Inp}_1} \rangle \sim_{\text{ps}} \langle \emptyset, 0, x_{|\text{Inp}_2} \rangle$ (Definition 4.3.24). We define *sub-circuit path-sum partial sequential equivalence* (SubCir-PS-Part-Seq) on such tuples. We write

$$\mathcal{QC}_1 \dot{=}_{(-, \text{ps})} \mathcal{QC}_2$$

if and only if (4.21) for $\mathcal{QC}_1$ the observed and discard parts share no variables and the discard part contains only path variables, and (4.22) the observed part of the sequence equals the input. Formally:

$$\left. \begin{aligned} & y^1 = y^{1'} y^{1''}, \\ & f^1(x, y^1)_{|\text{Obs}_1, \text{Di}_1} = f^{1'}(x, y^{1'})_{|\text{Obs}_1} f^{1''}(y^{1''})_{|\text{Di}_1}, \\ & p^1(x, y^1) = p^{1'}(x, y^{1'}) + p^{1''}(y^{1''}), \end{aligned} \right\} \quad (4.21)$$

$$\left. \begin{aligned} & \text{Obs}_1 \simeq \text{Obs}_2 \quad (\text{Notation 2.1.2}) \\ & \langle y', p'(x, y'), f'(x, y')_{|\text{Inp}_2} \rangle \sim_{\text{ps}} \langle \emptyset, 0, x_{|\text{Inp}_1} \rangle \quad (\text{Definition 4.3.24}) \end{aligned} \right\} \quad (4.22)$$

Lemme 4.3.30 (Correctness of $\dot{=}_{(-, \text{ps})}$ for $\dot{=}_{(-)}$). The relation $\dot{=}_{(-, \text{ps})}$, as defined in Definition 4.3.29, is a correct implementation of $\dot{=}_{(-)}$, as defined in Definition 4.3.13. Formally, for any two unitary circuit tuples $\mathcal{QC}_1$ and $\mathcal{QC}_2$ (Notation 4.3.8),

$$\mathcal{QC}_1 \dot{=}_{(-, \text{ps})} \mathcal{QC}_2 \implies \mathcal{QC}_1 \dot{=}_{(-)} \mathcal{QC}_2.$$

Proof. Let $\mathcal{QC}_1 = (\text{Init}_1, \text{Di}_1, \text{U}_1)$ and $\mathcal{QC}_2 = (\emptyset, \emptyset, \text{U}_2)$ be two unitary circuit tuples (Notation 4.3.8), and let $x \in \{0, 1\}^{\text{Inp}_1}$ be a bit-string such that

$$\begin{aligned} & \llbracket \text{U}_1 \rrbracket_{\text{PS}} (\langle \emptyset, 0, x_{|\text{Inp}_1} 0_{|\text{Init}_1} \rangle) = \langle y^1, p^1(x, y^1), f^1(x, y^1)_{|\text{Obs}_1, \text{Di}_1} \rangle, \\ & \llbracket \text{U}_1 - - \text{U}_2^\dagger \rrbracket_{\text{PS}} (\langle \emptyset, 0, x_{|\text{Inp}_1} 0_{|\text{Init}_1} \rangle) = \langle y, p(x, y), f(x, y)_{|\text{Inp}_2, \text{Di}_1} \rangle. \end{aligned}$$

Assume $\mathcal{QC}_1 \stackrel{\cdot}{=}_{(-, \text{ps})} \mathcal{QC}_2$. By Property 4.21:

$$\begin{aligned} & \left\langle y^1, p^1(x, y^1), f^1(x, y^1)_{|\text{Obs}_1, \text{Di}_1} \right\rangle \\ &= \left\langle y^{1'} y^{1''}, p^{1'}(x, y^{1'}) + p^{1''}(y^{1''}), f^{1'}(x, y^{1'})_{|\text{Obs}_1} f^{1''}(y^{1''})_{|\text{Di}_1} \right\rangle \\ &= \left\langle y^{1'}, p^{1'}(x, y^{1'}), f^{1'}(x, y^{1'})_{|\text{Obs}_1} \right\rangle \otimes \left\langle y^{1''}, p^{1''}(y^{1''}), f^{1''}(y^{1''})_{|\text{Di}_1} \right\rangle. \end{aligned}$$

Then, for all $x \in \{0, 1\}^{\text{Inp}_1}$,

$$\begin{aligned} & \mathcal{V} \left(\left\langle y^1, p^1(x, y^1), f^1(x, y^1)_{|\text{Obs}_1, \text{Di}_1} \right\rangle \right) (x) \\ &= \left(\mathcal{V} \left(\left\langle y^{1'}, p^{1'}(x, y^{1'}), f^{1'}(x, y^{1'})_{|\text{Obs}_1} \right\rangle \right) \otimes \mathcal{V} \left(\left\langle y^{1''}, p^{1''}(y^{1''}), f^{1''}(y^{1''})_{|\text{Di}_1} \right\rangle \right) \right) (x). \end{aligned}$$

Thus Property 4.10 holds. By Property 4.22:

$$\left\langle y', p'(x, y'), f'(x, y')_{|\text{Inp}_2} \right\rangle \sim_{\text{ps}} \langle \emptyset, 0, x_{|\text{Inp}_1} \rangle \quad (\text{Definition 4.3.24})$$

Then, for all $x \in \{0, 1\}^{\text{Inp}_1}$,

$$\mathcal{V} \left(\left\langle y', p'(x, y'), f'(x, y')_{|\text{Inp}_2} \right\rangle \right) (x) \sim_{(\text{Inp}_2, \text{Inp}_1)} \mathcal{V} \left(\langle \emptyset, 0, x_{|\text{Inp}_1} \rangle \right) (x) \quad (\text{Notation 2.2.23})$$

Thus Property 4.11 holds. Therefore, by Definition 4.3.13, $\mathcal{QC}_1 \stackrel{\cdot}{=}_{(-, \text{ps})} \mathcal{QC}_2$. $\square$

4.3.3.4 Global-Phase Path-Sum Partial Equivalence (GP-PS-Part)

This section follows the same scheme as Section 4.3.3.3, but up to global phase. In this setting, the global phase is dyadic (Definition 2.3.1).

First, we define global-phase path-sum partial parallel equivalence in Definition 4.3.31 and prove its correctness in Lemma 4.3.32. We then define global-phase path-sum partial sequential equivalence in Definition 4.3.33 and prove its correctness in Lemma 4.3.34.

Definition 4.3.31 (GP-PS-Part-Par). For $i \in \{1, 2\}$, let $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, \text{U}_i)$ be two unitary circuit tuples such that

$$\llbracket \text{U}_i \rrbracket_{\text{PS}} (\langle \emptyset, 0, x_{|\text{Inp}_i} 0_{|\text{Init}_i} \rangle) = \left\langle y^i, p^i(x, y^i), f^i(x, y^i)_{|\text{Obs}_i, \text{Di}_i} \right\rangle.$$

We assume that $\langle \emptyset, 0, x_{|\text{Inp}_1} \rangle \sim_{\text{ps}} \langle \emptyset, 0, x_{|\text{Inp}_2} \rangle$ (Definition 4.3.24). We define *global-phase path-sum partial parallel equivalence* (GP-PS-Part-Par) on such tuples. We write

$$\mathcal{QC}_1 \stackrel{\cdot}{=}_{(\text{gp}, \text{ps})} \mathcal{QC}_2$$

if and only if (4.19) the Obs and Di parts share no variables and the Di part contains

only path variables, and (4.20) the Obs parts are equal up to global phase. Formally:

$$\left. \begin{aligned} \forall i \in \{1, 2\}, \\ y^i = y^{i'} y^{i''}, \\ f^i(x, y^i)_{|\text{Obs}_i, \text{Di}_i} = f^{i'}(x, y^{i'})_{|\text{Obs}_i} f^{i''}(y^{i''})_{|\text{Di}_i}, \\ p^i(x, y^i) = p^{i'}(x, y^{i'}) + p^{i''}(y^{i''}), \\ \exists p : \{0, 1\}^{|y|} \rightarrow \mathbb{D}, \end{aligned} \right\} \quad (4.23)$$

$$\left. \begin{aligned} \text{Obs}_1 \simeq \text{Obs}_2 & \quad (\text{Notation 2.1.2}) \\ \left\langle y^{1'}, p^{1'}(x, y^{1'}), f^{1'}(x, y^{1'})_{|\text{Obs}_1} \right\rangle \\ \sim_{\text{ps}} \left\langle y^{2'}, p^{2'}(x, y^{2'}) + p(y), f^{2'}(x, y^{2'})_{|\text{Obs}_2} \right\rangle & \quad (\text{Definition 4.3.24}) \end{aligned} \right\} \quad (4.24)$$

Lemme 4.3.32 (Correctness of $\dot{\equiv}_{(\text{gp}, \text{ps})}$ for $\dot{\equiv}_{\text{gp}}$). The relation $\dot{\equiv}_{(\text{gp}, \text{ps})}$, as defined in Definition 4.3.31, is a correct implementation of $\dot{\equiv}_{\text{gp}}$, as defined in Definition 4.3.15. Formally, for any two unitary circuit tuples $\mathcal{QC}_1$ and $\mathcal{QC}_2$ (Notation 4.3.8),

$$\mathcal{QC}_1 \dot{\equiv}_{(\text{gp}, \text{ps})} \mathcal{QC}_2 \implies \mathcal{QC}_1 \dot{\equiv}_{\text{gp}} \mathcal{QC}_2.$$

Proof. Let $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, \text{U}_i)$ for $i \in \{1, 2\}$. Let $x \in \{0, 1\}^{\text{Inp}_i}$ be a bit-string such that

$$\llbracket \text{U}_i \rrbracket_{\text{PS}} (\langle \emptyset, 0, x_{|\text{Inp}_i} 0_{|\text{Init}_i} \rangle) = \langle y^i, p^i(x, y^i), f^i(x, y^i)_{|\text{Obs}_i, \text{Di}_i} \rangle.$$

Assume $\mathcal{QC}_1 \dot{\equiv}_{(\text{gp}, \text{ps})} \mathcal{QC}_2$. By Property 4.23, Property 4.12 holds (see Lemma 4.3.28 for details). By Property 4.24:

$$\begin{aligned} \exists p : \{0, 1\}^{|y|} \rightarrow \mathbb{D}, \\ \left\langle y^{1'}, p^{1'}(x, y^{1'}), f^{1'}(x, y^{1'})_{|\text{Obs}_1} \right\rangle \sim_{\text{ps}} \left\langle y^{2'}, p^{2'}(x, y^{2'}) + p(y), f^{2'}(x, y^{2'})_{|\text{Obs}_2} \right\rangle \end{aligned} \quad (4.25)$$

(Definition 4.3.24)

Then, for all $x \in \{0, 1\}^{\text{Inp}}$,

$$\begin{aligned} \exists p : \{0, 1\}^{|y|} \rightarrow \mathbb{D}, \\ \mathcal{V} \left(\left\langle y^{1'}, p^{1'}(x, y^{1'}), f^{1'}(x, y^{1'})_{|\text{Obs}_1} \right\rangle \right) (x) \\ \sim_{(\text{Obs}_1, \text{Obs}_2)} \mathcal{V} \left(\left\langle y^{2'}, p^{2'}(x, y^{2'}) + p(y), f^{2'}(x, y^{2'})_{|\text{Obs}_2} \right\rangle \right) (x) & \quad (\text{Notation 2.2.23}) \\ \approx_{\text{gp}} \mathcal{V} \left(\left\langle y^{2'}, p^{2'}(x, y^{2'}), f^{2'}(x, y^{2'})_{|\text{Obs}_2} \right\rangle \right) (x) & \quad (\text{Definition 2.2.12}) \end{aligned}$$

Thus Property 4.13 holds. Therefore, by Definition 4.3.15, $\mathcal{QC}_1 \dot{\equiv}_{\text{gp}} \mathcal{QC}_2$. $\square$

Definition 4.3.33 (GP-PS-Part-Seq). Let $\mathcal{QC}_1 = (\text{Init}_1, \text{Di}_1, \text{U}_1)$ and $\mathcal{QC}_2 = (\emptyset, \emptyset, \text{U}_2)$ be two unitary circuit tuples such that

$$\begin{aligned} \llbracket \text{U}_1 \rrbracket_{\text{PS}} (\langle \emptyset, 0, x_{|\text{Inp}_1} 0_{|\text{Init}_1} \rangle) &= \langle y^1, p^1(x, y^1), f^1(x, y^1)_{|\text{Obs}_1, \text{Di}_1} \rangle, \\ \llbracket \text{U}_1 - - \text{U}_2^\dagger \rrbracket_{\text{PS}} (\langle \emptyset, 0, x_{|\text{Inp}_1} 0_{|\text{Init}_1} \rangle) &= \langle y, p(x, y), f(x, y)_{|\text{Inp}_2, \text{Di}_1} \rangle. \end{aligned}$$

We assume that $\langle \emptyset, 0, x_{| \text{Inp}_1} \rangle \sim_{\text{ps}} \langle \emptyset, 0, x_{| \text{Inp}_2} \rangle$ (Definition 4.3.24).

We define *global-phase path-sum partial sequential equivalence* (GP-PS-Part-Seq) on such tuples. We write

$$\mathcal{QC}_1 \dot{=}_{(-, \text{gp}, \text{ps})} \mathcal{QC}_2$$

if and only if (4.26) for $\mathcal{QC}_1$ the observed and discard parts share no variables and the discard part contains only path variables, and (4.27) the observed part of the sequence equals the input up to global-phase. Formally:

$$\left. \begin{aligned} y^1 &= y^{1'} y^{1''}, \\ f^1(x, y^1)_{| \text{Obs}_1, \text{Di}_1} &= f^{1'}(x, y^{1'})_{| \text{Obs}_1} f^{1''}(y^{1''})_{| \text{Di}_1}, \\ p^1(x, y^1) &= p^{1'}(x, y^{1'}) + p^{1''}(y^{1''}), \end{aligned} \right\} \quad (4.26)$$

$$\left. \begin{aligned} \text{Obs}_1 &\simeq \text{Obs}_2 \\ y &= y' y^{1''} \\ \exists p : \{0, 1\}^{|y'|} &\rightarrow \mathbb{D}, \\ \langle y', p'(x, y'), f'(x, y')_{| \text{Inp}_2} \rangle &\sim_{\text{ps}} \langle y', p(y'), x_{| \text{Inp}_1} \rangle \quad (\text{Definition 4.3.24}) \end{aligned} \right\} \quad (\text{Notation 2.1.2}) \quad (4.27)$$

Lemme 4.3.34 (Correctness of $(\dot{=}_{(-, \text{gp}, \text{ps})})$ for $(\dot{=}_{(-, \text{gp})})$). The relation $(\dot{=}_{(-, \text{gp}, \text{ps})})$, as defined in Definition 4.3.33, is a correct implementation of $(\dot{=}_{(-, \text{gp})})$, as defined in Definition 4.3.16. Formally, for any two unitary circuit tuples $\mathcal{QC}_1$ and $\mathcal{QC}_2$ (Notation 4.3.8),

$$\mathcal{QC}_1 \dot{=}_{(-, \text{gp}, \text{ps})} \mathcal{QC}_2 \implies \mathcal{QC}_1 \dot{=}_{(-, \text{gp})} \mathcal{QC}_2.$$

Proof. Let $\mathcal{QC}_1 = (\text{Init}_1, \text{Di}_1, \text{U}_1)$ and $\mathcal{QC}_2 = (\emptyset, \emptyset, \text{U}_2)$. Let $x \in \{0, 1\}^{\text{Inp}_1}$ be a bit-string such that

$$\begin{aligned} \llbracket \text{U}_1 \rrbracket_{\text{PS}} (\langle \emptyset, 0, x_{| \text{Inp}_1} 0_{| \text{Init}_1} \rangle) &= \langle y^1, p^1(x, y^1), f^1(x, y^1)_{| \text{Obs}_1, \text{Di}_1} \rangle, \\ \llbracket \text{U}_1 - \text{U}_2^\dagger \rrbracket_{\text{PS}} (\langle \emptyset, 0, x_{| \text{Inp}_1} 0_{| \text{Init}_1} \rangle) &= \langle y, p(x, y), f(x, y)_{| \text{Inp}_2, \text{Di}_1} \rangle. \end{aligned}$$

Assume $\mathcal{QC}_1 \dot{=}_{(-, \text{gp}, \text{ps})} \mathcal{QC}_2$. By Property 4.26, Property 4.14 holds (see Lemma 4.3.30). By Property 4.27:

$$\begin{aligned} y &= y' y^{1''}, \\ \exists p : \{0, 1\}^{|y'|} &\rightarrow \mathbb{D}, \\ \langle y', p'(x, y'), f'(x, y')_{| \text{Inp}_2} \rangle &\sim_{\text{ps}} \langle y', p(y'), x_{| \text{Inp}_1} \rangle \quad (\text{Definition 4.3.24}) \end{aligned}$$

Then, for all $x \in \{0, 1\}^{\text{Inp}_1}$,

$$\begin{aligned} \mathcal{V} \left(\langle y', p'(x, y'), f'(x, y')_{| \text{Inp}_2} \rangle \right) (x) &\sim_{(\text{Inp}_2, \text{Inp}_1)} \mathcal{V} \left(\langle y', p(y'), x_{| \text{Inp}_1} \rangle \right) (x) \quad (\text{Notation 2.2.23}) \\ &\approx_{\text{gp}} \mathcal{V} \left(\langle \emptyset, 0, x_{| \text{Inp}_1} \rangle \right) (x) \quad (\text{Definition 2.2.12}) \end{aligned}$$

Thus Property 4.15 holds. Therefore, by Definition 4.3.16, $\mathcal{QC}_1 \dot{=}_{(-, \text{gp})} \mathcal{QC}_2$. $\square$

4.3.3.5 Full-Circuit-Path-Sum-Partial-Equivalence (FullCir-PS-Part)

This section mirrors the structure of Sections 4.3.3.3 and 4.3.3.4, but focuses on conditional phase (Definition 2.2.14). We define full-circuit path-sum partial parallel equivalence in Definition 4.3.35 and prove its correctness in Lemma 4.3.36. We do not define full-circuit path-sum partial sequential equivalence, since full-circuit-sequence-equivalence cannot be defined (Lemma 4.2.16).

Definition 4.3.35 (FullCir-PS-Part-Par). For all $i \in \{1, 2\}$, let $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, \text{U}_i)$ be unitary circuit tuples such that

$$\llbracket \text{U}_i \rrbracket_{\text{PS}} (\langle \emptyset, 0, x_{|\text{Inp}_i} 0_{|\text{Init}_i} \rangle) = \langle y^i, p^i(x, y^i), f^i(x, y^i)_{|\text{Obs}_i, \text{Di}_i} \rangle.$$

We define *full-circuit path-sum partial parallel equivalence* on such tuples. We write

$$\mathcal{QC}_1 \dot{\equiv}_{(\text{f}, \text{ps})} \mathcal{QC}_2$$

if and only if (4.28) for all $i \in \{1, 2\}$ the observed and discard parts share no variables and the discard part contains only path variables, and (4.29) the observed part is injective with respect to the path variables. Formally:

$$\left. \begin{aligned} \forall i \in \{1, 2\}, \\ \left. \begin{aligned} y^i &= y^{i'} y^{i''}, \\ f^i(x, y^i)_{|\text{Obs}_i, \text{Di}_i} &= f^{i'}(x, y^{i'})_{|\text{Obs}_i} f^{i''}(y^{i''})_{|\text{Di}_i}, \\ p^i(x, y^i) &= p^{i'}(x, y^{i'}) + p^{i''}(y^{i''}), \end{aligned} \right\} \end{aligned} \right\} \quad (4.28)$$

$$\left. \begin{aligned} \text{Obs}_1 &\simeq \text{Obs}_2 \\ |y^{1'}| &= |y^{2'}| \\ \forall y^{i'}, y^{i''} \in \{0, 1\}^{|y^{i'}|}, \\ y^{i'} \not\equiv y^{i''} &\implies f^{i'}(x, y^{i'})_{|\text{Obs}_i} \not\equiv_{(\text{Obs}_i, \text{Obs}_i)} f^{i''}(x, y^{i''})_{|\text{Obs}_i} \end{aligned} \right\} \quad \text{(Notation 2.1.2)} \quad (4.29)$$

Lemme 4.3.36 (Correctness of $\dot{\equiv}_{(\text{f}, \text{ps})}$ for $\dot{\equiv}_{\text{f}}$). The relation $\dot{\equiv}_{(\text{f}, \text{ps})}$, as defined in Definition 4.3.35, correctly implements $\dot{\equiv}_{\text{f}}$, as defined in Definition 4.3.18. Formally, for any two unitary circuit tuples $\mathcal{QC}_1$ and $\mathcal{QC}_2$,

$$\mathcal{QC}_1 \dot{\equiv}_{(\text{f}, \text{ps})} \mathcal{QC}_2 \implies \mathcal{QC}_1 \dot{\equiv}_{\text{f}} \mathcal{QC}_2.$$

Proof. Let $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, \text{U}_i)$ for $i \in \{1, 2\}$. Let $x \in \{0, 1\}^{\text{Inp}_i}$. Assume

$$\llbracket \text{U}_i \rrbracket_{\text{PS}} (\langle \emptyset, 0, x_{|\text{Inp}_i} 0_{|\text{Init}_i} \rangle) = \langle y^i, p^i(x, y^i), f^i(x, y^i)_{|\text{Obs}_i, \text{Di}_i} \rangle.$$

Assume $\mathcal{QC}_1 \dot{\equiv}_{(\text{f}, \text{ps})} \mathcal{QC}_2$. By Definition 4.3.35 and Theorem 4.3.26, the concretisation

of the observed part and the discarded part factorises as

$$\left(\frac{1}{\sqrt{2^{|y^{i'}|}}} \sum_{y^{i'} \in \{0,1\}^{|y^{i'}|}} e^{2\pi i p^{i'}(x, y^{i'})} \left| f^{i'}(x, y^{i'}) \right\rangle_{\text{Obs}_i} \right) \otimes \left(\frac{1}{\sqrt{2^{|y^{i''}|}}} \sum_{y^{i''} \in \{0,1\}^{|y^{i''}|}} e^{2\pi i p^{i''}(y^{i''})} \left| f^{i''}(y^{i''}) \right\rangle_{\text{Di}_i} \right).$$

By Property 4.16, the observed components coincide. Hence $\mathcal{QC}_1 \stackrel{\cdot}{\equiv}_{\mathbf{f}} \mathcal{QC}_2$. $\square$

Remark 4.3.37 (Full-Circuit-Path-Sum-Partial-Sequence-Equivalence Impossible). A sequential version cannot be defined, by Lemma 4.2.16.

Remark 4.3.38 (On the verification complexity of $(\stackrel{\cdot}{\equiv}_{(\mathbf{f}, \text{ps})})$). In practice, verifying Property (4.29) requires ensuring that all symbolic paths lead to distinct basis states $f^i(x, y^i)_{|\text{Obs}_i}$. This check has a worst-case complexity of $\mathcal{O}(2^{|y^{1'}|} \cdot 2^{|y^{2'}|})$, since every pair of path values must be compared. As a result, the full-circuit path-sum partial parallel equivalence relation $(\stackrel{\cdot}{\equiv}_{(\mathbf{f}, \text{ps})})$ is less efficient than purely symbolic equivalences such as sub-circuit path-sum partial parallel equivalence or global-phase path-sum partial parallel equivalence, for which verification is entirely symbolic and can be performed through term rewriting or symbolic unification.

Nevertheless, $(\stackrel{\cdot}{\equiv}_{(\mathbf{f}, \text{ps})})$ has an advantage: it can successfully establish equivalence between circuits that do not admit a normalised or canonical symbolic form, for example, due to an incomplete equational theory for the underlying gate set. Hence, $(\stackrel{\cdot}{\equiv}_{(\mathbf{f}, \text{ps})})$ extends the range of equivalences that can be automatically verified, at the cost of a higher (potentially exponential) verification complexity.

However, our experimental evaluation (Section 5.4) shows that this is not an issue for achieving our goal.

4.3.3.6 Relating Path-Sum Partial Equivalences

This section considers only parallel relations. Whenever a sequential relation is defined, it coincides with the corresponding parallel relation. Therefore, restricting the discussion to the parallel case entails no loss of generality.

Lemme 4.3.39 (SubCir-PS-Part-Par implies GP-PS-Part-Par). Let $\mathcal{QC}_1$ and $\mathcal{QC}_2$ be two unitary circuit tuples as defined in Notation 4.3.8.

$$\mathcal{QC}_1 \stackrel{\cdot}{\equiv}_{(\text{ps})} \mathcal{QC}_2 \implies \mathcal{QC}_1 \stackrel{\cdot}{\equiv}_{(\text{gp}, \text{ps})} \mathcal{QC}_2.$$

Proof. Assume $\mathcal{QC}_1 \stackrel{\cdot}{\equiv}_{(\text{ps})} \mathcal{QC}_2$. By Definition 4.3.27, Conditions (4.19) and (4.20) hold.

1. Condition (4.19) is exactly the structural requirement of GP-PS-Part-Par, namely Condition (4.23) in Definition 4.3.31.

2. For Condition (4.24), let $p : \{0, 1\}^{|y|} \rightarrow \mathbb{D}$ be the constant function $p(y) = 0$. Then Condition (4.20) implies

$$\left\langle y^{1'}, p^{1'}(x, y^{1'}), f^{1'}(x, y^{1'})|_{\text{Obs}_1} \right\rangle \sim_{\text{ps}} \left\langle y^{2'}, p^{2'}(x, y^{2'}) + p(y), f^{2'}(x, y^{2'})|_{\text{Obs}_2} \right\rangle,$$

since $p(y) = 0$ and $\text{Obs}_1 \simeq \text{Obs}_2$ already holds in (4.20).

3. Hence Conditions (4.23) and (4.24) hold, and therefore $\mathcal{QC}_1 \stackrel{\text{gp,ps}}{\equiv} \mathcal{QC}_2$ by Definition 4.3.31. $\square$

Lemma 4.3.40 (From GP-PS-Part-Par to FullCir-PS-Part-Par). Let $\mathcal{QC}_1$ and $\mathcal{QC}_2$ be two unitary circuit tuples as defined in Notation 4.3.8.

$$\mathcal{QC}_1 \stackrel{\text{gp,ps}}{\equiv} \mathcal{QC}_2 \implies \mathcal{QC}_1 \stackrel{\text{f,ps}}{\equiv} \mathcal{QC}_2.$$

Proof. Assume $\mathcal{QC}_1 \stackrel{\text{gp,ps}}{\equiv} \mathcal{QC}_2$. By Definition 4.3.31, (4.23) and (4.24) hold.

1. Condition (4.28) holds. Indeed, (4.28) is the same structural decomposition as (4.23).
2. Condition (4.29) holds. Fix $i \in \{1, 2\}$. By (4.24), the observed path-sum is related by $\sim_{\text{ps}}$ (up to an additive phase term in the exponent). By Definition 4.3.24, $\sim_{\text{ps}}$ compares symbolic paths on the observed interface without merging distinct paths into the same observed basis state. Hence, for any two valuations $y^{i'}, y^{i''} \in \{0, 1\}^{|y^{i'}|}$,

$$y^{i'} \not\equiv y^{i''} \implies f^{i'}(x, y^{i'})|_{\text{Obs}_i} \not\equiv_{(\text{Obs}_i, \text{Obs}_i)} f^{i''}(x, y^{i''})|_{\text{Obs}_i},$$

which is exactly (4.29).

3. Therefore $\mathcal{QC}_1 \stackrel{\text{f,ps}}{\equiv} \mathcal{QC}_2$ by Definition 4.3.35. $\square$

Theorem 4.3.41 (From SubCir-PS-Part-Par to FullCir-PS-Part-Par). Let $\mathcal{QC}_1$ and $\mathcal{QC}_2$ be two unitary circuit tuples as defined in Notation 4.3.8.

$$\mathcal{QC}_1 \stackrel{\text{ps}}{\equiv} \mathcal{QC}_2 \implies \mathcal{QC}_1 \stackrel{\text{f,ps}}{\equiv} \mathcal{QC}_2. \quad (\text{Definition 4.3.33 and 4.3.27})$$

Proof. Assume $(\mathcal{QC}_1 \stackrel{\text{ps}}{\equiv} \mathcal{QC}_2)$. By Lemma 4.3.39, this implies $(\mathcal{QC}_1 \stackrel{\text{gp,ps}}{\equiv} \mathcal{QC}_2)$. Therefore, by Lemma 4.3.40, we obtain $(\mathcal{QC}_1 \stackrel{\text{f,ps}}{\equiv} \mathcal{QC}_2)$. $\square$

4.3.4 Hybrid-Partial-Equivalences (Hyb-Part)

This section introduces our hybrid partial equivalence relations. Hybrid partial equivalence relations constitute our most general equivalence framework and enable the verification of hybrid circuit transformations such as One-Way Measurement (MBQC).

We begin by introducing the required notations in Section 4.3.4.1. Then, we define the hybrid partial equivalence relations in Section 4.3.4.2. Figure 4.12 summarises the dependencies between the hybrid partial equivalence relations. It illustrates two hybrid

circuits, C_1 and C_2 , together with their corresponding tuples. Let C_i transform an extended mixed state (Def. 2.2.20) $\{p_c, |\xi_c\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i}, c\}_{c \in \{0,1\}^m}$ into $\{p_{c_i}^i, |f_{c_i}^i(\xi_c)\rangle_{\text{Obs}_i, \text{Di}_i}, c_i\}_{c_i \in \{0,1\}^{m_i}}$. The observed part Obs_i and the discarded part Di_i may be entangled. The hybrid partial equivalence relations are defined on such tuples, by comparing their induced semantics under the appropriate equivalence notion.

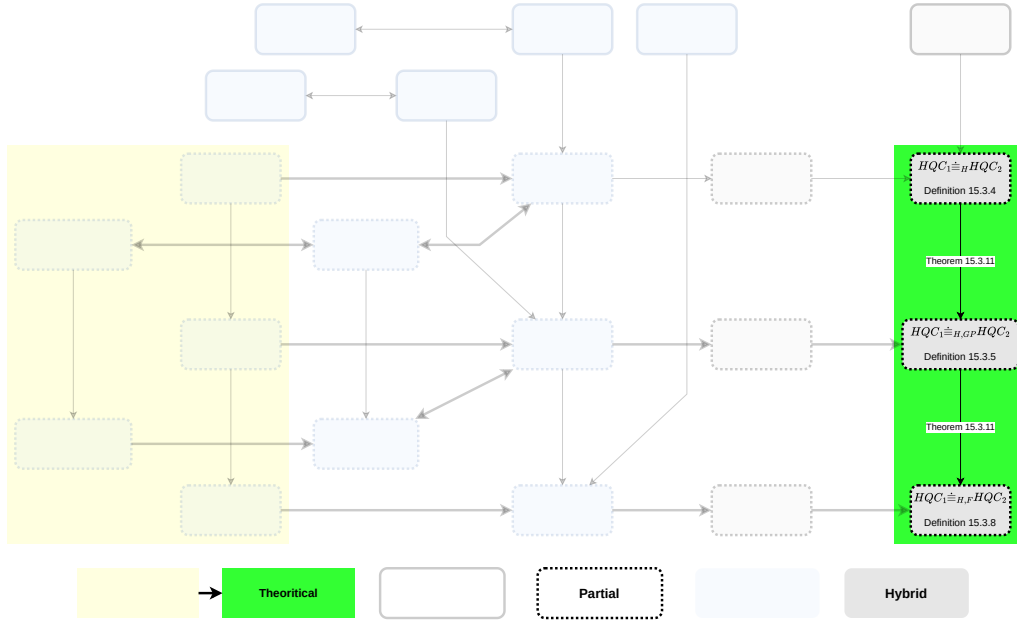


Figure 4.11: Partial Hybrid Equivalence Relations Overview.

$$\{p_c, |\xi_c\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i}, c\}_{c \in \{0,1\}^m} \left\{ \begin{array}{c} \boxed{C_i} \\ \hline \end{array} \right\} \{p_{c_i}^i, |f_{c_i}^i(\xi_c)\rangle_{\text{Obs}_i, \text{Di}_i}, c_i\}_{c_i \in \{0,1\}^{m_i}}$$

$$(\text{Init}_1, \text{Di}_1, C_1) \dot{=}_{\text{H}} (\text{Init}_2, \text{Di}_2, C_2) \quad (\text{Definition 4.3.43})$$

$$(\text{Init}_1, \text{Di}_1, C_1) \dot{=}_{(\text{H}, \text{gp})} (\text{Init}_2, \text{Di}_2, C_2) \quad (\text{Definition 4.3.44})$$

$$(\text{Init}_1, \text{Di}_1, C_1) \dot{=}_{(\text{H}, \text{f})} (\text{Init}_2, \text{Di}_2, C_2) \quad (\text{Definition 4.3.47})$$

Figure 4.12: Hybrid Partial Equivalence Summary.

4.3.4.1 Prerequisites

We introduce notations for classical measurement outcomes (Notation 4.3.42).

Notation 4.3.42 (Measurement Result Projection). Let n be the number of qubits in the quantum register, partitioned into Obs: indices of observed qubits, and Di: indices

of discarded qubits, such that $\text{Obs} \cup \text{Di} = \{0, 1, \dots, n-1\}$ and $\text{Obs} \cap \text{Di} = \emptyset$. Let c be a bit-string representing the classical measurement outcomes of a set of qubit indices. We denote by:

- $c_{|\text{Obs}}$ (Notation 2.1.3) the substring of c corresponding to the measurement outcomes of the qubits in Obs ,
- $c_{|\text{Di}}$ (Notation 2.1.3) the substring of c corresponding to the measurement outcomes of the qubits in Di .

where $c_{|\text{Obs}}c_{|\text{Di}} = c$. For example, if $\text{Obs} = \{0, 2, 4\}$, $\text{Di} = \{1, 3\}$, and $c = 0101$ corresponds to the outcomes of qubits $[0, 1, 2, 3]$ (in order), then $c_{|\text{Obs}} = 00$ and $c_{|\text{Di}} = 11$. Here, qubit $4 \in \text{Obs}$ is not measured and thus does not contribute to c .

4.3.4.2 Hybrid Partial Equivalence (Hyb-Part)

We define sub-circuit hybrid partial parallel equivalence (Definition 4.3.43), global-phase hybrid partial parallel equivalence (Definition 4.3.44), and full-circuit hybrid partial parallel equivalence (Definition 4.3.47). Theorem 4.3.48 establishes the implications between these hybrid partial equivalence relations.

In contrast to the unitary setting, where both sequential and parallel equivalence relations are introduced to support the implementation, the hybrid partial equivalence is defined only in its parallel form. It serves as the target specification of our method. The unitary sequential and parallel equivalence relations are used at the implementation level. The reduction of hybrid partial equivalence to the unitary setting, based on deferred measurement, is formalised in Section 4.3.5.

Definition 4.3.43 (SubCir-Hyb-Part-Par). Let $\mathcal{HQC}_1 = (\text{Init}_1, \text{Di}_1, C_1)$ and $\mathcal{HQC}_2 = (\text{Init}_2, \text{Di}_2, C_2)$ be two hybrid-circuit tuples (Notation 4.3.8). For $i \in \{1, 2\}$, their semantics is given by

$$\llbracket C_i \rrbracket_{\text{H}} \left(\left\{ p_c, |\xi_c^i\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i}, c \right\}_{c \in \{0,1\}^m} \right) = \left\{ p_{c_i}, |f_{c_i}(\xi_c^i)\rangle_{\text{Obs}_i, \text{Di}_i}, c_i \right\}_{c_i \in \{0,1\}^{m_i}}.$$

We assume that $|\xi^1\rangle_{\text{Inp}_1} \sim_{(\text{Inp}_1, \text{Inp}_2)} |\xi^2\rangle_{\text{Inp}_2}$ (Notation 2.2.23). We define *sub-circuit hybrid partial parallel equivalence* (SubCir-Hyb-Part-Par) on such hybrid circuit tuples. We write $\mathcal{HQC}_1 \dot{=} \mathcal{HQC}_2$ if and only if the corresponding extended mixed states (Definition 2.2.20) satisfy the following conditions:

$$\forall i \in \{1, 2\}, \forall \mathbf{c}_i \in \{0, 1\}^{m_i}, \exists |\phi_{\mathbf{c}_i}\rangle \in \mathcal{H}^{\text{Di}_i}, \forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \exists |\psi_{\mathbf{c}_i}(\xi)\rangle \in \mathcal{H}^{\text{Obs}_i},$$

$$|f_{\mathbf{c}_i}(\xi)\rangle_{\text{Obs}_i, \text{Di}_i} = |\psi_{\mathbf{c}_i}(\xi)\rangle_{\text{Obs}_i} \otimes |\phi_{\mathbf{c}_i}\rangle_{\text{Di}_i}, \quad (4.30)$$

$$\left. \begin{array}{l} \text{Obs}_1 \simeq \text{Obs}_2 \quad (\text{Notation 2.1.2}) \\ \forall \mathbf{c}_1 \in \{0, 1\}^{m_1}, \exists \mathbf{c}_2 \in \{0, 1\}^{m_2}, \\ \mathbf{c}_1|_{\text{Obs}_1} \cong_{(\text{Obs}_1, \text{Obs}_2)} \mathbf{c}_2|_{\text{Obs}_2} \quad (\text{Notation 2.1.3}) \\ |\psi_{\mathbf{c}_1}(\xi)\rangle_{\text{Obs}_1} \sim_{(\text{Obs}_1, \text{Obs}_2)} |\psi_{\mathbf{c}_2}(\xi)\rangle_{\text{Obs}_2} \quad (\text{Notation 2.2.23}) \\ \sum_{\mathbf{c}_1|_{\text{Di}_1}} p_{\mathbf{c}_1|_{\text{Di}_1}} \mathbf{c}_1|_{\text{Obs}_1} = \sum_{\mathbf{c}_2|_{\text{Di}_2}} p_{\mathbf{c}_2|_{\text{Di}_2}} \mathbf{c}_2|_{\text{Obs}_2} \quad (\text{Notation 2.1.3}) \end{array} \right\} \quad (4.31)$$

Definition 4.3.44 (GP-Hyb-Part-Par). Let $\mathcal{HQC}_1 = (\text{Init}_1, \text{Di}_1, \mathbf{C}_1)$ and $\mathcal{HQC}_2 = (\text{Init}_2, \text{Di}_2, \mathbf{C}_2)$ be two hybrid-circuit tuples (Notation 4.3.8).

For $i \in \{1, 2\}$, their semantics is given by

$$\llbracket \mathbf{C}_i \rrbracket_{\mathbb{H}} \left(\left\{ p_{\mathbf{c}_i}, |\xi_{\mathbf{c}_i}^i\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i}, \mathbf{c}_i \right\}_{\mathbf{c}_i \in \{0,1\}^{m_i}} \right) = \left\{ p_{\mathbf{c}_i}, |f_{\mathbf{c}_i}(\xi_{\mathbf{c}_i}^i)\rangle_{\text{Obs}_i, \text{Di}_i}, \mathbf{c}_i \right\}_{\mathbf{c}_i \in \{0,1\}^{m_i}}.$$

We assume that $|\xi^1\rangle_{\text{Inp}_1} \sim_{(\text{Inp}_1, \text{Inp}_2)} |\xi^2\rangle_{\text{Inp}_2}$ (Notation 2.2.23). We define *global-phase hybrid partial parallel equivalence* (GP-Hyb-Part-Par) on such hybrid circuit tuples. We write $\mathcal{HQC}_1 \equiv_{\mathbb{H}} \mathcal{HQC}_2$ if and only if the corresponding extended mixed states (Definition 2.2.20) satisfy the following conditions up to global phase:

$$\forall i \in \{1, 2\}, \forall \mathbf{c}_i \in \{0, 1\}^{m_i}, \exists |\phi_{\mathbf{c}_i}\rangle \in \mathcal{H}^{\text{Di}_i}, \forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \exists |\psi_{\mathbf{c}_i}(\xi)\rangle \in \mathcal{H}^{\text{Obs}_i},$$

$$|f_{\mathbf{c}_i}(\xi)\rangle_{\text{Obs}_i, \text{Di}_i} = |\psi_{\mathbf{c}_i}(\xi)\rangle_{\text{Obs}_i} \otimes |\phi_{\mathbf{c}_i}\rangle_{\text{Di}_i}, \quad (4.32)$$

$$\left. \begin{array}{l} \text{Obs}_1 \simeq \text{Obs}_2 \quad (\text{Notation 2.1.2}) \\ \forall \mathbf{c}_1 \in \{0, 1\}^{m_1}, \exists \mathbf{c}_2 \in \{0, 1\}^{m_2}, \\ \mathbf{c}_1|_{\text{Obs}_1} \cong_{(\text{Obs}_1, \text{Obs}_2)} \mathbf{c}_2|_{\text{Obs}_2} \quad (\text{Notation 2.1.3}) \\ |\psi_{\mathbf{c}_1}(\xi)\rangle_{\text{Obs}_1} \sim_{(\text{Obs}_1, \text{Obs}_2, \text{gp})} |\psi_{\mathbf{c}_2}(\xi)\rangle_{\text{Obs}_2} \quad (\text{Notation 2.2.23}) \\ \sum_{\mathbf{c}_1|_{\text{Di}_1}} p_{\mathbf{c}_1|_{\text{Di}_1}} \mathbf{c}_1|_{\text{Obs}_1} = \sum_{\mathbf{c}_2|_{\text{Di}_2}} p_{\mathbf{c}_2|_{\text{Di}_2}} \mathbf{c}_2|_{\text{Obs}_2} \quad (\text{Notation 2.1.3}) \end{array} \right\} \quad (4.33)$$

Definition 4.3.45 (Comp-Meas-Part-Obs-Hyb). We redefine d (Definition 4.3.5) as a function taking as parameters a real number $\mathbf{p} \in [0, 1]$, a state vector $|\psi\rangle$, and a set of qubit indices Obs .

The *complete register measurement with partial observation for hybrid circuits* (Comp-Meas-Part-Obs-Hyb) is defined as follows:

$$d(\mathbf{p}, \psi, \text{Obs}) := \left\{ \mathbf{p} \cdot \mathbf{p}(\psi, \text{Obs}, x), |x\rangle, x|_{\text{Obs}} \right\}_{x \in \{0,1\}^{|\text{Obs}|}}, \quad (\text{Notation 2.1.3})$$

where $\mathbf{p}(\psi, \text{Obs}, x)$ is the probability of obtaining the outcome $x|_{\text{Obs}}$ when measuring the subsystem Obs of the state $|\psi\rangle$, as defined in Definition 4.3.4.

Definition 4.3.46 (Equivalence of Comp-Meas-Part-Obs-Hyb). For all $i \in \{1, 2\}$, let $p^i \in [0, 1]$, be a probabilistic coefficient, $|\psi^i\rangle$ be a quantum state, and Obs_i be a set of qubit indices observed then, let $d(p^i, \psi^i, \text{Obs}_i)$ be two Comp-Meas-Part-Obs-Hyb (Definition 4.3.45), where $\text{Obs}_1, \text{Obs}_2 \subseteq [0, 1, \dots, n-1]$ are sets of qubit indices, and $x_{\text{Obs}_1} : \text{Obs}_1 \rightarrow \{0, 1\}$ and $x_{\text{Obs}_2} : \text{Obs}_2 \rightarrow \{0, 1\}$ are bit-strings for Obs_1 and Obs_2 , respectively. We define the *equivalence of complete register measurement with partial observation for hybrid-circuit*, denoted as

$$d(p^1, \psi^1, \text{Obs}_1) = d(p^2, \psi^2, \text{Obs}_2)$$

if and only if

$$\text{Obs}_1 \simeq \text{Obs}_2 \quad (\text{Notation 2.1.2}) \quad (4.34)$$

$$\forall x^i \in \{0, 1\}^{m_i}, \quad p^1 \cdot p(p^1, \text{Obs}_1, x^1|_{\text{Obs}_1}) = p^2 \cdot p(p^2, \text{Obs}_2, x^2|_{\text{Obs}_2}). \quad (4.35)$$

Definition 4.3.47 (FullCir-Hyb-Part-Par). Let $\mathcal{HQC}_1 = (\text{Init}_1, \text{Di}_1, \text{C}_1)$ and $\mathcal{HQC}_2 = (\text{Init}_2, \text{Di}_2, \text{C}_2)$ be two hybrid-circuit tuples (Notation 4.3.8).

For $i \in \{1, 2\}$, their semantics is given by

$$\llbracket \text{C}_i \rrbracket_{\text{H}} \left(\left\{ p_c, |\xi_c^i\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i}, c \right\}_{c \in \{0,1\}^m} \right) = \left\{ p_{c_i}, |f_{c_i}(\xi_c^i)\rangle_{\text{Obs}_i, \text{Di}_i}, c_i \right\}_{c_i \in \{0,1\}^{m_i}}.$$

We assume that $|\xi^1\rangle_{\text{Inp}_1} \sim_{(\text{Inp}_1, \text{Inp}_2)} |\xi^2\rangle_{\text{Inp}_2}$ (Notation 2.2.23). We define *full-circuit hybrid partial parallel equivalence* (FullCir-Hyb-Part-Par) on such hybrid circuit tuples. We write $\mathcal{HQC}_1 \dot{=}_{(\text{H}, \text{f})} \mathcal{HQC}_2$ if and only if complete register measurements with partial observations coincide, that is,

$$\begin{aligned} \forall c_1 \in \{0, 1\}^{m_1}, \exists c_2 \in \{0, 1\}^{m_2}, \\ c_1|_{\text{Obs}_1} = c_2|_{\text{Obs}_2}, \\ \forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, d(p_{c_1}, f_{c_1}(\xi), \text{Obs}_1) = d(p_{c_2}, f_{c_2}(\xi), \text{Obs}_2). \end{aligned} \quad (4.36)$$

Theorem 4.3.48 (Hybrid Partial Equivalence Implications). Let $\mathcal{HQC}_1 = (\text{Init}_1, \text{Di}_1, \text{C}_1)$ and $\mathcal{HQC}_2 = (\text{Init}_2, \text{Di}_2, \text{C}_2)$ be two hybrid-circuit tuples (Notation 4.3.8). Then the following implications hold:

$$\mathcal{HQC}_1 \dot{=}_{\text{H}} \mathcal{HQC}_2 \implies \mathcal{HQC}_1 \dot{=}_{(\text{H}, \text{gp})} \mathcal{HQC}_2 \implies \mathcal{HQC}_1 \dot{=}_{(\text{H}, \text{f})} \mathcal{HQC}_2.$$

Proof. For $i \in \{1, 2\}$, their semantics is given by

$$\llbracket \text{C}_i \rrbracket_{\text{H}} \left(\left\{ p_c, |\xi_c^i\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i}, c \right\}_{c \in \{0,1\}^m} \right) = \left\{ p_{c_i}, |f_{c_i}(\xi_c^i)\rangle_{\text{Obs}_i, \text{Di}_i}, c_i \right\}_{c_i \in \{0,1\}^{m_i}}.$$

We assume that $|\xi^1\rangle_{\text{Inp}_1} \sim_{(\text{Inp}_1, \text{Inp}_2)} |\xi^2\rangle_{\text{Inp}_2}$ (Notation 2.2.23). We prove the two implications.

1. Assume $\mathcal{HQC}_1 \dot{=}_{\text{H}} \mathcal{HQC}_2$. By Property 4.30, Property 4.32 holds. By Property 4.31 and Lemma 2.2.13, Property 4.33 holds. Hence, by Def. 4.3.44, $\mathcal{HQC}_1 \dot{=}_{(\text{H}, \text{gp})} \mathcal{HQC}_2$.
2. Assume $\mathcal{HQC}_1 \dot{=}_{(\text{H}, \text{gp})} \mathcal{HQC}_2$. By Property 4.33 and Lemma 2.2.15, Property 4.36 holds. Hence, by Definition 4.3.47, $\mathcal{HQC}_1 \dot{=}_{(\text{H}, \text{f})} \mathcal{HQC}_2$. $\square$

4.3.5 Reduction from Hyb-Part to Unit-Part (Hyb-Part-DM)

This section contains the main contribution of this chapter: verifying the partial equivalence of two hybrid circuits using our unitary equivalence relations associated with the deferred measurement (Section 4.3.5.4). This contribution extends the unitary tool to the hybrid case.

To reach this, we first demonstrate in Section 4.3.5.1 that sub-circuit hybrid partial parallel equivalence and global-phase hybrid partial parallel equivalence preserve semantics through temporal composition. This preservation is necessary for proving that our deferred measurement preserves semantics. Next, in Section 4.3.5.2, we define our deferred measurement implementation and show that it preserves circuit semantics and isolates the unitary part. Next, in Section 4.3.5.3, we establish that if two unitary circuits satisfy unitary partial equivalence, they also satisfy hybrid partial equivalence. Figure 4.13 illustrates **our method for transitioning from unitary partial equivalence to hybrid partial equivalence**. The main idea is that this process **requires an intermediate equivalence using hybrid partial equivalence over the unitary circuit**.

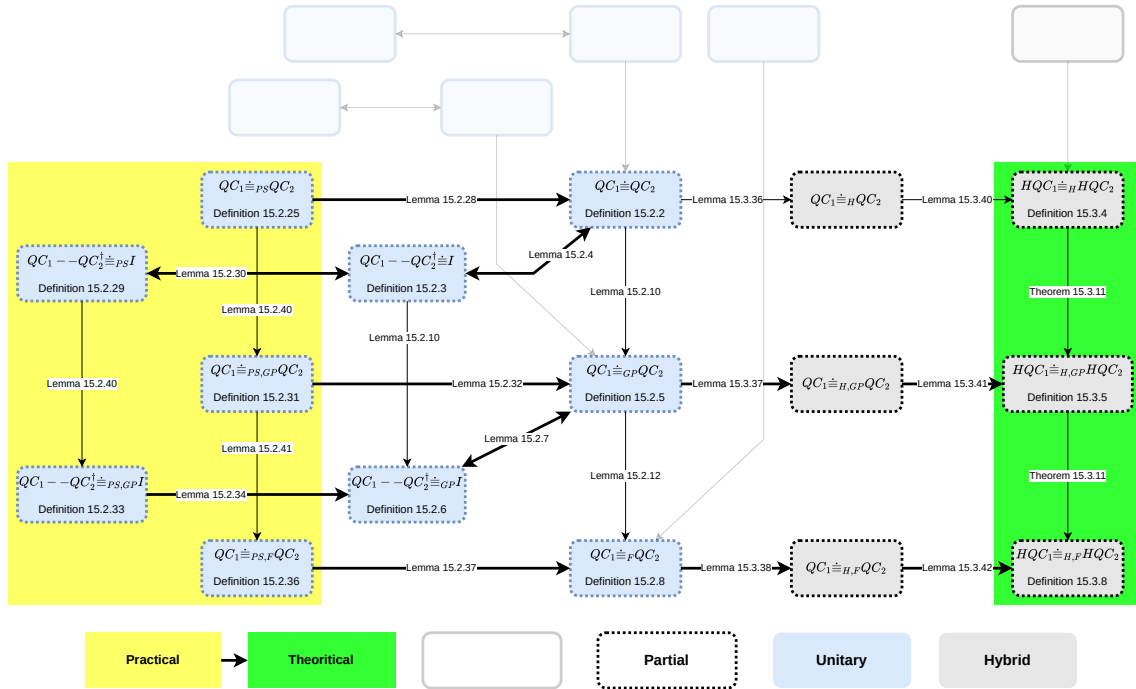


Figure 4.13: Hybrid Partial Equivalence Under Deferred Measurement

4.3.5.1 Equivalence Preservation by Temporal Composition

In this section, we establish semantic preservation under sub-circuit hybrid partial parallel equivalence (Lemma 4.3.49) and under global-phase hybrid partial parallel equiv-

alence (Lemma 4.3.50). These compositionality results are used in Section 4.3.5.2 to prove that our deferred measurement implementation preserves the intended semantics.

Lemme 4.3.49 (SubCir-Hyb-Part-Par Preservation by Temporal Composition). For $i \in \{0, 1\}$, let $\mathcal{HQC}_i = (\text{Init}_i, \text{Di}_i, \text{C}_i)$, and $\mathcal{HQC}'_i = (\text{Init}'_i, \text{Di}'_i, \text{C}'_i)$ be four hybrid circuit tuples (Notation 4.3.8) such that

$$\mathcal{HQC}_1 \dot{=} \mathcal{HQC}_2 \quad \text{and} \quad \mathcal{HQC}'_1 \dot{=} \mathcal{HQC}'_2, \quad (\text{See Definition 4.3.43. for } (\dot{=}))$$

Then the temporally composed circuit tuples (Definition 4.3.9) satisfy:

$$(\mathcal{HQC}_1 \text{---} \mathcal{HQC}'_1) \dot{=} (\mathcal{HQC}_2 \text{---} \mathcal{HQC}'_2).$$

Proof. We prove the preservation of equivalence by temporal composition.

1. Assume:

$$\begin{aligned} \llbracket \text{C}_i \rrbracket_{\text{H}} \left(\left\{ p_{\text{c}}, |\xi_{\text{c}}^i\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i}, \text{c} \right\}_{\text{c} \in \{0,1\}^m} \right) &= \left\{ p_{\text{c}_i}, |f_{\text{c}_i}(\xi_{\text{c}}^i)\rangle_{\text{Obs}_i, \text{Di}_i}, \text{c}_i \right\}_{\text{c}_i \in \{0,1\}^{m_i}} \\ \llbracket \text{C}'_i \rrbracket_{\text{H}} \left(\left\{ p_{\text{c}'}, |\xi_{\text{c}'}^i\rangle_{\text{Inp}_i'} |0\rangle_{\text{Init}_i'}, \text{c}' \right\}_{\text{c}' \in \{0,1\}^{m'}} \right) &= \left\{ p_{\text{c}'_i}, |f_{\text{c}'_i}(\xi_{\text{c}'}^i)\rangle_{\text{Obs}_i', \text{Di}_i'}, \text{c}'_i \right\}_{\text{c}'_i \in \{0,1\}^{m'_i}}. \end{aligned}$$

2. Assume that $(\mathcal{HQC}_1 \dot{=} \mathcal{HQC}_2)$, and $(\mathcal{HQC}'_1 \dot{=} \mathcal{HQC}'_2)$, then:

$$\left. \begin{aligned} \forall i \in \{1, 2\}, \forall \text{c}_i \in \{0, 1\}^{m_i}, \exists |\phi_{\text{c}_i}\rangle \in \mathcal{H}^{\text{Di}_i}, \forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \exists |\psi_{\text{c}_i}(\xi)\rangle \in \mathcal{H}^{\text{Obs}_i}, \\ |f_{\text{c}_i}(\xi)\rangle_{\text{Obs}_i, \text{Di}_i} = |\psi_{\text{c}_i}(\xi)\rangle_{\text{Obs}_i} \otimes |\phi_{\text{c}_i}\rangle_{\text{Di}_i}, \\ \text{Obs}_1 \simeq \text{Obs}_2 \\ \forall \text{c}_1 \in \{0, 1\}^{m_1}, \exists \text{c}_2 \in \{0, 1\}^{m_2}, \\ \left. \begin{aligned} \text{c}_1 |_{\text{Obs}_1} &\cong_{(\text{Obs}_1, \text{Obs}_2)} \text{c}_2 |_{\text{Obs}_2}, \\ |\psi_{\text{c}_1}(\xi)\rangle_{\text{Obs}_1} &\sim_{(\text{Obs}_1, \text{Obs}_2)} |\psi_{\text{c}_2}(\xi)\rangle_{\text{Obs}_2}, \\ \sum_{\text{c}_1 |_{\text{Di}_1}} p_{\text{c}_1 |_{\text{Di}_1}} \text{c}_1 |_{\text{Obs}_1} &= \sum_{\text{c}_2 |_{\text{Di}_2}} p_{\text{c}_2 |_{\text{Di}_2}} \text{c}_2 |_{\text{Obs}_2} \end{aligned} \right\} \end{aligned}$$

$$\left. \begin{aligned} \forall i \in \{1, 2\}, \forall \text{c}_i \in \{0, 1\}^{m_i}, \exists |\phi'_{\text{c}_i}\rangle \in \mathcal{H}^{\text{Di}'_i}, \forall |\xi\rangle \in \mathcal{H}^{\text{Inp}'}, \exists |\psi'_{\text{c}_i}(\xi)\rangle \in \mathcal{H}^{\text{Obs}'_i}, \\ |f'_{\text{c}_i}(\xi)\rangle_{\text{Obs}_i', \text{Di}'_i} = |\psi'_{\text{c}_i}(\xi)\rangle_{\text{Obs}'_i} \otimes |\phi'_{\text{c}_i}\rangle_{\text{Di}'_i}, \\ \text{Obs}'_1 \simeq \text{Obs}'_2 \\ \forall \text{c}_1 \in \{0, 1\}^{m_1}, \exists \text{c}_2 \in \{0, 1\}^{m_2}, \\ \left. \begin{aligned} \text{c}_1 |_{\text{Obs}'_1} &\cong_{(\text{Obs}'_1, \text{Obs}'_2)} \text{c}_2 |_{\text{Obs}'_2}, \\ |\psi'_{\text{c}_1}(\xi)\rangle_{\text{Obs}'_1} &\sim_{(\text{Obs}'_1, \text{Obs}'_2)} |\psi'_{\text{c}_2}(\xi)\rangle_{\text{Obs}'_2}, \\ \sum_{\text{c}_1 |_{\text{Di}'_1}} p_{\text{c}_1 |_{\text{Di}'_1}} \text{c}_1 |_{\text{Obs}'_1} &= \sum_{\text{c}_2 |_{\text{Di}'_2}} p_{\text{c}_2 |_{\text{Di}'_2}} \text{c}_2 |_{\text{Obs}'_2} \end{aligned} \right\} \end{aligned}$$

3. Then Obs_i and Di_i are separated. Thus, the inputs to $\mathcal{HQC}'_1$ and $\mathcal{HQC}'_2$ are the observed parts of $\mathcal{HQC}_1$ and $\mathcal{HQC}_2$, respectively. Formally:

$$\forall i \in \{1, 2\},$$

$$c' = c_i|_{\text{Obs}_i} \quad \text{and} \quad |\xi_{c'}^i\rangle_{\text{Inp}'_i} \sim_{(\text{Inp}'_i, \text{Obs}'_i)} |\psi_{c_i}(\xi_c^i)\rangle_{\text{Obs}_i} \quad \text{and} \quad p_{c'} = \sum_{c_i|_{\text{Di}_i}} p_{c_i|_{\text{Di}_i} c_i|_{\text{Obs}_i}}.$$

4. Then, using this input state, we can compose with $\mathcal{HQC}'_i$, resulting in:

$$\begin{aligned} \llbracket C'_i \rrbracket_{\mathbb{H}} \left(\left\{ p_{c'}, |\xi_{c'}^i\rangle_{\text{Inp}'_i} |0\rangle_{\text{Init}'_i}, c' \right\}_{c' \in \{0,1\}^{m'_i}} \right) \\ = \left\{ p_{c'_i}, |f_{c'_i}(\xi_{c'})\rangle_{\text{Obs}'_i, \text{Di}'_i}, c'_i \right\}_{c'_i \in \{0,1\}^{m'_i}}. \end{aligned}$$

5. Therefore: $(\mathcal{HQC}_1 - \mathcal{HQC}'_1) \stackrel{\text{H}}{\equiv} (\mathcal{HQC}_2 - \mathcal{HQC}'_2)$. □

Lemma 4.3.50 (GP-Hyb-Part-Par Preservation by Temporal Composition). For $i \in \{0, 1\}$, let $\mathcal{HQC}_i = (\text{Init}_i, \text{Di}_i, C_i)$, and $\mathcal{HQC}'_i = (\text{Init}'_i, \text{Di}'_i, C'_i)$ be four hybrid-circuit tuples (Notation 4.3.8) such that:

$$\mathcal{HQC}_1 \stackrel{\text{H, gp}}{\equiv} \mathcal{HQC}_2 \quad \text{and} \quad \mathcal{HQC}'_1 \stackrel{\text{H, gp}}{\equiv} \mathcal{HQC}'_2. \quad (\text{See Definition 4.3.44, for } (\stackrel{\text{H, gp}}{\equiv}))$$

Then the temporally composed circuit tuples (Definition 4.3.9) satisfy:

$$(\mathcal{HQC}_1 - \mathcal{HQC}'_1) \stackrel{\text{H, gp}}{\equiv} (\mathcal{HQC}_2 - \mathcal{HQC}'_2).$$

Proof. The proof proceeds as in Lemma 4.3.49, replacing the use of $\sim$ by $\sim_{(\text{gp})}$ (Notation 2.2.23). □

4.3.5.2 Deferred Measurement (DM)

This section presents our implementation of the deferred measurement principle (introduced in Section 2.2.6). We implement a function DM (Definition 4.3.69) which takes as a parameter a hybrid-circuit and returns a triplet composed of a set of qubit indices to initialise, a unitary circuit and a map whose keys are the bit indices receiving measurement results and the values are the qubit indices measured.

DM is composed of dl (Definition 4.3.53), which takes a hybrid-circuit (C) as parameter and returns the set of qubit indices to initialise (Inits), and a circuit without initialisations (C'). We show that C' contains no initialisation operations (Lem. 4.3.55) and that the tuple (Init, Di, C) is semantically equivalent to the tuple (Init, Di, C') (Lem. 4.3.57).

The second function that constitutes DM is tCQ (Definition 4.3.60). tCQ takes as parameter a hybrid-circuit C and returns a pair consisting of a circuit without measurements C' and a map recording the measurement addressing m_C . Moreover, tCQ preserves the structural property established by dl. In particular, if the input circuit of

tCQ (namely the output of dl) contains no initialization commands, then the resulting circuit C' also contains no initialization commands (Lemma 4.3.61). Hence, the absence of initializations ensured by dl is preserved throughout the deferred-measurement construction.

We define a function which applies the measurements stored in m_C at the end of C ($mA(C, m_C)$). We show C' doesn't content measurement operations (Lemma 4.3.60) and that the tuple $(Init, Di, C)$ is semantically equivalent to the tuple $(Init, Di, mA(C', m_C))$ (Lemma 4.3.57). Finally, when we apply dl, then tCQ on the result of dl, we obtain a unitary circuit (U) , a set of qubit indices to initialise (Inits) and m_C , and we show in the Theorem 4.3.71 that the tuple $(Init, Di, C)$ is semantically equivalent to the tuple $(Init, Di, mA(U, m_C))$. Figure 4.14 presents the general process of our deferred measurement implementation (DM). On the left, we show the source hybrid syntax. On the right, we display the target QASM unitary syntax. The central diagram illustrates the process of transforming the source syntax ($\mathcal{HQC}$) into the QASM unitary syntax ($\mathcal{QC}$). Figure 4.15 presents the results of our deferred measurement implementation, DM.

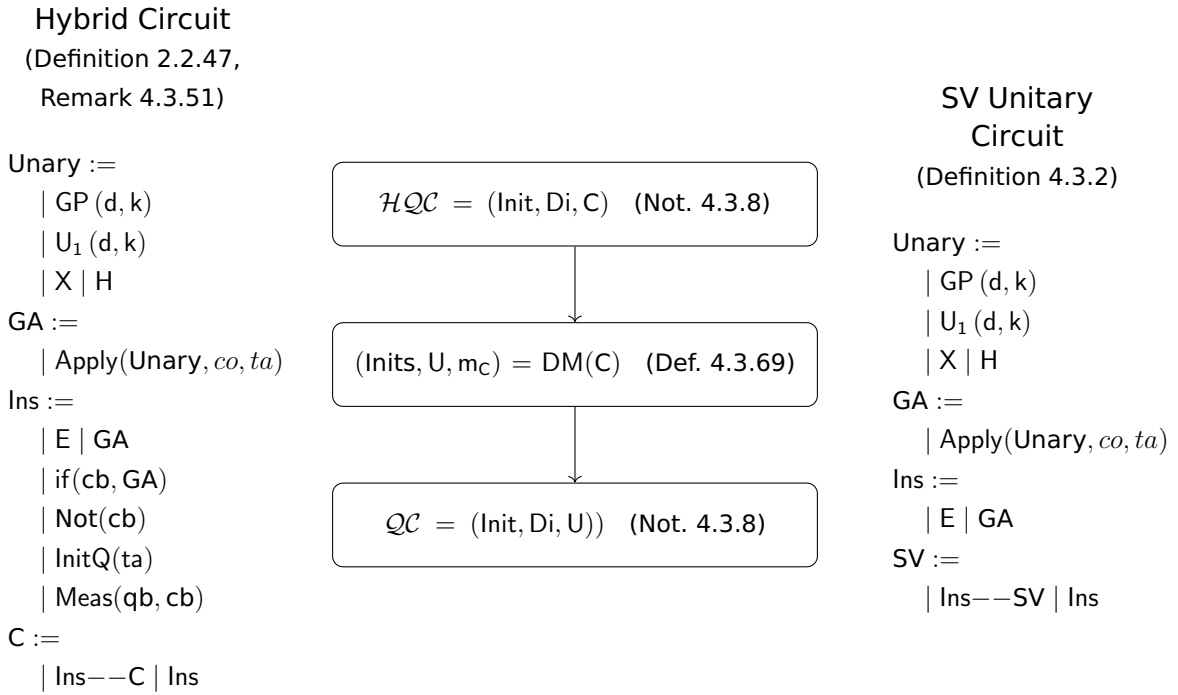


Figure 4.14: Deferred Measurement Overview

Figure 4.15a shows the original hybrid circuit, C . Figure 4.15b displays the intermediate circuit, C' , without initialisation. Finally, Figure 4.15c illustrates the final unitary circuit, U .

Remark 4.3.51 (Hybrid Circuit Syntax Used). We use the syntax and semantics from our hybrid circuit (Section 2.2.5), and to simplify the proof, we limit our gate set to a

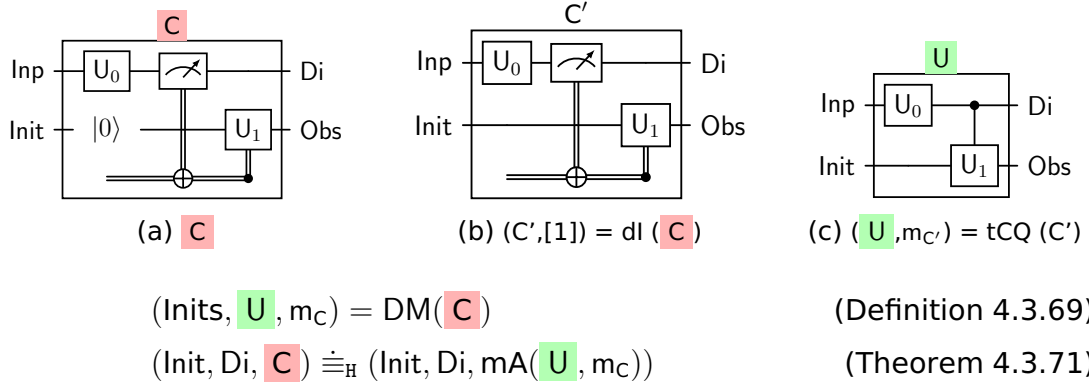


Figure 4.15: Deferred Measurement Function, DM.

minimal universal set: $\{GP(d, k), U_1(d, k), X, H\}$.

Definition 4.3.52 (Count the Number of Initializations (Clnits)). For a hybrid-circuit C (Definition 2.2.47, Remark 4.3.51), we define $Clnits(C)$ as the count of the number of initialisation commands in C . Formally:

$$Clnits(C) := \begin{cases} 1 & \text{if } C = \text{InitQ}(qb) \\ Clnits(C_1) + Clnits(Ins_1) & \text{if } C = C_1 \text{---} Ins_1 \\ 0 & \text{else} \end{cases}$$

Definition 4.3.53 (Decompose Initializations (dl)). Let C be a hybrid-circuit (Definition 2.2.47, Remark 4.3.51) and $\emptyset$ an empty set. We define the function dl as follows:

$$aux_dl(C, Inits) := \begin{cases} (E, qb \cup Inits) & \text{if } C = \text{InitQ}(qb) \\ (C, Inits) & \text{if } C = Ins \\ (C_2 \text{---} Ins_2, Inits_2) & \text{if } C = C_1 \text{---} Ins_1, \text{ then} \\ & (C_2, Inits_1) = aux_dl(C_1, Inits) \\ & (Ins_2, Inits_2) = aux_dl(Ins_1, Inits_1) \end{cases}$$

$dl(C) := aux_dl(C, \emptyset)$.

Lemme 4.3.54 (Decomposition of Initializations (aux_dl)). Let C be a hybrid-circuit (Def. 2.2.47, Rem. 4.3.51) and $Inits$ a set of qubit indices. Assume that $(C', Inits') = aux_dl(C, Inits)$, then

$$Clnits(C') = 0, \quad (4.37)$$

$$Inits' = \{qb \mid \text{InitQ}(qb) \in C\} \cup Inits. \quad (4.38)$$

Proof. We proceed by structural induction on C .

1. If $C = \text{InitQ}(\text{qb})$:

(a) By Definition 4.3.53: $\text{aux_dl}(\text{InitQ}(\text{qb}), \text{Inits}) = (E, \text{qb} \cup \text{Inits})$.

(b) By Definition 4.3.52: $\text{CInits}(E) = 0$.

(c) Therefore, Properties 4.37 and 4.38 hold.

2. If $C = C_1 \text{---} \text{Ins}_1$:

(a) By Definition 4.3.53:

$$(C_2, \text{Inits}_1) = \text{aux_dl}(C_1, \text{Inits})$$

$$(\text{Ins}_2, \text{Inits}_2) = \text{aux_dl}(\text{Ins}_1, \text{Inits}_1)$$

$$(C_2 \text{---} \text{Ins}_2, \text{Inits}_2) = \text{aux_dl}(C_1 \text{---} \text{Ins}_1, \text{Inits})$$

(b) By induction hypothesis:

$$\text{CInits}(C_2) = 0,$$

$$\text{CInits}(\text{Ins}_2) = 0,$$

$$\text{Inits}_1 = \{\text{qb} \mid \text{InitQ}(\text{qb}) \in C_1\} \cup \text{Inits},$$

$$\text{Inits}_2 = \{\text{qb} \mid \text{InitQ}(\text{qb}) \in C_1\} \cup \{\text{qb} \mid \text{InitQ}(\text{qb}) \in \text{Ins}_1\} \cup \text{Inits},$$

(c) Then,

$$\text{CInits}(C_2 \text{---} \text{Ins}_2) = \text{CInits}(C_2) + \text{CInits}(\text{Ins}_2) \quad (\text{By Definition 4.3.52})$$

$$= 0 + 0 = 0,$$

$$\text{Inits}_2 = \{\text{qb} \mid \text{InitQ}(\text{qb}) \in C\} \cup \text{Inits}.$$

(d) Therefore, Properties 4.37 and 4.38 hold.

3. Else:

(a) By Definition 4.3.53: $\text{aux_dl}(C, \text{Inits}) = (C, \text{Inits})$.

(b) In addition, $\text{Ins} \neq \text{InitQ}(\text{qb})$, then $\text{CInits}(C) = 0$, and $\text{Inits} = \emptyset \cup \text{Inits}$.

(c) Therefore, Properties 4.37 and 4.38 hold. $\square$

Lemme 4.3.55 (Decomposition of Initializations (dl)). Let C be a hybrid-circuit (Definition 2.2.47, Remark 4.3.51). Assume that $(C', \text{Inits}') = \text{dl}(C)$, then:

$$\text{CInits}(C') = 0, \tag{4.39}$$

$$\text{Inits}' = \{\text{qb} \mid \text{InitQ}(\text{qb}) \in C\}. \tag{4.40}$$

where dl is defined in the Definition 4.3.53.

Proof. We prove Properties 4.39 and 4.40 hold. By Definition 4.3.53: $\text{dl}(C) = \text{aux_dl}(C, \emptyset) = (C', \text{Inits}')$. By instantiating Lemma 4.3.54 with $\text{Inits} = \emptyset$:

$$\begin{aligned} \text{CInits}(C') &= 0 \\ \text{Inits}' &= \{\text{qb} \mid \text{InitQ}(\text{qb}) \in C\} \cup \emptyset \\ &= \{\text{qb} \mid \text{InitQ}(\text{qb}) \in C\}. \end{aligned} \quad \square$$

Lemme 4.3.56 (SubCir-Hyb-Part-Par Under aux_dl). Let $(\text{Init}, \text{Di}, C)$ be a hybrid-circuit tuple (Notation 4.3.8, Remark 4.3.51), and Inits a set of qubit indices representing the qubits to be initialised. Assume that $(C', \text{Inits}') = \text{aux_dl}(C, \text{Inits})$, and $\text{Inits} \subseteq \text{Init}$ it follows that

$$(\text{Init}, \text{Di}, C) \dot{=}_{\text{H}} (\text{Init}, \text{Di}, C'),$$

where aux_dl is defined in the Definition 4.3.53, and $(\dot{=}_{\text{H}})$ in the Definition 4.3.43.

Proof. We proceed by structural induction on C .

1. If $C = \text{InitQ}(\text{qb})$:

- (a) By Definition 4.3.53, then $(E, \text{Inits}') = \text{aux_dl}(C, \text{Inits})$.
- (b) By Property 4.38: $\text{Inits}' = \text{qb} \cup \text{Inits}$.
- (c) By assumption: $\text{Inits} \subseteq \text{Init}$.
- (d) By Property 4.5: $\text{qb} \subseteq \text{Init}$.
- (e) Then $\text{Inits}' \subseteq \text{Init}$.
- (f) Thus, by Definition 2.2.53:

$$\begin{aligned} \llbracket \text{InitQ}(\text{qb}) \rrbracket_{\text{H}}(\{p_c, |\xi_c\rangle_{\text{Inp}} | 0\rangle_{\text{Init}}, c\}_{c \in \{0,1\}^m}) &= \{p_c, |\xi_c\rangle_{\text{Obs}} | 0\rangle_{\text{Di}}, c\}_{c \in \{0,1\}^m} \\ \llbracket E \rrbracket_{\text{H}}(\{p_c, |\xi_c\rangle_{\text{Inp}} | 0\rangle_{\text{Init}}, c\}_{c \in \{0,1\}^m}) &= \{p_c, |\xi_c\rangle_{\text{Obs}} | 0\rangle_{\text{Di}}, c\}_{c \in \{0,1\}^m} \end{aligned}$$

- (g) Therefore, by Definition 4.3.43: $(\text{Init}, \text{Di}, E) \dot{=}_{\text{H}} (\text{Init}, \text{Di}, \text{InitQ}(\text{qb}))$.

2. If $C = C_1 \text{---} \text{Ins}_1$:

- (a) By Definition 4.3.53:

$$\begin{aligned} (C_2, \text{Inits}_1) &= \text{aux_dl}(C_1, \text{Inits}), \\ (\text{Ins}_2, \text{Inits}_2) &= \text{aux_dl}(\text{Ins}_1, \text{Inits}_1), \\ (C_2 \text{---} \text{Ins}_2, \text{Inits}_2) &= \text{aux_dl}(C, \text{Inits}) \end{aligned}$$

- (b) By induction hypothesis:

$$\begin{aligned} (\text{Init}, \text{Di}, C_1) &\dot{=}_{\text{H}} (\text{Init}, \text{Di}, C_2) \\ (\text{Init}, \text{Di}, \text{Ins}_1) &\dot{=}_{\text{H}} (\text{Init}, \text{Di}, \text{Ins}_2) \end{aligned}$$

(c) By Lemma 4.3.49: $(\text{Init}, \text{Di}, C) \dot{\equiv}_H (\text{Init}, \text{Di}, C_2 \text{---} \text{Ins}_2)$.

3. Else, by Definition 4.3.53: $(C', \text{Inits}') = \text{aux_dl}(C, \text{Inits})$, and $(C', \text{Inits}') = (C, \text{Inits})$, then $(\text{Init}, \text{Di}, C) \dot{\equiv}_H (\text{Init}, \text{Di}, C')$. $\square$

Lemme 4.3.57 (SubCir-Hyb-Part-Par Under dl). Let $(\text{Init}, \text{Di}, C)$ be a hybrid-circuit tuple (Notation 4.3.8, Remark 4.3.51). Assume that $(C', \text{Inits}') = \text{dl}(C)$, it follows that

$$(\text{Init}, \text{Di}, C) \dot{\equiv}_H (\text{Init}, \text{Di}, C'),$$

where dl is defined in the Definition 4.3.53, and $(\dot{\equiv}_H)$ is defined in the Definition 4.3.43.

Proof. By Def. 4.3.53: $\text{dl}(C) = \text{aux_dl}(C, []) = (C', \text{Inits}')$. By instantiating Lem. 4.3.56 with $\text{Inits} = \emptyset$: $(\text{Init}, \text{Di}, C) \dot{\equiv}_H (\text{Init}, \text{Di}, C')$ $\square$

Definition 4.3.58 (Count the Number of Classical Operations (CMeass)). For a hybrid-circuit C (Definition 2.2.47, Remark 4.3.51), $\text{CMeass}(C)$ is defined as the count of classical operations, including measurements. Formally:

$$\text{CMeass}(C) := \begin{cases} 1 & \text{if } C \in \{\text{Meas}(\text{qb}, \text{cb}), \text{if}(\text{cb}, \text{Apply}(G, \text{qb}, \text{qb})), \text{Not}(\text{cb})\} \\ \text{CMeass}(C_1) + & \text{if } C = C_1 \text{---} \text{Ins}_1 \\ \text{CMeass}(\text{Ins}_1) & \\ 0 & \text{else} \end{cases}$$

We define (Definition 4.3.59) a map from bit indices to qubit indices. This map is used in our deferred measurement transformations (DM) to convert classical control commands into quantum control commands.

Definition 4.3.59 (Map from Bit to Qubit (m_C)). m_C is a non-empty map from a set of classical bit indices cb to a set of qubit indices qb for a hybrid circuit C (Definition 2.2.47, Remark 4.3.51). Let $\text{aux_}m_C$ be a non-empty map from a classical bit index b to a qubit index q for a circuit C . Then we define m_C as follows:

$$\begin{aligned} m_C([b_1, b_2, \dots, b_n]) &:= [\text{aux_}m_C(b_1), \text{aux_}m_C(b_2), \dots, \text{aux_}m_C(b_n)] \\ \forall b \in \text{cb}, \exists q \in \text{C} \quad &| \quad q = \text{aux_}m_C(b). \end{aligned}$$

The function tCQ (Definition 4.3.60) transforms classical operations into quantum operations and removes measurements. It returns a unitary operation along with a map, as defined in Definition 4.3.59, which maps bits that have received measurement results to the source qubits of these results.

Definition 4.3.60 (Transform Classical to Quantum Operations (tCQ)). Let C be a hybrid-circuit (Definition 2.2.47, Remark 4.3.51), the function tCQ is defined recursively

as follows:

$$\text{aux_tCQ}(C, m_C) := \begin{cases} (\text{Apply}(G, \text{co} \cup m_C(\text{cb}), \text{ta}), m_C) & \text{if } C = \text{if}(\text{cb}, \text{Apply}(G, \text{co}, \text{ta})) \\ (\text{Apply}(X, \emptyset, m_C(\text{cb})), m_C) & \text{if } C = \text{Not}(\text{cb}) \\ (E, m_C[\text{cb} \leftarrow \text{qb}]) & \text{if } C = \text{Meas}(\text{qb}, \text{cb}) \\ (C_2 \text{---} \text{Ins}_2, m_{C_2}) & \text{if } C = C_1 \text{---} \text{Ins}_1, \text{ then} \\ & (C_2, m_{C_1}) = \text{aux_tCQ}(C_1, m_C) \\ & (\text{Ins}_2, m_{C_2}) = \text{aux_tCQ}(\text{Ins}_1, m_{C_1}) \\ (C, m_C) & \text{else} \end{cases}$$

$$\text{tCQ}(C) := \text{aux_tCQ}(C, \emptyset)$$

where $\emptyset$ represents the empty set, $\emptyset$ denotes an empty map, $m_C[\text{cb} \leftarrow \text{qb}]$ represents adding the mapping of a bit indices set (cb) to a qubit indices set (qb) in the map m_C .

Lemme 4.3.61 (No Introduction of Initializations by tCQ). Let C be a hybrid-circuit (Definition 2.2.47, Remark 4.3.51). Assume that $\text{CInits}(C) = 0$. Then $\text{CInits}(\text{tCQ}(C)) = 0$.

Proof. The proof is immediate by structural induction on C , following Definition 4.3.60. No clause of tCQ introduces an initialization command, and the sequential case preserves this property by induction. $\square$

Lemme 4.3.62 (Transformation to Quantum Operations (aux_tCQ)). Let C be a hybrid-circuit (Definition 2.2.47, Remark 4.3.51), and m_C a map as defined in the Definition 4.3.59.

Assume that $(C', m_{C'}) = \text{aux_tCQ}(C, m_C)$, then the following properties hold:

$$\text{CMeass}(C') = 0 \tag{4.41}$$

$$\forall \text{cb}, (m_{C'}(\text{cb}) = \text{qb}) \iff (\text{Meas}(\text{qb}, \text{cb}) \in C \vee m_C(\text{cb}) = \text{qb}). \tag{4.42}$$

Proof. We proceed by structural induction on C .

1. If $C = \text{if}(\text{cb}, \text{Apply}(G, \text{co}, \text{ta}))$:

- (a) By Definition 4.3.60: $(C', m_{C'}) = (\text{Apply}(G, \text{co} \cup m_C(\text{cb}), \text{ta}), m_C)$.
- (b) Then, $\text{CMeass}(C') = 0$, and $m_{C'} = m_C$.
- (c) Therefore, Properties 4.41 and 4.42 hold.

2. If $C = \text{Not}(\text{cb})$:

- (a) By Definition 4.3.60: $(C', m_{C'}) = (\text{Apply}(X, \emptyset, m_C(\text{cb})), m_C)$.
- (b) Then, $\text{CMeass}(C') = 0$, and $m_{C'} = m_C$.
- (c) Therefore, Properties 4.41 and 4.42 hold.

3. If $C = \text{Meas}(\text{qb}, \text{cb})$:

- (a) By Definition 4.3.60: $(C', m_C') = (E, m_C[cb \leftarrow qb])$.
- (b) Then, $C\text{Meass}(C') = 0$, and $m_C' = m_C[cb \leftarrow qb]$.
- (c) Therefore, Properties 4.41 and 4.42 hold.

4. Inductive case, $C = C_1 \dashv\vdash \text{Ins}_1$:

- (a) By Definition 4.3.60:

$$\begin{aligned} (C_2, m_{C_1}) &= \text{aux_tCQ}(C_1, m_C) \\ (\text{Ins}_2, m_{C_2}) &= \text{aux_tCQ}(\text{Ins}_1, m_{C_1}) \\ (C_2 \dashv\vdash \text{Ins}_2, m_{C_2}) &= \text{aux_tCQ}(C_1 \dashv\vdash \text{Ins}_1, m_C) \\ (C_2 \dashv\vdash \text{Ins}_2, m_{C_2}) &= (C', m_C') \end{aligned}$$

- (b) Inductive hypothesis:

$$\begin{aligned} C\text{Meass}(C_2) &= 0 \\ \forall cb. (m_{C_1}(cb) = qb) &\iff (\text{Meas}(qb, cb) \in C_1 \vee m_C(cb) = qb) \\ C\text{Meass}(\text{Ins}_2) &= 0 \\ \forall cb. (m_{C_2}(cb) = qb) &\iff (\text{Meas}(qb, cb) \in \text{Ins}_1 \vee m_{C_1}(cb) = qb). \end{aligned}$$

- (c) By inductive hypothesis and by Definition 4.3.58:

$$C\text{Meass}(C') = C\text{Meass}(C_2) + C\text{Meass}(\text{Ins}_2) = 0.$$

- (d) By inductive hypothesis:

$$\begin{aligned} \forall cb, (m_{C_2}(cb) = qb) \\ &\iff (\text{Meas}(qb, cb) \in \text{Ins}_1 \vee \text{Meas}(qb, cb) \in C_1 \vee m_C(cb) = qb) \\ &\iff (\text{Meas}(qb, cb) \in C \vee m_C(cb) = qb). \end{aligned}$$

- (e) Therefore, Properties 4.41 and 4.42 hold.

5. Else, by Def. 4.3.60: $(C', m_C') = (\text{Ins}, m_C)$. Therefore, Prop. 4.41 and 4.42 hold. $\square$

Lemme 4.3.63 (Transformation to Quantum Operations). Let C be a hybrid-circuit (Definition 2.2.47, Remark 4.3.51). Assume that $(C', m_C') = \text{tCQ}(C)$, then the following properties hold:

$$C\text{Meass}(C') = 0 \tag{4.43}$$

$$\forall cb, (\text{Meas}(qb, cb) \in C) \iff (m_C'(cb) = qb). \tag{4.44}$$

Proof. By Definition 4.3.60: $\text{tCQ} = \text{aux_tCQ}(C, \emptyset)$. Then, by instantiating Lemma 4.3.62 with $\text{Inits} = \emptyset$, properties 4.41 and 4.42 hold, thus properties 4.43 and 4.44 hold. $\square$

Definition 4.3.64 (Measurement Application). Let $\text{find}(m_C)$ be a function that, given a mapping m_C (Definition 4.3.59), removes an arbitrary entry from m_C and returns both the removed entry and the updated mapping. Then, the function $\text{mA}(C, m_C)$ is defined recursively to apply measurements (based on m_C) to a hybrid-circuit C (Definition 2.2.47, Remark 4.3.51). The definition is as follows:

$$\text{mA}(C, m_C) := \begin{cases} C & \text{if } m_C = \emptyset \\ \text{mA}(C - \text{Meas}(\text{qb}, \text{cb}), m_C') & \text{if } (\text{qb}, \text{cb}, m_C') = \text{find}(m_C). \end{cases}$$

Notation 4.3.65 (Measurement Result with Fixed Bit Value). Let cb be a set of qubit indices. We introduce the notation $c_{[\text{cb}=i]}$ where $i \in \{0, 1\}$ to denote a bit string that matches c in all bit positions except at indices cb , for which it has the value i . Formally, given a bit string $c = [b_0, b_1, \dots, b_{n-1}]$ representing a measurement result and a set of bit index cb (with $\forall c \in \text{cb}, 0 \leq c \leq n-1$), the notation $c_{[\text{cb}=i]}$ represents the bit string where:

$$\forall j \in [0, n-1], \quad c_{[\text{cb}=i]}(j) = \begin{cases} i & \text{if } j \in \text{cb}, \\ b_j & \text{else.} \end{cases}$$

Notation 4.3.66 (Qubit State with Fixed Qubit Value). We introduce the notation $|\psi\rangle_{S[\text{qb}=i]}$, where S is a set of qubit indices and $i \in \{0, 1\}$. This denotes a quantum state in S that matches $|\psi\rangle$ except at indices qb , which is set to $|i\rangle$. Formally, given $|\psi\rangle_S = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle$ with n qubits in S and a set of qubit indices qb :

$$|\psi\rangle_{S[\text{qb}=i]} = \begin{cases} \sum_{x \in \{0,1\}^n, x_q=i} \alpha_x |x\rangle & \text{if } q \in \text{qb}, \\ |\psi\rangle_S & \text{else.} \end{cases}$$

Lemme 4.3.67 (SubCir-Hyb-Part-Par Under aux_tCQ). Let $(\text{Init}, \text{Di}, C)$ be a hybrid-circuit tuple (Notation 4.3.8, Remark 4.3.51), and m_C a map as defined in Definition 4.3.59. Assume that $(C', m_C') = \text{aux_tCQ}(C, m_C)$, then the following property holds:

$$(\text{Init}, \text{Di}, \text{mA}(C', m_C')) \dot{=}_{\text{H}} (\text{Init}, \text{Di}, \text{mA}(C, m_C)).$$

Proof. We proceed by structural induction on C .

1. If $C = \text{Meas}(\text{qb}, \text{cb})$:

- (a) By Definition 4.3.60: $(C', m_C') = (E, m_C[\text{cb} \leftarrow \text{qb}])$.
- (b) By Definition 4.3.64: $\text{mA}(C', m_C') = \text{mA}(C, m_C)$.
- (c) Therefore, $(\text{Init}, \text{Di}, \text{mA}(C', m_C')) \dot{=}_{\text{H}} (\text{Init}, \text{Di}, \text{mA}(C, m_C))$.

2. If $C = \text{if}(\text{cb}, \text{Apply}(G, \text{co}, \text{ta}))$:

- (a) By Definition 4.3.60: $(C', m_C') = (\text{Apply}(G, \text{co} \cup m_C(\text{cb}), \text{ta}), m_C)$.

(b) By Definition 2.2.53:

On one side, we have:

$$\begin{aligned}
& \llbracket \text{if}(\text{cb}, \text{Apply}(\text{G}, \text{co}, \text{ta})) \rrbracket_{\text{H}} \left(\left\{ p_{\text{c}}, |\xi_{\text{c}}\rangle_{\text{Inp}} |0\rangle_{\text{Init}}, \text{c} \right\}_{\text{c} \in \{0,1\}^m} \right) \\
&= \left\{ \left(p_{\text{c}_{[\text{cb}=0]}}, |\xi_{\text{c}_{[\text{cb}=0]}}\rangle_{\text{Obs}[\text{qb}=0]} |0\rangle_{\text{Di}}, \text{c}_{[\text{cb}=0]} \right), \right. \\
&\quad \left. \left(p_{\text{c}_{[\text{cb}=1]}}, \llbracket \text{Apply}(\text{G}, \text{co}, \text{ta}) \rrbracket_{\text{H}} \left(|\xi_{\text{c}_{[\text{cb}=1]}}\rangle_{\text{Inp}[\text{qb}=1]} |0\rangle_{\text{Init}} \right), \text{c}_{[\text{cb}=1]} \right) \right\}_{\text{c} \in \{0,1\}^m} \\
&= \rho_1.
\end{aligned}$$

On the other side we have:

$$\begin{aligned}
& \llbracket \text{Apply}(\text{G}, \text{co} \cup \text{m}_{\text{C}(\text{cb})}, \text{ta}) \rrbracket_{\text{H}} \left(\left\{ p_{\text{c}}, \left(|\xi_{\text{c}}\rangle_{\text{Inp}[\text{qb}=0]} + |\xi_{\text{c}}\rangle_{\text{Inp}[\text{qb}=1]} \right) |0\rangle_{\text{Init}}, \text{c} \right\}_{\text{c} \in \{0,1\}^m} \right) \\
&= \left\{ p_{\text{c}}, |\xi_{\text{c}}\rangle_{\text{Obs}[\text{qb}=0]} |0\rangle_{\text{Di}} + \llbracket \text{Apply}(\text{G}, \text{co}, \text{ta}) \rrbracket_{\text{H}} \left(|\xi_{\text{c}}\rangle_{\text{Inp}[\text{qb}=1]} |0\rangle_{\text{Init}} \right), \text{c} \right\}_{\text{c} \in \{0,1\}^m} \\
&= \rho_2, \\
& \llbracket \text{Meas}(\text{qb}, \text{cb}) \rrbracket_{\text{H}}(\rho_2) = \rho_1.
\end{aligned}$$

(c) Therefore, by Definition 4.3.43:

$$\begin{aligned}
& (\text{Init}, \text{Di}, \text{mA}(\text{if}(\text{cb}, \text{Apply}(\text{G}, \text{co}, \text{ta})), \text{m}_{\text{C}})) \stackrel{\text{H}}{=} \\
& (\text{Init}, \text{Di}, \text{mA}(\text{Apply}(\text{G}, \text{co} \cup \text{m}_{\text{C}(\text{cb})}, \text{ta}), \text{m}_{\text{C}})).
\end{aligned}$$

3. If $\text{C} = \text{Not}(\text{cb})$:

(a) By Definition 4.3.60: $(\text{C}', \text{m}_{\text{C}'}) = (\text{Apply}(\text{X}, \emptyset, \text{m}_{\text{C}(\text{cb})}), \text{m}_{\text{C}})$.

(b) By Definition 4.3.64: $\text{mA}(\text{C}', \text{m}_{\text{C}'}) = \text{Apply}(\text{X}, \emptyset, \text{m}_{\text{C}(\text{cb})})$.

(c) By Definition 2.2.53:

On one side, we have:

$$\begin{aligned}
& \llbracket \text{Not}(\text{cb}) \rrbracket_{\text{H}} \left(\left\{ p_{\text{c}}, |\xi_{\text{c}}\rangle_{\text{Inp}} |0\rangle_{\text{Init}}, \text{c} \right\}_{\text{c} \in \{0,1\}^{|\text{c}|}} \right) \\
&= \left\{ p_{\text{c}'}, |\xi_{\text{c}'}\rangle_{\text{Obs}} |0\rangle_{\text{Di}}, \text{c}' \right\}_{\text{c}' = \text{c}_{[\text{cb} \leftarrow \overline{\text{c}(\text{cb})]}} \\
&= \rho_1,
\end{aligned}$$

By well-formed conditions of hybrid-circuit (Definition 2.2.47), the application of classical control is only permitted on the results of measurements performed on qubits to be discarded.

On the other side, we have:

$$\begin{aligned}
& \llbracket \text{Apply}(\text{X}, \emptyset, \text{m}_{\text{C}(\text{cb})}) \rrbracket_{\text{H}} \left(\left\{ p_{\text{c}}, |\xi_{\text{c}}\rangle_{\text{Inp}} |0\rangle_{\text{Init}}, \text{c} \right\}_{\text{c} \in \{0,1\}^m} \right) = \rho_2, \\
& \llbracket \text{Meas}(\text{m}_{\text{C}(\text{cb})}, \text{cb}) \rrbracket_{\text{H}}(\rho_2) = \rho_1.
\end{aligned}$$

4. If $C = C_1 \text{---} \text{Ins}_1$:

(a) By Definition 4.3.60:

$$\begin{aligned} (C_2, m_{C1}) &= \text{aux_tCQ}(C_1, m_C) \\ (\text{Ins}_2, m_{C2}) &= \text{aux_tCQ}(\text{Ins}_1, m_{C1}) \\ (C_2 \text{---} \text{Ins}_2, m_{C2}) &= \text{aux_tCQ}(C, m_C). \end{aligned}$$

(b) By induction hypothesis:

$$\begin{aligned} (\text{Init}, \text{Di}, \text{mA}(C_2, m_{C1})) &\dot{=}_{\text{H}} (\text{Init}, \text{Di}, \text{mA}(C_1, m_C)) \\ (\text{Init}, \text{Di}, \text{mA}(\text{Ins}_2, m_{C2})) &\dot{=}_{\text{H}} (\text{Init}, \text{Di}, \text{mA}(\text{Ins}_1, m_{C1})). \end{aligned}$$

(c) By Lemma 4.3.49:

$$(\text{Init}, \text{Di}, \text{mA}(C_2 \text{---} \text{Ins}_2, m_{C2})) \dot{=}_{\text{H}} (\text{Init}, \text{Di}, \text{mA}(C, m_C)).$$

5. Else, by Definition 4.3.60: $(C', m_{C'}) = (C, m_C)$, then

$$(\text{Init}, \text{Di}, \text{mA}(C, m_C)) \dot{=}_{\text{H}} (\text{Init}, \text{Di}, \text{mA}(C', m_{C'})). \quad \square$$

Lemme 4.3.68 (SubCir-Hyb-Part-Par Under tCQ). Let $(\text{Init}, \text{Di}, C)$ be a hybrid-circuit tuple (Notation 4.3.8, Remark 4.3.51), and m_C a map as defined in Definition 4.3.59. Assume that $(C', m_{C'}) = \text{tCQ}(C)$, then the following property holds:

$$(\text{Init}, \text{Di}, \text{mA}(C', m_{C'})) \dot{=}_{\text{H}} (\text{Init}, \text{Di}, C).$$

Proof. By Definition 4.3.60: $(C', m_{C'}) = \text{tCQ}(C) = \text{aux_tCQ}(C, \emptyset)$. Then, by instantiating Lemma 4.3.67 with $m_C = \emptyset$: $(\text{Init}, \text{Di}, \text{mA}(C', m_{C'})) \dot{=}_{\text{H}} (\text{Init}, \text{Di}, C)$. $\square$

Our implementation of the deferred measurement principle (DM, Definition 4.3.69) rearranges a hybrid-circuit by moving initialisation commands to the beginning and measurements to the end of the circuit.

Definition 4.3.69 (Deferred Measurement Transformation (DM)). Let C be a hybrid-circuit (Notation 4.3.8, Remark 4.3.51). The transformation DM is defined as follows:

$$\text{DM}(C) := \begin{cases} (\text{Inits}, U, m_C) & \text{where } (C', \text{Inits}) = \text{dl}(C) \text{ (Definition 4.3.53)} \\ & \text{and } (U, m_C) = \text{tCQ}(C') \text{ (Definition 4.3.60)} \end{cases}$$

Definition 4.3.70 (Initialisation-Unitary-Measurement-Circuit (IUM)). Let Inits be a set of qubit indices to initialise, U a unitary circuit (Definition 2.2.41), and m_C a map as defined in Definition 4.3.59. We define an *initialisation-unitary-measurement-circuit* as

$$\text{IUM} = (\text{Inits}, U, m_C).$$

Theorem 4.3.71 (SubCir-Hyb-Part-Par By Deferred Measurement). Let $(\text{Init}, \text{Di}, C)$ be a hybrid-circuit tuple (Notation 4.3.8, Remark 4.3.51). Assume that $(\text{Inits}, U, m_C) = \text{DM}(C)$, the following properties hold:

$$(\text{Init}, \text{Di}, m_A(U, m_C)) \dot{=}_{\text{H}} (\text{Init}, \text{Di}, C), \quad (4.45)$$

$$\text{CInits}(U) = 0, \quad (4.46)$$

$$\text{CMeass}(U) = 0, \quad (4.47)$$

where $(\dot{=}_{\text{H}})$ is the sub-circuit hybrid partial parallel equivalence defined in 4.3.43.

Proof. We prove that the properties 4.45, 4.46, 4.47 are satisfied.

1. By Definition 4.3.69: $(C', \text{Inits}) = \text{dl}(C)$, and $(U, m_C) = \text{tCQ}(C')$.
2. By Lemma 4.3.55: $\text{CInits}(C') = 0$.
3. By Lemma 4.3.61: $\text{CInits}(U) = 0$, thus 4.46 is satisfied.
4. By Lemma 4.3.63: $\text{CMeass}(U) = 0$, then Property 4.47 is satisfied.
5. By Lemma 4.3.57: $(\text{Init}, \text{Di}, C) \dot{=}_{\text{H}} (\text{Init}, \text{Di}, C')$.
6. By Lemma 4.3.68: $(\text{Init}, \text{Di}, C') \dot{=}_{\text{H}} (\text{Init}, \text{Di}, m_A(U, m_C))$.
7. Therefore, by transitivity: $(\text{Init}, \text{Di}, C) \dot{=}_{\text{H}} (\text{Init}, \text{Di}, m_A(U, m_C))$. By symmetry of $\dot{=}_{\text{H}}$, we obtain $(\text{Init}, \text{Di}, m_A(U, m_C)) \dot{=}_{\text{H}} (\text{Init}, \text{Di}, C)$, hence Property 4.45 holds. $\square$

Corollary 4.3.72 (GP-Hyb-Part-Par By Deferred Measurement). Let $(\text{Init}, \text{Di}, C)$ be a hybrid-circuit tuple (Notation 4.3.8, Remark 4.3.51). Assume that $(\text{Inits}, U, m_C) = \text{DM}(C)$, the following properties hold:

$$(\text{Init}, \text{Di}, m_A(U, m_C)) \dot{=}_{(\text{H}, \text{gp})} (\text{Init}, \text{Di}, C), \quad (4.48)$$

$$\text{CInits}(U) = 0, \quad (4.49)$$

$$\text{CMeass}(U) = 0, \quad (4.50)$$

where $(\dot{=}_{(\text{H}, \text{gp})})$ is the global-phase-partial-hybrid-equivalence defined in 4.3.44.

Proof. We prove that the properties 4.48, 4.49, 4.50 are satisfied. By Theorem 4.3.71: $(\text{Init}, \text{Di}, m_A(U, m_C)) \dot{=}_{\text{H}} (\text{Init}, \text{Di}, C)$, $\text{CInits}(U) = 0$, and $\text{CMeass}(U) = 0$. Then, by Theorem 4.3.48: $(\text{Init}, \text{Di}, m_A(U, m_C)) \dot{=}_{(\text{H}, \text{gp})} (\text{Init}, \text{Di}, C)$ $\square$

Corollary 4.3.73 (FullCir-Hyb-Part-Par By Deferred Measurement). Let $(\text{Init}, \text{Di}, C)$ be a hybrid-circuit tuple (Notation 4.3.8, Remark 4.3.51). Assume that $(\text{Inits}, U, m_C) = \text{DM}(C)$, the following properties hold:

$$(\text{Init}, \text{Di}, m_A(U, m_C)) \dot{=}_{(\text{H}, \text{f})} (\text{Init}, \text{Di}, C), \quad (4.51)$$

$$\text{CInits}(U) = 0, \quad (4.52)$$

$$\text{CMeass}(U) = 0, \quad (4.53)$$

where $(\dot{=}_{(\text{H}, \text{f})})$ is the full-circuit-partial-hybrid-equivalence defined in 4.3.47.

Proof. We prove that the properties 4.51, 4.52, 4.53 are satisfied. By Theorem 4.3.71: $(\text{Init}, \text{Di}, \text{mA}(\text{U}, \text{m}_\text{C})) \dot{=}_{\text{H}} (\text{Init}, \text{Di}, \text{C})$, $\text{CInits}(\text{U}) = 0$ and $\text{CMeass}(\text{U}) = 0$. Therefore, by Theorem 4.3.48: $(\text{Init}, \text{Di}, \text{mA}(\text{U}, \text{m}_\text{C})) \dot{=}_{(\text{H}, \text{f})} (\text{Init}, \text{Di}, \text{C})$. $\square$

4.3.5.3 Reduction from Hyb-Par to Unit-Par for Unitary Circuit

This section presents a method for verifying the **hybrid partial equivalence** of **unitary circuit**, as defined in Section 4.3.4, by using the **unitary partial equivalence** defined in Section 4.3.2. This is a necessary step to prove our method: verifying the hybrid partial equivalence of hybrid-circuit, as defined in Section 4.3.4, by using the unitary partial equivalence defined in Section 4.3.2. Figure 4.16 provides an overview

$$\begin{aligned}
 & \left\{ p_c, |\xi_c^i\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i}, c \right\}_{c \in \{0,1\}^m} \left\{ \begin{array}{c} \text{---} \\ \boxed{\text{U}_i} \\ \text{---} \end{array} \right\} \left\{ p_c, |f_c^i(\xi_c^i)\rangle_{\text{Obs}_i, \text{Di}_i}, c \right\}_{c \in \{0,1\}^m} \\
 & \forall i \in \{1, 2\}, \mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, \text{C}_i) \\
 & (\mathcal{QC}_1 \dot{=} \mathcal{QC}_2) \implies (\mathcal{QC}_1 \dot{=}_{\text{H}} \mathcal{QC}_2) \quad \text{(Lemma 4.3.74)} \\
 & (\mathcal{QC}_1 \dot{=}_{\text{gp}} \mathcal{QC}_2) \implies (\mathcal{QC}_1 \dot{=}_{(\text{H}, \text{gp})} \mathcal{QC}_2) \quad \text{(Lemma 4.3.75)} \\
 & (\mathcal{QC}_1 \dot{=}_{\text{f}} \mathcal{QC}_2) \iff (\mathcal{QC}_1 \dot{=}_{(\text{H}, \text{f})} \mathcal{QC}_2) \quad \text{(Lemma 4.3.76)}
 \end{aligned}$$

Figure 4.16: Reduction of Hyb-Part of Unitary Circuit to Unit-Part

of our method, since U_1 and U_2 are unitary circuits, we can use either hybrid semantics (Definition 2.2.53) or QASM unitary semantics (Definition 2.2.44), as a result, in both cases, only the quantum state evolves. Lemma 4.3.74 proves our method for sub-circuit, Lemma 4.3.75 for the global-phase, and Lemma 4.3.76 for full-circuit.

Lemme 4.3.74 (Reduction from SubCir-Hyb-Comp-Par to SubCir-Unit-Comp-Par for U). Let $\mathcal{QC}_1 = (\text{Init}_1, \text{Di}_1, \text{U}_1)$ and $\mathcal{QC}_2 = (\text{Init}_2, \text{Di}_2, \text{U}_2)$ be two unitary circuit tuples (Notation 4.3.8). If $\mathcal{QC}_1$ is SubCir-Unit-Comp-Par to $\mathcal{QC}_2$ (Definition 4.3.12) then, $\mathcal{QC}_1$ is SubCir-Hyb-Comp-Par to $\mathcal{QC}_2$ (Definition 4.3.43), denoted by: $(\mathcal{QC}_1 \dot{=} \mathcal{QC}_2) \implies (\mathcal{QC}_1 \dot{=}_{\text{H}} \mathcal{QC}_2)$.

Proof. Let $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, \text{U}_i)$ be two unitary circuit tuples (Notation 4.3.8). Assume that $(\mathcal{QC}_1 \dot{=} \mathcal{QC}_2)$, by Property 4.8, for all $i \in \{0, 1\}$:

$$\llbracket \text{U}_i \rrbracket_{\text{H}} \left(\left\{ p_c, |\xi_c^i\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i}, c \right\}_{c \in \{0,1\}^m} \right) = \left\{ p_{c_i}, |\psi_{c_i}(\xi_c^i)\rangle_{\text{Obs}_i} \otimes |\phi_{c_i}\rangle_{\text{Di}_i}, c_i \right\}_{c_i \in \{0,1\}^{m_i}}.$$

By unitarity of U_i : $\forall i \in \{1, 2\}$, $m = m_i$. Then by Property 4.9 the output extended mixed states are partially equal, thus Property 4.31 holds, therefore $(\mathcal{QC}_1 \dot{=}_{\text{H}} \mathcal{QC}_1)$. $\square$

Lemme 4.3.75 (Reduction from GP-Hyb-Comp-Par to GP-Unit-Comp-Par for U). Let $\mathcal{QC}_1 = (\text{Init}_1, \text{Di}_1, U_1)$ and $\mathcal{QC}_2 = (\text{Init}_2, \text{Di}_2, U_2)$ be two unitary circuit tuples (Notation 4.3.8). If $\mathcal{QC}_1$ is GP-Unit-Comp-Par to $\mathcal{QC}_2$ (Definition 4.3.15), then, $\mathcal{QC}_1$ is GP-Hyb-Comp-Par to $\mathcal{QC}_2$ (Definition 4.3.44), denoted by $(\mathcal{QC}_1 \dot{=}_{\text{gp}} \mathcal{QC}_2) \implies (\mathcal{QC}_1 \dot{=}_{(\text{H}, \text{gp})} \mathcal{QC}_2)$.

Proof. The proof is identical to that of Lemma 4.3.74, replacing the equality of vectors ($=$), Definition 2.2.10) with equivalence up to a global-phase ($\approx_{\text{gp}}$), Definition 2.2.12). $\square$

Lemme 4.3.76 (Reduction from FullCir-Hyb-Par to FullCir-Unit-Par for U). Let $\mathcal{QC}_1 = (\text{Init}_1, \text{Di}_1, U_1)$ and $\mathcal{QC}_2 = (\text{Init}_2, \text{Di}_2, U_2)$ be two unitary circuit tuples (Notation 4.3.8). If $\mathcal{QC}_1$ is FullCir-Unit-Par to $\mathcal{QC}_2$ (Definition 4.3.18), if and only if, $\mathcal{QC}_1$ is FullCir-Hyb-Par to $\mathcal{QC}_2$ (Definition 4.3.47), denoted by: $(\mathcal{QC}_1 \dot{=}_{\text{f}} \mathcal{QC}_2) \iff (\mathcal{QC}_1 \dot{=}_{(\text{H}, \text{f})} \mathcal{QC}_2)$.

Proof. We prove the two implications.

For all $i \in \{0, 1\}$, let $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, U_i)$ be two unitary circuit tuples, such that:

$$\llbracket U_i \rrbracket_{\text{H}} \left(\left\{ p_{\text{c}}, |\xi_{\text{c}}^i\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i}, \text{c} \right\}_{\text{c} \in \{0,1\}^m} \right) = \left\{ p_{\text{c}_i}, |f_{\text{c}_i}(\xi_{\text{c}}^i)\rangle_{\text{Obs}_i, \text{Di}_i}, \text{c}_i \right\}_{\text{c}_i \in \{0,1\}^{m_i}}.$$

By unitarity of U_i : $\forall i \in \{1, 2\}$, $m = m_i$.

1. Assume that $(\mathcal{QC}_1 \dot{=}_{\text{f}} \mathcal{QC}_2)$:

(a) By Definition 4.3.18:

$$\forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \forall \text{c} \in \{0, 1\}^m, d(\psi_{\text{c}}^1(\xi), \text{Obs}_1) = d(\psi_{\text{c}}^2(\xi), \text{Obs}_2).$$

(b) Then, by Definition 4.3.46:

$$\forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \forall \text{c} \in \{0, 1\}^m, d(p_{\text{c}}, \psi_{\text{c}}^1(\xi), \text{Obs}) = d(p_{\text{c}}, \psi_{\text{c}}^2(\xi), \text{Obs})$$

(c) Thus, by Definition 4.3.47: $(\mathcal{QC}_1 \dot{=}_{(\text{H}, \text{f})} \mathcal{QC}_1)$.

2. Assume that $(\mathcal{QC}_1 \dot{=}_{(\text{H}, \text{f})} \mathcal{QC}_1)$:

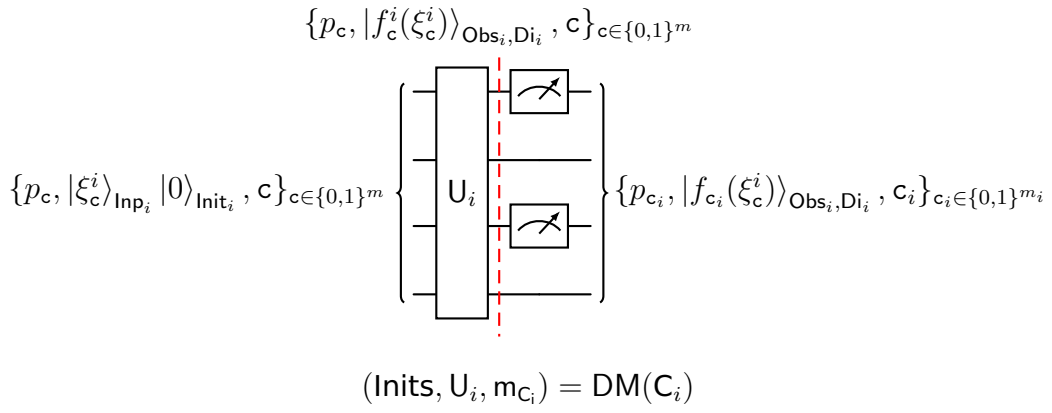
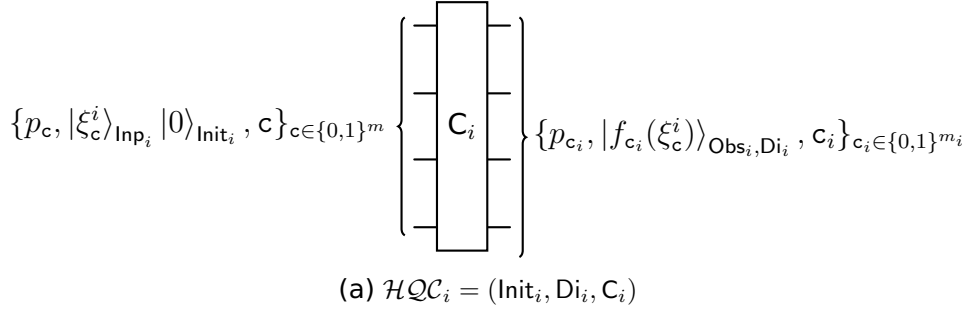
(a) By Definition 4.3.47:

$$\forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \forall \text{c} \in \{0, 1\}^m, d(p_{\text{c}}, \psi_{\text{c}}^1(\xi), \text{Obs}_1) = d(p_{\text{c}}, \psi_{\text{c}}^2(\xi), \text{Obs}_2)$$

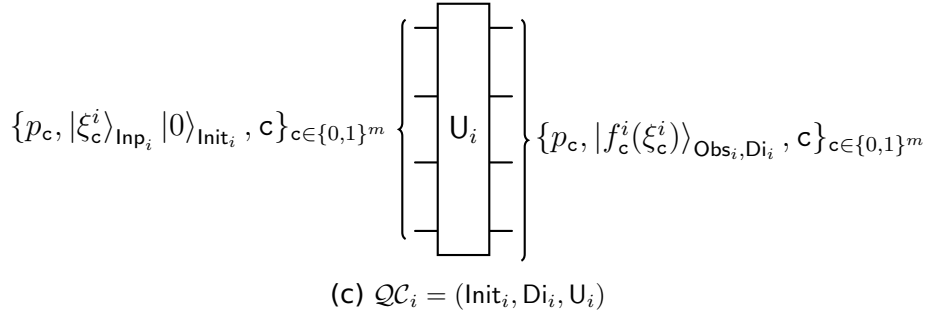
(b) Then, by Definition 4.3.6:

$$\forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \forall \text{c} \in \{0, 1\}^m, d(\psi_{\text{c}}^1(\xi), \text{Obs}_1) = d(\psi_{\text{c}}^2(\xi), \text{Obs}_2).$$

(c) Therefore, by Definition 4.3.18: $(\mathcal{QC}_1 \dot{=}_{\text{f}} \mathcal{QC}_1)$. $\square$



(b) $(\text{Init}_i, \text{Di}_i, \text{m}(\text{C}_i, \text{m}_{\text{C}_i}))$



$$\begin{aligned}
 (\text{m}_{\text{C}_1} \dot{=}_{\text{Obs}} \text{m}_{\text{C}_2} \wedge \mathcal{QC}_1 \dot{=} \mathcal{QC}_2) &\implies (\mathcal{HQC}_1 \dot{=}_{\text{H}} \mathcal{HQC}_2) && \text{(Lemma 4.3.78)} \\
 (\text{m}_{\text{C}_1} \dot{=}_{\text{Obs}} \text{m}_{\text{C}_2} \wedge \mathcal{QC}_1 \dot{=}_{\text{gp}} \mathcal{QC}_2) &\implies (\mathcal{HQC}_1 \dot{=}_{(\text{H}, \text{gp})} \mathcal{HQC}_2) && \text{(Lemma 4.3.79)} \\
 (\mathcal{QC}_1 \dot{=}_{\text{f}} \mathcal{QC}_2) &\iff (\mathcal{HQC}_1 \dot{=}_{(\text{H}, \text{f})} \mathcal{HQC}_2) && \text{(Lemma 4.3.80)}
 \end{aligned}$$

Figure 4.17: Reduction from Unit-Part to Hyb-Part (Hyb-Part-DM)

4.3.5.4 Reduction from Hyb-Part to Unit-Part (Hyb-Part-DM)

This section presents the goal of this chapter: our method for verifying **hybrid partial equivalence** of **hybrid-circuit** by **unitary partial equivalence** under deferred measurement. Figure 4.17 illustrates this method:

1. Assume, we want to check the hybrid partial equivalence between two hybrid circuits C_1 and C_2 (Figure 4.17a).
2. We defer the measurement (Definition 4.3.69), as illustrated in Figure 4.17b.
3. Finally, we check the equivalence of the unitary part, Figure 4.17c.

Definition 4.3.77 (Map Partial Equivalence). For $i \in \{0, 1\}$, let m_{C_i} be two maps as defined in Definition 4.3.59, and let S be a set of qubit indices. The maps m_{C_1} and m_{C_2} are *partially equal* with respect to S , denoted by $m_{C_1} \dot{=}^S m_{C_2}$ if and only if for all classical bit indices c :

$$(m_{C_1}([c]) \in S \implies m_{C_1}([c]) = m_{C_2}([c])) \quad \wedge \quad (m_{C_2}([c]) \in S \implies m_{C_1}([c]) = m_{C_2}([c])).$$

Informally, Two functions m_{C_1} and m_{C_2} are partially equal with respect to a set of qubits S if they coincide on all classical bit sets whose image belongs to S for either function.

Lemme 4.3.78 (Reduction from SubCir-Hyb-Part-Par to SubCir-Unit-Part-Par). For $i \in \{0, 1\}$, let $\mathcal{HQC}_i = (\text{Init}_i, \text{Di}_i, C_i)$ be two hybrid-circuit tuples (Notation 4.3.8). Assume that $(\text{Init}_i, U_i, m_{C_i}) = \text{DM}(C_i)$ (Definition 4.3.69), then $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, U_i)$. If $(m_{C_1} \dot{=}^{\text{Obs}} m_{C_2})$ and $\mathcal{QC}_1$ is SubCir-Unit-Part-Par to $\mathcal{QC}_2$ (Definition 4.3.12), then $\mathcal{HQC}_1$ is SubCir-Hyb-Part-Par to $\mathcal{HQC}_2$ (Definition 4.3.43), denoted by

$$(m_{C_1} \dot{=}^{\text{Obs}} m_{C_2} \wedge \mathcal{QC}_1 \dot{=} \mathcal{QC}_2) \implies (\mathcal{HQC}_1 \dot{=}^{\text{H}} \mathcal{HQC}_2)$$

Proof. We prove the implication.

1. Assume that $(m_{C_1} \dot{=}^{\text{Obs}} m_{C_2} \wedge \mathcal{QC}_1 \dot{=} \mathcal{QC}_2)$ (Definition 4.3.12, and Definition 4.3.77).
2. Then, by Lemma 4.3.74: $(\mathcal{QC}_1 \dot{=}^{\text{H}} \mathcal{QC}_2)$.
3. Assume:

$$\llbracket C_i \rrbracket_{\text{H}} \left(\left\{ p_c, |\xi_c^i\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i}, c \right\}_{c \in \{0,1\}^m} \right) = \left\{ p_{c_i}, |f_{c_i}(\xi_c^i)\rangle_{\text{Obs}_i, \text{Di}_i}, c_i \right\}_{c_i \in \{0,1\}^{m_i}}.$$

Then:

$$\left. \begin{aligned} &\forall i \in \{1, 2\}, \forall c_i \in \{0, 1\}^{m_i}, \exists |\phi_{c_i}\rangle \in \mathcal{H}^{\text{Di}_i}, \forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \exists |\psi_{c_i}(\xi)\rangle \in \mathcal{H}^{\text{Obs}_i}, \\ &|f_{c_i}(\xi)\rangle_{\text{Obs}_i, \text{Di}_i} = |\psi_{c_i}(\xi)\rangle_{\text{Obs}_i} \otimes |\phi_{c_i}\rangle_{\text{Di}_i}, \\ &\text{Obs}_1 \simeq \text{Obs}_2 \\ &\forall c_1 \in \{0, 1\}^{m_1}, \exists c_2 \in \{0, 1\}^{m_2}, \\ &\left. \begin{aligned} &c_1|_{\text{Obs}_1} \cong_{(\text{Obs}_1, \text{Obs}_2)} c_2|_{\text{Obs}_2}, \\ &|\psi_{c_1}(\xi)\rangle_{\text{Obs}_1} \sim_{(\text{Obs}_1, \text{Obs}_2)} |\psi_{c_2}(\xi)\rangle_{\text{Obs}_2}, \\ &\sum_{c_1|_{\text{Di}_1}} p_{c_1|_{\text{Di}_1}} c_1|_{\text{Obs}_1} = \sum_{c_2|_{\text{Di}_2}} p_{c_2|_{\text{Di}_2}} c_2|_{\text{Obs}_2} \end{aligned} \right\} \end{aligned}$$

4. By $(m_{C_1} \dot{=}_{\text{Obs}} m_{C_2})$, there exists a bijection between the observed part of the map (the part whose codomain is mapped to qubits in Obs), formally:

$$\forall c \in \text{domain}(m_{C_i}), \quad m_{C_i}([c]) \in \text{Obs}_i \implies m_{C_1}([c]) = m_{C_2}([c]),$$

where $\text{domain}(m_{C_i})$ is the domain of m_{C_i} .

5. Then, by Definition 2.2.53:

$$\begin{aligned} &\forall i \in \{1, 2\}, \forall \text{cb}_i \mid m_{C_i}(\text{cb}_i) = \text{qb}_i \\ &\llbracket \text{Meas}(\text{qb}_i, \text{cb}_i) \rrbracket_{\text{H}} \left(\{p_c, |\psi_c^i(\xi_c^i)\rangle_{\text{Obs}_i} \otimes |\phi_c^i(\xi_c^i)\rangle_{\text{Di}_i}, c\}_{c \in \{0,1\}^m} \right) \\ &= \left\{ p_{c_i}, |\psi_{c_i}^i(\xi_c^i)\rangle_{\text{Obs}_i} \otimes |\phi_{c_i}^i(\xi_c^i)\rangle_{\text{Di}_i}, c_i \right\}_{c_i \in \{0,1\}^{m_i}}, \end{aligned}$$

6. Thus, since the observed parts before measurement are equal up to a bit and qubit mapping, and since measurement does not introduce entanglement, the observed parts after measurement remain equal up to the same mapping. Furthermore, the output and discard parts, being separable before measurement, remain separable afterwards. So Property 4.30, and 4.31 hold.

7. By Definition 4.3.43: $(\text{Init}_1, \text{Di}_1, \text{mA}(U_1, m_{C_1})) \dot{=}_{\text{H}} (\text{Init}_2, \text{Di}_2, \text{mA}(U_2, m_{C_2}))$.

8. Next, by Theorem 4.3.71, we have:

$$\begin{aligned} &(\text{Init}_1, \text{Di}_1, \text{mA}(U_1, m_{C_1})) \dot{=}_{\text{H}} (\text{Init}_1, \text{Di}_1, C_1) \\ &(\text{Init}_2, \text{Di}_2, \text{mA}(U_2, m_{C_2})) \dot{=}_{\text{H}} (\text{Init}_2, \text{Di}_2, C_2). \end{aligned}$$

9. By transitivity: $(\text{Init}_1, \text{Di}_1, C_1) \dot{=}_{\text{H}} (\text{Init}_2, \text{Di}_2, C_2)$, therefore $(\mathcal{HQC}_1) \dot{=}_{\text{H}} (\mathcal{HQC}_2)$. $\square$

Lemme 4.3.79 (Reduction from GP-Hyb-Part-Par to GP-Unit-Part-Par). For $i \in \{0, 1\}$, let $\mathcal{HQC}_i = (\text{Init}_i, \text{Di}_i, C_i)$ be two hybrid-circuit tuples (Notation 4.3.8). Assume that $(\text{Inits}_i, U_i, m_{C_i}) = \text{DM}(C_i)$ (Definition 4.3.69), then $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, U_i)$. If $\mathcal{QC}_1$ is GP-Unit-Part-Par to $\mathcal{QC}_2$, denoted as $\mathcal{QC}_1 \dot{=}_{\text{gp}} \mathcal{QC}_2$ (Definition 4.3.44), then $\mathcal{HQC}_1$ is GP-Hyb-Part-Par to $\mathcal{HQC}_2$, denoted as $\mathcal{HQC}_1 \dot{=}_{(\text{H}, \text{gp})} \mathcal{HQC}_2$ (Definition 4.3.44). Formally:

$$(\mathcal{QC}_1 \dot{=}_{\text{gp}} \mathcal{QC}_2) \implies (\mathcal{HQC}_1 \dot{=}_{(\text{H}, \text{gp})} \mathcal{HQC}_2)$$

Proof. The proof follows the same structure as that of Lemma 4.3.78, with the vector equality ($=$) (Definition 2.2.10) replaced by the global-phase equivalence ($\approx_{\text{gp}}$) (Definition 2.2.12), and Theorem 4.3.71 replaced by Corollary 4.3.72. $\square$

Lemme 4.3.80 (Reduction from FullCir-Hyb-Part-Par to FullCir-Unit-Part-Par). For $i \in \{0, 1\}$, let $\mathcal{HQC}_i = (\text{Init}_i, \text{Di}_i, \text{C}_i)$ be two hybrid-circuit tuples (Notation 4.3.8). Assume that $(\text{Inits}_i, \text{U}_i, \text{m}_{\text{C}_i}) = \text{DM}(\text{C}_i)$ (Definition 4.3.69), then $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, \text{U}_i)$. $\mathcal{QC}_1$ is FullCir-Unit-Part-Par to $\mathcal{QC}_2$ (Definition 4.3.47), if and only if $\mathcal{HQC}_1$ is FullCir-Hyb-Part-Par to $\mathcal{HQC}_2$ (Definition 4.3.47), denoted by

$$(\mathcal{QC}_1 \dot{=}_{\text{f}} \mathcal{QC}_2) \iff (\mathcal{HQC}_1 \dot{=}_{(\text{H}, \text{f})} \mathcal{HQC}_2)$$

Proof. We prove the two implications. Assume:

- Assume that

$$\llbracket \text{U}_i \rrbracket_{\text{H}}(\{p_{\text{c}}, |\xi_{\text{c}}^i\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i}, \text{c}\}_{\text{c} \in \{0,1\}^m}) = \{p_{\text{c}_i}, |f_{\text{c}_i}^i(\xi_{\text{c}}^i)\rangle_{\text{Obs}_i, \text{Di}_i}, \text{c}_i\}_{\text{c}_i \in \{0,1\}^{m_i}}.$$

- By unitarity of U_i : $\forall i \in \{1, 2\}$, $m = m_i$.
- Then, we have

$$\llbracket \text{U}_i \rrbracket_{\text{H}}(\{p_{\text{c}}, |\xi_{\text{c}}^i\rangle_{\text{Inp}_i} |0\rangle_{\text{Init}_i}, \text{c}\}_{\text{c} \in \{0,1\}^m}) = \{p_{\text{c}}, |f_{\text{c}}^i(\xi_{\text{c}}^i)\rangle_{\text{Obs}_i, \text{Di}_i}, \text{c}\}_{\text{c} \in \{0,1\}^m}.$$

- Then, by Definition 2.2.53:

$$\begin{aligned} \forall i \in \{1, 2\}, \forall \text{cb}_i \mid \text{m}_{\text{C}_i}(\text{cb}_i) = \text{qb}_i \\ \llbracket \text{Meas}(\text{qb}_i, \text{cb}_i) \rrbracket_{\text{H}} \left(\{p_{\text{c}}, |f_{\text{c}}^i(\xi_{\text{c}}^i)\rangle_{\text{Obs}_i, \text{Di}_i}, \text{c}\}_{\text{c} \in \{0,1\}^m} \right) \\ = \left\{ p_{\text{c}_i}, |f_{\text{c}_i}^i(\xi_{\text{c}}^i)\rangle_{\text{Obs}_i, \text{Di}_i}, \text{c}_i \right\}_{\text{c}_i \in \{0,1\}^{m_i}}. \end{aligned}$$

1. We prove that $(\mathcal{QC}_1 \dot{=}_{\text{f}} \mathcal{QC}_2) \implies (\mathcal{HQC}_1 \dot{=}_{(\text{H}, \text{f})} \mathcal{HQC}_2)$:

- Assume that: $(\text{Init}_1, \text{Di}_1, \text{U}_1) \dot{=}_{\text{f}} (\text{Init}_2, \text{Di}_2, \text{U}_2)$
- By Lemma 4.3.76: $(\text{Init}_1, \text{Di}_1, \text{U}_1) \dot{=}_{(\text{H}, \text{f})} (\text{Init}_2, \text{Di}_2, \text{U}_2)$.
- Thus, by Definition 4.3.47 the following properties are satisfied before measurement:

$$\begin{aligned} \forall |\xi\rangle \in \mathcal{H}^{\text{Inp}}, \forall \text{c}_1 \in \{0,1\}^{m_1}, \exists \text{c}_2 \in \{0,1\}^{m_2}, \\ \text{d}(p_{\text{c}_1}, \psi_{\text{c}_1}^1(\xi), \text{Obs}_1) = \text{d}(p_{\text{c}_2}, \psi_{\text{c}_2}^2(\xi), \text{Obs}_2), \end{aligned}$$

- Thus, the Property 4.36 holds.
- Then, by Definition 4.3.47: $(\text{Init}_1, \text{Di}_1, \text{mA}(\text{U}_1, \text{C}_1)) \dot{=}_{(\text{H}, \text{f})} (\text{Init}_2, \text{Di}_2, \text{mA}(\text{U}_2, \text{C}_2))$.

(f) Next, by Corollary 4.3.73, we have:

$$\begin{aligned} (\text{Init}_1, \text{Di}_1, \text{mA}(U_1, C_1)) &\dot{\equiv}_{(\mathcal{H}, \mathcal{F})} (\text{Init}_1, \text{Di}_1, C_2) \\ (\text{Init}_2, \text{Di}_1, \text{mA}(U_2, C_2)) &\dot{\equiv}_{(\mathcal{H}, \mathcal{F})} (\text{Init}_2, \text{Di}_2, C_2) \end{aligned}$$

(g) By transitivity: $(\text{Init}_1, \text{Di}_1, C_1) \dot{\equiv}_{(\mathcal{H}, \mathcal{F})} (\text{Init}_2, \text{Di}_2, C_2)$.

2. We prove that $(\mathcal{QC}_1 \dot{\equiv}_{\mathcal{F}} \mathcal{QC}_2)$ is implied by $(\mathcal{HQC}_1 \dot{\equiv}_{(\mathcal{H}, \mathcal{F})} \mathcal{HQC}_2)$ by reversing the steps of the previous proof, without detailing them again.

(a) Assume $(\text{Init}_1, \text{Di}_1, C_1) \dot{\equiv}_{(\mathcal{H}, \mathcal{F})} (\text{Init}_2, \text{Di}_2, C_2)$ (Definition 4.3.47).

(b) Then, by Corollary 4.3.73, we have

$$\begin{aligned} (\text{Init}_1, \text{Di}_1, \text{mA}(U_1, m_{C_1})) &\dot{\equiv}_{(\mathcal{H}, \mathcal{F})} (\text{Init}_1, \text{Di}_1, C_2) \\ (\text{Init}_2, \text{Di}_1, \text{mA}(U_2, m_{C_2})) &\dot{\equiv}_{(\mathcal{H}, \mathcal{F})} (\text{Init}_2, \text{Di}_2, C_2) \end{aligned}$$

(c) By transitivity, we obtain

$$(\text{Init}_1, \text{Di}_1, \text{mA}(U_1, m_{C_1})) \dot{\equiv}_{(\mathcal{H}, \mathcal{F})} (\text{Init}_2, \text{Di}_1, \text{mA}(U_2, m_{C_2})).$$

(d) Consequently, by Definition 4.3.47: $(\text{Init}_1, \text{Di}_1, U_1) \dot{\equiv}_{(\mathcal{H}, \mathcal{F})} (\text{Init}_2, \text{Di}_2, U_2)$.

(e) Therefore, by Lemma 4.3.76: $(\text{Init}_1, \text{Di}_1, U_1) \dot{\equiv}_{\mathcal{F}} (\text{Init}_2, \text{Di}_2, U_2)$. $\square$

4.4 Relating Complete and Partial Equivalences

This section bridges the concepts of complete equivalence and partial equivalence. We detail the relationships between complete equivalence (Section 4.2) and partial equivalence (Section 4.3), starting with unitary relations (Section 4.4.1) and concluding with hybrid relations (Section 4.4.2). Figure 4.18 illustrates that complete relations can imply all partial equivalence relations.

4.4.1 Unitary-Equivalences

This section gives the relationships between Complete and Partial equivalences. In Lemma 4.4.2 we show the implication of sub-circuit unitary partial equivalence by sub-circuit unitary complete equivalence. In Lemma 4.4.3 we show the implication of global phase unitary partial equivalence by global-phase unitary complete equivalence. Finally, in Lemma 4.4.4 we show the implication of full-circuit unitary partial equivalence by full-circuit unitary complete equivalence.

Remark 4.4.1. The equivalence relations are stated only for their parallel forms. The sequential variants are not introduced separately, as they have already been shown to coincide with the parallel ones. Hence, defining the relations in parallel form suffices.

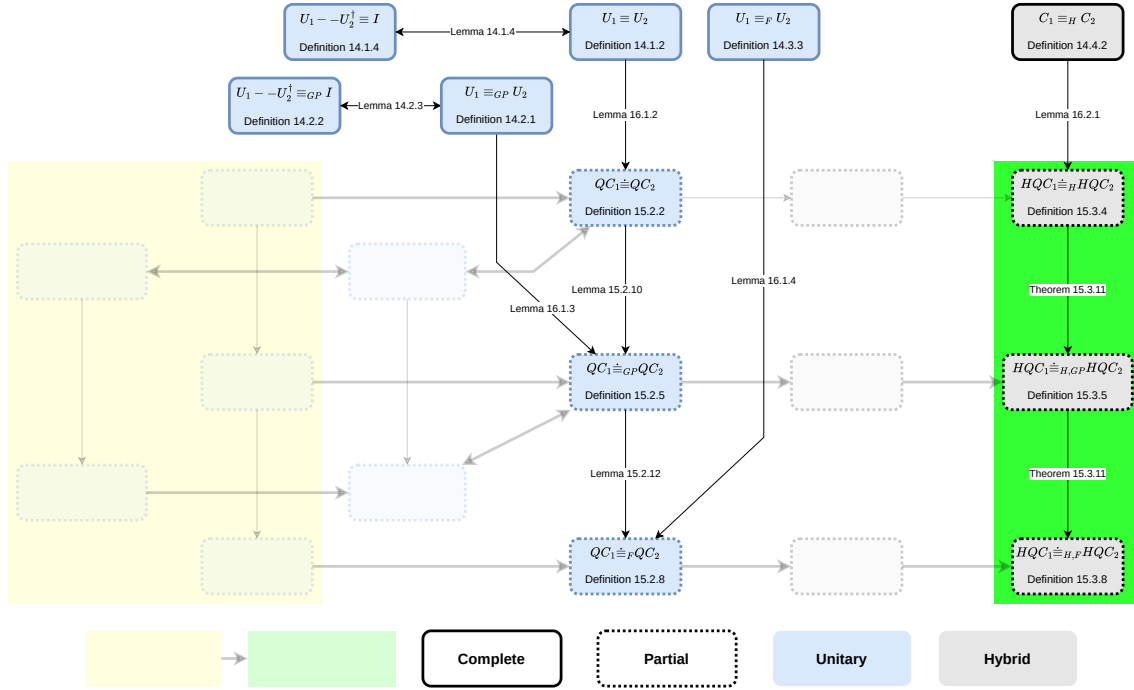


Figure 4.18: Relating Complete and Partial Equivalences

Lemma 4.4.2 (SubCir-Unit-Comp-Par Implies SubCir-Unit-Part-Par). Let $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, U_i)$ for $i \in \{1, 2\}$ be two tuples of unitary circuits (Notation 4.3.8). Assume that $\text{Di}_i = \emptyset$ and that the observed part coincides with the whole register on which U_i acts. If $U_1 \equiv U_2$ (Definition 4.2.2), then $\mathcal{QC}_1 \equiv \mathcal{QC}_2$ (Definition 4.3.12).

Proof. Let Q_i be the complete quantum register on which U_i acts, for $i \in \{1, 2\}$. In the complete setting, the observed part is $\text{Obs}_i = Q_i$ and $\text{Di}_i = \emptyset$. By Definition 4.3.12, SubCir-Unit-Part-Par compares the induced semantics on Obs_i . When $\text{Obs}_i = Q_i$, this reduces exactly to SubCir-Unit-Comp-Par (Definition 4.2.2). Hence, $U_1 \equiv U_2$ implies $\mathcal{QC}_1 \equiv \mathcal{QC}_2$. $\square$

Lemma 4.4.3 (GP-Unit-Comp-Par Implies GP-Unit-Part-Par). Let $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, U_i)$ for $i \in \{1, 2\}$ be two tuples of unitary circuits (Notation 4.3.8). Assume that $\text{Di}_i = \emptyset$ and that the observed part coincides with the whole register on which U_i acts. If $U_1 \equiv_{GP} U_2$ (Definition 4.2.6), then $\mathcal{QC}_1 \equiv_{GP} \mathcal{QC}_2$ (Definition 4.3.15).

Proof. Let Q_i be the complete quantum register on which U_i acts, for $i \in \{1, 2\}$. In the complete setting, the observed part is $\text{Obs}_i = Q_i$ and $\text{Di}_i = \emptyset$. By Definition 4.3.15, GP-Unit-Part-Par compares the induced semantics on Obs_i up to global phase. When $\text{Obs}_i = Q_i$, this reduces exactly to GP-Unit-Comp-Par (Definition 4.2.6). Hence, $U_1 \equiv_{GP} U_2$ implies $\mathcal{QC}_1 \equiv_{GP} \mathcal{QC}_2$. $\square$

Lemma 4.4.4 (FullCir-Unit-Comp-Par Implies FullCir-Unit-Part-Par). Let $\mathcal{QC}_i = (\text{Init}_i, \text{Di}_i, U_i)$ for $i \in \{1, 2\}$ be two tuples of unitary circuits (Notation 4.3.8). Assume that $\text{Di}_i = \emptyset$

and that the observed part coincides with the whole register on which U_i acts. If $U_1 \equiv_f U_2$ (Definition 4.2.12), then $\mathcal{QC}_1 \dot{\equiv}_f \mathcal{QC}_2$ (Definition 4.3.18).

Proof. Let Q_i be the complete quantum register on which U_i acts, for $i \in \{1, 2\}$. In the complete setting, the observed part is $\text{Obs}_i = Q_i$ and $\text{Di}_i = \emptyset$. By Definition 4.3.18, FullCir-Unit-Part-Par compares the induced semantics on Obs_i . When $\text{Obs}_i = Q_i$, this reduces exactly to FullCir-Unit-Comp-Par (Definition 4.2.12). Hence, $U_1 \equiv_f U_2$ implies $\mathcal{QC}_1 \dot{\equiv}_f \mathcal{QC}_2$. $\square$

4.4.2 Hybrid-Equivalence

This section gives Lemma 4.4.5, which ensures that if two hybrid circuits are completely equivalent, then they are also partially equivalent⁴.

Lemme 4.4.5 (SubCir-Hyb-Comp-Par Implies SubCir-Hyb-Part-Par). Let $\mathcal{HQC}_i = (\text{Init}_i, \text{Di}_i, C_i)$ for $i \in \{1, 2\}$ be two tuples of hybrid circuits (Notation 4.3.8). Assume that $\text{Di}_i = \emptyset$ and that the observed part coincides with the whole register on which C_i acts. If $C_1 \equiv_H C_2$ (Definition 4.2.18), then $\mathcal{HQC}_1 \dot{\equiv}_H \mathcal{HQC}_2$ (Definition 4.3.43).

Proof. Let Q_i be the complete quantum register on which C_i acts, for $i \in \{1, 2\}$. In the complete setting, the observed part is $\text{Obs}_i = Q_i$ and $\text{Di}_i = \emptyset$. By Definition 4.3.43, SubCir-Hyb-Part-Par compares the induced semantics on Obs_i . When $\text{Obs}_i = Q_i$, this reduces exactly to SubCir-Hyb-Comp-Par (Definition 4.2.18). Hence, $C_1 \equiv_H C_2$ implies $\mathcal{HQC}_1 \dot{\equiv}_H \mathcal{HQC}_2$. $\square$

⁴Recall that the only complete hybrid relation found in the state of the art is within the context of sub-circuit, as discussed in Section 4.2.2.

Chapter 5

Implementation

Our objective is to develop a practical method for establishing the correctness of circuit transformations. In Chapter 3, we presented a proof of correctness for a transformation. However, even with a semi-automatic tool, the proof remains lengthy and complex.

An alternative approach consists in reducing the correctness of a transformation to instances of circuit equivalence between the original and the transformed circuits. To this end, we reduce the problem to instances of partial path-sum equivalence, introduced in Section 4.3.3.1. We prove that this equivalence is sound with respect to standard unitary partial equivalences (Section 4.3.2). Additionally, we define and prove the correctness of a method extending unitary equivalence to the hybrid case in Section 4.3.5.4.

In this chapter, we focus on the practical implementation. To evaluate our method on non-trivial circuit transformations (i.e., those that introduce discard), we proceed as follows:

1. We implement such transformations. In particular, we implement the One-Way Measurement transformation (O), detailed in Section 2.5.3, and Teleportation Insertion (T), described in Section 2.5.4. We provide proofs of correctness for these implementations.
2. We implement our partial path-sum equivalence relations¹ together with our method for extending unitary equivalence to the hybrid case.
3. We define an evaluation protocol.

This chapter is structured as follows. Section 5.1 presents the proofs of correctness for the implemented transformations. Section 5.2 describes our tool for checking hybrid circuit partial equivalence. Section 5.3 outlines the evaluation protocol, and Section 5.4 reports the evaluation results. Finally, Section 5.5 discusses our approach.

¹We exclude the full-circuit equivalence due to scalability issues (Remark 4.3.38).

5.1 Circuit Transformations Correctness

This section presents the correctness proofs for our One-Way Measurement implementation (O, Section 2.5.3) and our Teleportation Insertion implementation (T, Section 2.5.4).

These proofs serve as independent theoretical case studies for the equivalence relations introduced in Chapter 4. They are not required for the execution of our evaluation protocol. Rather, they provide a formal reference against which the automated equivalence checks can be interpreted. The transformations themselves are applied within our evaluation protocol (Section 5.3) to assess the ability of our framework to verify hybrid circuit equivalence using tools that support only unitary operations. The proof for O is detailed in Section 5.1.1, and the proof for T is provided in Section 5.1.2.

5.1.1 One-Way Measurement (O)

Our implementation of OWM preserves the semantics up to global phase (Def. 2.2.12), whereas its deferred-measurement version relies on vector equality (Def. 2.2.10).

In this section, we establish two results. First, we prove that OWM produces a circuit whose semantics is equal up to global phase to that of the original unitary circuit, thereby establishing global-phase hybrid partial parallel equivalence (Def. 4.3.44). Second, we analyse the version obtained after applying deferred measurement. This transformation eliminates intermediate measurements and yields a circuit that is SubCir-Unit-Part equivalent to the original one. Hence, correctness of the deferred-measurement construction is characterised under sub-circuit unitary partial equivalence (Sec. 4.3.2.1). Therefore, in our evaluation framework, it suffices to verify SubCir-Unit-Part equivalence. We first prove global-phase preservation for OWM (Thm. 5.1.1), and then establish the corresponding result for its deferred-measurement version (Thm 5.1.2).

Theorem 5.1.1 (Correctness Of Our Implementation of OWM, O). Let U be a unitary circuit. Assume that $(\text{Inp}, \text{Obs}, C) = O(U)$, let Q be the set of qubit indices of C , and define $\text{Init} := Q \setminus \text{Inp}$, and $\text{Di} := Q \setminus \text{Obs}$, then

$$(\emptyset, \emptyset, U) \dot{\equiv}_{(\text{H}, \text{gp})} (\text{Init}, \text{Di}, C). \quad (\text{Definition 4.3.44.})$$

Proof. Let $(\emptyset, \emptyset, U)$ be a unitary circuit tuple (Definition 4.3.7). Assume that $(\text{Inp}, \text{Obs}, C) = O(U)$ where Q is the quantum register of C such that $\text{Init} := Q \setminus \text{Inp}$, and $\text{Di} := Q \setminus \text{Obs}$. We prove by structural induction that: $(\emptyset, \emptyset, U) \dot{\equiv}_{(\text{H}, \text{gp})} (\text{Init}, \text{Di}, C)$.

1. If $U = \text{Apply}(X, \emptyset, \{0\})$, (Figure 2.11a):

$$\llbracket O(\text{Apply}(X, \emptyset, \{0\})) \rrbracket_{\text{H}}(\{1, |x_0\rangle_{\text{Inp}}, *\}) = \{1, |1 \oplus x_0\rangle_{\text{Obs}}, *\}.$$

Therefore, by Definition 4.3.44:

$$(\emptyset, \emptyset, \text{Apply}(X, \emptyset, \{0\})) \dot{\equiv}_{(\text{H}, \text{gp})} (\text{Init}, \text{Di}, O(\text{Apply}(X, \emptyset, \{0\}))).$$

2. If $U = \text{Apply}(H, \emptyset, \{0\})$, (Figure 2.11b):

Input state: $\rho = \{1, |x_0\rangle_{\text{Inp}} |0\rangle_{\text{Init}}, *\}$.

Preparation (P):

$$\llbracket \text{Apply}(H, \emptyset, \{1\}) \rrbracket_{\mathbb{H}}(\rho) = \left\{ 1, \frac{1}{\sqrt{2}} \sum_{y_0 \in \{0,1\}} |x_0\rangle_{\text{Inp}} |y_0\rangle_{\text{Init}}, * \right\} = \rho_1.$$

Entanglement (E):

$$\llbracket \text{Apply}(CZ, \{0\}, \{1\}) \rrbracket_{\mathbb{H}}(\rho_1) = \left\{ 1, \frac{1}{\sqrt{2}} \sum_{y_0 \in \{0,1\}} (-1)^{x_0 y_0} |x_0\rangle_{\text{Inp}} |y_0\rangle_{\text{Init}}, * \right\} = \rho_2.$$

Measurement (M):

$$\begin{aligned} \llbracket \text{Apply}(H, \emptyset, \{0\}) \rrbracket_{\mathbb{H}}(\rho_2) &= \left\{ 1, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} (-1)^{x_0 y_0 + x_0 y_1} |y_1\rangle_{\text{Inp}} |y_0\rangle_{\text{Init}}, * \right\} = \rho_3 \\ \llbracket \text{Meas}(\{0\}, \{0\}) \rrbracket_{\mathbb{H}}(\rho_3) &= \left\{ \left(\frac{1}{2}, \frac{1}{\sqrt{2}} \sum_{y_0 \in \{0,1\}} (-1)^{x_0 y_0} |0\rangle_{\text{Inp}} |y_0\rangle_{\text{Init}}, 0 \right), \right. \\ &\quad \left. \left(\frac{1}{2}, \frac{1}{\sqrt{2}} \sum_{y_0 \in \{0,1\}} (-1)^{x_0 y_0 + x_0} |1\rangle_{\text{Inp}} |y_0\rangle_{\text{Init}}, 1 \right) \right\} = \rho_4. \end{aligned}$$

Correction (C):

$$\begin{aligned} &\llbracket \text{if}(\{0\}, \text{Apply}(X, \emptyset, \{1\})) \rrbracket_{\mathbb{H}}(\rho_4) \\ &= \left\{ \left(\frac{1}{2}, \frac{1}{\sqrt{2}} \sum_{y_0 \in \{0,1\}} (-1)^{x_0 y_0} |0\rangle_{\text{Inp}} |y_0\rangle_{\text{Init}}, 0 \right), \right. \\ &\quad \left. \left(\frac{1}{2}, \frac{1}{\sqrt{2}} \sum_{y_0 \in \{0,1\}} (-1)^{x_0(1+y_0)} |1\rangle_{\text{Inp}} |1 \oplus y_0\rangle_{\text{Init}}, 1 \right) \right\} = \rho_5. \end{aligned}$$

Replacement of y_0 by $(1 \oplus y_0)$:

$$\begin{aligned} \rho_5 &= \left\{ \left(\frac{1}{2}, \frac{1}{\sqrt{2}} \sum_{y_0 \in \{0,1\}} (-1)^{x_0 y_0} |0\rangle_{\text{Inp}} |y_0\rangle_{\text{Init}}, 0 \right), \right. \\ &\quad \left. \left(\frac{1}{2}, \frac{1}{\sqrt{2}} \sum_{y_0 \in \{0,1\}} (-1)^{x_0 y_0} |1\rangle_{\text{Inp}} |y_0\rangle_{\text{Init}}, 1 \right) \right\}. \end{aligned}$$

Full process:

$$\llbracket O(\text{Apply}(\mathbf{H}, \emptyset, \{0\})) \rrbracket_{\mathbf{H}}(\rho) = \left\{ \left(\frac{1}{2}, |0\rangle_{\text{Di}} \otimes \frac{1}{\sqrt{2}} \sum_{y_0 \in \{0,1\}} (-1)^{x_0 y_0} |y_0\rangle_{\text{Obs}}, 0 \right), \left(\frac{1}{2}, |1\rangle_{\text{Di}} \otimes \frac{1}{\sqrt{2}} \sum_{y_0 \in \{0,1\}} (-1)^{x_0 y_0} |y_0\rangle_{\text{Obs}}, 1 \right) \right\}.$$

Thus, the output state (Obs) is:

$$\left\{ \left(\frac{1}{2}, \sum_{y_0 \in \{0,1\}} (-1)^{x_0 y_0} |y_0\rangle_{\text{Obs}}, 0 \right), \left(\frac{1}{2}, \sum_{y_0 \in \{0,1\}} (-1)^{x_0 y_0} |y_0\rangle_{\text{Obs}}, 1 \right) \right\}.$$

Thus, by Def. 4.3.44: $(\emptyset, \emptyset, \text{Apply}(\mathbf{H}, \emptyset, \{0\})) \dot{=}_{(\mathbf{H}, \mathbf{gp})} (\text{Init}, \text{Di}, O(\text{Apply}(\mathbf{H}, \emptyset, \{0\})))$.

3. If $\mathbf{U} = \text{Apply}(\mathbf{U}_1(\theta), \emptyset, \{0\})$ (Figure 2.11d), input state: $\rho = \{1, |x_0\rangle_{\text{Inp}} |0, 0\rangle_{\text{Init}}, **\}$.

Preparation (P):

$$\begin{aligned} & \llbracket \text{Apply}(\mathbf{H}, \emptyset, [1, 2]) \rrbracket_{\mathbf{H}}(\rho) \\ &= \left\{ 1, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} |x_0\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}}, ** \right\} = \rho_1. \end{aligned}$$

Entanglement (E):

$$\begin{aligned} & \llbracket \text{Apply}(\mathbf{Z}, \{0\}, \{1\}) - \text{Apply}(\mathbf{Z}, \{1\}, \{2\}) \rrbracket_{\mathbf{H}}(\rho_1) \\ &= \left\{ 1, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1))} |x_0\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}}, ** \right\} = \rho_2. \end{aligned}$$

Measurement (M):

$$\begin{aligned} & \llbracket \text{Apply}(\mathbf{U}_1(\theta), \emptyset, \{0\}) \rrbracket_{\mathbf{H}}(\rho_2) \\ &= \left\{ 1, \frac{1}{2\sqrt{2}} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1) + \theta x_0)} |x_0\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}}, ** \right\} = \rho_3 \\ & \llbracket \text{Apply}(\mathbf{H}, \emptyset, \{0\}) \rrbracket_{\mathbf{H}}(\rho_3) \\ &= \left\{ 1, \frac{1}{2\sqrt{2}} \sum_{y_0, y_1, y_2 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + x_0 y_2) + \theta x_0)} |y_2\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}}, ** \right\} = \rho_4 \end{aligned}$$

$$\begin{aligned}
& \llbracket \text{Meas}(\{0\}, \{0\}) \rrbracket_{\mathbb{H}}(\rho_3) \\
&= \left\{ \left(\frac{1}{2}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1) + \theta x_0)} |0\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}}, 0, * \right), \right. \\
&\quad \left. \left(\frac{1}{2}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + x_0) + \theta x_0)} |1\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}}, 1, * \right) \right\}
\end{aligned}$$

$$\begin{aligned}
& \llbracket \text{Apply}(\mathbb{H}, \emptyset, \{1\}) \rrbracket_{\mathbb{H}}(\rho_4) \\
&= \left\{ \left(\frac{1}{2}, \frac{1}{2\sqrt{2}} \sum_{y_0, y_1, y_3 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + y_0 y_3) + \theta x_0)} |0\rangle_{\text{Inp}} |y_3, y_1\rangle_{\text{Init}}, 0, * \right), \right. \\
&\quad \left. \left(\frac{1}{2}, \frac{1}{2\sqrt{2}} \sum_{y_0, y_1, y_3 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + x_0 + y_0 y_3) + \theta x_0)} |1\rangle_{\text{Inp}} |y_3, y_1\rangle_{\text{Init}}, 1, * \right) \right\} = \rho_5
\end{aligned}$$

$$\begin{aligned}
& \llbracket \text{Meas}(\{1\}, \{1\}) \rrbracket_{\mathbb{H}}(\rho_5) \\
&= \left\{ \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1) + \theta x_0)} |0\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 0, 0 \right), \right. \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + y_0) + \theta x_0)} |0\rangle_{\text{Inp}} |1, y_1\rangle_{\text{Init}}, 0, 1 \right), \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + x_0) + \theta x_0)} |1\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 1, 0 \right) \\
&\quad \left. \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + x_0 + y_0) + \theta x_0)} |1\rangle_{\text{Inp}} |1, y_1\rangle_{\text{Init}}, 1, 1 \right) \right\} = \rho_6
\end{aligned}$$

Correction (C):

$$\begin{aligned}
& \llbracket \text{if}(\{0\}, \text{Apply}(\mathbf{Z}, \emptyset, \{2\})) \rrbracket_{\mathbf{H}}(\rho_6) \\
&= \left\{ \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1) + \theta x_0)} |0\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 0, 0 \right), \right. \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + y_0) + \theta x_0)} |0\rangle_{\text{Inp}} |1, y_1\rangle_{\text{Init}}, 0, 1 \right), \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + x_0 + y_1) + \theta x_0)} |1\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 1, 0 \right) \\
&\quad \left. \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + x_0 + y_0 + y_1) + \theta x_0)} |1\rangle_{\text{Inp}} |1, y_1\rangle_{\text{Init}}, 1, 1 \right) \right\} = \rho_7
\end{aligned}$$

$$\begin{aligned}
& \llbracket \text{if}(\{1\}, \text{Apply}(\mathbf{X}, \emptyset, \{2\})) \rrbracket_{\mathbf{H}}(\rho_7) \\
&= \left\{ \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1) + \theta x_0)} |0\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 0, 0 \right), \right. \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + y_0) + \theta x_0)} |0\rangle_{\text{Inp}} |1, 1 \oplus y_1\rangle_{\text{Init}}, 0, 1 \right), \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + x_0 + y_1) + \theta x_0)} |1\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 1, 0 \right) \\
&\quad \left. \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + x_0 + y_0 + y_1) + \theta x_0)} |1\rangle_{\text{Inp}} |1, 1 \oplus y_1\rangle_{\text{Init}}, 1, 1 \right) \right\} = \rho_8
\end{aligned}$$

$$\begin{aligned}
\rho_8 &= \left\{ \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(y_0(x_0 + y_1) + \theta x_0))} |0\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 0, 0 \right), \right. \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(y_0(y_1 + x_0 + 1) + \theta x_0))} |0\rangle_{\text{Inp}} |1, 1 \oplus y_1\rangle_{\text{Init}}, 0, 1 \right), \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(y_0(x_0 + y_1) + x_0 + y_1) + \theta x_0)} |1\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 1, 0 \right) \\
&\quad \left. \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(y_0(y_1 + x_0 + 1) + x_0 + y_1) + \theta x_0)} |1\rangle_{\text{Inp}} |1, 1 \oplus y_1\rangle_{\text{Init}}, 1, 1 \right) \right\}
\end{aligned}$$

Application of the reduction rules HH and $Elim$ (Definition 2.3.10):

$$\rho_8 = \left\{ \left(\frac{1}{4}, e^{i\theta x_0} |0\rangle_{\text{Inp}} |0, x_0\rangle_{\text{Init}}, 0, 0 \right), \left(\frac{1}{4}, e^{i\theta x_0} |0\rangle_{\text{Inp}} |1, x_0\rangle_{\text{Init}}, 0, 1 \right), \right. \\ \left. \left(\frac{1}{4}, e^{i\theta x_0} |1\rangle_{\text{Inp}} |0, x_0\rangle_{\text{Init}}, 1, 0 \right), \left(\frac{1}{4}, -e^{i\theta x_0} |1\rangle_{\text{Inp}} |1, x_0\rangle_{\text{Init}}, 1, 1 \right) \right\}$$

Then the full process gives:

$$\llbracket O(\text{Apply}(U_1(\theta), \emptyset, \{0\})) \rrbracket_{\text{H}}(\rho) \\ = \left\{ \left(\frac{1}{4}, e^{i\theta x_0} |0, 0\rangle_{\text{Di}} |x_0\rangle_{\text{Obs}}, 0, 0 \right), \left(\frac{1}{4}, e^{i\theta x_0} |0, 1\rangle_{\text{Di}} |x_0\rangle_{\text{Obs}}, 0, 1 \right), \right. \\ \left. \left(\frac{1}{4}, e^{i\theta x_0} |1, 0\rangle_{\text{Di}} |x_0\rangle_{\text{Obs}}, 1, 0 \right), \left(\frac{1}{4}, -e^{i\theta x_0} |1, 1\rangle_{\text{Di}} |x_0\rangle_{\text{Obs}}, 1, 1 \right) \right\}$$

Thus, the output state (Obs) is:

$$\left\{ \left(\frac{1}{4}, e^{i\theta x_0} |x_0\rangle_{\text{Obs}}, 0, 0 \right), \left(\frac{1}{4}, e^{i\theta x_0} |x_0\rangle_{\text{Obs}}, 0, 1 \right), \right. \\ \left. \left(\frac{1}{4}, e^{i\theta x_0} |x_0\rangle_{\text{Obs}}, 1, 0 \right), \left(\frac{1}{4}, -e^{i\theta x_0} |x_0\rangle_{\text{Obs}}, 1, 1 \right) \right\}$$

Therefore, by Definition 4.3.44:

$$(\emptyset, \emptyset, \text{Apply}(U_1(\theta), \emptyset, \{0\})) \doteq_{(\text{H}, \text{gp})} (\text{Init}, \text{Di}, O(\text{Apply}(U_1(\theta), \emptyset, \{0\}))).$$

4. If $U = \text{Apply}(CX, \emptyset, \{0\})$, (Figure 2.11c), input state: $\rho = \{1, |x_0, x_1\rangle_{\text{Inp}} |0, 0\rangle_{\text{Init}}, **\}$.

Preparation (P):

$$\llbracket \text{Apply}(H, \emptyset, \{1\}) - \text{Apply}(H, \emptyset, \{2\}) \rrbracket_{\text{H}}(\rho) \\ = \left\{ 1, \frac{1}{2} \sum_{y_0, y_1 \in \{0, 1\}} |x_0, x_1\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}}, ** \right\} = \rho_1.$$

Entanglement (E):

$$\llbracket \text{Apply}(Z, \{2\}, \{3\}) - \text{Apply}(Z, \{1\}, \{2\}) - \text{Apply}(Z, \{0\}, \{2\}) \rrbracket_{\text{H}}(\rho_1) \\ = \left\{ 1, \frac{1}{2} \sum_{y_0, y_1 \in \{0, 1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0)} |x_0, x_1\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}}, ** \right\} = \rho_2.$$

Measurement (M):

$$\llbracket \text{Apply}(H, \emptyset, \{1\}) \rrbracket_{\text{H}}(\rho_2) \\ = \left\{ 1, \frac{1}{2\sqrt{2}} \sum_{y_0, y_1, y_2 \in \{0, 1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + x_1 y_2)} |x_0, y_2\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}}, ** \right\} = \rho_3$$

$$\begin{aligned}
& \llbracket \text{Meas}(\{1\}, \{0\}) \rrbracket_{\mathbb{H}}(\rho_3) \\
&= \left\{ \left(\frac{1}{2}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0)} |x_0, 0\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}}, 0* \right), \right. \\
&\quad \left. \left(\frac{1}{2}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + x_1)} |x_0, 1\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}}, 1* \right) \right\} = \rho_4.
\end{aligned}$$

$$\begin{aligned}
& \llbracket \text{Apply}(\mathbb{H}, \emptyset, \{2\}) \rrbracket_{\mathbb{H}}(\rho_4) \\
&= \left\{ \left(\frac{1}{2}, \frac{1}{\sqrt{2}} \sum_{y_0, y_1, y_2 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + y_0 y_2)} |x_0, 0\rangle_{\text{Inp}} |y_2, y_1\rangle_{\text{Init}}, 0* \right), \right. \\
&\quad \left. \left(\frac{1}{2}, \frac{1}{\sqrt{2}} \sum_{y_0, y_1, y_2 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + x_1 + y_0 y_2)} |x_0, 1\rangle_{\text{Inp}} |y_2, y_1\rangle_{\text{Init}}, 1* \right) \right\} = \rho_5.
\end{aligned}$$

$$\begin{aligned}
& \llbracket \text{Meas}(\{2\}, \{1\}) \rrbracket_{\mathbb{H}}(\rho_5) \\
&= \left\{ \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0)} |x_0, 0\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 00 \right), \right. \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + y_0)} |x_0, 0\rangle_{\text{Inp}} |1, y_1\rangle_{\text{Init}}, 01 \right), \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + x_1)} |x_0, 1\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 10 \right), \\
&\quad \left. \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + x_1 + y_0)} |x_0, 1\rangle_{\text{Inp}} |1, y_1\rangle_{\text{Init}}, 11 \right) \right\} = \rho_6.
\end{aligned}$$

Correction (C):

$$\begin{aligned}
& \llbracket \text{if}(\{0\}, \text{Apply}(\mathbf{Z}, \emptyset, [0, 3])) \rrbracket_{\mathbf{H}}(\rho_6) \\
&= \left\{ \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0)} |x_0, 0\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 00 \right), \right. \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + y_0)} |x_0, 0\rangle_{\text{Inp}} |1, y_1\rangle_{\text{Init}}, 01 \right), \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + x_1 + x_0 + y_1)} |x_0, 1\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 10 \right), \\
&\quad \left. \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + x_1 + y_0 + x_0 + y_1)} |x_0, 1\rangle_{\text{Inp}} |1, y_1\rangle_{\text{Init}}, 11 \right) \right\} = \rho_6.
\end{aligned}$$

$$\begin{aligned}
& \llbracket \text{if}(\{1\}, \text{Apply}(\mathbf{X}, \emptyset, \{3\})) \rrbracket_{\mathbf{H}}(\rho_7) \\
&= \left\{ \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0)} |x_0, 0\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 00 \right), \right. \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + y_0)} |x_0, 0\rangle_{\text{Inp}} |1, 1 \oplus y_1\rangle_{\text{Init}}, 01 \right), \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + x_1 + x_0 + y_1)} |x_0, 1\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 10 \right), \\
&\quad \left. \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + x_1 + y_0 + x_0 + y_1)} |x_0, 1\rangle_{\text{Inp}} |1, 1 \oplus y_1\rangle_{\text{Init}}, 11 \right) \right\} \\
&= \left\{ \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0(y_1 + x_1 + x_0))} |x_0, 0\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 00 \right), \right. \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0(y_1 + x_1 + x_0 + 1))} |x_0, 0\rangle_{\text{Inp}} |1, 1 \oplus y_1\rangle_{\text{Init}}, 01 \right), \\
&\quad \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0(y_1 + x_1 + x_0) + x_1 + x_0 + y_1)} |x_0, 1\rangle_{\text{Inp}} |0, y_1\rangle_{\text{Init}}, 10 \right), \\
&\quad \left. \left(\frac{1}{4}, \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(y_0(y_1 + x_1 + x_0 + 1) + x_1 + x_0 + y_1)} |x_0, 1\rangle_{\text{Inp}} |1, 1 \oplus y_1\rangle_{\text{Init}}, 11 \right) \right\} = \rho_8.
\end{aligned}$$

Application of the reduction rules *HH* and *Elim* (Definition 2.3.10):

$$\rho_8 = \left\{ \left(\frac{1}{4}, |x_0, 0\rangle_{\text{Inp}} |0, x_0 \oplus x_1\rangle_{\text{Init}}, 00 \right), \left(\frac{1}{4}, |x_0, 0\rangle_{\text{Inp}} |1, x_0 \oplus x_1\rangle_{\text{Init}}, 01 \right), \right. \\ \left. \left(\frac{1}{4}, |x_0, 1\rangle_{\text{Inp}} |0, x_0 \oplus x_1\rangle_{\text{Init}}, 10 \right), \left(\frac{1}{4}, -|x_0, 1\rangle_{\text{Inp}} |1, x_0 \oplus x_1\rangle_{\text{Init}}, 11 \right) \right\}$$

Full process:

$$\llbracket \text{O}(\text{Apply}(\text{X}, \{0\}, \{1\})) \rrbracket_{\text{H}}(\rho) \\ = \left\{ \left(\frac{1}{4}, |x_0\rangle_{\text{Obs}_0} |0, 0\rangle_{\text{Di}} |x_0 \oplus x_1\rangle_{\text{Obs}_2}, 00 \right), \left(\frac{1}{4}, |x_0\rangle_{\text{Obs}_0} |0, 1\rangle_{\text{Di}} |x_0 \oplus x_1\rangle_{\text{Obs}_2}, 01 \right), \right. \\ \left. \left(\frac{1}{4}, |x_0\rangle_{\text{Obs}_0} |1, 0\rangle_{\text{Di}} |x_0 \oplus x_1\rangle_{\text{Obs}_2}, 10 \right), \left(\frac{1}{4}, -|x_0\rangle_{\text{Obs}_0} |1, 1\rangle_{\text{Di}} |x_0 \oplus x_1\rangle_{\text{Obs}_2}, 11 \right) \right\}$$

Thus, the output state (Obs) is:

$$\llbracket \text{O}(\text{Apply}(\text{X}, \{0\}, \{1\})) \rrbracket_{\text{H}}(\rho) \\ = \left\{ \left(\frac{1}{4}, |x_0, x_0 \oplus x_1\rangle_{\text{Obs}}, 00 \right), \left(\frac{1}{4}, |x_0, x_0 \oplus x_1\rangle_{\text{Obs}}, 01 \right), \right. \\ \left. \left(\frac{1}{4}, |x_0, x_0 \oplus x_1\rangle_{\text{Obs}}, 10 \right), \left(\frac{1}{4}, -|x_0, x_0 \oplus x_1\rangle_{\text{Obs}}, 11 \right) \right\}$$

By Definition 4.3.44:

$$(\emptyset, \emptyset, \text{Apply}(\text{CX}, \emptyset, \{0\})) \doteq_{(\text{H}, \text{gp})} (\text{Init}, \text{Di}, \text{O}(\text{Apply}(\text{CX}, \emptyset, \{0\}))).$$

5. If $U = U_1 - U_2$.

For each $i \in \{1, 2\}$, assume that $(\text{Inp}_i, \text{Obs}_i, \text{C}_i) = \text{O}(U_i)$, with $\text{Inp}_2 = \text{Obs}_1$. Let Q_i denote the lists of qubit indices for C_i , and define $\text{Init}_i := Q_i \setminus \text{Inp}_i$ and $\text{Di}_i := Q_i \setminus \text{Obs}_i$. By the induction hypothesis, we have:

$$(\emptyset, \emptyset, U_1) \doteq_{(\text{H}, \text{gp})} (\text{Init}_1, \text{Di}_1, \text{C}_1) \\ (\emptyset, \emptyset, U_2) \doteq_{(\text{H}, \text{gp})} (\text{Init}_2, \text{Di}_2, \text{C}_2).$$

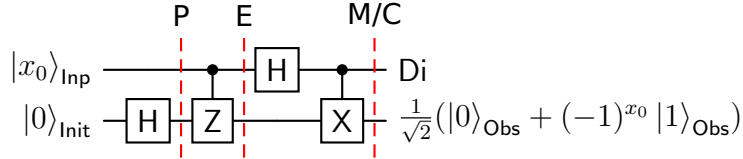
Therefore, by Lemma 4.3.50: $(\emptyset, \emptyset, U) \doteq_{(\text{H}, \text{gp})} (\text{Init}_1 \cup \text{Init}_2, \text{Di}_1 \cup \text{Di}_2, \text{C}_1 - \text{C}_2)$.

Where $\forall k \in \mathbb{N}, (-1)^k = e^{k\pi i}$, and $x \leftarrow x'$ represent a variable replacement of x by x' . $\square$

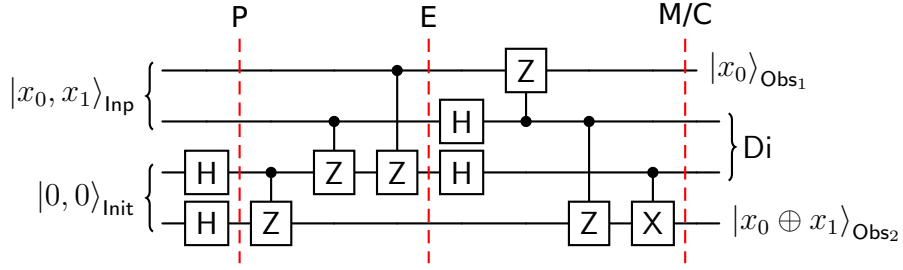
We now turn to the deferred-measurement version of OWM. Let U be a unitary circuit and $(\text{Inp}, \text{Obs}, \text{C}) = \text{O}(U)$. Applying deferred measurement yields $(\text{Inits}, U', \text{m}_\text{C}) = \text{DM}(\text{C})$ (Definition 4.3.69), where U' is unitary. Let Q be the set of qubit indices of C , and define $\text{Init} := Q \setminus \text{Inp}$ and $\text{Di} := Q \setminus \text{Obs}$. We prove that $(\text{Init}, \text{Di}, U') \doteq (\emptyset, \emptyset, U)$ (Theorem 5.1.2).

$$|x_0\rangle_{\text{Inp}} \text{---} \boxed{\text{X}} \text{---} |1 \oplus x_0\rangle_{\text{Obs}}$$

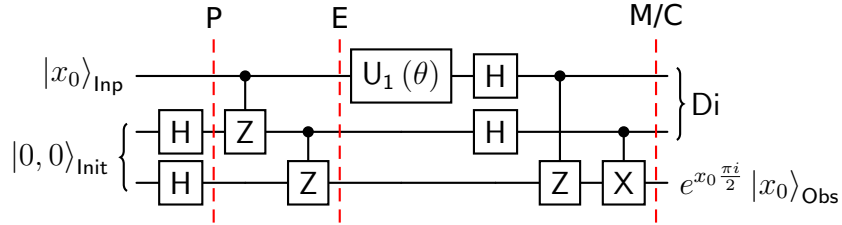
(a) $\text{DM}(\text{O}(\text{Apply}(\text{X}, \emptyset, \{0\})))$



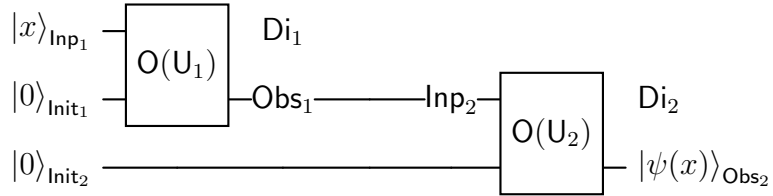
(b) $\text{DM}(\text{O}(\text{Apply}(\text{H}, \emptyset, \{0\})))$



(c) $\text{DM}(\text{O}(\text{Apply}(\text{X}, \{0\}, \{1\})))$



(d) $\text{DM}(\text{O}(\text{Apply}(\text{U}_1(\theta), \emptyset, \{0\})))$



(e) $\text{DM}(\text{O}(U_1 \text{---} U_2))$

Figure 5.1: One-Way Measurement under Deferred Measurement ($\text{DM}(\text{O}(U))$)

Theorem 5.1.2 (Correctness of O under our deferred measurement, DM). Let U be a unitary circuit. Assume that $(\text{Inp}, \text{Obs}, C) = O(U)$ and $(\text{Inits}, U', m_C) = DM(C)$ (Def. 4.3.69). Let Q be the set of qubit indices of C , and define $\text{Init} := Q \setminus \text{Inp}$, and $\text{Di} := Q \setminus \text{Obs}$:

$$(\text{Init}, \text{Di}, U') \triangleq (\emptyset, \emptyset, U). \quad (\text{Definition 4.3.12.})$$

Proof. Verification of the correctness of the one-way measurement under deferred measurement. Assume that $(\text{Inp}, \text{Obs}, C) = O(U)$ and $(\text{Inits}, U', m_C) = DM(C)$. For each $i \in \{1, 2\}$, let Q_i be the list of qubit indices for U_i , we define $\text{Init}_i := Q_i \setminus \text{Inp}_i$, and $\text{Di}_i := Q_i \setminus \text{Di}_i$. We prove by structural induction that $(\text{Init}, \text{Di}, U') \triangleq (\emptyset, \emptyset, U)$.

1. If $U = \text{Apply}(X, \emptyset, \{0\})$: $U' = \text{Apply}(X, \emptyset, \{0\})$, therefore $(\text{Init}, \text{Di}, U') \triangleq (\emptyset, \emptyset, U)$.
2. If $U = \text{Apply}(H, \emptyset, \{0\})$, input state: $|\xi\rangle = |x_0\rangle_{\text{Inp}} |0\rangle_{\text{Init}}$.

Preparation (P):

$$\llbracket \text{Apply}(H, \emptyset, \{1\}) \rrbracket (|\xi\rangle) = \frac{1}{\sqrt{2}} \sum_{y_0 \in \{0,1\}} |x_0\rangle_{\text{Inp}} |y_0\rangle_{\text{Init}} = |\xi_1\rangle.$$

Entanglement (E):

$$\llbracket \text{Apply}(CZ, \{0\}, \{1\}) \rrbracket (|\xi_1\rangle) = \frac{1}{\sqrt{2}} \sum_{y_0 \in \{0,1\}} (-1)^{x_0 y_0} |x_0\rangle_{\text{Inp}} |y_0\rangle_{\text{Init}} = |\xi_2\rangle.$$

Measurement/Correction (M/C):

$$\begin{aligned} \llbracket \text{Apply}(H, \emptyset, \{0\}) \rrbracket (|\xi_2\rangle) &= \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} (-1)^{x_0 y_0 + x_0 y_1} |y_1\rangle_{\text{Inp}} |y_0\rangle_{\text{Init}} = |\xi_3\rangle \\ \llbracket \text{Apply}(X, \{0\}, \{1\}) \rrbracket (|\xi_3\rangle) &= \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} (-1)^{x_0(y_0 + y_1)} |y_1\rangle_{\text{Inp}} |y_0 \oplus y_1\rangle_{\text{Init}} = |\xi_4\rangle. \end{aligned}$$

Replacement of $(y_0 \oplus y_1)$ by y_0 : $|\xi\rangle_4 = \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} (-1)^{x_0 y_0} |y_1\rangle_{\text{Inp}} |y_0\rangle_{\text{Init}}$.

Full process:

$$\begin{aligned} \llbracket \text{Apply}(U', \emptyset, \{0\}) \rrbracket (|\xi\rangle) &= \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} (-1)^{x_0 y_0} |y_1\rangle_{\text{Di}} |y_0\rangle_{\text{Obs}} \\ &= \frac{1}{\sqrt{2}} \sum_{y_1 \in \{0,1\}} |y_1\rangle_{\text{Di}} \otimes \frac{1}{\sqrt{2}} \sum_{y_0 \in \{0,1\}} (-1)^{x_0 y_0} |y_0\rangle_{\text{Obs}} \end{aligned}$$

Thus, the output state is: $\frac{1}{\sqrt{2}} \sum_{y_0 \in \{0,1\}} (-1)^{x_0 y_0} |y_0\rangle_{\text{Obs}}$.

Therefore, by Definition 4.3.27: $(\text{Init}, \text{Di}, U') \triangleq (\emptyset, \emptyset, U)$.

3. If $U = \text{Apply}(U_1(\theta), \emptyset, \{0\})$:

Input state: $|\xi\rangle = |x_0\rangle_{\text{Inp}} |0, 0\rangle_{\text{Init}}$.

Preparation (P)

$$\llbracket \text{Apply} (H, \emptyset, \{1, 2\}) \rrbracket (|\xi\rangle) = \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} |x_0\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}} = |\xi_1\rangle.$$

Entanglement (E)

$$\llbracket \text{Apply} (Z, \{0\}, \{1\}) \rrbracket (|\xi_1\rangle) = \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(x_0 y_0)} |x_0\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}} = |\xi_2\rangle.$$

$$\llbracket \text{Apply} (Z, \{1\}, \{2\}) \rrbracket (|\xi_2\rangle) = \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i\pi(x_0 y_0 + y_0 y_1)} |x_0\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}} = |\xi_3\rangle.$$

Measurement/Correction (M/C)

$$\begin{aligned} & \llbracket \text{Apply} (U_1(\theta), \emptyset, \{0\}) \rrbracket (|\xi_3\rangle) \\ &= \frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1) + \theta x_0)} |x_0\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}} = |\xi_4\rangle, \\ & \llbracket \text{Apply} (H, \emptyset, [0, 1]) \rrbracket (|\xi_4\rangle) \\ &= \frac{1}{4} \sum_{y_0, y_1, y_2, y_3 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + x_0 y_2 + y_0 y_3) + \theta x_0)} |y_2\rangle_{\text{Inp}} |y_3, y_1\rangle_{\text{Init}} = \text{ket}\xi_5, \end{aligned}$$

$$\begin{aligned} & \llbracket \text{Apply} (Z, \{0\}, \{2\}) \rrbracket (|\xi_5\rangle) \\ &= \frac{1}{4} \sum_{y_0, y_1, y_2, y_3 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + x_0 y_2 + y_0 y_3 + y_1 y_2) + \theta x_0)} |y_2\rangle_{\text{Inp}} |y_3, y_1\rangle_{\text{Init}} = |\xi_6\rangle, \end{aligned}$$

$$\begin{aligned} & \llbracket \text{Apply} (X, \{1\}, \{2\}) \rrbracket (|\xi_6\rangle) \\ &= \frac{1}{4} \sum_{y_0, y_1, y_2, y_3 \in \{0,1\}} e^{i(\pi(x_0 y_0 + y_0 y_1 + x_0 y_2 + y_0 y_3 + y_1 y_2) + \theta x_0)} |y_2\rangle_{\text{Inp}} |y_3, y_1 \oplus y_3\rangle_{\text{Init}} \\ &= \frac{1}{4} \sum_{y_0, y_1, y_2, y_3 \in \{0,1\}} e^{i(\pi(y_0(y_1 + x_0 + y_3) + x_0 y_2 + y_1 y_2) + \theta x_0)} |y_2\rangle_{\text{Inp}} |y_3, y_1 \oplus y_3\rangle_{\text{Init}} \\ &= |\xi_7\rangle. \end{aligned}$$

Application of the reduction rules *HH* and *Elim* (Definition 2.3.10):

$$|\xi_7\rangle = \frac{1}{2} \sum_{y_2, y_3 \in \{0,1\}} e^{i(\pi y_2 y_3 + \theta x_0)} |y_2\rangle_{\text{Inp}} |y_3, x_0\rangle_{\text{Init}}.$$

Full process:

$$\llbracket U_0 \rrbracket (|\xi\rangle) = \left(\frac{1}{2} \sum_{y_2, y_3 \in \{0,1\}} e^{i\pi y_2 y_3} |y_2, y_3\rangle_{\text{Di}} \right) \otimes e^{i\theta x_0} |x_0\rangle_{\text{Obs}}.$$

Thus, the output state (Obs) is: $e^{i\theta x_0} |x_0\rangle_{\text{Obs}}$.

Therefore, by Definition 4.3.27: $(\text{Init}, \text{Di}, U') \triangleq (\emptyset, \emptyset, U)$.

4. If $U = \text{Apply}(U_1(\theta), \emptyset, \{0\})$:

Input state: $|\xi\rangle = |x_0, x_1\rangle_{\text{Inp}} |0, 0\rangle_{\text{Init}}$.

Preparation (P):

$$\llbracket \text{Apply}(H, \emptyset, [2, 3]) \rrbracket(|\xi\rangle) = \frac{1}{2} \sum_{y_0, y_1 \in \{0, 1\}} |x_0, x_1\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}} = |\xi_1\rangle.$$

Entanglement (E):

$$\llbracket \text{Apply}(Z, \{2\}, \{3\}) \rrbracket(|\xi_1\rangle) = \frac{1}{2} \sum_{y_0, y_1 \in \{0, 1\}} e^{i\pi y_0 y_1} |x_0, x_1\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}} = |\xi_2\rangle$$

$$\llbracket \text{Apply}(Z, \{1\}, \{2\}) \rrbracket(|\xi_2\rangle) = \frac{1}{2} \sum_{y_0, y_1 \in \{0, 1\}} e^{i\pi(y_0 y_1 + x_1 y_0)} |x_0, x_1\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}} = |\xi_3\rangle$$

$$\llbracket \text{Apply}(Z, \{0\}, \{2\}) \rrbracket(|\xi_3\rangle) = \frac{1}{2} \sum_{y_0, y_1 \in \{0, 1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0)} |x_0, x_1\rangle_{\text{Inp}} |y_0, y_1\rangle_{\text{Init}} = |\xi_4\rangle.$$

Measurement/Correction (M/C):

$$\begin{aligned} & \llbracket \text{Apply}(H, \emptyset, \{1, 2\}) \rrbracket(|\xi_4\rangle) \\ &= \frac{1}{4} \sum_{y_0, y_1, y_2, y_3 \in \{0, 1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + x_1 y_2 + y_0 y_3)} |x_0, y_2\rangle_{\text{Inp}} |y_3, y_1\rangle_{\text{Init}} = |\xi_5\rangle. \end{aligned}$$

$$\begin{aligned} & \llbracket \text{Apply}(Z, \{0\}, \{1\}) \rrbracket(|\xi_5\rangle) \\ &= \frac{1}{4} \sum_{y_0, y_1, y_2, y_3 \in \{0, 1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + x_1 y_2 + y_0 y_3 + x_0 y_2)} |x_0, y_2\rangle_{\text{Inp}} |y_3, y_1\rangle_{\text{Init}} = |\xi_6\rangle \end{aligned}$$

$$\begin{aligned} & \llbracket \text{Apply}(Z, \{1\}, \{3\}) \rrbracket(|\xi_6\rangle) \\ &= \frac{1}{4} \sum_{y_0, y_1, y_2, y_3 \in \{0, 1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + x_1 y_2 + y_0 y_3 + x_0 y_2 + y_1 y_2)} |x_0, y_2\rangle_{\text{Inp}} |y_3, y_1\rangle_{\text{Init}} = |\xi_7\rangle \end{aligned}$$

$$\begin{aligned} & \llbracket \text{Apply}(X, \{2\}, \{3\}) \rrbracket(|\xi_7\rangle) \\ &= \frac{1}{4} \sum_{y_0, y_1, y_2, y_3 \in \{0, 1\}} e^{i\pi(y_0 y_1 + x_1 y_0 + x_0 y_0 + x_1 y_2 + y_0 y_3 + x_0 y_2 + y_1 y_2)} |x_0, y_2\rangle_{\text{Inp}} |y_3, y_1 \oplus y_3\rangle_{\text{Init}} \\ &= \frac{1}{4} \sum_{y_0, y_1, y_2, y_3 \in \{0, 1\}} e^{i\pi(y_0(y_1 + x_0 + x_1 + y_3) + x_1 y_2 + x_0 y_2 + y_1 y_2)} |x_0, y_2\rangle_{\text{Inp}} |y_3, y_1 \oplus y_3\rangle_{\text{Init}} \\ &= |\xi_8\rangle. \end{aligned}$$

Application of the reduction rules *HH* and *Elim* (Definition 2.3.10):

$$|\xi_8\rangle = \frac{1}{2} \sum_{y_2, y_3 \in \{0, 1\}} e^{i\pi(y_3 y_2)} |x_0, y_2\rangle_{\text{Inp}} |y_3, x_0 + x_1\rangle_{\text{Init}}$$

Full process:

$$\llbracket U' \rrbracket(|\xi\rangle) = |x_0\rangle_{\text{Obs}_1} \otimes \left(\frac{1}{2} \sum_{y_2, y_3 \in \{0,1\}} e^{i\pi(y_3 y_2)} |y_2, y_3\rangle_{\text{Di}} \right) \otimes |x_0 + x_1\rangle_{\text{Obs}_2}$$

Thus, the output state (Obs) is: $|x_0, x_0 + x_1\rangle_{\text{Obs}}$.

Therefore, by Definition 4.3.27: $(\text{Init}, \text{Di}, U') \doteq (\emptyset, \emptyset, U)$.

5. If $U = U_1 - - U_2$:

For each $i \in \{1, 2\}$, assume that $(\text{Inp}_i, \text{Obs}_i, C_i) = O(U_i)$, with $\text{Inp}_2 = \text{Obs}_1$, and $(\text{Inits}_i, U'_i, m_{C_i}) = \text{DM}(C_i)$. Let Q_i denote the lists of qubit indices for U'_i , and define $\text{Init}_i := Q_i \setminus \text{Inp}_i$ and $\text{Di}_i := Q_i \setminus \text{Obs}_i$. By construction $\text{Inits}_i \subseteq \text{Init}_i$. By the induction hypothesis, we have: $(\emptyset, \emptyset, U_1) \doteq (\text{Init}_1, \text{Di}_1, U'_1)$ and $(\emptyset, \emptyset, U_2) \doteq (\text{Init}_2, \text{Di}_2, U'_2)$.

Therefore, by unitarity: $(\emptyset, \emptyset, U) \doteq (\text{Init}_1 \cup \text{Init}_2, \text{Di}_1 \cup \text{Di}_2, U'_1 - - U'_2)$. $\square$

5.1.2 Teleportation Insertion (T)

This section proves the correctness of the implementation presented in Section 2.5.4.

Theorem 5.1.3 (Correctness of Our Implementation of the Teleportation, T). Let U be a unitary circuit. Assume that $(\text{Inp}, \text{Obs}, C) = T(U)$. Let Q be the set of qubit indices used in C , and define $\text{Init} := Q \setminus \text{Inp}$ and $\text{Di} := Q \setminus \text{Obs}$. Then, we have:

$$(\text{Init}, \text{Di}, C) \doteq_{\text{H}} (\emptyset, \emptyset, U). \quad (\text{Definition 4.3.43})$$

Proof. The proof follows directly from Definition 4.3.43 and Example 2.2.54. Example 2.2.54 can be generalised to any quantum state by utilising unitarity and can be extended to n qubits by separating the teleported states. $\square$

5.2 Hyb-Part-DM Implementation: SQbricks

Our goal is to verify that a circuit transformation preserves the semantics of the original circuit. Chapter 3 provides formal correctness proofs for specific transformations, while Chapter 4 introduces equivalence relations enabling practical verification between original and transformed circuits.

To this end, we define efficiently encodable partial equivalence relations for unitary circuits based on path-sum semantics (Section 4.3.3) and extend them to the hybrid setting via deferred measurement (Section 4.3.5.4). This allows us to verify hybrid circuit transformations, including non-trivial cases such as One-Way Measurement (Section 2.5.3), which introduce discarded qubits.

This section presents the implementation of this method in SQbricks, composed of two components: *SQbricks-Verif* and *SQbricks-Lift*. SQbricks-Verif implements SubCir-PS-Part (Def. 4.3.27, 4.3.29) and GP-PS-Part equivalences (Def. 4.3.31, 4.3.33). We do

not implement FullCir-PS-Part-Par (Def. 4.3.35), as its verification requires comparing all symbolic paths, yielding potentially exponential complexity (Rem. 4.3.38). In contrast, sub-circuit and global-phase variants remain purely symbolic and efficiently verifiable, which suffices for our evaluation goals. SQbricks-Lift implements deferred measurement DM (Sec. 4.3.5.2).

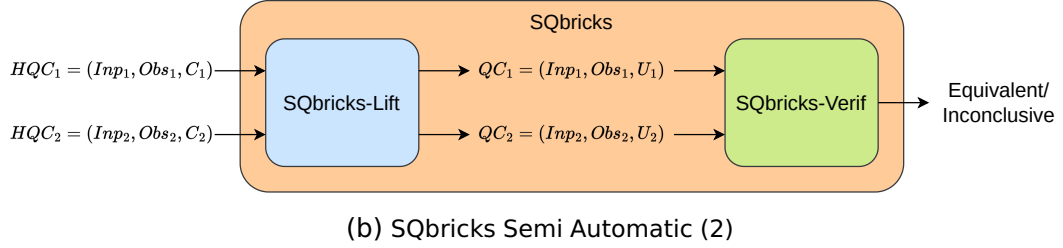
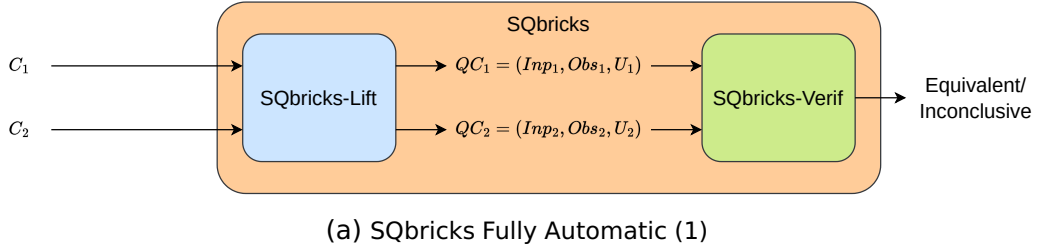


Figure 5.2: SQbricks Overview

SQbricks integrates these components into two operational modes, which are illustrated in Figure 5.2:

1. *Fully automatic mode* takes as parameters two hybrid circuits C_1 and C_2 . It applies our deferred measurement (DM) and returns the unitary tuples QC_1 and QC_2 . This mode is fully automatic because the outputs Obs_1 and Obs_2 are deduced from the measurement of C_1 and C_2 . Finally, it checks the unitary partial equivalence between QC_1 and QC_2 . It returns 'Equivalent' when the circuits are equivalent and 'Inconclusive' when it cannot conclude due to a timeout or unequal path-sums².
2. A *semi-automatic* takes two HQC as inputs (Figure 5.2b). Each HQC consists of a hybrid circuit (C), an input list (Inp), and an output list (Obs). The deferred measurement returns the unitary part and the input list without modifying them, and SQbricks-Verif checks the unitary partial equivalence, as in the full automatic mode (Item 1).

Section 5.2.1 introduces SQbricks-Verif. Section 5.2.2 describes SQbricks-Lift.

²Two path-sums can represent the same quantum state (Lemma 2.3.9).

5.2.1 PS-Part Implementation: SQbricks-Verif

Addressing non-trivial circuit transformations requires handling partial hybrid equivalence, which remains a gap in the state of the art. Our approach reduces this problem to the verification of partial unitary equivalence as an intermediate step. To this end, SQbricks-Verif implements sub-circuit path-sum partial equivalence (Definition 4.3.27) and global-phase path-sum partial equivalence (Definition 4.3.31). We did not implement full-circuit path-sum partial equivalence (Definition 4.3.35), as its verification requires comparing all symbolic paths, leading to a potentially exponential worst-case complexity (Remark 4.3.38). In contrast, sub-circuit path-sum partial equivalence and global-phase path-sum partial equivalence are purely symbolic and can be verified efficiently via rewriting or symbolic unification, which suffices for our experimental objectives.

We begin by introducing the main differences with Feynman, which implements related path-sum techniques, while SQbricks-Verif is an independent reimplementa- tion in a different language with additional features (Section 5.2.1.1). Then we discuss the equivalence test approaches used in SQbricks-Verif (Section 5.2.1.2). Next, we define the syntax of SQbricks-Verif in Section 5.2.1.3. Finally, we explain how SQbricks-Verif is implemented.

5.2.1.1 Feynman Comparison

Feynman is limited to complete unitary equivalence (Section 4.2.1) and does not imple- ment quantum multi-control. To recall, quantum multi-control is necessary for a unitary tool to be lifted to the hybrid case using deferred measurement, so Feynman requires preprocessing to be lifted as well. Furthermore, it is impossible to lift Feynman to the hybrid case with discard (the non-trivial case) due to the absence of partial equivalence with separation testing. SQbricks-Verif extends the functionality of Feynman to multi-control without preprocessing and equivalence relations to the partial case with separation tests. This extension enables us to achieve the hybrid case with discard using deferred measurement. Thus, we can verify non-trivial transformations, such as the One-Way Measurement.

5.2.1.2 Equivalence Verification Approaches

SQbricks-Verif (SV) implements the parallel and the sequential approaches (Sec. 4.1.4) to check partial unitary equivalence. These approaches are commonly used in previous work, each with its own advantages and disadvantages. The sequential approach ($--$), illustrated in Fig. 4.2, checks if a unitary circuit (U_1) temporally composed with the inverse of a second (U_2) acts as the identity on the observable part. However, this approach is limited to checking with at least one circuit without discarding. The parallel approach ($//$), illustrated in Fig. 4.3, checks if two circuits create the same observed state. This approach can handle two unitary circuits with discard but may be more

limited by the non-injectivity of path-sum concretisation (Lem. 2.3.9). Both algorithms check the separation (Def. 2.2.16) between Di and Obs.

5.2.1.3 Syntax

SQbricks-Verif incorporates a minimal gate set ($\{\text{GP}(d, k), \text{U}_1(d, k), \text{X}, \text{H}\}$) together with multi-controlled versions of these gates, enabling the expression of any unitary circuit. It relies on our unitary QASM representation (Section 2.2.4.3). To facilitate encoding and symbolic verification, real-valued angles θ are represented exactly using two integers d and k such that $\theta = \frac{d}{2^k}$. This encoding is not an approximation: angles are stored as dyadic rationals, which are well suited to path-sum manipulation and easy to encode as integer pairs. However, this choice restricts the supported rotations to dyadic angles and does not allow arbitrary real parameters. The formal syntax of SQbricks-Verif is defined in Definition 4.3.2. It is designed to match the target form produced by our deferred measurement construction (Figure 4.14).

5.2.1.4 Implementation

Figure 5.3 shows the flowchart of SQbricks-Verif. SQbricks-Verif takes two OpenQASM circuits as input, constructs two programs, evaluates them into path-sums, reduces them, and checks the equivalence of the reduced path-sums. The result is either equivalent or inconclusive³. SQbricks-Verif is composed of 15k lines of OCaml code, with 9k lines in the main library (lib/) and 6k lines in the unitary and integration tests (test/).

Figure 5.4 illustrates the dependencies between the modules of SQbricks-Verif. The program’s entry point is the Main_lib module. The modules Lexer_OpenQASM and Parser_OpenQASM utilise the Program module to parse two circuits, written in OpenQASM, into the Program type of SQbricks-Verif, resulting in two programs. The following listing introduces the Program type:

```

1  type Gate = GP of Q * int | U1 of Q * int | X | H
2
3  type Program =
4    | E
5    | Apply of Gates * int list * int list
6    | Sequence of Program * Program
7    | Measure of int * int
8    | It of int list * Program
9    | InitQ of int
10   | Not of int

```

Program type directly matches the restricted hybrid type (Def. 2.2.47, Rem. 4.3.51), with the only difference being that sequences are not given in infix notation (—) but by Sequence. We provide macros to simplify program writing and gate decompositions,

³Our test cannot answer non-equivalent because of the incomplete equational theory of our path-sum semantics extended to multi-control(Section 4.3.3.2).

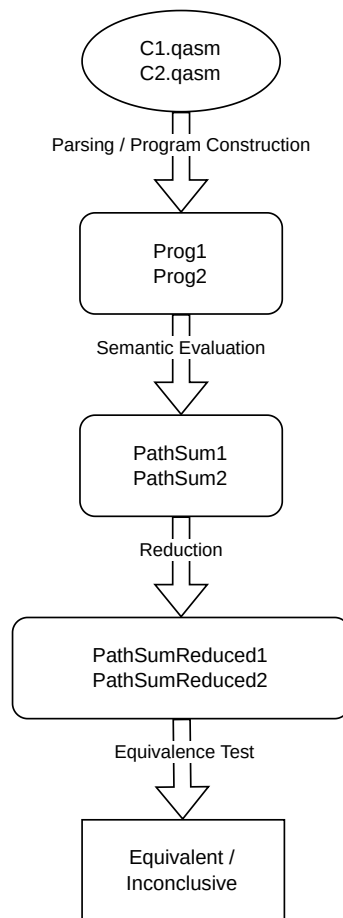


Figure 5.3: Flowchart of SQbricks-Verif

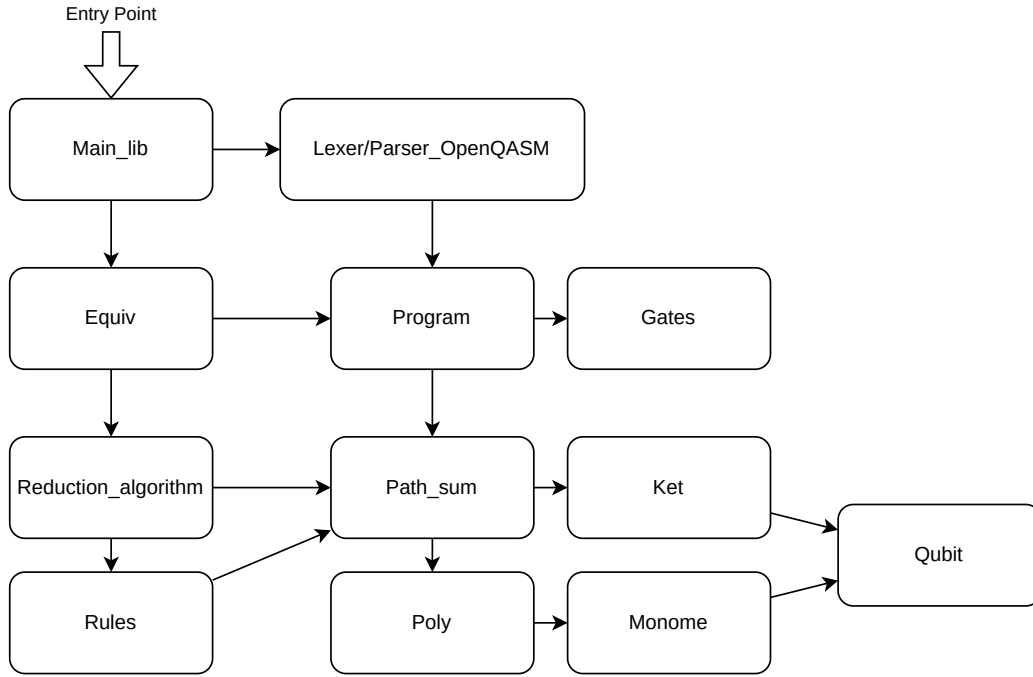


Figure 5.4: Dependencies Diagram of SQbricks-Verif

enabling the use of specific state-of-the-art tools, such as Feynman. In the following, we give an example of a controlled $U_1(\theta)$ gate decomposed for Feynman⁴:

```

1  let (--) p1 p2 : Program = Sequence (p1, p2)
2
3  let u1 ?(s = 1) k ta : Program =
4    if k < 0 then failwith "k < 0, forbidden";
5    Apply (U1 (Q.of_int s, k), [], [ ta ])
6
7  let cx co ta : Program = Apply (X, [ co ], [ ta ])
8
9  let crzdecomp ?(s = 1) k co ta : Program =
10   u1 ~s (k + 1) ta -- cx co ta -- u1 ~s:(-s) (k + 1) ta -- cx co
11   ta
12
13  let culdecomp ?(s = 1) k co ta : Program =
14   u1 ~s (k + 1) co -- crzdecomp ~s k co ta

```

We recall that s and k are elements of our dyadic representation of angles such that $\theta = \frac{s}{2^k}$. The notation $?(s = 1)$ indicates that s is optional and is replaced by 1 when not provided. The following listing gives the `Path_sum` type and an example of path-sum (H in Definition 2.3.7):

⁴Feynman lacks generic gate control and does not implement controlled $R_z(\theta)$ or $U_1(\theta)$ gates.

```

1  type Qubit =
2      Zero | One | Var of int | Prod of Qubit * Qubit |
3      SumMod2 of Qubit * Qubit
4
5  type Monome =
6      Scal of Q | Qubit of Monome | Prod of Monome * Monome
7
8  type Ket = Qubit array
9
10 module Poly = BatHeap.Make (struct
11     type t = Monome
12     let compare m1 m2 = -Monome.compare m1 m2
13 end)
14
15 type Path_sum = { phase : Poly; ket : Ket; path_var : int list }
16
17 let xx ta w : Qubit =
18     if ta < w then Var ta
19     else failwith
20         (sprintf "w = %d < ta = %d\n" w ta)
21
22 let yy n w : Qubit = Var (n + w)
23
24 let h ta w =
25 {
26     phase = Prod (
27         Scal div2,
28         Prod (Qubit (xx ta w), Qubit (yy 0 w))) ++ empty;
29     ket = [| yy 0 w |];
30     path_var = [ w ];
31 }

```

It is a record type consisting of Poly, Ket, and a list of path variables. Each subtype, including Path_sum itself, has a defined canonical form to facilitate the comparison of path-sums. A path-sum in canonical form has its phase polynomial decomposed into a sum of monomials. These monomials are sorted first by the denominator in ascending order, then by the type of variable, with input variables (x_i) first, followed by path variables (y_i), and within the same type, by indices. Qubits are sorted using the same principle. For example, in concretisation form (Definition 2.3.4):

$$\frac{1}{\sqrt{2^3}} \sum_{y_0, y_1, y_2} e^{2\pi i \left(\frac{x_0 \cdot y_1}{2} + \frac{y_0 \cdot y_1}{2} + \frac{y_1 \cdot y_2}{2^3} \right)} |y_0(x_1 \oplus x_2 \cdot y_2)(x_0 \oplus y_1)\rangle,$$

or in algebraic form (Definition 2.3.3):

$$\left\langle y_0 y_1 y_2, \frac{x_0 \cdot y_1}{2} + \frac{y_0 \cdot y_1}{2} + \frac{y_1 \cdot y_2}{2^3}, |y_0(x_1 \oplus x_2 \cdot y_2)(x_0 \oplus y_1)\rangle \right\rangle.$$

The Qubit type includes Var for defining symbolic variables. We distinguish input variables from path variables using the variable index: if the index i is less than or equal

to the number n of qubits in the circuit ($i \leq n$), it is an input variable (denoted x_i); otherwise, it is a path variable (denoted y_{i-n}). The operations `Prod` and `SumMod2` in the `Qubit` type are Boolean operations, with `Zero` corresponding to `false` and `1` to `true`. The `Monome` type comprises a product of monomials (`Prod`), qubits (`Qubit`), and rationals (`Scal`). The output signature $f(x, y)$ is represented by the `Ket` type, which is an array of qubits. The phase polynomial $p(x, y)$ is represented by the `Poly` type, a heap of monomials. This structure is ideal for maintaining the canonical form, allowing $O(\log m)$ insertion time and $O(1)$ removal time⁵. An example of a path-sum is given by `(h ta w)`, the path-sum of the Hadamard gate without control (Definition 2.3.7) applied to the qubit indexed by `ta`, where `w` represents the width, i.e., the number of qubits.

The `Rules` module contains our reduction functions. It implements the `HH` (in module `HH`) and `Elim` (in module `Elim`) rules from Feynman (Definition 2.3.10). It also includes standard simplification rules such as $x_0 \oplus x_0 = 0$ (in module `Simplification`) and a renaming function (e.g., if 7 is the smaller index in the path-sum, then x_7 is renamed to x_0) in module `Rename`. Additionally, it provides variable replacement functions in module `Variable_replacement`, as shown in the transformation of

$$\frac{1}{2} \sum_{y_0, y_1 \in \{0,1\}} e^{2\pi i \frac{x_0 \cdot y_0 + x_0 \cdot y_1}{2}} |y_0 \oplus y_1\rangle$$

by replacing y_0 with $y_0 \oplus y_1 = y_0 + y_1 - 2 \cdot y_0 \cdot y_1$, resulting in

$$\frac{1}{\sqrt{2}} \sum_{y_0 \in \{0,1\}} e^{2\pi i \frac{x_0 \cdot y_0}{2}} |y_0\rangle.$$

We did not implement the Feynman rules ω and `Case`. In our experiments with Feynman, these rules were never triggered on the benchmark circuits we considered. Moreover, in the instances where path-sum reduction failed, this was either due to time limitations or because no applicable rewriting rule matched the reduction pattern, including ω and `Case`. Therefore, their implementation was not required for our evaluation. The following listing presents the main function of our implementation of the `HH` rule:

```

1  let hh (ps : Path_sum) : Path_sum =
2    let width = Array.length ps.ket in
3    (* Try y0 in order of arrival *)
4    let rec aux (acc : Path_sum) = function
5      | y0 :: y0_remain →
6        if y0_accepted y0 acc then (
7          match hh_aux y0 acc with
8          | Some acc_reduced →
9            aux
10             (Path_sum.remove_path_var
11              (Simplification.simplify acc_reduced)
12              y0))

```

⁵OCaml Library, Module `BatHeap`: <https://ocaml.org/p/batteries/3.5.0/doc/batteries.unthreaded/BatHeap/index.html>

```

13         y0_remain
14         | None → aux acc y0_remain)
15     else aux acc y0_remain
16 | _ → Elim.elim acc
17 in
18 aux ps ps.path_var

```

We use the simplest heuristic for removing path variables: processing them in the order of their appearance (line 5). This approach yielded better results in our experiments than more complex strategies, such as those based on variable frequency in the path-sum. Before attempting elimination, we filter path variables that are syntactically non-removable (line 6, function `y0_accepted`). Concretely, a variable is excluded if it appears either in the Ket component, or in the phase polynomial `Poly` with scalar coefficient $\frac{1}{2}$. At line 7, we attempt to remove the variable. If successful, we simplify the path-sum, remove the variable from the set of path variables, and recursively process the remaining variables. Finally, we apply *Elim* to remove from the set any path variables that no longer occur in the path-sum. Among the reduction rules, *HH* is the most crucial and computationally demanding in terms of both time and memory.

Our reduction algorithm is defined in the `Reduction_algorithm` module and is presented in the following listing:

```

1  let _condition_to_continue (input : Path_sum) (output : Path_sum)
   =
2      let len_in = List.length input.path_var in
3      let len_out = List.length output.path_var in
4      let nb_of_sum_in =
5          Ket.number_of_sum input.ket + Poly.size input.phase in
6      let nb_of_sum_out =
7          Ket.number_of_sum output.ket + Poly.size output.phase in
8      (len_out < len_in && 0 < len_out) ||
9      nb_of_sum_out < nb_of_sum_in
10
11  let reduction_algorithm input =
12      let rec aux acc =
13          let state_simpl = Rules.Simplification.simplify acc in
14          let state_hh = Rules.HH.hh state_simpl in
15          let state_elim = Rules.Elim.elim state_hh in
16          match Rules.Variable_replacement.variable_replacement
17              state_elim with
18          | Some state_replace →
19              aux state_replace
20          | None →
21              let state_fact =
22                  let rec aux state_in =
23                      let state_out =
24                          Rules.Variable_replacement.
25                              variable_replacement_factorisation
26                              state_in
27                      in

```

```

26         let condition = _condition_to_continue state_in
27           state_out in
28         if condition then aux state_out else state_in
29       in
30       aux state_elim
31     in
32     let state_repl =
33       Rules.Variable_replacement.replace_not_path_var_by_var
34       state_fact
35     in
36     if _condition_to_continue acc state_repl then aux
37       state_repl
38     else state_repl
39   in
40   Rename.rename (aux input)

```

We experimentally developed this recursive algorithm. Its continuation condition (`condition_to_continue`, lines 1-9) ensures it continues to run as path variables are removed or the path-sum shrinks. The reduction begins by simplifying and applying *HH* and *Elim* (lines 13-15), followed by variable substitutions (lines 16-36), where any success triggers recursion. Finally, variable renaming (line 37) yields a canonical form. The *Equiv* module manages testing: it reduces two *Path_sum* programs to check equivalence or report an inconclusive result.

5.2.2 DM Implementation: SQbricks-Lift

We aim to bridge the gap in the state of the art by implementing a tool for checking the partial equivalence of hybrid circuits. In Section 4.3.5.4, we define an approach using partial unitary equivalence to check hybrid circuits. Section 5.2.1 presents our implementation of partial unitary equivalence. This section introduces SQbricks-Lift (SL), our implementation of deferred measurement (Section 4.3.5.2). Figure 5.5a gives the flowchart of SQbricks-Lift. The deferred measurement function (`to_deferred_measurements`) replaces classical control with quantum control, removes initialisations and measurements, and returns a triplet. More formally,

$$(\text{Inits}, \text{mC}, \text{Prog}') = \text{to_deferred_measurements Prog}.$$

Here, *Inits* is the list of qubit indices that are initialised in *Prog*; *mC* is the mapping from measurement result indices to the corresponding measured qubit indices (Definition 4.3.59); and *Prog'* is the unitary circuit obtained after the transformation. SQbricks-Lift consists of 150 lines of code. Figure 5.5b illustrates the dependencies diagram of SQbricks-Lift. The entry point is *Main_lib*, which takes as input a circuit in OpenQASM. The lexer and parser are used to create the program. Then, the DM function of the module *To_deferred_measurement* is used to create the triple: initialisation, a map of measurement results, and a program without initialisation or measurement.

The following listing presents the function implementing our deferred measurement (DM) defined in Section 4.3.5.2:

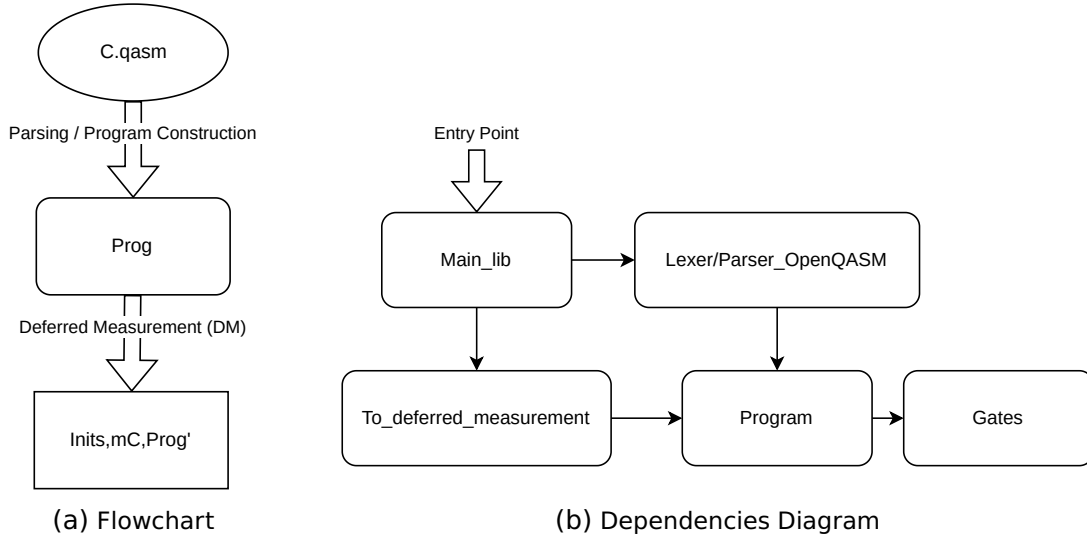


Figure 5.5: Overview of SQbricks-Lift

```

1  let to_deferred_measurements p =
2    let wc, wq = Program.widths p in
3    let bit_to_qubit = Array.make wc (-1) in
4    let is_wires_available wires_unavailable wires_to_check =
5      let rec aux wires_to_check =
6        match wires_to_check with
7        | [] -> true
8        | wire :: _ when Array.mem wire wires_unavailable -> false
9        | _ :: wires_to_check_remain -> aux wires_to_check_remain
10     in
11     aux wires_to_check
12   in
13
14   let rec aux p inits meas =
15     match p with
16     | Apply (_, co, ta) when is_wires_available bit_to_qubit (co @ ta) ->
17       (p, inits, meas)
18     | Measure (qubit_indice, bit_indice) ->
19       bit_to_qubit.(bit_indice) <- qubit_indice;
20       (E, inits, qubit_indice :: meas)
21     | It (bits_indices, Apply (g, qubits_indices_co, qubits_indices_ta))
22       when is_wires_available bit_to_qubit (qubits_indices_co @
23         qubits_indices_ta) ->
24       let rec bits_to_qubits bits_indices =
25         match bits_indices with
26         | [] -> []
27         | bit_indice :: bits_indices_remain ->
28           let qubit_indice = bit_to_qubit.(bit_indice) in
29           qubit_indice :: bits_to_qubits bits_indices_remain
30       in
31       let qubits_indices = bits_to_qubits bits_indices in
32       Apply ( g, qubits_indices @ qubits_indices_co, qubits_indices_ta ),
33       inits,
34       meas )
35     | It (_, E) -> (E, inits, meas)
36     | It ([ bit_indice ], Sequence (p1, p2)) ->
37       aux

```

```

38         (Sequence (It ([ bit_indice ], p1), It ([ bit_indice ], p2)))
39         inits meas
40     | Sequence (p1, p2) →
41         let p1', l1, l1' = aux p1 inits meas in
42         let p2', l2, l2' = aux p2 l1 l1' in
43         (Sequence (p1', p2'), l2, l2')
44     | E → (E, inits, meas)
45     | Not bit_indice → (x bit_to_qubit.(bit_indice), inits, meas)
46     | InitQ ta → (E, ta :: inits, meas)
47     | _ → failwith (sprintf "Forbidden")
48 in
49 let dm, inits, meas = aux p [] [] in
50 (dm, inits, meas)

```

It takes as input a hybrid program p and returns a unitary program dm , a list of initialised qubit indices ($inits$), and a list of measured qubits ($meas$). It begins by extracting the width of the classical and quantum registers, wc and wq , from p (lines 1 and 2). It then initialises an array (bit_to_qubit) to store measured qubits and their indices linked to the classical bit index (line 3). This allows us to determine the location of the control qubit when transforming a classical control into a quantum control. From lines 4 to 12, an internal function is declared to verify that a gate is not applied to a qubit that has already been measured. We recall that in the well-formedness conditions of our hybrid circuits, reusing already measured qubits is prohibited⁶ (Definition 2.2.47). From lines 14 to 48, the internal recursive function aux over the program p is declared to effectively apply the deferred measurement.

5.2.3 Usage

SQbricks comes with a Docker container and scripts to reproduce our evaluations. It allows automatic verification that two hybrid circuits in OpenQASM format are partially equivalent. It also enables the execution of One-Way Measurement (O, Section 2.5.3) and Teleportation Insertion (T, Section 2.5.4) transformations. To facilitate its use, SQbricks will be made available on GitHub in this archive⁷. The archive includes a README file that explains how to create and use the Docker container, along with a mini-tutorial containing usage examples.

To ensure the correctness of our implementation, we perform a sanity check presented in Section 5.4.1. Additionally, we have 127 unit and integration tests for low-level functions. There are 381 circuit equivalence tests for unitary circuits and 108 for hybrid circuits. We also include 874 circuit equivalence tests of hybrid circuits with discards. Finally, there are 13 circuit verification tests against a specification (Hoare style 1). These tests are located in the `test` folder at the repository root.

⁶Not all conditions for verifying the well-formedness of circuits are presented in this listing, to simplify the presentation.

⁷SQbricks GitHub archive: <https://github.com/Qbricks/qbricks.github.io/tree/main/Artifacts/SQbricks>. The public release of the repository is subject to internal approval procedures and may therefore occur after a short delay.

5.3 Evaluation Protocol

Our objective is to verify that circuit transformations preserve semantics, including non-trivial transformations such as One-Way Measurement, which introduce discarded qubits (Section 4.1). Our approach enables the verification of hybrid circuit equivalence by reducing it to partial unitary equivalence, combined with a separability test between output and discard and a deferred-measurement construction that allows the use of unitary verification tools in the hybrid setting.

In this section, we evaluate our technique for verifying equivalence between original and transformed circuits. We consider lifting existing unitary tools to DisFree (the hybrid case without discard, corresponding to the state of the art), and lifting our tool, which integrates a separability test, to Mix and Dis (the hybrid case with discard).

To this end, we address the following research questions:

- RQ1: Can unitary verification tools be lifted to check DisFree equivalence?
- RQ2: How does our approach compare to prior work on DisFree equivalence?
- RQ3: Can unitary verification tools be lifted to check Dis and Mix equivalence?

We start by giving our experimental setup in Section 5.3.1, then we discuss the considered tools in Section 5.3.2, next we give the circuit collections used, and finally we define our equivalence challenges in Section 5.3.4.

5.3.1 Experimental setup

We conducted all benchmarks on a laptop running Linux Mint 21.2, equipped with an Intel Core i9-9880H CPU (2.3 GHz x 8), 31 GB of RAM, and a 500 GB SSD.

5.3.2 Considered tools

We evaluate state-of-the-art tools for verifying unitary circuits (Section 2.2.4). These tools include AutoQ [21, 1, 7] (tree automata)⁸, Feynman [2, 3, 4, 34] (path-sums)⁹, PyZX [47, 57] (ZX calculus)¹⁰, and QCEC [12, 13, 60] (ZX calculus, decision diagrams, and simulation)¹¹. All these tools implement sub-circuit unitary complete equivalence. We also evaluate our tool SQbricks-Verif, which is based on path-sums with its sub-circuit unitary complete equivalence and sub-circuit unitary partial equivalence implementations¹².

⁸AutoQ-2: version 2.0 (resp. POPL25ae), commit 24 (resp. 19) October 2024.

⁹Feyn-22: commit 13 October 2022; Feyn-24: commit 28 November 2024.

¹⁰PyZX: version 0.9.0.

¹¹QCEC: version 2.8.1.

¹²We do not use global-phase unitary complete equivalence and global phase unitary partial equivalence in our implementation because they are not necessary to achieve our goal.

Remark 5.3.1 (Note on SliQEC). SliQEC (Section 2.4.2.6) implements global-phase unitary complete equivalence (using his *total equivalence*) and full-circuit unitary partial equivalence (using his *partial equivalence*). Our comparison focuses on sub-circuit unitary complete equivalence and sub-circuit unitary partial equivalence. Since we already use QCEC, which employs the same technology (DD) and implements sub-circuit unitary complete equivalence, and since full-circuit unitary partial equivalence has not been implemented in SQbricks-Verif for scalable settings (see Remark 4.3.38), we did not include SliQEC in our evaluation.

We lift all tools using SQbricks-Lift to enable a uniform comparison in the unitary setting. We compare our overall performance for hybrid equivalence checking against QCEC [14] (in full hybrid mode) and VeriQC [42]. Since QCEC does not parse classical control from OpenQASM 2, we preprocess the circuits using `qiskit.qasm2.load` before sending them to QCEC. In the case of VeriQC, we cannot use the tool directly; therefore, we compare against the figures reported in [42]. We use the notation *standalone* when we apply QCEC directly and *lifting* when used in conjunction with SQbricks-Lift.

5.3.3 Circuit Collection

To evaluate the effectiveness of deferred measurement for verifying the equivalence of hybrid circuits, we selected two libraries of quantum circuits in OpenQASM 2 [27] commonly used in the literature: VeriQbench [17] and the Feynman library [2]. Additionally, we included a hybrid implementation of the Shor algorithm [71] inspired by the version provided in QASMBench [50]. We retained only the circuits that SQbricks could parse for each library, excluding circuits with named registers or macros. Note that several samples correspond to the same algorithms but with different parameter instantiations (typically size).

Overall, our collection encompasses all the leading families of quantum algorithms used in prior evaluations, including QFT, QPE, Shor, Grover, and Bernstein-Vazirani, among others. This results in a total of 420 unitary circuits and 204 hybrid circuits.

5.3.4 Circuit Equivalence Challenges

Once equipped with these circuits, we create equivalence checking challenges by either pairing functionally equivalent circuits or applying functionality-preserving transformations. We consider the following transformations:

1. Q: The `generate_preset_pass_manager` of IBM Qiskit¹³ [45] with an optimisation level of 3 (the highest level) to maximise circuit transformations (Section 2.5.2);
2. O: One-Way Measurement [29] is a circuit transformation based on measurement-based quantum computation, which restructures unitary circuits into a measurement-driven form and may introduce discarded qubits. Section 2.5.3 describes

¹³Qiskit: version 1.1.0.

our implementation O , while Section 5.1.1 establishes its correctness. We implement O in `owm.ml` (320 lines of code);

3. T : Teleportation Insertion refers to the transformation described in Section 2.5.4, which we implement in `teleportation.ml` with 130 lines of code.

We exclude circuits for which the verification process exceeds the time limit. Finally, our equivalence challenge categories are as follows:

- **DisFree (88+169)**: We consider two classes of challenges:
 - $U-C$: 169 pairs of unitary-hybrid circuits provided by VeriQbench that implement the same algorithms, either as a unitary U or as a hybrid circuit C (hybridisation, Section 2.5.5);
 - $C-Q(C)$: 88 pairs of a hybrid circuit C and its Qiskit-transformed version $Q(C)$.
- **Mix (347+406+88)**: We consider three classes of challenges:
 - $O(U)-U$: 347 pairs of a unitary circuit U and its OWM version $O(U)$;
 - $T(U)-U$: 406 pairs of a unitary circuit U and its Teleported version $T(U)$;
 - $O(C)-Q(C)$: 88 pairs of a hybrid circuit C transformed by $Q(C)$ and $O(C)$.
- **Dis** : 347 $O(U)-T(U)$ pairs from unitary circuits U transformed by $O(U)$ and $T(U)$.

5.4 Experimentation Evaluation

This section evaluates our approach for verifying hybrid circuits (Section 4.3.5.4). To achieve this, we begin by verifying the correctness of our approach and tools through a sanity check in Section 5.4.1. We then address the three research questions: RQ1 in Section 5.4.2, RQ2 in Section 5.4.3, and RQ3 in Section 5.4.4. Finally, we present additional findings in Section 5.4.5 regarding unexpected behaviours of Qiskit `pass_manager`.

5.4.1 Preliminary Experiment: Sanity Check

We conducted a sanity check for each equivalence verification task. This check consists of 73 equivalence tasks involving “mutants” (obtained by adding or removing a gate, modifying angles, or changing indices) of quantum circuits selected from the VeriQbench and QASMBench libraries. These mutations are constructed to ensure non-equivalence by design. Therefore, if a tool reports “equivalent”, this constitutes a false positive.

The results of the sanity checks are presented in Table 5.1. All versions of Feynman and SQbricks passed the sanity check thoroughly, as they did not report any false positives (i.e., they did not incorrectly prove equivalence for non-equivalent samples).

Table 5.1: Sanity check

Tool	Error	Total
AutoQ-2.0	1	63
AutoQ	1	63
Feyn-22	0	63
Feyn-24	0	63

Tool	Error	Total
PyZX	5	63
QCEC	8	63
SQbricks-Verif (---)	0	63
SQbricks-Verif (//)	0	63

All the other tools made at least one error in the sanity check. The failures were due to issues with rotation gates, either because of tiny angles, as seen with QCEC (5/63) and PyZX (5/63), or because of the inability to distinguish between control $R_z(\theta)$ and $R_z(\theta)$ gates, as seen with AutoQ (1/63). Especially:

- PyZX fails on slight angle modifications, specifically angles less than or equal to $\pi/2^{26}$, where it considers $R_z(\pi/67108864)$ as an identity operation;
- QCEC fails on slight angle modifications, specifically angles less than or equal to $\pi/2^{27}$, where it considers $U_1(\pi/134217728)$ ($R_z(\pi/134217728)$ up to global phase) as an identity operation;
- Both versions of AutoQ fail to distinguish between control $R_z(k)$ with and without control.

In conclusion, it appears that path-sums are less sensitive to errors than other representations, particularly with respect to rotation gates. While in the latter experiments we focus on the positive results, it is important to keep in mind that results from Feynman and SQbricks-Verif are more reliable.

5.4.2 RQ1: Evaluation on Hybrid Equivalence DisFree

We consider the DisFree hybrid equivalence challenges previously defined in our benchmark, together with our selected set of unitary verification tools.

Results: Table 5.2 presents a summary of the results. We are interested here in the lifted approaches (denoted L in the "Lift" column) for DisFree challenges. We can see that all methods based on path-sums, once lifted, achieve very high success rates here: both versions of Feynman and SV¹⁴ achieve perfect scores (169/169 and 88/88).

PyZX with lifting verifies equivalence across all families of circuits except for the Shor implementation inspired by QASMBench (C-Q(C)), where it cannot verify equivalence between two versions of Fredkin, as illustrated in Figure 5.6. PyZX also encounters more scalability issues than path-sum approaches. For example, it can verify the best

¹⁴SV (//) is inconclusive in two cases due to the incompleteness of the equational theory of the extension of path-sum to quantum multi-control (Section 4.3.3.2). Feynman uses the (---) approach, suggesting that this approach is more reliable at least in this experiment. This is confirmed in Section 5.4.4.

Table 5.2: Evaluation results of the generic application of deferred

Tool	Lift	Success							TO
		DisFree U-C	DisFree C-Q(C)	Mix O(U)-U	Mix T(U)-U	Mix O(C)-Q(C)	Dis O(U)-T(U)	Total	
QCEC	S	0 Wrong: 17	87	NA	NA	NA	NA	87 Wrong: 17	128
VeriQC (Cf. Table 5.3)	S	NW	NW	NA	NA	NA	NA	NA	NA
AutoQ-2.0	L	10	16	NA	NA	NA	NA	26	16
AutoQ	L	8	14	NA	NA	NA	NA	22	13
Feyn-22	L	169	88	NA	NA	NA	NA	257	0
Feyn-24	L	169	88	NA	NA	NA	NA	257	0
PyZX	L	80	81	NA	NA	NA	NA	161	95
QCEC	L	151	69	NA	NA	NA	NA	220	19
SV (—)	L	169	88	183	338	53	NA	831	128
SV (//)	L	169	86	137	343	5	137	877	364
#challenges		169	88	347	406	88	347	1445	

O: OWM, T: Tele, Q: Qiskit pass_manager, TO: Time Out (10 min) Wrong: equivalence check returns not equivalent when it should return equivalent, NA: Not Applicable, NW: Not Working.

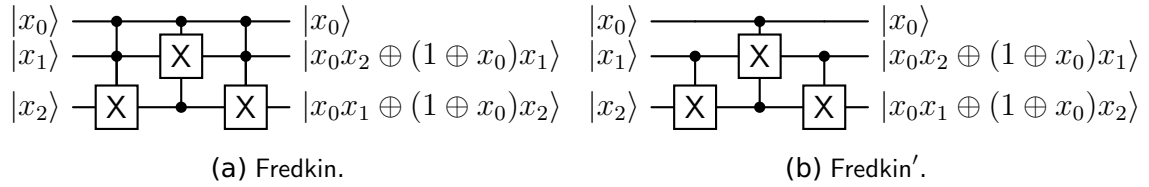


Figure 5.6: Minimal Inconclusive Example with PyZX: Fredkin $\neq$ Fredkin'.

QPE instance 41 in 517 seconds, while Feyn-24 verifies QPE instance 71 in only 7 seconds (U-C).

Finally, AutoQ verifies all families of circuits for small instances but encounters scaling issues on larger instances, resulting in timeouts (TO) or failures. For example, version 2.0 (resp. POPL25ae) in the C-Q(C) tasks for QPE is limited to size 6 (resp. 5). It takes 0.3 seconds (resp. 73 seconds) for size 5 and 296 (resp. TO) seconds for size 6.

Conclusion: Our technique does, in practice, lift unitary verification tools to the hybrid DisFree case. While path-sums allow reaching very high performance here, ZX-calcul offers reasonable performance, but AutoQ, which uses Tree Automata, seems to lag behind on these problems.

5.4.3 RQ2: Comparison with prior work on DisFree

As already stated, the two prior approaches that can handle DisFree hybrid equivalence are QCEC [14] and VeriQC [42]. For QCEC, we use a direct comparison on our benchmark, while for VeriQC, we use an indirect comparison against their published results.

Results for QCEC: The results of QCEC (standalone) and the lifted versions of Feynman and SV are comparable on the 88 challenges from C-Q(C) (see Table 5.2). On the other hand, QCEC appears unable to prove a single challenge from U-C (0/169), whereas Feynman and SV achieve perfect scores. Interestingly, all unitary methods, once lifted with our approach, perform better than QCEC (standalone) on this subcategory including QCEC itself! Moreover, QCEC standalone actually reports 17 *wrong* non-equivalences on these examples, including one for the C-Q(C) category, highlighting the correctness issues identified in the sanity checks. A minimal example of a circuit yielding a wrong non-equivalence proof is illustrated in Figure 5.7.



Figure 5.7: QCEC Minimal Example Wrong Non-Equivalence

Results for VeriQC: VeriQC [42] performs hybrid equivalence checking on DisFree¹⁵. Unfortunately, we were not able to use VeriQC for our experiments, despite having contacted the authors. We still perform some form of comparison with VeriQC, by comparing our methods against the published results on their benchmark. We have

¹⁵The paper claims the result over Dis [42, Definition 2], however Section [42, Section 4.5] explains how to check the equivalence of hybrid circuits with TDDs, limited to circuits without discard (our DisFree).

used the circuit collection available on the VeriQC Git¹⁶ [79]. This collection includes instances of bit/phase flip correction, teleportation, state_inj_S/T, and instances 11-16 of QFT (instances 11-21 in the paper) and 8-9 of PE (instances 8-14 in the paper). Results are presented in Table 5.3.

Table 5.3: VeriQC benchmark (time in seconds)

Task	VeriQC [42]	SQbricks-Lift					
		AutoQ	Feyn-24	PyZX	QCEC	SV (—)	SV (//)
qft_11	0.86	Err	0.01	0.34	0.00	0.01	0.01
qft_12	1.36	Err	0.01	0.62	0.00	0.02	0.01
qft_13	2.56	Err	0.01	1.08	0.00	0.03	0.01
qft_14	6.48	Err	0.01	1.67	0.00	0.03	0.01
qft_15	8.84	Err	0.02	2.20	0.00	0.05	0.02
qft_16	23.38	Err	0.02	2.97	0.02	0.06	0.02
pe_8	0.64	Err	0.01	0.14	0.01	0.02	0.01
pe_9	1.62	Err	0.01	0.27	0.01	0.02	0.01
bitflip	0.12	0.00	0.00	0.02	0.00	0.00	0.00
phaseflip	0.18	2.70	0.00	0.02	0.00	0.00	0.00
teleportation	0.01	0.00	0.00	0.00	0.00	0.00	0.00
state_inj_S	0.00	0.00	0.00	0.00	0.00	0.00	0.00
state_inj_T	0.01	0.00	0.00	0.00	0.00	0.00	0.00

With our lifting approach, all tools except AutoQ can verify the equivalence of the entire VeriQC benchmark faster than VeriQC itself. Especially, Feynman and SQbricks are several orders of magnitude faster on the hardest examples (qft_16: 23s vs. 0.02s and 0.06s). On average, for the most demanding examples here (qft_16 and pe_9), Feynman is 1250 times faster, and SV is between 312 times and 833 times faster, depending on the version.

Conclusion: Our lifting method turns out to be especially effective on DisFree challenges compared with prior work on hybrid equivalence checking. Indeed, while we manage to match QCEC performance on its best subcategory by lifting path-sum unitary verification, we beat both QCEC and VeriQC on all other cases with lifting any of the considered unitary verification tools but one (AutoQ), demonstrating the real power of our lifting approach.

5.4.4 RQ3: Hybrid equivalence evaluation beyond DisFree.

We consider here hybrid equivalence problems that fall outside the scope of prior work, i.e., challenges from Dis and Mix. As already explained, QCEC and VeriQC cannot handle them as they are limited to DisFree challenges. Moreover, as no unitary tools other than SV provide partial equivalence with separation check, only SV can be used with our lifting approach for these problems.

¹⁶Not all samples from the paper were available in the Git.

Results are reported in Table 5.2 for columns labelled Mix and Dis. Our method successfully addresses a significant portion of the Mix and Dis hybrid equivalence challenges, achieving 39% for Dis and between 58% (//) and 68% (—) for the subcategories of Mix. Regarding the differences between the two variants of SV, as previously mentioned, SV (//) is the only version capable of handling Dis. However, on the other hand, while both versions are mostly equivalent on all other subcategories, SV (—) performs better than SV (//) on O(U)-U (183 vs. 137) and completely outperforms it on O(C)-Q(C) (53 vs. 5).

Conclusion: Our approach, implemented in SV, effectively addresses hybrid equivalence-checking problems beyond the scope of current state-of-the-art tools, achieving a reasonable success rate. This establishes a firm baseline and provides an acceptable first solution for these problems. While the parallel equivalence checking (//) method can handle the entire Dis category, the sequential version (—) is limited to Mix but performs notably better in specific subcategories. This highlights the benefit of having both versions available.

5.4.5 Additional Findings

During our experiments, we observed unexpected behaviours with Q. Some circuit transformations are not validated. This leads us to conduct a more thorough investigation. We discovered two bugs in the Qiskit transpiler.

1. **Angle Approximation:** We observe that using Qiskit version 1.1.0 with its pass manager approximates small angles, such as $\pi/2^{42}$, to 0. This leads to incorrect circuit simplifications. Although we can reproduce this error with other circuits in Qiskit version 1.1.0, it appears to be fixed in Qiskit version 1.4.0.
2. **Introduction of Floats:** In version 1.4.0, the transformation process introduces floating-point values instead of rational numbers for at least two circuits (dqc_pe_22, dqc_qft_20, from VeriQbench). This results in a loss of equivalence. Further exploration shows that this process converts angles such as $3\pi/32$ into 0.294524. We contacted the Qiskit team about this issue. They respond that this is not a result of the circuit transformation but is linked to the OpenQASM 2 parser. This choice is made for performance reasons.

These observations underscore the practical utility of our method for identifying and mitigating approximation-related issues in quantum circuit transformations. Our approach provides a rigorous means to verify equivalence in such challenges and demonstrating its relevance to practical quantum computing workflows.

5.5 Discussion

Our study demonstrates the effectiveness of deferred measurement as an equivalence-checking mechanism for hybride-circuits. We applied three non-trivial transforma-

tions to various circuits and circuit families, including both unitary and hybrid-circuit. Through this approach, we successfully verified semantic equivalence with the original circuit using unitary tools.

5.5.1 Future Extensions and Research Directions

While our approach successfully addresses hybrid circuit equivalence under deferred measurement, several extensions remain as directions for future work:

1. Extending language support. The current implementation focuses on hybrid circuits involving measurement, initialisation, and classical control. Future work includes supporting richer classical features such as general classical writes, parametrised circuits, and structured program constructs such as loops.
2. Enriching the equational theory for multi-controlled gates. Although we extend the path-sum formalism to general multi-control gates, the equational theory has not been correspondingly extended. This may lead to inconclusive results (false negatives) in some cases. For instance, introducing dedicated reduction rules for controlled Hadamard gates would improve completeness.
3. Supporting qubit reuse after measurement. At present, measured qubits cannot be reused. Extending the framework to handle qubit recycling would broaden the applicability of the method to more general hybrid programming patterns.

5.5.2 New insights for the path-sum approach

Our study also yields observations regarding the path-sum (PS) formalism, which was initially developed for unitary circuits. This formalism is particularly well-suited for equivalence checking. Tools such as Feynman and SQbricks show reduced sensitivity to errors caused by small angles, unlike tools based on ZX calculus, such as QCEC or PyZX, as demonstrated in our sanity check. Furthermore, the PS formalism scales more effectively than AutoQ, which relies on tree automata. Path-sums also offer the flexibility to easily incorporate new features such as separation checks or partial equivalence. From a computational perspective, our experiments on path-sums indicate that strictly adhering to the Clifford+ $R_z(k)$ set, particularly when decomposing control $R_z(k)$ gates, can incur costs. Finally, within the PS calculus, preliminary exploratory tests conducted to determine which reduction rules were necessary for our benchmark suite showed that *HH* and *Elim* accounted for the vast majority of reductions. In these tests, the ω and *Case* rules were never triggered. This observation guided our implementation choices but is not intended as a general claim about their usefulness beyond the considered setting. What can be stated with confidence is that *HH* and *Elim* already enable substantial reductions in practice.

5.5.3 Insights on other approaches

Our experiments with other tools and technologies provide the following insights. Generally, all these tools lack partial equivalence and separation checks. This prevents them from being fully lifted to Dis equivalence with our method.

QCEC uses various methods for checking equivalence, including ZX calculus, simulation, and decision diagrams. It implements sub-circuit hybrid equivalence [14] by using deferred measurement and full-circuit unitary partial equivalence [61]. To determine which part of the register is observed, full-circuit unitary partial equivalence uses the final measurement: each wire with a final measurement is an observed qubit. The first idea is to combine sub-circuit hybrid equivalence and full-circuit unitary partial equivalence options to implement full-circuit hybrid partial parallel equivalence. However, there is no correspondence between the observed part and the deferred measurement. For example, in the teleportation case (Example 2.2.50), qubits q_0 and q_1 are measured, so QCEC considers them as observed qubits, although the qubit that must be considered as observed is q_2 . More fundamentally, we primarily want to check sub-circuit unitary partial equivalence, which is impossible with QCEC, particularly because of the lack of a separation check. Finally, our sanity check shows that it does not detect minor angle differences. AutoQ, which uses tree automata, does not support certain interesting features, such as partial equivalence and struggles to scale. PyZX uses ZX calculus, and our sanity check shows that it allows small-angle errors and errors with equivalence of two implementations of Fredkin.

Chapter 6

Conclusion

Our work aims to improve the practical usability of quantum computing in the circuit model. We focus on a central aspect of this model: circuit transformation. Our objective is to provide a practical method for establishing that a quantum circuit transformation preserves semantics. To this end, we develop SQbricks, a framework for verifying hybrid circuit equivalence. We then use this framework to validate circuit transformations. In particular, we observe that the most challenging transformations are those that produce hybrid circuits with discards, as they fall outside the scope of standard unitary verification tools.

To address the challenge of hybrid circuits, we demonstrated that **deferred measurement**, applied generically as a pre-processing step for arbitrary tools limited to unitary circuits, is effective. Furthermore, we showed that adding a **separation test** enables us to handle even the most difficult transformations (specifically those that introduce discards). We show that **path-sums** are a formalism well-suited for formal verification, applicable to both deductive verification (with Qbricks) and symbolic execution (with SQbricks). Path-sum allow a straightforward implementation of our separation test and the extension to multi-controlled gates. We also provide an exhaustive classification of equivalence relations. The results of the evaluation of our approach against the state of the art show that for cases already reachable by existing methods, our approach yields superior performance by extending tools limited to unitary circuits to the hybrid case. Moreover, for cases currently unreachable by state-of-the-art tools, our approach achieved promising results. Finally, we observed that among all rewrite rules, the HH rule performs the vast majority of the work by reducing the number of path variables.

As future work, extending the path-sum rewriting system to better handle certain multi-controlled gates would improve completeness. Although the HH rule accounts for the vast majority of reductions in our experiments, some patterns remain outside the current equational theory. In particular, controlled Hadamard gates are not reduced by the present rewriting system, which can prevent simplifications in constructions such as deferred-measurement coin-toss patterns.

In addition, two concrete extensions would increase the scope of the framework. First, supporting structured control flow such as loops would allow the verification of iterative quantum programs. Second, enabling qubit reuse after measurement would make it possible to handle hybrid programs that rely on qubit recycling patterns.

Bibliography

- [1] Parosh Aziz Abdulla, Yo-Ga Chen, Yu-Fang Chen, Lukáš Holík, Ondřej Lengál, Jyun-Ao Lin, Fang-Yi Lo, and Wei-Lun Tsai. Verifying quantum circuits with level-synchronized tree automata. *Proc. ACM Program. Lang.*, 9(POPL), January 2025. doi:10.1145/3704868.
- [2] Matthew Amy. Towards Large-scale Functional Verification of Universal Quantum Circuits. *Electron. Proc. Theor. Comput. Sci.*, 287:1–21, January 2019. arXiv:1805.06908, doi:10.4204/EPTCS.287.1.
- [3] Matthew Amy. Complete equational theories for the sum-over-paths with unbalanced amplitudes. *Electronic Proceedings in Theoretical Computer Science*, 384:127141, August 2023. URL: <http://dx.doi.org/10.4204/EPTCS.384.8>, doi:10.4204/eptcs.384.8.
- [4] Matthew Amy and Joseph Lunderville. Linear and non-linear relational analyses for quantum program optimization. *Proc. ACM Program. Lang.*, 9(POPL), January 2025. doi:10.1145/3704873.
- [5] Frank Arute, Kunal Arya, Ryan Babbush, Dave Bacon, Joseph C. Bardin, Rami Barends, Rupak Biswas, Sergio Boixo, Fernando G. S. L. Brandao, David A. Buell, Brian Burkett, Yu Chen, Zijun Chen, Ben Chiaro, Roberto Collins, William Courtney, Andrew Dunsworth, Edward Farhi, Brooks Foxen, Austin Fowler, Craig Gidney, Marissa Giustina, Rob Graff, Keith Guerin, Steve Habegger, Matthew P. Harrigan, Michael J. Hartmann, Alan Ho, Markus Hoffmann, Trent Huang, Travis S. Humble, Sergei V. Isakov, Evan Jeffrey, Zhang Jiang, Dvir Kafri, Kostyantyn Kechedzhi, Julian Kelly, Paul V. Klimov, Sergey Knysh, Alexander Korotkov, Fedor Kostritsa, David Landhuis, Mike Lindmark, Erik Lucero, Dmitry Lyakh, Salvatore Mandrà, Jarrod R. McClean, Matthew McEwen, Anthony Megrant, Xiao Mi, Kristel Michielsen, Masoud Mohseni, Josh Mutus, Ofer Naaman, Matthew Neeley, Charles Neill, Murphy Yuezhen Niu, Eric Ostby, Andre Petukhov, John C. Platt, Chris Quintana, Eleanor G. Rieffel, Pedram Roushan, Nicholas C. Rubin, Daniel Sank, Kevin J. Satzinger, Vadim Smelyanskiy, Kevin J. Sung, Matthew D. Trevithick, Amit Vainsencher, Benjamin Villalonga, Theodore White, Z. Jamie Yao, Ping Yeh, Adam Zalcman, Hartmut Neven, and John M. Martinis. Quantum supremacy using

a programmable superconducting processor. *Nature*, 574(7779):505510, October 2019. URL: <http://dx.doi.org/10.1038/s41586-019-1666-5>, doi:10.1038/s41586-019-1666-5.

- [6] AutoQ 2 Git. <https://github.com/fmlab-iis/AutoQ/tree/2.0>.
- [7] AutoQ Git. <https://github.com/fmlab-iis/AutoQ>.
- [8] Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. cvc5: A versatile and industrial-strength smt solver. In *Tools and Algorithms for the Construction and Analysis of Systems: 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 27, 2022, Proceedings, Part I*, page 415442, Berlin, Heidelberg, 2022. Springer-Verlag. doi:10.1007/978-3-030-99524-9_24.
- [9] Fabian Bauer-Marquart, Stefan Leue, and Christian Schilling. *symQV: Automated Symbolic Verification of Quantum Programs*, page 181198. Springer International Publishing, 2023. URL: http://dx.doi.org/10.1007/978-3-031-27481-7_12, doi:10.1007/978-3-031-27481-7_12.
- [10] Charles H. Bennett and Gilles Brassard. Quantum cryptography: Public key distribution and coin tossing. *Theoretical Computer Science*, 560:711, December 2014. URL: <http://dx.doi.org/10.1016/j.tcs.2014.05.025>, doi:10.1016/j.tcs.2014.05.025.
- [11] Charles H. Bennett, Gilles Brassard, Claude Crépeau, Richard Jozsa, Asher Peres, and William K. Wootters. Teleporting an unknown quantum state via dual classical and einstein-podolsky-rosen channels. *Phys. Rev. Lett.*, 70:1895–1899, Mar 1993. URL: <https://link.aps.org/doi/10.1103/PhysRevLett.70.1895>, doi:10.1103/PhysRevLett.70.1895.
- [12] Lukas Burgholzer and Robert Wille. Improved dd-based equivalence checking of quantum circuits. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 127–132, 2020. doi:10.1109/ASP-DAC47756.2020.9045153.
- [13] Lukas Burgholzer and Robert Wille. The power of simulation for equivalence checking in quantum computing. In *Proceedings of the 57th ACM/EDAC/IEEE Design Automation Conference, DAC '20*. IEEE Press, 2020.
- [14] Lukas Burgholzer and Robert Wille. Handling Non-Unitaries in Quantum Circuit Equivalence Checking. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*, pages 529–534, July 2022. arXiv:2106.01099, doi:10.1145/3489517.3530482.

- [15] Elisa Bäumer, Vinay Tripathi, Alireza Seif, Daniel Lidar, and Derek S. Wang. Quantum fourier transform using dynamic circuits, 2024. URL: <https://arxiv.org/abs/2403.09514>, arXiv:2403.09514.
- [16] Christophe Chareton, Sébastien Bardin, François Bobot, Valentin Perrelle, and Benoît Valiron. An automated deductive verification framework for circuit-building quantum programs. In *Programming Languages and Systems: 30th European Symposium on Programming, ESOP 2021, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2021, Luxembourg City, Luxembourg, March 27–April 1, 2021, Proceedings 30*, pages 148–177. Springer International Publishing, 2021.
- [17] Kean Chen, Wang Fang, Ji Guan, Xin Hong, Mingyu Huang, Junyi Liu, Qisheng Wang, and Mingsheng Ying. Veriqbench: A benchmark for multiple types of quantum circuits, 2022. URL: <https://arxiv.org/abs/2206.10880>, arXiv:2206.10880.
- [18] Tian-Fu Chen, Jie-Hong R. Jiang, and Min-Hsiu Hsieh. Partial equivalence checking of quantum circuits. In *2022 IEEE International Conference on Quantum Computing and Engineering (QCE)*, page 594604. IEEE, September 2022. URL: <http://dx.doi.org/10.1109/QCE53715.2022.00082>, doi:10.1109/qce53715.2022.00082.
- [19] Tian-Fu Chen, Jie-Hong R. Jiang, and Min-Hsiu Hsieh. Partial equivalence checking of quantum circuits. In *2022 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 594–604, 2022. doi:10.1109/QCE53715.2022.00082.
- [20] Yu-Fang Chen, Kai-Min Chung, Min-Hsiu Hsieh, Wei-Jia Huang, Ondřej Lengál, Jyun-Ao Lin, and Wei-Lun Tsai. Autoq 2.0: From verification of quantum circuits to verification of quantum programs, 2024. URL: <https://arxiv.org/abs/2411.09121>, arXiv:2411.09121.
- [21] Yu-Fang Chen, Kai-Min Chung, Ondřej Lengál, Jyun-Ao Lin, and Wei-Lun Tsai. Autoq: An automata-based quantum circuit verifier. In Constantin Enea and Akash Lal, editors, *Computer Aided Verification*, pages 139–153, Cham, 2023. Springer Nature Switzerland.
- [22] Sylvain Conchon, Mohamed Iguernlala, and Alain Mebsout. Altgr-ergo, a graphical user interface for the smt solver alt-ergo. *Electronic Proceedings in Theoretical Computer Science*, 239:113, January 2017. URL: <http://dx.doi.org/10.4204/EPTCS.239.1>, doi:10.4204/eptcs.239.1.
- [23] D. Coppersmith. An approximate fourier transform useful in quantum factoring, 2002. URL: <https://arxiv.org/abs/quant-ph/0201067>, arXiv:quant-ph/0201067.

- [24] CoqQ Git. <https://github.com/coq-quantum/CoqQ>.
- [25] Thierry Coquand and Gérard Huet. The calculus of constructions. *Information and Computation*, 76(2):95–120, 1988. URL: <https://www.sciencedirect.com/science/article/pii/0890540188900053>, doi:10.1016/0890-5401(88)90005-3.
- [26] A. D. Córcoles, Maika Takita, Ken Inoue, Scott Lekuch, Zlatko K. Mineev, Jerry M. Chow, and Jay M. Gambetta. Exploiting dynamic quantum circuits in a quantum algorithm with superconducting qubits. *Phys. Rev. Lett.*, 127:100501, Aug 2021. URL: <https://link.aps.org/doi/10.1103/PhysRevLett.127.100501>, doi:10.1103/PhysRevLett.127.100501.
- [27] Andrew W. Cross, Lev S. Bishop, John A. Smolin, and Jay M. Gambetta. Open quantum assembly language, 2017. URL: <https://arxiv.org/abs/1707.03429>, arXiv:1707.03429.
- [28] Pascal Cuoq, Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-c: a software analysis perspective. In *Proceedings of the 10th International Conference on Software Engineering and Formal Methods, SEFM’12*, page 233247, Berlin, Heidelberg, 2012. Springer-Verlag. doi:10.1007/978-3-642-33826-7_16.
- [29] Vincent Danos, Elham Kashefi, and Prakash Panangaden. The Measurement Calculus, April 2007. arXiv:0704.1263.
- [30] Leonardo De Moura and Nikolaj Bjørner. Z3: an efficient smt solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems, TACAS’08/ETAPS’08*, page 337340, Berlin, Heidelberg, 2008. Springer-Verlag.
- [31] Simon J Devitt, William J Munro, and Kae Nemoto. Quantum error correction for beginners. *Reports on Progress in Physics*, 76(7):076001, June 2013. URL: <http://dx.doi.org/10.1088/0034-4885/76/7/076001>, doi:10.1088/0034-4885/76/7/076001.
- [32] Clotilde Erard, Alain Giorgetti, and Jérôme Ricciardi. Towards random and enumerative testing for ocaml and whyml properties. *Software Quality Journal*, 30(1):253279, March 2022. doi:10.1007/s11219-021-09572-z.
- [33] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm, 2014. URL: <https://arxiv.org/abs/1411.4028>, arXiv:1411.4028.
- [34] Feynman Git. <https://github.com/meamy/feynman>.

- [35] Jean-Christophe Filliâtre and Andrei Paskevich. Why3 — where programs meet provers. In Matthias Felleisen and Philippa Gardner, editors, *Programming Languages and Systems*, pages 125–128, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.
- [36] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, STOC '96, page 212219, New York, NY, USA, 1996. Association for Computing Machinery. doi:10.1145/237814.237866.
- [37] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum algorithm for linear systems of equations. *Physical Review Letters*, 103(15), October 2009. URL: <http://dx.doi.org/10.1103/PhysRevLett.103.150502>, doi:10.1103/physrevlett.103.150502.
- [38] Patrick Hayden, Kevin Milner, and Mark M. Wilde. Two-message quantum interactive proofs and the quantum separability problem. In *2013 IEEE Conference on Computational Complexity*, page 156167. IEEE, June 2013. URL: <http://dx.doi.org/10.1109/CCC.2013.24>, doi:10.1109/ccc.2013.24.
- [39] Miguel Herrero-Collantes and Juan Carlos Garcia-Escartin. Quantum random number generators. *Reviews of Modern Physics*, 89(1), February 2017. URL: <http://dx.doi.org/10.1103/RevModPhys.89.015004>, doi:10.1103/revmodphys.89.015004.
- [40] Kesha Hietala, Robert Rand, Shih-Han Hung, Liyi Li, and Michael Hicks. Proving Quantum Programs Correct. In Liron Cohen and Cezary Kaliszyk, editors, *12th International Conference on Interactive Theorem Proving (ITP 2021)*, volume 193 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 21:1–21:19, Dagstuhl, Germany, 2021. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2021.21>, doi:10.4230/LIPIcs.ITP.2021.21.
- [41] C. A. R. Hoare. An axiomatic basis for computer programming. *Commun. ACM*, 12(10):576580, October 1969. doi:10.1145/363235.363259.
- [42] Xin Hong, Yuan Feng, Sanjiang Li, and Mingsheng Ying. Equivalence checking of dynamic quantum circuits. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, ICCAD '22, New York, NY, USA, 2022. Association for Computing Machinery. doi:10.1145/3508352.3549479.
- [43] Xin Hong, Xiangzhen Zhou, Sanjiang Li, Yuan Feng, and Mingsheng Ying. A tensor network based decision diagram for representation of quantum circuits. *ACM Trans. Des. Autom. Electron. Syst.*, 27(6), June 2022. doi:10.1145/3514355.

- [44] Yipeng Huang and Margaret Martonosi. Statistical assertions for validating patterns and finding bugs in quantum programs. In Srilatha Bobbie Manne, Hillery C. Hunter, and Erik R. Altman, editors, *Proceedings of the 46th International Symposium on Computer Architecture, ISCA 2019*, pages 541–553. ACM, 2019. doi: [10.1145/3307650.3322213](https://doi.org/10.1145/3307650.3322213).
- [45] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. Quantum computing with qiskit, 2024. URL: <https://arxiv.org/abs/2405.08810>, arXiv:2405.08810.
- [46] James C. King. Symbolic execution and program testing. *Commun. ACM*, 19(7):385394, July 1976. doi: [10.1145/360248.360252](https://doi.org/10.1145/360248.360252).
- [47] Aleks Kissinger and John van de Wetering. PyZX: Large Scale Automated Diagrammatic Reasoning. In Bob Coecke and Matthew Leifer, editors, *Proceedings 16th International Conference on Quantum Physics and Logic*, Chapman University, Orange, CA, USA., 10-14 June 2019, volume 318 of *Electronic Proceedings in Theoretical Computer Science*, pages 229–241. Open Publishing Association, 2020. doi: [10.4204/EPTCS.318.14](https://doi.org/10.4204/EPTCS.318.14).
- [48] Aleks Kissinger and John van de Wetering. Simulating quantum circuits with zx-calculus reduced stabiliser decompositions. *Quantum Science and Technology*, 7(4):044001, July 2022. URL: <http://dx.doi.org/10.1088/2058-9565/ac5d20>, doi: [10.1088/2058-9565/ac5d20](https://doi.org/10.1088/2058-9565/ac5d20).
- [49] Aleks Kissinger, John van de Wetering, and Renaud Vilmart. Classical Simulation of Quantum Circuits with Partial and Graphical Stabiliser Decompositions. In François Le Gall and Tomoyuki Morimae, editors, *17th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2022)*, volume 232 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 5:1–5:13, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TQC.2022.5>, doi: [10.4230/LIPIcs.TQC.2022.5](https://doi.org/10.4230/LIPIcs.TQC.2022.5).
- [50] Ang Li, Samuel Stein, Sriram Krishnamoorthy, and James Ang. Qasmbench: A low-level qasm benchmark suite for nisq evaluation and simulation, 2022. URL: <https://arxiv.org/abs/2005.13018>, arXiv:2005.13018.
- [51] Gushu Li, Li Zhou, Nengkun Yu, Yufei Ding, Mingsheng Ying, and Yuan Xie. Projection-based runtime assertions for testing and debugging quantum programs. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):150:1–150:29, 2020. doi: [10.1145/3428218](https://doi.org/10.1145/3428218).
- [52] Michael A. Nielsen, Isaac Chuang, and Lov K. Grover. Quantum computation and quantum information. *American Journal of Physics*, 70(5):558–

- 559, 05 2002. [arXiv:https://pubs.aip.org/aapt/ajp/article-pdf/70/5/558/7529938/558_2_online.pdf](https://pubs.aip.org/aapt/ajp/article-pdf/70/5/558/7529938/558_2_online.pdf), doi:10.1119/1.1463744.
- [53] Tobias Nipkow, Markus Wenzel, and Lawrence C. Paulson. *Isabelle/HOL: a proof assistant for higher-order logic*. Springer-Verlag, Berlin, Heidelberg, 2002.
 - [54] Tom Peham, Lukas Burgholzer, and Robert Wille. Equivalence checking of quantum circuits with the zx-calculus. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 12(3):662675, September 2022. URL: <http://dx.doi.org/10.1109/JETCAS.2022.3202204>, doi:10.1109/jetcas.2022.3202204.
 - [55] Yuxiang Peng, Kesha Hietala, Runzhou Tao, Liyi Li, Robert Rand, Michael Hicks, and Xiaodi Wu. A formally certified end-to-end implementation of shor’s factorization algorithm, 2022. URL: <https://arxiv.org/abs/2204.07112>, arXiv:2204.07112.
 - [56] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik, and Jeremy L. OBrien. A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 5(1), July 2014. URL: <http://dx.doi.org/10.1038/ncomms5213>, doi:10.1038/ncomms5213.
 - [57] PyZX documentation. <https://pyzx.readthedocs.io/en/latest/gettingstarted.html>.
 - [58] PyZX Git. <https://github.com/zxcalc/pyzx>.
 - [59] Qbricks Git. <https://github.com/Qbricks/qbricks.github.io/tree/main>.
 - [60] MQT-QCEC documentation. <https://mqt.readthedocs.io/projects/qcec/en/latest/>.
 - [61] MQT-QCEC observational partial equivalence documentation. https://mqt.readthedocs.io/projects/qcec/en/latest/library/configuration/Optimizations.html#mqt.qcec.Configuration.Optimizations.transform_dynamic_circuit.
 - [62] MQT-QCEC Git. <https://github.com/munich-quantum-toolkit/qcec>.
 - [63] Quasimodo Git. <https://github.com/trishullab/Quasimodo>.
 - [64] QuiZX Git. <https://github.com/zxcalc/quizx>.
 - [65] Neilson Carlos Leite Ramalho, Higor Amario de Souza, and Marcos Lordello Chaim. Testing and debugging quantum programs: The road to 2030, 2024. URL: <https://arxiv.org/abs/2405.09178>, arXiv:2405.09178.
 - [66] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM*, 21(2):120126, February 1978. doi:10.1145/359340.359342.

- [67] Rock Doc. <https://rocq-prover.org/>.
- [68] Joschka Roffe. Quantum error correction: an introductory guide. *Contemporary Physics*, 60(3):226245, July 2019. URL: <http://dx.doi.org/10.1080/00107514.2019.1667078>, doi:10.1080/00107514.2019.1667078.
- [69] Colin Runciman, Matthew Naylor, and Fredrik Lindblad. Smallcheck and lazy small-check: automatic exhaustive testing for small values. *SIGPLAN Not.*, 44(2):3748, September 2008. doi:10.1145/1543134.1411292.
- [70] Aaron Sander, Lukas Burgholzer, and Robert Wille. Equivalence checking of quantum circuits via intermediary matrix product operator, 2024. URL: <https://arxiv.org/abs/2410.10946>, arXiv:2410.10946.
- [71] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):14841509, October 1997. URL: <http://dx.doi.org/10.1137/S0097539795293172>, doi:10.1137/s0097539795293172.
- [72] Meghana Sistla, Swarat Chaudhuri, and Thomas Reps. Symbolic quantum simulation with quasimodo. In *Computer Aided Verification: 35th International Conference, CAV 2023, Paris, France, July 1722, 2023, Proceedings, Part III*, page 213225, Berlin, Heidelberg, 2023. Springer-Verlag. doi:10.1007/978-3-031-37709-9_11.
- [73] SliQEC Git. <https://github.com/NTU-ALComLab/SliQEC>.
- [74] Sliqsim Git. <https://github.com/NTU-ALComLab/SliQSim>.
- [75] Sqr Git. <https://github.com/inQWIRE/VQO>.
- [76] symqv Git. <https://github.com/fabianbauermarquart/symQV>.
- [77] Yuan-Hung Tsai, Jie-Hong R. Jiang, and Chiao-Shan Jhang. Bit-slicing the hilbert space: Scaling up accurate quantum circuit simulation to a new level, 2020. URL: <https://arxiv.org/abs/2007.09304>, arXiv:2007.09304.
- [78] Twist Git. <https://github.com/psg-mit/twist-popl22>.
- [79] VeriQC dynamic quantum circuits benchmarks. <https://github.com/Veriqc/EC-for-Dynamic-Quantum-Circuits/tree/main/Benchmarks2>.
- [80] VeriQC. <https://github.com/Veriqc>.
- [81] Chun-Yu Wei, Yuan-Hung Tsai, Chiao-Shan Jhang, and Jie-Hong R. Jiang. Accurate bdd-based unitary operator manipulation for scalable and robust quantum circuit verification. In *Proceedings of the 59th ACM/IEEE Design Automation Conference, DAC '22*, page 523528, New York, NY, USA, 2022. Association for Computing Machinery. doi:10.1145/3489517.3530481.

- [82] Why3 Git. <https://gitlab.inria.fr/why3/why3>.
- [83] Charles Yuan, Christopher McNally, and Michael Carbin. Twist: sound reasoning for purity and entanglement in quantum programs. *Proceedings of the ACM on Programming Languages*, 6(POPL):132, January 2022. URL: <http://dx.doi.org/10.1145/3498691>, doi:10.1145/3498691.
- [84] Jake Zappin, Trevor Stalnaker, Oscar Chaparro, and Denys Poshyvanyk. Challenges and practices in quantum software testing and debugging: Insights from practitioners, 2025. URL: <https://arxiv.org/abs/2506.17306>, arXiv:2506.17306.
- [85] Han-Sen Zhong, Hui Wang, Yu-Hao Deng, Ming-Cheng Chen, Li-Chao Peng, Yi-Han Luo, Jian Qin, Dian Wu, Xing Ding, Yi Hu, Peng Hu, Xiao-Yan Yang, Wei-Jun Zhang, Hao Li, Yuxuan Li, Xiao Jiang, Lin Gan, Guangwen Yang, Lixing You, Zhen Wang, Li Li, Nai-Le Liu, Chao-Yang Lu, and Jian-Wei Pan. Quantum computational advantage using photons. *Science*, 370(6523):14601463, December 2020. URL: <http://dx.doi.org/10.1126/science.abe8770>, doi:10.1126/science.abe8770.
- [86] Li Zhou, Gilles Barthe, Pierre-Yves Strub, Junyi Liu, and Mingsheng Ying. Coqq: Foundational verification of quantum programs. *Proc. ACM Program. Lang.*, 7(POPL), January 2023. doi:10.1145/3571222.

Tables of Symbols and Notations

This appendix compiles the symbols and notations used in this document. For clarity, we use the following abbreviations:

- Ch.: Chapter
- Sec.: Section
- Def.: Definition
- Not.: Notation
- Ex.: Example
- Rem.: Remark
- Pred.: Predicate

Table A.1: General

Symbol	Description	Reference
$\emptyset$	Empty Set	Not. 2.1.2
$\setminus$	Set Difference	Not. 2.1.2
$\simeq$	Set Equivalence	Not. 2.1.2
$\min(s)$	Return smallest element and the remain	Not. 2.1.2
$\emptyset$	Empty Bit-String	Not. 2.1.3
$\cong_{(s,s')}$	Bit-String Equivalence	Not. 2.1.3
$x_{[i \leftarrow b]}$	Bit-String Substitution	Not. 2.1.3
$x_{[cb \leftarrow b]}$	Bit-String Multi-Substitution	Not. 2.1.3
$*$	Undefined Bit-String	Not. 2.1.3
$x _s$	Sub-Bit-String	Not. 2.1.3
$\text{Indices}(x)$	Indices of x	Not. 2.1.3

Table A.2: Quantum State

Symbol	Description	Reference
$ z $	Modulus	Def. 2.2.1
$ v\rangle$	Ket (Dirac Notation)	Not. 2.2.2
$\langle v $	Bra (Dirac Notation)	Not. 2.2.2
$\otimes$	Tensor Product	Def. 2.2.3
$\langle v w\rangle$	Inner Product	Def. 2.2.4
$ v\rangle\langle w $	Outer Product	Def. 2.2.5
$\mathcal{H}^{\otimes n}$	n -Qubits Hilbert Space	Not. 2.2.6
$ x\rangle = x_0x_1 \dots x_i \dots x_{n-1}\rangle$	n -Qubits Basis State	Def. 2.2.7, Not. 2.2.23
$ \psi\rangle, \phi\rangle, \xi\rangle$	Quantum State (QS)	Def. 2.2.8, Not. 2.2.23
$ \psi\rangle_i$	QS indexed by i	Not. 2.2.23
$ \psi\phi\rangle, \psi \otimes \phi\rangle$	Separable States	Not. 2.2.23
$\{ x\rangle\}_{x \in \{0,1\}^n}$	n -Qubits Standard Basis	Def. 2.2.9
$=$	Vect-Equal	Not. 2.2.10
GP	Global-Phase	Not. 2.2.11
CP	Conditional-Phase	Def. 2.2.14
$\approx_{\text{gp}}$	GP-Equiv	Def. 2.2.12
$\approx_{\text{cp}}$	CP-Equiv	Def. 2.2.14
β_{00}	Bell State	Ex. 2.2.18
$\mathbf{c}$	Classical Data	Def. 2.2.19
ρ	Extended Mixed State	Def. 2.2.20, Not. 2.2.21
$=$	ρ Equality	Def. 2.2.22
$\sim_{(s_1, s_2)}$	Vect-Equiv-Map	Not. 2.2.23
$\sim_{(s_1, s_2, \text{gp})}$	GP-Equiv-Map	Not. 2.2.23
$\sim_{(s_1, s_2, \text{cp})}$	CP-Equiv-Map	Not. 2.2.23
Inp	Input	Not. 2.2.23
Obs	Observed	Not. 2.2.23
Init	Initialised	Not. 2.2.23
Di	Discarded	Not. 2.2.23

Table A.3: Semantics Common Elements

Symbol	Description	Reference
$\mathcal{H}^{\otimes n}$	n -Qubits Hilbert Space	Def. 2.2.6
$ x\rangle = x_0 x_1 \dots x_i \dots x_{n-1}\rangle$	n -qubits Separated State	Def. 2.2.16, 2.2.23
$ \psi\rangle, \phi\rangle, \xi\rangle$	General Quantum State	Not. 2.2.23
$\overline{M}$	Matrix Conjugate	Def. 2.2.27
M^T	Matrix Transpose	Def. 2.2.28
$M^\dagger$	Matrix Ajoint	Def. 2.2.29
I	Identity Matrix	Def. 2.2.31
U	Unitary Matrix	Def. 2.2.32
H	Hadamard Matrix	Def. 2.2.32
ρ	Extended Mixed State	Def. 2.2.1.5
PS	Path-Sum	Ch. 2.3
HOPS	High Order Path-Sum	Sec. 3.1
EPS	Extended Path-Sum	Sec. 4.3.3.2
c	Measurement Result	Sec. 2.2.2
$c _{\text{Obs}}$	Meas. Result Observed	Not. 4.3.42
$c _{\text{Di}}$	Meas. Result Discarded	Not. 4.3.42
p_c	Probability of Measured c	Not. 2.2.21
$p(\psi, \text{Obs}, x)$	Subset Projection Prob.	Def. 4.3.4
$d(\psi)$	Complete Reg. Meas. (CRM)	Def. 2.2.24
$d(\psi, \text{Obs})$	CRM Partial Obs. (CRMPO)	Def. 4.3.5
$d(p, \psi, \text{Obs})$	CRMPO For Hybrid Circuit	Def. 4.3.45
$ \psi_c\rangle, \phi_c\rangle, \xi_c\rangle$	QS Indexed by c	Def. 2.2.26
$\mathcal{T}$	Circuit Tuple	Def. 4.3.7
$\mathcal{QC}$	Unitary $\mathcal{T}$	Not. 4.3.8
$\mathcal{HQC}$	Hybrid $\mathcal{T}$	Not. 4.3.8
$(--)$	$\mathcal{T}$ Temp. Comp.	Def. 4.3.9
$(//)$	$\mathcal{T}$ Spat. Comp.	Def. 4.3.10
$\mathcal{QC}^\dagger$	$\mathcal{QC}$ Inverse	Def. 4.3.11

Table A.4: Graphical Representation of Unitary Circuits

Symbol	Description	Reference(s)
H	Hadamard Gate	Sec.2.2.4.1,
I	Identity Gate	Sec.2.2.4.1
X	Qubit Flip Gate	Sec. 2.2.4.1
Z	Phase Flip Gate	Sec. 2.2.4.1
CX	Control Not Gate	Def. 2.2.33
Toffoli, CCNOT	Control CX	Def. 2.2.34
Swap, SWAP	Swap quantum states	Def. 2.2.35
Fredkin, CSWAP	Control Swap	Def. 2.2.37

Table A.5: Brick-Based Representation of Unitary Circuit

Symbol	Description	Reference
G	Unary Gates	Def. 2.2.38
U	Unitary Circuit (Unit-Cir)	Def. 2.2.38
(—)	Temporal Composition	Sec. 2.2.4.2
(//)	Spatial Composition	Sec. 2.2.4.2
$U^\dagger$	Unit-Cir Inverse	Sec. 2.2.39
$\llbracket U \rrbracket$	Brick-Based Semantics	Sec. 2.2.40
H	Hadamard Gate	Sec. 2.2.4.2
I	Identity Gate	Sec. 2.2.4.2
GP	GP Gate	Sec. 2.2.4.2
$R_x(\theta)$	X Rotation Gate	Sec. 2.2.4.2
$R_y(\theta)$	Y Rotation Gate	Sec. 2.2.4.2
$R_z(\theta)$	Z Rotation Gate	Sec. 2.2.4.2
$U_1(\theta)$	$e^{2\pi i \frac{\theta}{2}} R_z(\theta)$	Sec. 2.2.4.2
X	Qubit Flip Gate	Sec. 2.2.4.2
Y	$iR_y(\pi)$ Gate	Sec. 2.2.4.2
Z	Phase Flip Gate	Sec. 2.2.4.2
S	$U_1\left(\frac{\pi}{2}\right)$	Sec. 2.2.4.2
T	$U_1\left(\frac{\pi}{4}\right)$ Gate	Sec. 2.2.4.2
CX	Control Not Gate	Sec. 2.2.4.2

Table A.6: QASM Representation of Unitary Circuit

Symbol	Description	Reference
G	Unary Gates (UG)	Def. 2.2.41
GA	UG Application	Sec. 2.2.4.3
E	Empty Circuit	Sec. 2.2.4.3
Ins	Unitary Instruction	Def. 2.2.41
U	Unitary Circuit	Def. 2.2.41
qb	Set of Qubit Indices	Sec. 2.2.4.3
G _{qb}	G Application to qb	Not. 2.2.43
(--)	Temporal Composition	Sec. 2.2.4.3
(//)	Spatial Composition	Sec. 2.2.4.3
[[U]]	QASM Semantics	Def. 2.2.44
Anc	Ancilla Qubit	Def. 2.2.45

Table A.7: Hybrid Circuit

Symbol	Description	Reference
C	Hybrid Circuit	Sec. 2.2.5
Ins	Hybrid Instruction(s)	Sec. 2.2.5
cb	Set of Bit Indices	Sec. 2.2.5
InitQ(qb)	Qubit(s) Initialisation	Sec. 2.2.5
if(cb, GA)	Classical Control	Sec. 2.2.5
Meas(qb, cb)	Measurement (M)	Sec. 2.2.5
Not(cb)	Classical Not	Sec. 2.2.5
[[C]] _H	Hybrid Semantics	Def. 2.2.53
Di	Discard	Def. 2.2.55

Table A.8: Path-Sum

Symbol	Description	Reference
$\mathcal{D}$	Dyadic Number	Def. 2.3.1
x	Input Signature	Def. 2.3.3
y	Path Variable(s)	Def. 2.3.3
$p(x, y)$	Phase Polynomial	Def. 2.3.3
$f(x, y)$	Output Signature	Def. 2.3.3
$\langle y, p(x, y), f(x, y) \rangle$	Path-Sum (PS)	Def. 2.3.3
$\mathcal{V}(PS)$	PS Concretisation Function	Def. 2.3.4
$\llbracket U \rrbracket_{PS}$	PS Semantics	Sec. 2.3.2
I	Identity	Def. 2.3.7
H	Hadamard	Def. 2.3.7
$R_z(k)$	Z Rotation	Def. 2.3.7
$R_z(k)^\dagger$	Inverse of $R_z(k)$	Def. 2.3.7
CX	Control Not	Def. 2.3.7
$(--)$	Temporal Composition	Def. 2.3.7
$(//)$	Spatial Composition	Def. 2.3.7
HH	Reduction Rule	Def. 2.3.10
$Elim$	Reduction Rule	Def. 2.3.10

Table A.9: Classifications

Symbol	Description	Reference
QC	Quantum Circuit	Sec. 2.4
U	Unitary Circuit	Sec. 2.4
C	Hybrid Circuit	Sec. 2.4
PQC	Parametrised QC	Def. 2.4.1
PUC	Parametrised U	Def. 2.4.1
PHC	Parametrised C	Def. 2.4.1
QP	Quantum Program	Def. 2.4.2
Sim	Simulator	Sec. 2.4
SE	Symbolic Execution	Sec. 2.4
P	Proof	Sec. 2.4
DisFree	No Discard	Sec. 4.1.2
Mix	One Circuits contains Discards	Sec. 4.1.2
Dis	Two Circuits contains Discards	Sec. 4.1.2
SC	Sub-Circuit	Sec. 4.1.3
FC	Full-Circuit	Sec. 4.1.3

Table A.10: Circuit Transformations

Symbol	Description	Reference
O	One-Way Measurement	Sec. 2.5.3
T	Teleportation Insertion	Sec. 2.5.4
CInits	Count Nb Init.	Def. 4.3.52
dI	Extract Initialisations	Def. 4.3.53
Inits	Set of Qubits Initialisation	Sec. 4.3.5.2
CMeass	Count Nb Meas.	Def. 4.3.58
m _C	Map of Measurements from C	Def. 4.3.59
tCQ	Extract Measurements	Def. 4.3.60
mA	Measurements Application	Def. 4.3.64
$ \psi\rangle_{S[\text{qb}=i]}$	State with Fixed Value	Not. 4.3.66
DM	Deferred Measurement	Def. 4.3.69
IUM	Init. Unit. Meas. Circuit	Def. 4.3.70

Table A.11: Qbricks

Symbol	Description	Reference
Unary	Qbricks Unary Gates	Def. 3.1.1
Skip	Qbricks Identity Gate	Sec. 3.1
Phase	Global Phase Gate	Sec. 3.1
$R_{zp}(k)$	Qbricks $U_1\left(\frac{2\pi i}{2^k}\right)$ Gate	Sec. 3.1
Binary	Qbricks Binary Gates	Def. 3.1.1
Cnot	Qbricks Cnot Gate	Sec. 3.1
Bricks_Cnot	Cnot (0,1)	Sec. 3.1
Swap	Qbricks Swap Gate	Sec. 3.1
Bricks_Swap	Swap (0,1)	Sec. 3.1
Ternary	Qbricks Ternary Gates	Def. 3.1.1
Toffoli	Qbricks Toffoli Gate	Sec. 3.1
Bricks_Toffoli	Toffoli (0,1,2)	Sec. 3.1
Fredkin	Qbricks Fredkin Gate	Sec. 3.1
Bricks_Fredkin	Fredkin (0,1,2)	Sec. 3.1
Gate	Qbricks Gates	Def. 3.1.1
Sequence	Qbricks Temporal Composition	Def. 3.1.1
Parallel	Qbricks Spatial Composition	Def. 3.1.1
(—)	Infix Sequence	Not. 3.1.2
(//)	Infix Parallel	Not. 3.1.2
Cont	Qbricks Sub-Circuit Control	Def. 3.1.1
Place	Qbricks Sub-Circuit Placement	Def. 3.1.1
Qb	Qbricks Circuit	Def. 3.1.1
$\llbracket U \rrbracket_{Qb}$	Qbricks Semantics	Sec. 3.1.2
G_i	Qbricks Unary Application to Qubit i	Not. 3.1.5

Table A.12: OpenQASM

Symbol	Description	Reference
$U(\theta, \phi, \lambda)$	Arbitrary Single-Qubit Gate	Def. 3.2.1
Cnot	Control Not Gate	Def. 3.2.1
$U(\theta, \phi, \lambda) \text{ } ta$	U Application	Not. 3.2.3
$Cnot \text{ } co \text{ } ta$	Cnot Application	Not. 3.2.3
OUnary	OpenQASM Unary Gates	Def. 3.2.8
OBinary	OpenQASM Binary Gates	Def. 3.2.8
OQ	OpenQASM Unitary Circuit	Def. 3.2.8

Table A.13: Qbricks to OpenQASM

Symbol	Description	Reference
parallel_del parallel_free	Removing Parallel Function Parallel Free Predicate	Def. 3.2.9 Pred. 3.2.11
crz_decomp	Control R_z Decomposition	Def. 3.2.13
crzp_decomp	Control R_{zp} Decomposition	Def. 3.2.15
ccz_decomp	Control Control Z Decomposition	Def. 3.2.17
toffoli_decomp	Toffoli Decomposition	Def. 3.2
bricks_toffoli_decomp	Toffoli Decomposition	Def. 3.2
fredkin_decomp	Fredkin Decomposition	Def. 3.5
bricks_fredkin_decomp	Fredkin Decomposition	Def. 3.5
ternary_gates_del	Removing Ternary Gates	Def. 3.2.23
ternary_gates_free	Ternary Gates Free	Pred. 3.2.25
placed_atomic	Remov. Sub-Circ. Param. Placement	Def. 3.2.29
atomic_place	Sub-Circ. Param. Plac. Free	Pred. 3.2.31
place_atomic	Remov. Sub-Circ. Placement	Def. 3.2.34
cont_depth	Control Nesting Depth	Def. 3.2.38
control_del_step	Controls Decomposition	Def. 3.2.39
control_del	Controls Decomposition	Def. 3.2.43
into_qasm	Complete Conversion	Def. 3.2.46
conv_rot	Angle Conversion	Def. 3.2.49
to_qasm	From Qb to OQ	Def. 3.2.50

Table A.14: Equivalences/Equalities Symbols

Symbol	Description	Reference
$\equiv$	Complete Equiv.	Not. 4.0.1
$\not\equiv$	Not Comp. Equiv.	Not. 4.0.1
$\dot{\equiv}$	Partial Equiv.	Not. 4.0.1
$//$	Parallel Algorithm	Sec. 4.1.4
$--$	Sequential Algorithm	Sec. 4.1.4
$\equiv$	SubCir-Unit-Comp-Par	Def. 4.2.2
$\equiv(--)$	SubCir-Unit-Comp-Seq	Def. 4.2.3
$\equiv_{gp}$	GP-Unit-Comp-Comp-Par	Def. 4.2.2
$\equiv(--,gp)$	GP-Unit-Comp-Comp-Seq	Def. 4.2.7
$\equiv_f$	FullCir-Unit-Comp-Par	Def. 4.3.18
$\equiv_H$	SubCir-Hyb-Comp-Par	Def. 4.2.18
SV	QASM Unit. Cir. Restriction	Def. 4.3.2
$\dot{\equiv}$	SubCir-Unit-Part-Par	Def.4.3.12
$\dot{\equiv}(--)$	SubCir-Unit-Part-Seq	Def.4.3.13
$\dot{\equiv}_{gp}$	GP-Unit-Comp-Part-Par	Def.4.3.12
$\dot{\equiv}(--,gp)$	GP-Unit-Comp-Part-Seq	Def.4.3.13
$\dot{\equiv}_f$	FullCir-Unit-Part-Par	Def.4.3.18
$\dot{\equiv}_H$	SubCir-Hyb-Part-Par	Def.4.3.43
$\dot{\equiv}_{(H,gp)}$	GP-Hyb-Part-Par	Def.4.3.44
$\dot{\equiv}_{(H,f)}$	FullCir-Hyb-Part-Par	Def.4.3.47
$\dot{\equiv}(--,ps)$	SubCir-PS-Part-Seq	Def.4.3.29
$\dot{\equiv}_{(ps)}$	SubCir-PS-Part-Par	Def.4.3.27
$\dot{\equiv}(--,gp,ps)$	GP-PS-Part-Seq	Def.4.3.33
$\dot{\equiv}_{(gp,ps)}$	GP-PS-Part-Par	Def.4.3.31
$\dot{\equiv}_{(f,ps)}$	FullCir-PS-Part-Par	Def.4.3.35
$\dot{\equiv}_H$	SubCir-Hyb-Part-Par	Def.4.3.43
$\dot{\equiv}_{(H,gp)}$	GP-Hyb-Part-Par	Def.4.3.44
$\dot{\equiv}_{(H,f)}$	FullCir-Hyb-Part-Par	Def.4.3.18
$\dot{\equiv}_S$	Map Partial Equiv.	Def.4.3.77